main.py

001-01

```python
from sj_generator.app.bootstrap import main


if __name__ == "__main__":
    main()

```

sj_generator/__init__.py

002-01

```python
__all__ = []

```

sj_generator/app/__init__.py

003-01

```python


```

sj_generator/app/bootstrap.py

004-01

```python
from sj_generator.presentation.qt.main_wizard import main as run_desktop_app


def main() -> None:
    run_desktop_app()

```

sj_generator/application/__init__.py

005-01

```python


```

sj_generator/application/dedupe/__init__.py

006-01

```python


```

sj_generator/application/dedupe/service.py

007-01

```python
from __future__ import annotations

import math
import re
from dataclasses import dataclass
from pathlib import Path

import jieba

from sj_generator.domain.entities import Question
from sj_generator.infrastructure.persistence.excel_repo import load_questions
from sj_generator.infrastructure.persistence.sqlite_repo import load_all_questions


_WS_RE = re.compile(r"\s+")
_TOKEN_RE = re.compile(r"[\u4e00-\u9fffA-Za-z0-9]+")
_JIEBA_READY = False


@dataclass(frozen=True)
class DedupeHit:
    left_file: Path
    left_number: str
    left_stem: str
    right_file: Path
    right_number: str
    right_stem: str
    similarity: float
    right_level_path: str = ""


def list_xlsx_in_folder(folder: Path) -> list[Path]:
    if not folder.exists() or not folder.is_dir():
        return []
    return sorted([p for p in folder.rglob("*.xlsx") if p.is_file()])


def dedupe_between_repos(
    *,
    left_repo: Path,
    other_repos: list[Path],
    threshold: float,
    limit: int = 200,
) -> list[DedupeHit]:
    left_questions = [q for q in load_questions(left_repo) if q.stem.strip()]
    other_files = [p for p in other_repos if p.resolve() != left_repo.resolve()]

    right_questions_by_file: list[tuple[Path, list]] = []
    for repo in other_files:
        qs = [q for q in load_questions(repo) if q.stem.strip()]
        if qs:
            right_questions_by_file.append((repo, qs))

    corpus: list[str] = []
    corpus.extend([q.stem for q in left_questions])
    for _, qs in right_questions_by_file:
        corpus.extend([q.stem for q in qs])

    tfidf, norms = _build_tfidf(corpus)

    left_count = len(left_questions)
    idx = 0
    left_vecs = tfidf[:left_count]
    left_norms = norms[:left_count]
    idx += left_count

    hits: list[DedupeHit] = []
    for file_path, qs in right_questions_by_file:
        right_vecs = tfidf[idx : idx + len(qs)]
        right_norms = norms[idx : idx + len(qs)]
```

```python
        idx += len(qs)

        for li, lq in enumerate(left_questions):
            for ri, rq in enumerate(qs):
                sim = _cosine(left_vecs[li], left_norms[li], right_vecs[ri], right_norms[ri])
                if sim >= threshold:
                    hits.append(
                        DedupeHit(
                            left_file=left_repo,
                            left_number=lq.number,
                            left_stem=lq.stem,
                            right_file=file_path,
                            right_number=rq.number,
                            right_stem=rq.stem,
                            similarity=sim,
                        )
                    )

    hits.sort(key=lambda x: x.similarity, reverse=True)
    if limit > 0:
        hits = hits[:limit]
    return hits


def dedupe_between_questions_and_repos(
    *,
    left_questions: list[Question],
    left_file: Path,
    other_repos: list[Path],
    threshold: float,
    limit: int = 200,
) -> list[DedupeHit]:
    filtered_left_questions = [q for q in left_questions if q.stem.strip()]
    other_files = [p for p in other_repos if p.resolve() != left_file.resolve()]

    right_questions_by_file: list[tuple[Path, list[Question]]] = []
    for repo in other_files:
        qs = [q for q in load_questions(repo) if q.stem.strip()]
        if qs:
            right_questions_by_file.append((repo, qs))

    corpus: list[str] = []
    corpus.extend([q.stem for q in filtered_left_questions])
    for _, qs in right_questions_by_file:
        corpus.extend([q.stem for q in qs])

    tfidf, norms = _build_tfidf(corpus)

    left_count = len(filtered_left_questions)
    idx = 0
    left_vecs = tfidf[:left_count]
    left_norms = norms[:left_count]
    idx += left_count

    hits: list[DedupeHit] = []
    for file_path, qs in right_questions_by_file:
        right_vecs = tfidf[idx : idx + len(qs)]
        right_norms = norms[idx : idx + len(qs)]
        idx += len(qs)

        for li, lq in enumerate(filtered_left_questions):
            for ri, rq in enumerate(qs):
                sim = _cosine(left_vecs[li], left_norms[li], right_vecs[ri], right_norms[ri])
                if sim >= threshold:
                    hits.append(
                        DedupeHit(
                            left_file=left_file,
                            left_number=lq.number,
                            left_stem=lq.stem,
                            right_file=file_path,
```

```python
                            right_number=rq.number,
                            right_stem=rq.stem,
                            similarity=sim,
                        )
                    )

    hits.sort(key=lambda x: x.similarity, reverse=True)
    if limit > 0:
        hits = hits[:limit]
    return hits


def dedupe_between_questions_and_db(
    *,
    left_questions: list[Question],
    left_file: Path,
    db_path: Path,
    threshold: float,
    limit: int = 200,
) -> list[DedupeHit]:
    filtered_left_questions = [q for q in left_questions if q.stem.strip()]
    db_questions = [q for q in load_all_questions(db_path) if q.stem.strip()]

    corpus: list[str] = []
    corpus.extend([q.stem for q in filtered_left_questions])
    corpus.extend([q.stem for q in db_questions])

    tfidf, norms = _build_tfidf(corpus)

    left_count = len(filtered_left_questions)
    left_vecs = tfidf[:left_count]
    left_norms = norms[:left_count]
    right_vecs = tfidf[left_count:]
    right_norms = norms[left_count:]

    hits: list[DedupeHit] = []
    for li, lq in enumerate(filtered_left_questions):
        for ri, rq in enumerate(db_questions):
            sim = _cosine(left_vecs[li], left_norms[li], right_vecs[ri], right_norms[ri])
            if sim >= threshold:
                hits.append(
                    DedupeHit(
                        left_file=left_file,
                        left_number=lq.number,
                        left_stem=lq.stem,
                        right_file=db_path,
                        right_number=rq.id,
                        right_stem=rq.stem,
                        right_level_path=rq.level_path,
                        similarity=sim,
                    )
                )

    hits.sort(key=lambda x: x.similarity, reverse=True)
    if limit > 0:
        hits = hits[:limit]
    return hits


def _normalize_text(text: str) -> str:
    return _WS_RE.sub(" ", text).strip()

def _tokenize(text: str) -> list[str]:
    global _JIEBA_READY
    if not _JIEBA_READY:
        jieba.setLogLevel(20)
        jieba.initialize()
        _JIEBA_READY = True
    text = _normalize_text(text)
    if not text:
```

```python
        return []
    cleaned = " ".join(_TOKEN_RE.findall(text))
    if not cleaned:
        return []
    tokens = [t.strip() for t in jieba.cut(cleaned) if t.strip()]
    return [t for t in tokens if len(t) > 1]


def _build_tfidf(texts: list[str]) -> tuple[list[dict[str, float]], list[float]]:
    docs = [_tokenize(t) for t in texts]

    df: dict[str, int] = {}
    for tokens in docs:
        seen = set(tokens)
        for tok in seen:
            df[tok] = df.get(tok, 0) + 1

    n = max(1, len(docs))
    idf: dict[str, float] = {}
    for tok, d in df.items():
        idf[tok] = math.log((1 + n) / (1 + d)) + 1.0

    vectors: list[dict[str, float]] = []
    norms: list[float] = []
    for tokens in docs:
        if not tokens:
            vectors.append({})
            norms.append(0.0)
            continue

        tf: dict[str, int] = {}
        for tok in tokens:
            tf[tok] = tf.get(tok, 0) + 1

        total = sum(tf.values())
        vec: dict[str, float] = {}
        for tok, c in tf.items():
            vec[tok] = (c / total) * idf.get(tok, 0.0)

        vectors.append(vec)
        norms.append(math.sqrt(sum(v * v for v in vec.values())))

    return vectors, norms


def _cosine(
    a: dict[str, float],
    a_norm: float,
    b: dict[str, float],
    b_norm: float,
) -> float:
    if a_norm <= 0 or b_norm <= 0:
        return 0.0
    if len(a) > len(b):
        a, b = b, a
        a_norm, b_norm = b_norm, a_norm
    dot = 0.0
    for k, v in a.items():
        dot += v * b.get(k, 0.0)
    return dot / (a_norm * b_norm)

```

sj_generator/application/exporting/__init__.py

008-01

```python


```

sj_generator/application/exporting/batch_folderize.py

009-01

```python
from __future__ import annotations

from dataclasses import dataclass
from datetime import date
from pathlib import Path

from openpyxl import load_workbook

from sj_generator.infrastructure.persistence.excel_repo import save_questions
from sj_generator.infrastructure.exporting.export_md import export_questions_to_markdown
from sj_generator.domain.entities import Question


@dataclass(frozen=True)
class BatchFolderizeResult:
    source_path: Path
    target_dir: Path
    target_xlsx: Path
    target_md: Path
    question_count: int


def process_excel_to_folder_mode(path: Path, *, export_date: date | None = None) -> BatchFolderizeResult:
    src = Path(path)
    if not src.exists():
        raise FileNotFoundError(src)
    if src.suffix.lower() != ".xlsx":
        raise ValueError(f"仅支持 xlsx 文件：{src}")

    target_dir = src.parent / src.stem
    target_dir.mkdir(parents=True, exist_ok=True)
    target_xlsx = target_dir / src.name
    if src.resolve() != target_xlsx.resolve():
        if target_xlsx.exists():
            target_xlsx.unlink()
        src.replace(target_xlsx)

    questions = load_compatible_questions(target_xlsx)
    save_questions(target_xlsx, questions)

    target_md = target_dir / f"{target_xlsx.stem}.md"
    md_text = export_questions_to_markdown(
        excel_file_name=target_xlsx.stem,
        export_date=export_date or date.today(),
        questions=questions,
    )
    target_md.write_text(md_text, encoding="utf-8")
    return BatchFolderizeResult(
        source_path=src,
        target_dir=target_dir,
        target_xlsx=target_xlsx,
        target_md=target_md,
        question_count=len(questions),
    )


def process_excel_files_to_folder_mode(
    paths: list[Path], *, export_date: date | None = None
) -> list[BatchFolderizeResult]:
    return [process_excel_to_folder_mode(path, export_date=export_date) for path in paths]


def load_compatible_questions(path: Path) -> list[Question]:
    wb = load_workbook(path)
    ws = wb["questions"] if "questions" in wb.sheetnames else wb.active
    header_row = [cell.value for cell in next(ws.iter_rows(min_row=1, max_row=1))]
    header_map: dict[str, int] = {}
    for idx, value in enumerate(header_row):
        if isinstance(value, str) and value.strip():
            header_map[value.strip()] = idx
```

```python
    required = ["编号", "题目", "答案", "解析"]
    missing = [name for name in required if name not in header_map]
    if missing:
        raise ValueError(f"题库表头缺少列：{', '.join(missing)}")

    result: list[Question] = []
    for row in ws.iter_rows(min_row=2, values_only=True):
        number = _to_text(row[header_map["编号"]])
        stem = _to_text(row[header_map["题目"]])
        options = _to_text(row[header_map["选项"]]) if "选项" in header_map else ""
        answer = _to_text(row[header_map["答案"]])
        analysis = _to_text(row[header_map["解析"]])
        if not any([number, stem, options, answer, analysis]):
            continue
        result.append(
            Question(
                number=number,
                stem=stem,
                options=options,
                answer=answer,
                analysis=analysis,
            )
        )
    return result


def _to_text(value: object) -> str:
    if value is None:
        return ""
    if isinstance(value, str):
        return value.strip()
    return str(value).strip()
```

sj_generator/application/importing/__init__.py

010-01

```python
from .batch_ai_import import BatchAiImportResult, BatchAiProgress, process_source_files_to_folders
from .services import commit_questions_to_db, import_questions_for_sources, load_docx_sources, resolve_question_refs_for_sources

__all__ = [
    "BatchAiImportResult",
    "BatchAiProgress",
    "commit_questions_to_db",
    "import_questions_for_sources",
    "load_docx_sources",
    "process_source_files_to_folders",
    "resolve_question_refs_for_sources",
]

```

```python
from __future__ import annotations

from concurrent.futures import ThreadPoolExecutor, as_completed
from dataclasses import dataclass
from datetime import date
from pathlib import Path
import re
from typing import Callable

from sj_generator.infrastructure.llm.client import LlmClient
from sj_generator.infrastructure.llm.explanations import ExplanationInputs, generate_explanation_result
from sj_generator.infrastructure.llm.import_questions import import_questions_from_sources
from sj_generator.infrastructure.persistence.excel_repo import save_questions
from sj_generator.infrastructure.exporting.export_md import export_questions_to_markdown
from sj_generator.infrastructure.document.source_reader import read_source_text
from sj_generator.domain.entities import Question
from sj_generator.shared.paths import app_paths, common_mistakes_md_path


_INVALID_FILENAME_CHARS = re.compile(r'[<>:"/\\\\|?*]+')


@dataclass(frozen=True)
class BatchAiImportResult:
    source_path: Path
    target_dir: Path
    target_xlsx: Path
    target_md: Path
    question_count: int


@dataclass(frozen=True)
class BatchAiProgress:
    file_index: int
    total_files: int
    file_name: str
    stage: str
    message: str
    question_current: int = 0
    question_total: int = 0
    question_count: int = 0


def process_source_files_to_folders(
    *,
    paths: list[Path],
    client: LlmClient | None = None,
    analysis_client: LlmClient | None = None,
    kimi_client: LlmClient | None = None,
    qwen_client: LlmClient | None = None,
    client_factory: Callable[[], LlmClient] | None = None,
    analysis_client_factory: Callable[[], LlmClient] | None = None,
    kimi_client_factory: Callable[[], LlmClient] | None = None,
    qwen_client_factory: Callable[[], LlmClient] | None = None,
    max_workers: int = 1,
    max_question_workers: int = 1,
    max_analysis_workers: int = 1,
    export_date: date | None = None,
    progress_cb: Callable[[str], None] | None = None,
    progress_info_cb: Callable[[BatchAiProgress], None] | None = None,
) -> list[BatchAiImportResult]:
    total = len(paths)
    if total <= 0:
        return []

    worker_count = max(1, int(max_workers))
    if worker_count == 1:
        if client is None or kimi_client is None or qwen_client is None:
            raise ValueError("串行模式需要提供 client、kimi_client 和 qwen_client。")
        results: list[BatchAiImportResult] = []
```

```python
        for idx, raw_path in enumerate(paths, start=1):
            results.append(
                _process_single_source(
                    path=Path(raw_path),
                    idx=idx,
                    total=total,
                    client=client,
                    analysis_client=analysis_client or client,
                    kimi_client=kimi_client,
                    qwen_client=qwen_client,
                    client_factory=client_factory,
                    analysis_client_factory=analysis_client_factory or client_factory,
                    kimi_client_factory=kimi_client_factory,
                    qwen_client_factory=qwen_client_factory,
                    max_question_workers=max_question_workers,
                    max_analysis_workers=max_analysis_workers,
                    export_date=export_date,
                    progress_cb=progress_cb,
                    progress_info_cb=progress_info_cb,
                )
            )
        return results

    if client_factory is None or kimi_client_factory is None or qwen_client_factory is None:
        raise ValueError("并发模式需要提供 client_factory、kimi_client_factory 和 qwen_client_factory。")

    results_by_index: dict[int, BatchAiImportResult] = {}
    with ThreadPoolExecutor(max_workers=min(worker_count, total)) as executor:
        future_map = {
            executor.submit(
                _process_single_source,
                path=Path(raw_path),
                idx=idx,
                total=total,
                client=client_factory(),
                analysis_client=(analysis_client_factory or client_factory)(),
                kimi_client=kimi_client_factory(),
                qwen_client=qwen_client_factory(),
                client_factory=client_factory,
                analysis_client_factory=analysis_client_factory or client_factory,
                kimi_client_factory=kimi_client_factory,
                qwen_client_factory=qwen_client_factory,
                max_question_workers=max_question_workers,
                max_analysis_workers=max_analysis_workers,
                export_date=export_date,
                progress_cb=progress_cb,
                progress_info_cb=progress_info_cb,
            ): idx
            for idx, raw_path in enumerate(paths, start=1)
        }
        for future in as_completed(future_map):
            idx = future_map[future]
            results_by_index[idx] = future.result()
    return [results_by_index[idx] for idx in sorted(results_by_index.keys())]


def _process_single_source(
    *,
    path: Path,
    idx: int,
    total: int,
    client: LlmClient,
    analysis_client: LlmClient,
    kimi_client: LlmClient,
    qwen_client: LlmClient,
    client_factory: Callable[[], LlmClient] | None,
    analysis_client_factory: Callable[[], LlmClient] | None,
    kimi_client_factory: Callable[[], LlmClient] | None,
    qwen_client_factory: Callable[[], LlmClient] | None,
    max_question_workers: int,
```

```python
    max_analysis_workers: int,
    export_date: date | None,
    progress_cb: Callable[[str], None] | None,
    progress_info_cb: Callable[[BatchAiProgress], None] | None,
) -> BatchAiImportResult:
    _emit_progress(
        idx=idx,
        total=total,
        file_name=path.name,
        stage="reading",
        message=f"({idx}/{total}) {path.name}：读取资料…",
        progress_cb=progress_cb,
        progress_info_cb=progress_info_cb,
    )
    text = read_source_text(path).strip()
    _emit_progress(
        idx=idx,
        total=total,
        file_name=path.name,
        stage="counting",
        message=f"({idx}/{total}) {path.name}：AI 解析准备中…",
        progress_cb=progress_cb,
        progress_info_cb=progress_info_cb,
    )
    imported = import_questions_from_sources(
        client=client,
        kimi_client=kimi_client,
        qwen_client=qwen_client,
        client_factory=client_factory,
        kimi_client_factory=kimi_client_factory,
        qwen_client_factory=qwen_client_factory,
        sources=[(path, text)],
        strategy="per_question",
        max_question_workers=max_question_workers,
        progress_cb=lambda msg: _emit_progress(
            idx=idx,
            total=total,
            file_name=path.name,
            stage="processing",
            message=msg,
            progress_cb=progress_cb,
            progress_info_cb=progress_info_cb,
        ),
        progress_count_cb=lambda current, total_questions: _emit_progress(
            idx=idx,
            total=total,
            file_name=path.name,
            stage="processing",
            message=f"({idx}/{total}) {path.name}：已完成 {current}/{total_questions} 题",
            question_current=current,
            question_total=total_questions,
            progress_cb=None,
            progress_info_cb=progress_info_cb,
        ),
    )
    questions_with_analysis = _fill_missing_explanations(
        questions=imported.questions,
        client=analysis_client,
        client_factory=analysis_client_factory,
        source_name=path.name,
        idx=idx,
        total=total,
        max_analysis_workers=max_analysis_workers,
        progress_cb=progress_cb,
        progress_info_cb=progress_info_cb,
    )
    safe_name = _sanitize_filename(path.stem) or path.stem or f"未命名_{idx}"
    target_dir = path.parent / safe_name
    target_dir.mkdir(parents=True, exist_ok=True)
    target_xlsx = target_dir / f"{safe_name}.xlsx"
```

```python
    _emit_progress(
        idx=idx,
        total=total,
        file_name=path.name,
        stage="saving_xlsx",
        message=f"({idx}/{total}) {path.name}：写入 xlsx…",
        question_count=len(questions_with_analysis),
        progress_cb=progress_cb,
        progress_info_cb=progress_info_cb,
    )
    save_questions(target_xlsx, questions_with_analysis)
    target_md = target_dir / f"{safe_name}.md"
    md_text = export_questions_to_markdown(
        excel_file_name=safe_name,
        export_date=export_date or date.today(),
        questions=questions_with_analysis,
    )
    _emit_progress(
        idx=idx,
        total=total,
        file_name=path.name,
        stage="saving_md",
        message=f"({idx}/{total}) {path.name}：生成 Markdown…",
        question_count=len(questions_with_analysis),
        progress_cb=progress_cb,
        progress_info_cb=progress_info_cb,
    )
    target_md.write_text(md_text, encoding="utf-8")
    _emit_progress(
        idx=idx,
        total=total,
        file_name=path.name,
        stage="done",
        message=f"({idx}/{total}) {path.name}：完成，输出 {len(questions_with_analysis)} 题。",
        question_current=len(questions_with_analysis),
        question_total=len(questions_with_analysis),
        question_count=len(questions_with_analysis),
        progress_cb=progress_cb,
        progress_info_cb=progress_info_cb,
    )
    return BatchAiImportResult(
        source_path=path,
        target_dir=target_dir,
        target_xlsx=target_xlsx,
        target_md=target_md,
        question_count=len(questions_with_analysis),
    )


def _sanitize_filename(name: str) -> str:
    s = (name or "").strip()
    s = _INVALID_FILENAME_CHARS.sub("_", s)
    s = s.replace("\n", " ").replace("\r", " ").replace("\t", " ")
    return s.strip(" .")


def _fill_missing_explanations(
    *,
    questions: list[Question],
    client: LlmClient,
    client_factory: Callable[[], LlmClient] | None,
    source_name: str,
    idx: int,
    total: int,
    max_analysis_workers: int,
    progress_cb: Callable[[str], None] | None,
    progress_info_cb: Callable[[BatchAiProgress], None] | None,
) -> list[Question]:
    if not questions:
        return questions
```

```python
    resource_paths = app_paths()
    root_dir = resource_paths.base_dir
    ref_dir = resource_paths.reference_resource_dir
    reference_md_paths = sorted(ref_dir.glob("*.md"), key=lambda p: p.name) if ref_dir.exists() else []
    include_common_mistakes = common_mistakes_md_path(root_dir).exists()

    tasks = [(i, q) for i, q in enumerate(questions) if not q.analysis.strip()]
    if not tasks:
        return questions

    updated = list(questions)
    total_tasks = len(tasks)
    worker_count = max(1, int(max_analysis_workers))
    if worker_count <= 1:
        for current, (row, q) in enumerate(tasks, start=1):
            result = _generate_one_explanation(
                client=client,
                question=q,
                reference_md_paths=reference_md_paths,
                include_common_mistakes=include_common_mistakes,
                root_dir=root_dir,
            )
            updated[row] = Question(
                number=q.number,
                stem=q.stem,
                options=q.options,
                answer=result.answer_text or q.answer,
                analysis=result.analysis_text,
                question_type=q.question_type,
                choice_1=q.choice_1,
                choice_2=q.choice_2,
                choice_3=q.choice_3,
                choice_4=q.choice_4,
            )
            _emit_progress(
                idx=idx,
                total=total,
                file_name=source_name,
                stage="generating_analysis",
                message=f"({idx}/{total}) {source_name}：已生成解析 {current}/{total_tasks}",
                question_current=current,
                question_total=total_tasks,
                question_count=len(questions),
                progress_cb=progress_cb,
                progress_info_cb=progress_info_cb,
            )
        return updated

    if client_factory is None:
        raise ValueError("解析并发需要提供 client_factory。")

    _emit_progress(
        idx=idx,
        total=total,
        file_name=source_name,
        stage="generating_analysis",
        message=f"({idx}/{total}) {source_name}：并发生成解析中…",
        question_current=0,
        question_total=total_tasks,
        question_count=len(questions),
        progress_cb=progress_cb,
        progress_info_cb=progress_info_cb,
    )
    completed = 0
    with ThreadPoolExecutor(max_workers=min(worker_count, total_tasks)) as executor:
        future_map = {
            executor.submit(
                _generate_one_explanation,
                client=client_factory(),
                question=q,
```

```python
                reference_md_paths=reference_md_paths,
                include_common_mistakes=include_common_mistakes,
                root_dir=root_dir,
            ): (row, q)
            for row, q in tasks
        }
        for future in as_completed(future_map):
            row, q = future_map[future]
            result = future.result()
            updated[row] = Question(
                number=q.number,
                stem=q.stem,
                options=q.options,
                answer=result.answer_text or q.answer,
                analysis=result.analysis_text,
                question_type=q.question_type,
                choice_1=q.choice_1,
                choice_2=q.choice_2,
                choice_3=q.choice_3,
                choice_4=q.choice_4,
            )
            completed += 1
            _emit_progress(
                idx=idx,
                total=total,
                file_name=source_name,
                stage="generating_analysis",
                message=f"({idx}/{total}) {source_name}：已生成解析 {completed}/{total_tasks}",
                question_current=completed,
                question_total=total_tasks,
                question_count=len(questions),
                progress_cb=progress_cb,
                progress_info_cb=progress_info_cb,
            )
    return updated


def _generate_one_explanation(
    *,
    client: LlmClient,
    question: Question,
    reference_md_paths: list[Path],
    include_common_mistakes: bool,
    root_dir: Path,
) -> str:
    inp = ExplanationInputs(
        question_text=_question_text_for_explanation(question),
        answer_text=question.answer,
        reference_md_paths=reference_md_paths,
        include_common_mistakes=include_common_mistakes,
        root_dir=root_dir,
    )
    return generate_explanation_result(client, inp)


def _question_text_for_explanation(q: Question) -> str:
    stem = q.stem.strip()
    options = q.options.strip()
    if not options:
        return stem
    return f"{stem}\n{options}"


def _emit_progress(
    *,
    idx: int,
    total: int,
    file_name: str,
    stage: str,
    message: str,
```

```
    question_current: int = 0,
    question_total: int = 0,
    question_count: int = 0,
    progress_cb: Callable[[str], None] | None,
    progress_info_cb: Callable[[BatchAiProgress], None] | None,
) -> None:
    if progress_cb is not None:
        progress_cb(message)
    if progress_info_cb is not None:
        progress_info_cb(
            BatchAiProgress(
                file_index=idx,
                total_files=total,
                file_name=file_name,
                stage=stage,
                message=message,
                question_current=question_current,
                question_total=question_total,
                question_count=question_count,
            )
        )

```

```python
from __future__ import annotations

from pathlib import Path
from typing import Callable

from sj_generator.domain.entities import Question
from sj_generator.infrastructure.document.source_reader import read_source_text
from sj_generator.infrastructure.llm.client import LlmClient
from sj_generator.infrastructure.llm.import_questions import (
    import_questions_from_sources,
    question_content_llm_config,
)
from sj_generator.infrastructure.llm.question_ref_scan import resolve_question_refs_with_scan
from sj_generator.infrastructure.persistence.draft_db_import import (
    import_draft_questions_to_db,
)


def load_docx_sources(paths: list[Path]) -> list[tuple[Path, str]]:
    sources: list[tuple[Path, str]] = []
    for path in paths:
        if path.suffix.lower() != ".docx":
            raise RuntimeError(f"当前仅支持 Word 文档导入：{path.name}")
        sources.append((path, read_source_text(path)))
    return sources


def resolve_question_refs_for_sources(
    *,
    paths: list[Path],
    progress_cb: Callable[[str], None] | None = None,
    compare_cb: Callable[[object], None] | None = None,
    scan_progress_cb: Callable[[object], None] | None = None,
    progress_count_cb: Callable[[int, int], None] | None = None,
    stop_cb: Callable[[], bool] | None = None,
):
    return resolve_question_refs_with_scan(
        sources=load_docx_sources(paths),
        progress_cb=progress_cb,
        compare_cb=compare_cb,
        scan_progress_cb=scan_progress_cb,
        progress_count_cb=progress_count_cb,
        stop_cb=stop_cb,
    )


def import_questions_for_sources(
    *,
    model_specs: list[dict[str, str]],
    paths: list[Path],
    question_refs_by_source: dict[str, list[dict[str, str]]],
    strategy: str,
    max_question_workers: int,
    progress_cb: Callable[[str], None] | None = None,
    question_cb: Callable[[Question], None] | None = None,
    compare_cb: Callable[[dict], None] | None = None,
    progress_count_cb: Callable[[int, int], None] | None = None,
    stop_cb: Callable[[], bool] | None = None,
):
    client_factories = {
        str(spec.get("key") or ""): _build_question_content_client_factory(
            provider=str(spec.get("provider") or ""),
            model_name=str(spec.get("model_name") or ""),
        )
        for spec in model_specs
        if str(spec.get("key") or "").strip()
    }
    return import_questions_from_sources(
        model_specs=model_specs,
        client_factories=client_factories,
```

```python
        sources=load_docx_sources(paths),
        strategy=strategy,
        max_question_workers=max_question_workers,
        progress_cb=progress_cb,
        question_cb=question_cb,
        compare_cb=compare_cb,
        progress_count_cb=progress_count_cb,
        stop_cb=stop_cb,
        question_refs_by_source=question_refs_by_source,
    )


def commit_questions_to_db(
    *,
    db_path: Path,
    questions: list[Question],
    level_path: str,
    source_files: list[Path],
    textbook_version: str,
) -> int:
    return import_draft_questions_to_db(
        db_path=db_path,
        questions=questions,
        level_path=level_path,
        source_files=source_files,
        textbook_version=textbook_version,
    )


def _build_question_content_client_factory(
    *,
    provider: str,
    model_name: str,
) -> Callable[[], LlmClient]:
    return lambda provider=provider, model_name=model_name: LlmClient(
        question_content_llm_config(provider, model_name)
    )

```

sj_generator/application/settings/__init__.py

013-01

```python
from sj_generator.application.settings import provider_settings as _provider_settings
from sj_generator.application.settings import storage as _storage
from sj_generator.application.settings.program_settings import (
    load_program_settings,
    save_program_analysis_target,
    save_program_settings,
    save_program_settings_merged,
)
from sj_generator.application.settings.project_parse_settings import (
    default_project_parse_model_rows,
    load_project_parse_model_rows,
    normalize_project_parse_model_rows,
    project_parse_model_override,
    save_project_parse_model_rows,
)
from sj_generator.application.settings.provider_settings import (
    DeepSeekConfig,
    KimiConfig,
    QwenConfig,
    default_available_models,
    load_available_models,
    load_deepseek_config,
    load_kimi_config,
    load_qwen_config,
    normalize_available_models,
    save_available_models,
    save_deepseek_config,
    save_kimi_config,
    save_qwen_config,
    set_user_environment_variable,
    to_analysis_llm_config,
    to_kimi_llm_config,
    to_kimi_question_number_llm_config,
    to_llm_config,
    to_question_number_llm_config,
    to_qwen_llm_config,
    to_qwen_question_number_llm_config,
    with_capped_timeout,
)
from sj_generator.application.settings.view_settings import (
    load_welcome_table_column_visibility,
    load_welcome_table_font_point_size,
    load_welcome_tree_expanded_prefixes,
    save_welcome_table_column_visibility,
    save_welcome_table_font_point_size,
    save_welcome_tree_expanded_prefixes,
)


def _config_path():
    return _storage.deepseek_config_path()


def _kimi_config_path():
    return _storage.kimi_config_path()


def _qwen_config_path():
    return _storage.qwen_config_path()


def _program_settings_path():
    return _storage.program_settings_path()


def _read_json_dict(path):
    return _storage.read_json_dict(path)


def sync_deepseek_runtime_env(cfg: DeepSeekConfig) -> None:
```

```python
    _provider_settings.set_user_environment_variable = set_user_environment_variable
    _provider_settings.sync_deepseek_runtime_env(cfg)


def sync_kimi_runtime_env(cfg: KimiConfig) -> None:
    _provider_settings.set_user_environment_variable = set_user_environment_variable
    _provider_settings.sync_kimi_runtime_env(cfg)


def sync_qwen_runtime_env(cfg: QwenConfig) -> None:
    _provider_settings.set_user_environment_variable = set_user_environment_variable
    _provider_settings.sync_qwen_runtime_env(cfg)

__all__ = [
    "DeepSeekConfig",
    "KimiConfig",
    "QwenConfig",
    "default_available_models",
    "default_project_parse_model_rows",
    "load_available_models",
    "load_deepseek_config",
    "load_kimi_config",
    "load_program_settings",
    "load_project_parse_model_rows",
    "load_qwen_config",
    "load_welcome_table_column_visibility",
    "load_welcome_table_font_point_size",
    "load_welcome_tree_expanded_prefixes",
    "normalize_available_models",
    "normalize_project_parse_model_rows",
    "project_parse_model_override",
    "save_available_models",
    "save_deepseek_config",
    "save_kimi_config",
    "save_program_analysis_target",
    "save_program_settings",
    "save_program_settings_merged",
    "save_project_parse_model_rows",
    "save_qwen_config",
    "save_welcome_table_column_visibility",
    "save_welcome_table_font_point_size",
    "save_welcome_tree_expanded_prefixes",
    "set_user_environment_variable",
    "sync_deepseek_runtime_env",
    "sync_kimi_runtime_env",
    "sync_qwen_runtime_env",
    "sync_deepseek_runtime_env",
    "sync_kimi_runtime_env",
    "sync_qwen_runtime_env",
    "to_analysis_llm_config",
    "to_kimi_llm_config",
    "to_kimi_question_number_llm_config",
    "to_llm_config",
    "to_question_number_llm_config",
    "to_qwen_llm_config",
    "to_qwen_question_number_llm_config",
    "with_capped_timeout",
    "_config_path",
    "_kimi_config_path",
    "_qwen_config_path",
    "_program_settings_path",
    "_read_json_dict",
]
```

```python
from __future__ import annotations

from datetime import datetime
from decimal import Decimal, InvalidOperation

from sj_generator.application.settings.storage import import_cost_history_path, load_json_config_file, save_js
on_config_file

_MAX_IMPORT_COST_HISTORY_ROWS = 200
_PROVIDER_KEYS = ("deepseek", "kimi", "qwen")


def load_import_cost_history_rows(*, limit: int | None = None) -> list[dict[str, str]]:
    data = load_json_config_file(import_cost_history_path())
    raw_rows = data.get("rows")
    if not isinstance(raw_rows, list):
        return []
    rows = [_normalize_history_row(item) for item in raw_rows if isinstance(item, dict)]
    rows = _coalesce_same_run_rows([item for item in rows if item])
    _save_history_rows(rows)
    rows.reverse()
    if limit is not None and limit > 0:
        rows = rows[:limit]
    return rows


def append_import_cost_history_entry(
    *,
    run_at: str | None = None,
    source_label: str = "",
    before_provider_balances: dict[str, str] | None = None,
    provider_balances: dict[str, str] | None = None,
    total_balance: str = "",
    total_cost: str = "",
    cost_summary: str = "",
    cost_detail: str = "",
) -> dict[str, str]:
    data = load_json_config_file(import_cost_history_path())
    raw_rows = data.get("rows")
    rows = [item for item in raw_rows if isinstance(item, dict)] if isinstance(raw_rows, list) else []
    rows = _coalesce_same_run_rows([_normalize_history_row(item) for item in rows])
    entry = _normalize_history_row(
        {
            "run_at": _format_run_at_text(run_at=run_at, source_label=source_label),
            "deepseek_balance": str((provider_balances or {}).get("deepseek") or ""),
            "kimi_balance": str((provider_balances or {}).get("kimi") or ""),
            "qwen_balance": str((provider_balances or {}).get("qwen") or ""),
        }
    )
    if rows and str(rows[-1].get("run_at") or "").strip() == entry["run_at"]:
        rows[-1] = _prefer_history_row(rows[-1], entry)
        entry = rows[-1]
    else:
        rows.append(entry)
    if len(rows) > _MAX_IMPORT_COST_HISTORY_ROWS:
        rows = rows[-_MAX_IMPORT_COST_HISTORY_ROWS :]
    _save_history_rows(rows)
    return entry


def append_balance_history_for_provider_results(
    results: list[tuple[str, dict[str, object]]],
    *,
    source_label: str = "",
) -> None:
    provider_balances: dict[str, str] = {}
    for provider_key, result in results:
        provider = str(provider_key or "").strip().lower()
        if provider not in _PROVIDER_KEYS:
            continue
```

```python
        if not isinstance(result, dict):
            continue
        provider_balances[provider] = str(result.get("balance_value") or "").strip()
    if not any(provider_balances.values()):
        return
    append_import_cost_history_entry(provider_balances=provider_balances, source_label=source_label)


def clear_import_cost_history() -> None:
    _save_history_rows([])


def _format_run_at_text(*, run_at: str | None, source_label: str) -> str:
    base = str(run_at or datetime.now().strftime("%Y-%m-%d %H:%M:%S")).strip()
    label = str(source_label or "").strip()
    if not label:
        return base
    if base.endswith(f" [{label}]"):
        return base
    return f"{base} [{label}]"


def _save_history_rows(rows: list[dict[str, str]]) -> None:
    save_json_config_file(import_cost_history_path(), {"rows": rows})


def build_total_balance_text(provider_balances: dict[str, str]) -> str:
    totals_by_currency: dict[str, Decimal] = {}
    text_parts: list[str] = []
    for provider in _PROVIDER_KEYS:
        balance_text = str((provider_balances or {}).get(provider) or "").strip()
        amount, currency = _parse_amount_text(balance_text)
        if amount is None or not currency:
            continue
        totals_by_currency[currency] = totals_by_currency.get(currency, Decimal("0")) + amount
    if totals_by_currency:
        for currency, amount in sorted(totals_by_currency.items(), key=lambda item: item[0]):
            prefix = {"CNY": "￥", "USD": "$"}.get(currency, "")
            text_parts.append(f"{prefix}{amount:.4f}" if prefix else f"{currency} {amount:.4f}")
    if text_parts:
        return "；".join(text_parts)
    values = [str((provider_balances or {}).get(provider) or "").strip() for provider in _PROVIDER_KEYS]
    values = [value for value in values if value]
    return "；".join(values) if values else ""


def _normalize_history_row(row: dict) -> dict[str, str]:
    return {
        "run_at": str(row.get("run_at") or "").strip(),
        "deepseek_balance": str(row.get("deepseek_balance") or "").strip(),
        "kimi_balance": str(row.get("kimi_balance") or "").strip(),
        "qwen_balance": str(row.get("qwen_balance") or "").strip(),
    }


def _coalesce_same_run_rows(rows: list[dict[str, str]]) -> list[dict[str, str]]:
    merged: list[dict[str, str]] = []
    for row in rows:
        if not merged:
            merged.append(row)
            continue
        if str(merged[-1].get("run_at") or "").strip() == str(row.get("run_at") or "").strip():
            merged[-1] = _prefer_history_row(merged[-1], row)
            continue
        merged.append(row)
    return merged


def _prefer_history_row(left: dict[str, str], right: dict[str, str]) -> dict[str, str]:
    left_score = _history_row_score(left)
```

```python
    right_score = _history_row_score(right)
    return right if right_score >= left_score else left


def _history_row_score(row: dict[str, str]) -> tuple[int, int]:
    provider_values = [
        str(row.get("deepseek_balance") or "").strip(),
        str(row.get("kimi_balance") or "").strip(),
        str(row.get("qwen_balance") or "").strip(),
    ]
    non_empty_provider_count = sum(1 for value in provider_values if value)
    non_zero_provider_count = sum(1 for value in provider_values if _is_non_zero_amount_text(value))
    return (
        non_empty_provider_count,
        non_zero_provider_count,
    )


def _is_non_zero_amount_text(text: str) -> bool:
    amount, _currency = _parse_amount_text(text)
    return amount is not None and amount != Decimal("0")


def _parse_amount_text(text: str) -> tuple[Decimal | None, str]:
    value = str(text or "").strip()
    if not value:
        return None, ""
    currency = "CNY" if value.startswith("￥") else "USD" if value.startswith("$") else ""
    if not currency and " " in value:
        prefix, _, remainder = value.partition(" ")
        currency = prefix.strip().upper()
        value = remainder.strip()
    elif currency:
        value = value[1:].strip()
    try:
        return Decimal(value), currency
    except (InvalidOperation, ValueError):
        return None, ""

```

sj_generator/application/settings/program_settings.py

015-01

```python
from __future__ import annotations

from sj_generator.application.settings.storage import (
    load_json_config_file,
    program_settings_path,
    save_json_config_file,
)


def load_program_settings() -> dict:
    data = load_json_config_file(program_settings_path())
    return data if isinstance(data, dict) else {}


def save_program_settings(settings: dict) -> None:
    save_json_config_file(program_settings_path(), dict(settings))


def save_program_settings_merged(settings: dict) -> dict:
    data = load_program_settings()
    data.update(settings)
    save_program_settings(data)
    return data


def save_program_analysis_target(*, provider: str, model_name: str) -> dict:
    return save_program_settings_merged(
        {
            "analysis_provider": str(provider or "").strip().lower() or "deepseek",
            "analysis_model_name": str(model_name or "").strip() or "deepseek-reasoner",
        }
    )

```

```python
from __future__ import annotations

from sj_generator.application.settings.storage import (
    load_json_config_file,
    program_settings_path,
    save_json_config_file,
)

_PROJECT_PARSE_ROW_KEYS = ("question_number_parse", "question_content_parse")


def default_project_parse_model_rows() -> list[dict]:
    return [
        {"key": "question_number_parse", "round": "1", "ratio": "1/4", "models": []},
        {"key": "question_content_parse", "round": "2", "ratio": "1/4", "models": []},
    ]


def load_project_parse_model_rows() -> list[dict]:
    data = load_json_config_file(program_settings_path())
    return normalize_project_parse_model_rows(data.get("project_parse_model_rows"))


def save_project_parse_model_rows(rows: list[dict]) -> dict:
    data = load_json_config_file(program_settings_path())
    data["project_parse_model_rows"] = normalize_project_parse_model_rows(rows)
    save_json_config_file(program_settings_path(), data)
    return data


def normalize_project_parse_model_rows(rows: object) -> list[dict]:
    raw_rows = rows if isinstance(rows, list) else []
    normalized: list[dict] = []
    defaults = default_project_parse_model_rows()
    for index, key in enumerate(_PROJECT_PARSE_ROW_KEYS):
        raw = raw_rows[index] if index < len(raw_rows) and isinstance(raw_rows[index], dict) else {}
        default_row = defaults[index]
        round_value = str(raw.get("round") or default_row["round"]).strip()
        if round_value not in {"1", "2", "3", "4", "5", "6", "7", "8"}:
            round_value = default_row["round"]
        if key == "question_number_parse":
            round_value = "1"
        ratio_value = str(raw.get("ratio") or default_row["ratio"]).strip() or default_row["ratio"]
        normalized.append(
            {
                "key": key,
                "round": round_value,
                "ratio": ratio_value,
                "models": _normalize_project_parse_models(raw.get("models")),
            }
        )
    return normalized


def project_parse_model_override(row_key: str, provider: str) -> str:
    provider_key = _normalize_project_model_provider(provider)
    for row in load_project_parse_model_rows():
        if str(row.get("key") or "").strip() != row_key:
            continue
        for item in row.get("models") or []:
            if str(item.get("provider") or "").strip() == provider_key:
                return str(item.get("model_name") or "").strip()
    return ""


def _normalize_project_parse_models(models: object) -> list[dict]:
    raw_models = models if isinstance(models, list) else []
    normalized: list[dict] = []
    for raw in raw_models[:8]:
        if not isinstance(raw, dict):
```

```python
            continue
        provider = _normalize_project_model_provider(raw.get("provider"))
        model_name = str(raw.get("model_name") or "").strip()
        if not provider or not model_name:
            continue
        normalized.append({"provider": provider, "model_name": model_name})
    return normalized


def _normalize_project_model_provider(value: object) -> str:
    raw = str(value or "").strip().lower()
    aliases = {
        "deepseek": "deepseek",
        "deep seek": "deepseek",
        "kimi": "kimi",
        "moonshot": "kimi",
        "qwen": "qwen",
        "千问": "qwen",
    }
    return aliases.get(raw, "")

```

sj_generator/application/settings/provider_settings.py

017-01

```python
from __future__ import annotations

import os
import sys
from dataclasses import dataclass, replace
from typing import TYPE_CHECKING

from sj_generator.application.settings.project_parse_settings import project_parse_model_override
from sj_generator.application.settings.storage import (
    clean_base_url,
    deepseek_config_path,
    kimi_config_path,
    load_json_config_file,
    provider_config_path,
    qwen_config_path,
    save_json_config_file,
)

if TYPE_CHECKING:
    from sj_generator.infrastructure.llm.client import LlmConfig


@dataclass(frozen=True)
class DeepSeekConfig:
    base_url: str = "https://api.deepseek.com"
    api_key: str = ""
    number_model: str = "deepseek-chat"
    model: str = "deepseek-chat"
    analysis_model: str = "deepseek-reasoner"
    timeout_s: float = 120.0

    def is_ready(self) -> bool:
        return bool(
            self.api_key.strip()
            and self.base_url.strip()
            and self.number_model.strip()
            and self.model.strip()
            and self.analysis_model.strip()
        )


@dataclass(frozen=True)
class KimiConfig:
    base_url: str = "https://api.moonshot.cn/v1"
    api_key: str = ""
    number_model: str = "kimi-k2.6"
    model: str = "kimi-k2.6"
    timeout_s: float = 120.0

    def is_ready(self) -> bool:
        return bool(self.api_key.strip() and self.base_url.strip() and self.number_model.strip() and self.model.strip())


@dataclass(frozen=True)
class QwenConfig:
    base_url: str = "https://dashscope.aliyuncs.com/compatible-mode/v1"
    api_key: str = ""
    number_model: str = "qwen-max"
    model: str = "qwen-max"
    account_access_key_id: str = ""
    account_access_key_secret: str = ""
    timeout_s: float = 120.0

    def is_ready(self) -> bool:
        return bool(self.api_key.strip() and self.base_url.strip() and self.number_model.strip() and self.model.strip())

    def has_account_balance_credentials(self) -> bool:
        return bool(self.account_access_key_id.strip() and self.account_access_key_secret.strip())
```

```python


_DEFAULT_AVAILABLE_MODELS: dict[str, list[str]] = {
    "deepseek": ["deepseek-v4-pro", "deepseek-chat", "deepseek-reasoner"],
    "kimi": [
        "kimi-k2.6",
        "kimi-k2.5",
        "kimi-k2-thinking",
        "kimi-k2-thinking-turbo",
        "kimi-k2-0905-preview",
        "kimi-k2-0711-preview",
        "kimi-k2-turbo-preview",
        "moonshot-v1-32k",
        "moonshot-v1-8k",
        "moonshot-v1-128k",
    ],
    "qwen": [
        "qwen-max",
        "qwen-max-latest",
        "qwen3.6-plus",
        "qwen3.5-plus",
        "qwen-plus",
        "qwen-turbo",
        "qwen-long",
        "qwen-deep-research",
        "qwen-ocr",
        "gui-plus",
    ],
}


def with_capped_timeout(cfg, max_timeout_s: float):
    capped_timeout_s = min(float(getattr(cfg, "timeout_s", max_timeout_s)), float(max_timeout_s))
    if float(getattr(cfg, "timeout_s", capped_timeout_s)) <= capped_timeout_s:
        return cfg
    return replace(cfg, timeout_s=capped_timeout_s)


def load_deepseek_config() -> DeepSeekConfig:
    env_base_url = os.getenv("DEEPSEEK_BASE_URL", "").strip()
    env_api_key = os.getenv("DEEPSEEK_API_KEY", "").strip()
    env_model = os.getenv("DEEPSEEK_QUESTION_UNIT_MODEL", "").strip() or os.getenv("DEEPSEEK_MODEL", "").strip()
    env_number_model = os.getenv("DEEPSEEK_QUESTION_NUMBER_MODEL", "").strip()
    env_analysis_model = os.getenv("DEEPSEEK_ANALYSIS_MODEL", "").strip()
    env_timeout = os.getenv("DEEPSEEK_TIMEOUT_S", "").strip()
    project_number_model = project_parse_model_override("question_number_parse", "deepseek")
    project_unit_model = project_parse_model_override("question_content_parse", "deepseek")

    file_cfg = load_json_config_file(deepseek_config_path())
    file_timeout = file_cfg.get("timeout_s")
    timeout_s = DeepSeekConfig.timeout_s
    try:
        if env_timeout:
            timeout_s = float(env_timeout)
        elif file_timeout is not None:
            timeout_s = float(file_timeout)
    except Exception:
        timeout_s = DeepSeekConfig.timeout_s

    return DeepSeekConfig(
        base_url=clean_base_url(env_base_url or file_cfg.get("base_url") or DeepSeekConfig.base_url),
        api_key=env_api_key,
        number_model=(
            project_number_model
            or env_number_model
            or str(file_cfg.get("question_number_model") or file_cfg.get("number_model") or DeepSeekConfig.number_model).strip()
        ),
        model=(project_unit_model or env_model or file_cfg.get("model") or DeepSeekConfig.model).strip(),
```

```python
        analysis_model=(env_analysis_model or file_cfg.get("analysis_model") or DeepSeekConfig.analysis_model).strip(),
        timeout_s=timeout_s,
    )


def save_deepseek_config(cfg: DeepSeekConfig) -> None:
    path = deepseek_config_path()
    data = load_json_config_file(path)
    data.pop("api_key", None)
    data.update(
        {
            "base_url": cfg.base_url.strip(),
            "question_number_model": cfg.number_model.strip(),
            "model": cfg.model.strip(),
            "analysis_model": cfg.analysis_model.strip(),
            "timeout_s": cfg.timeout_s,
        }
    )
    save_json_config_file(path, data)


def to_llm_config(cfg: DeepSeekConfig) -> LlmConfig:
    from sj_generator.infrastructure.llm.client import LlmConfig

    return LlmConfig(
        base_url=cfg.base_url.strip(),
        api_key=cfg.api_key.strip(),
        model=cfg.model.strip(),
        timeout_s=float(cfg.timeout_s),
    )


def to_question_number_llm_config(cfg: DeepSeekConfig) -> LlmConfig:
    from sj_generator.infrastructure.llm.client import LlmConfig

    return LlmConfig(
        base_url=cfg.base_url.strip(),
        api_key=cfg.api_key.strip(),
        model=cfg.number_model.strip(),
        timeout_s=float(cfg.timeout_s),
    )


def to_analysis_llm_config(cfg: DeepSeekConfig) -> LlmConfig:
    from sj_generator.infrastructure.llm.client import LlmConfig

    return LlmConfig(
        base_url=cfg.base_url.strip(),
        api_key=cfg.api_key.strip(),
        model=cfg.analysis_model.strip(),
        timeout_s=float(cfg.timeout_s),
    )


def load_kimi_config() -> KimiConfig:
    env_base_url = os.getenv("KIMI_BASE_URL", "").strip()
    env_api_key = os.getenv("KIMI_API_KEY", "").strip()
    env_model = os.getenv("KIMI_QUESTION_UNIT_MODEL", "").strip() or os.getenv("KIMI_MODEL", "").strip()
    env_number_model = os.getenv("KIMI_QUESTION_NUMBER_MODEL", "").strip()
    env_timeout = os.getenv("KIMI_TIMEOUT_S", "").strip()
    project_number_model = project_parse_model_override("question_number_parse", "kimi")
    project_unit_model = project_parse_model_override("question_content_parse", "kimi")

    file_cfg = load_json_config_file(kimi_config_path())
    file_timeout = file_cfg.get("timeout_s")
    timeout_s = KimiConfig.timeout_s
    try:
        if env_timeout:
            timeout_s = float(env_timeout)
```

```python
        elif file_timeout is not None:
            timeout_s = float(file_timeout)
    except Exception:
        timeout_s = KimiConfig.timeout_s

    return KimiConfig(
        base_url=clean_base_url(env_base_url or file_cfg.get("base_url") or KimiConfig.base_url),
        api_key=env_api_key,
        number_model=(
            project_number_model
            or env_number_model
            or str(file_cfg.get("question_number_model") or file_cfg.get("number_model") or file_cfg.get("model") or KimiConfig.number_model).strip()
        ),
        model=(project_unit_model or env_model or file_cfg.get("model") or KimiConfig.model).strip(),
        timeout_s=timeout_s,
    )


def save_kimi_config(cfg: KimiConfig) -> None:
    path = kimi_config_path()
    data = load_json_config_file(path)
    data.pop("api_key", None)
    data.update(
        {
            "base_url": clean_base_url(cfg.base_url),
            "question_number_model": cfg.number_model.strip(),
            "model": cfg.model.strip(),
            "timeout_s": cfg.timeout_s,
        }
    )
    save_json_config_file(path, data)


def to_kimi_llm_config(cfg: KimiConfig) -> LlmConfig:
    from sj_generator.infrastructure.llm.client import LlmConfig

    return LlmConfig(
        base_url=clean_base_url(cfg.base_url),
        api_key=cfg.api_key.strip(),
        model=cfg.model.strip(),
        timeout_s=float(cfg.timeout_s),
    )


def to_kimi_question_number_llm_config(cfg: KimiConfig) -> LlmConfig:
    from sj_generator.infrastructure.llm.client import LlmConfig

    return LlmConfig(
        base_url=clean_base_url(cfg.base_url),
        api_key=cfg.api_key.strip(),
        model=cfg.number_model.strip(),
        timeout_s=float(cfg.timeout_s),
    )


def load_qwen_config() -> QwenConfig:
    env_base_url = os.getenv("QWEN_BASE_URL", "").strip()
    env_api_key = os.getenv("QWEN_API_KEY", "").strip()
    env_model = os.getenv("QWEN_QUESTION_UNIT_MODEL", "").strip() or os.getenv("QWEN_MODEL", "").strip()
    env_number_model = os.getenv("QWEN_QUESTION_NUMBER_MODEL", "").strip()
    env_account_access_key_id = os.getenv("QWEN_ACCOUNT_ACCESS_KEY_ID", "").strip() or os.getenv("ALIBABA_CLOUD_ACCESS_KEY_ID", "").strip()
    env_account_access_key_secret = os.getenv("QWEN_ACCOUNT_ACCESS_KEY_SECRET", "").strip() or os.getenv("ALIBABA_CLOUD_ACCESS_KEY_SECRET", "").strip()
    env_timeout = os.getenv("QWEN_TIMEOUT_S", "").strip()
    project_number_model = project_parse_model_override("question_number_parse", "qwen")
    project_unit_model = project_parse_model_override("question_content_parse", "qwen")

    file_cfg = load_json_config_file(qwen_config_path())
```

```python
    file_timeout = file_cfg.get("timeout_s")
    timeout_s = QwenConfig.timeout_s
    try:
        if file_timeout is not None:
            timeout_s = float(file_timeout)
        elif env_timeout:
            timeout_s = float(env_timeout)
    except Exception:
        timeout_s = QwenConfig.timeout_s

    return QwenConfig(
        base_url=clean_base_url(file_cfg.get("base_url") or env_base_url or QwenConfig.base_url),
        api_key=str(file_cfg.get("api_key") or "").strip() or env_api_key,
        number_model=(
            project_number_model
            or str(file_cfg.get("question_number_model") or file_cfg.get("number_model") or file_cfg.get("model") or "").strip()
            or env_number_model
            or QwenConfig.number_model
        ),
        model=(project_unit_model or str(file_cfg.get("model") or "").strip() or env_model or QwenConfig.model).strip(),
        account_access_key_id=str(file_cfg.get("account_access_key_id") or "").strip() or env_account_access_key_id,
        account_access_key_secret=str(file_cfg.get("account_access_key_secret") or "").strip() or env_account_access_key_secret,
        timeout_s=timeout_s,
    )


def save_qwen_config(cfg: QwenConfig) -> None:
    path = qwen_config_path()
    data = load_json_config_file(path)
    data.update(
        {
            "base_url": clean_base_url(cfg.base_url),
            "api_key": cfg.api_key.strip(),
            "question_number_model": cfg.number_model.strip(),
            "model": cfg.model.strip(),
            "account_access_key_id": cfg.account_access_key_id.strip(),
            "account_access_key_secret": cfg.account_access_key_secret.strip(),
            "timeout_s": cfg.timeout_s,
        }
    )
    save_json_config_file(path, data)


def to_qwen_llm_config(cfg: QwenConfig) -> LlmConfig:
    from sj_generator.infrastructure.llm.client import LlmConfig

    return LlmConfig(
        base_url=clean_base_url(cfg.base_url),
        api_key=cfg.api_key.strip(),
        model=cfg.model.strip(),
        timeout_s=float(cfg.timeout_s),
    )


def to_qwen_question_number_llm_config(cfg: QwenConfig) -> LlmConfig:
    from sj_generator.infrastructure.llm.client import LlmConfig

    return LlmConfig(
        base_url=clean_base_url(cfg.base_url),
        api_key=cfg.api_key.strip(),
        model=cfg.number_model.strip(),
        timeout_s=float(cfg.timeout_s),
    )


def default_available_models(provider: str) -> list[str]:
```

```python
    key = str(provider or "").strip().lower()
    return list(_DEFAULT_AVAILABLE_MODELS.get(key, []))


def normalize_available_models(models: object, provider: str) -> list[str]:
    default_models = default_available_models(provider)
    if not isinstance(models, list):
        return default_models
    normalized: list[str] = []
    seen: set[str] = set()
    for item in models:
        model_name = str(item or "").strip()
        if not model_name:
            continue
        key = model_name.lower()
        if key in seen:
            continue
        seen.add(key)
        normalized.append(model_name)
    return normalized or default_models


def load_available_models(provider: str) -> list[str]:
    path = provider_config_path(provider)
    data = load_json_config_file(path)
    return normalize_available_models(data.get("available_models"), provider)


def save_available_models(provider: str, models: list[str]) -> list[str]:
    normalized = normalize_available_models(models, provider)
    path = provider_config_path(provider)
    data = load_json_config_file(path)
    data["available_models"] = normalized
    save_json_config_file(path, data)
    return normalized


def sync_deepseek_runtime_env(cfg: DeepSeekConfig) -> None:
    set_user_environment_variable("DEEPSEEK_BASE_URL", clean_base_url(cfg.base_url))
    set_user_environment_variable("DEEPSEEK_API_KEY", cfg.api_key.strip())
    set_user_environment_variable("DEEPSEEK_QUESTION_NUMBER_MODEL", cfg.number_model.strip())
    set_user_environment_variable("DEEPSEEK_QUESTION_UNIT_MODEL", cfg.model.strip())
    set_user_environment_variable("DEEPSEEK_MODEL", cfg.model.strip())
    set_user_environment_variable("DEEPSEEK_ANALYSIS_MODEL", cfg.analysis_model.strip())
    set_user_environment_variable("DEEPSEEK_TIMEOUT_S", str(float(cfg.timeout_s)))


def sync_kimi_runtime_env(cfg: KimiConfig) -> None:
    set_user_environment_variable("KIMI_BASE_URL", clean_base_url(cfg.base_url))
    set_user_environment_variable("KIMI_API_KEY", cfg.api_key.strip())
    set_user_environment_variable("KIMI_QUESTION_NUMBER_MODEL", cfg.number_model.strip())
    set_user_environment_variable("KIMI_QUESTION_UNIT_MODEL", cfg.model.strip())
    set_user_environment_variable("KIMI_MODEL", cfg.model.strip())
    set_user_environment_variable("KIMI_TIMEOUT_S", str(float(cfg.timeout_s)))


def sync_qwen_runtime_env(cfg: QwenConfig) -> None:
    set_user_environment_variable("QWEN_BASE_URL", clean_base_url(cfg.base_url))
    set_user_environment_variable("QWEN_API_KEY", cfg.api_key.strip())
    set_user_environment_variable("QWEN_QUESTION_NUMBER_MODEL", cfg.number_model.strip())
    set_user_environment_variable("QWEN_QUESTION_UNIT_MODEL", cfg.model.strip())
    set_user_environment_variable("QWEN_MODEL", cfg.model.strip())
    set_user_environment_variable("QWEN_TIMEOUT_S", str(float(cfg.timeout_s)))
    set_user_environment_variable("QWEN_ACCOUNT_ACCESS_KEY_ID", cfg.account_access_key_id.strip())
    set_user_environment_variable("QWEN_ACCOUNT_ACCESS_KEY_SECRET", cfg.account_access_key_secret.strip())
    set_user_environment_variable("ALIBABA_CLOUD_ACCESS_KEY_ID", cfg.account_access_key_id.strip())
    set_user_environment_variable("ALIBABA_CLOUD_ACCESS_KEY_SECRET", cfg.account_access_key_secret.strip())


def set_user_environment_variable(name: str, value: str) -> None:
```

```
    value = (value or "").strip()
    if value:
        os.environ[name] = value
    else:
        os.environ.pop(name, None)

    if sys.platform != "win32":
        return

    import ctypes
    import winreg

    with winreg.OpenKey(winreg.HKEY_CURRENT_USER, "Environment", 0, winreg.KEY_SET_VALUE) as key:
        if value:
            winreg.SetValueEx(key, name, 0, winreg.REG_SZ, value)
        else:
            try:
                winreg.DeleteValue(key, name)
            except FileNotFoundError:
                pass

    HWND_BROADCAST = 0xFFFF
    WM_SETTINGCHANGE = 0x001A
    ctypes.windll.user32.SendNotifyMessageW(HWND_BROADCAST, WM_SETTINGCHANGE, 0, "Environment")

```

sj_generator/application/settings/storage.py

018-01

```python
from __future__ import annotations

import json
import os
from pathlib import Path


def clean_base_url(url: str) -> str:
    value = (url or "").strip()
    value = value.strip("`").strip('"').strip("'").strip()
    if value.endswith("/"):
        value = value[:-1]
    return value


def default_config_dir() -> Path:
    app_data = os.getenv("APPDATA", "").strip()
    if app_data:
        return Path(app_data) / "sj_generator"
    return Path.home() / ".sj_generator"


def legacy_config_path(file_name: str) -> Path | None:
    base_dir = Path(__file__).resolve().parents[3]
    path = base_dir / ".local" / file_name
    return path if path.exists() else None


def read_json_dict(path: Path) -> dict:
    try:
        data = json.loads(path.read_text(encoding="utf-8"))
    except Exception:
        return {}
    return data if isinstance(data, dict) else {}


def load_json_config_file(path: Path) -> dict:
    if not path.exists():
        legacy = legacy_config_path(path.name)
        if legacy is not None and legacy.exists():
            path.parent.mkdir(parents=True, exist_ok=True)
            data = read_json_dict(legacy)
            if data:
                path.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")
                return data
        return {}
    return read_json_dict(path)


def save_json_config_file(path: Path, data: dict) -> dict:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")
    return data


def merge_json_config_values(path: Path, values: dict) -> dict:
    data = load_json_config_file(path)
    data.update(values)
    return save_json_config_file(path, data)


def deepseek_config_path() -> Path:
    env = os.getenv("SJ_GENERATOR_CONFIG_PATH", "").strip()
    return Path(env) if env else default_config_dir() / "deepseek.json"


def kimi_config_path() -> Path:
    env = os.getenv("SJ_GENERATOR_KIMI_CONFIG_PATH", "").strip()
    return Path(env) if env else default_config_dir() / "kimi.json"

```

```python

def qwen_config_path() -> Path:
    env = os.getenv("SJ_GENERATOR_QWEN_CONFIG_PATH", "").strip()
    return Path(env) if env else default_config_dir() / "qwen.json"


def provider_config_path(provider: str) -> Path:
    key = str(provider or "").strip().lower()
    if key == "kimi":
        return kimi_config_path()
    if key == "qwen":
        return qwen_config_path()
    return deepseek_config_path()


def welcome_view_config_path() -> Path:
    env = os.getenv("SJ_GENERATOR_WELCOME_VIEW_CONFIG_PATH", "").strip()
    return Path(env) if env else default_config_dir() / "welcome_view.json"


def program_settings_path() -> Path:
    env = os.getenv("SJ_GENERATOR_PROGRAM_SETTINGS_PATH", "").strip()
    return Path(env) if env else default_config_dir() / "program_settings.json"


def import_cost_history_path() -> Path:
    env = os.getenv("SJ_GENERATOR_IMPORT_COST_HISTORY_PATH", "").strip()
    return Path(env) if env else default_config_dir() / "import_cost_history.json"

```

sj_generator/application/settings/view_settings.py

019-01

```python
from __future__ import annotations

from sj_generator.application.settings.storage import (
    load_json_config_file,
    save_json_config_file,
    welcome_view_config_path,
)


def load_welcome_table_column_visibility() -> dict[str, bool]:
    data = load_json_config_file(welcome_view_config_path())
    raw = data.get("table_column_visibility")
    if not isinstance(raw, dict):
        return {}
    result: dict[str, bool] = {}
    for key, value in raw.items():
        if isinstance(key, str):
            result[key] = bool(value)
    return result


def save_welcome_table_column_visibility(visibility: dict[str, bool]) -> None:
    _save_welcome_view_config_values(
        {"table_column_visibility": {str(key): bool(value) for key, value in visibility.items()}}
    )


def load_welcome_table_font_point_size() -> int | None:
    data = load_json_config_file(welcome_view_config_path())
    raw = data.get("table_font_point_size")
    try:
        value = int(raw)
    except Exception:
        return None
    return value if value > 0 else None


def save_welcome_table_font_point_size(point_size: int) -> None:
    try:
        value = int(point_size)
    except Exception:
        return
    if value <= 0:
        return
    _save_welcome_view_config_values({"table_font_point_size": value})


def load_welcome_tree_expanded_prefixes() -> list[str] | None:
    data = load_json_config_file(welcome_view_config_path())
    raw = data.get("tree_expanded_prefixes")
    if raw is None:
        return None
    if not isinstance(raw, list):
        return []
    return [str(value).strip() for value in raw if str(value).strip()]


def save_welcome_tree_expanded_prefixes(prefixes: list[str]) -> None:
    normalized: list[str] = []
    seen: set[str] = set()
    for value in prefixes:
        text = str(value).strip()
        if not text or text in seen:
            continue
        seen.add(text)
        normalized.append(text)
    _save_welcome_view_config_values({"tree_expanded_prefixes": normalized})


def _save_welcome_view_config_values(values: dict) -> None:
```

```
    path = welcome_view_config_path()
    data = load_json_config_file(path)
    data.update(values)
    save_json_config_file(path, data)

```

sj_generator/application/state/__init__.py

020-01

```python
from sj_generator.application.state.wizard_state import (
    AI_CONCURRENCY_OPTIONS,
    ANALYSIS_PROVIDER_OPTIONS,
    DEFAULT_ANALYSIS_MODEL_NAME,
    DEFAULT_EXPORT_INCLUDE_ANALYSIS,
    DEFAULT_EXPORT_INCLUDE_ANSWERS,
    DEFAULT_LIBRARY_DB_FILE_NAME,
    DEFAULT_PREFERRED_TEXTBOOK_VERSION,
    DEFAULT_REPO_PARENT_DIR_NAME,
    AiSourceFileItem,
    ImportDraftState,
    ImportExecutionState,
    ImportQuestionRefState,
    ImportSourceState,
    ImportWizardSession,
    WizardState,
    build_import_flow_session,
    default_import_source_dir,
    default_repo_parent_dir,
    desktop_import_source_dir,
    library_db_path_from_repo_parent_dir_text,
    normalize_ai_concurrency,
    normalize_analysis_model_name,
    normalize_analysis_provider,
    normalize_default_repo_parent_dir_text,
    normalize_export_convertible_multi_mode,
    normalize_export_include_analysis,
    normalize_export_include_answers,
    normalize_import_source_dir_text,
    normalize_preferred_textbook_version,
)

__all__ = [
    "AI_CONCURRENCY_OPTIONS",
    "ANALYSIS_PROVIDER_OPTIONS",
    "DEFAULT_ANALYSIS_MODEL_NAME",
    "DEFAULT_EXPORT_INCLUDE_ANALYSIS",
    "DEFAULT_EXPORT_INCLUDE_ANSWERS",
    "DEFAULT_LIBRARY_DB_FILE_NAME",
    "DEFAULT_PREFERRED_TEXTBOOK_VERSION",
    "DEFAULT_REPO_PARENT_DIR_NAME",
    "AiSourceFileItem",
    "ImportDraftState",
    "ImportExecutionState",
    "ImportQuestionRefState",
    "ImportSourceState",
    "ImportWizardSession",
    "WizardState",
    "build_import_flow_session",
    "default_import_source_dir",
    "default_repo_parent_dir",
    "desktop_import_source_dir",
    "library_db_path_from_repo_parent_dir_text",
    "normalize_ai_concurrency",
    "normalize_analysis_model_name",
    "normalize_analysis_provider",
    "normalize_default_repo_parent_dir_text",
    "normalize_export_convertible_multi_mode",
    "normalize_export_include_analysis",
    "normalize_export_include_answers",
    "normalize_import_source_dir_text",
    "normalize_preferred_textbook_version",
]
```

```python
from __future__ import annotations

from dataclasses import dataclass, field
from pathlib import Path
import threading
from typing import Optional, TYPE_CHECKING

if TYPE_CHECKING:
    from sj_generator.domain.entities import Question
    from sj_generator.application.dedupe.service import DedupeHit

AI_CONCURRENCY_OPTIONS = tuple(range(1, 21))
ANALYSIS_PROVIDER_OPTIONS = ("deepseek", "kimi", "qwen")
EXPORT_CONVERTIBLE_MULTI_MODE_OPTIONS = ("keep_combo", "as_multi")
DEFAULT_ANALYSIS_MODEL_NAME = "deepseek-reasoner"
DEFAULT_REPO_PARENT_DIR_NAME = "思政题库"
DEFAULT_LIBRARY_DB_FILE_NAME = "思政题库.db"
DEFAULT_PREFERRED_TEXTBOOK_VERSION = "2026年春"
DEFAULT_EXPORT_INCLUDE_ANSWERS = True
DEFAULT_EXPORT_INCLUDE_ANALYSIS = True


def normalize_ai_concurrency(value: int | None) -> int:
    try:
        workers = int(value or 0)
    except Exception:
        workers = 0
    return workers if workers in AI_CONCURRENCY_OPTIONS else 3


def normalize_analysis_provider(value: str | None) -> str:
    provider = (value or "").strip().lower()
    return provider if provider in ANALYSIS_PROVIDER_OPTIONS else "deepseek"


def normalize_analysis_model_name(value: str | None) -> str:
    model_name = (value or "").strip()
    return model_name or DEFAULT_ANALYSIS_MODEL_NAME


def normalize_export_convertible_multi_mode(value: str | None) -> str:
    mode = (value or "").strip().lower()
    return mode if mode in EXPORT_CONVERTIBLE_MULTI_MODE_OPTIONS else "keep_combo"


def default_repo_parent_dir() -> Path:
    return Path.home() / "Desktop" / DEFAULT_REPO_PARENT_DIR_NAME


def default_import_source_dir() -> Path:
    return Path.home() / "Downloads"


def desktop_import_source_dir() -> Path:
    return Path.home() / "Desktop"


def normalize_default_repo_parent_dir_text(value: str | None) -> str:
    text = (value or "").strip()
    return text or str(default_repo_parent_dir())


def normalize_import_source_dir_text(value: str | None) -> str:
    text = (value or "").strip()
    return text or str(default_import_source_dir())


def normalize_preferred_textbook_version(value: str | None) -> str:
    text = (value or "").strip()
    return text or DEFAULT_PREFERRED_TEXTBOOK_VERSION
```

```python


def normalize_export_include_answers(value: object) -> bool:
    return bool(DEFAULT_EXPORT_INCLUDE_ANSWERS if value is None else value)


def normalize_export_include_analysis(value: object) -> bool:
    return bool(DEFAULT_EXPORT_INCLUDE_ANALYSIS if value is None else value)


def library_db_path_from_repo_parent_dir_text(value: str | None) -> Path:
    return Path(normalize_default_repo_parent_dir_text(value)) / DEFAULT_LIBRARY_DB_FILE_NAME


@dataclass
class AiSourceFileItem:
    path: str
    version: str = ""
    level_path: str = ""


@dataclass
class ImportSourceState:
    files: list[Path] = field(default_factory=list)
    files_text: str = ""
    file_items: list[AiSourceFileItem] = field(default_factory=list)
    import_level_path: str = ""


@dataclass
class ImportQuestionRefState:
    question_refs_by_source: dict[str, list[dict[str, str]]] = field(default_factory=dict)
    revision: int = 0


@dataclass
class ImportDraftState:
    questions: list[Question] = field(default_factory=list)
    dedupe_hits: Optional[list[DedupeHit]] = None

    def clear_questions(self) -> None:
        self.questions = []
        self.dedupe_hits = None

    def replace_questions(self, questions: list[Question]) -> None:
        self.questions = list(questions)
        self.dedupe_hits = None


@dataclass
class ImportExecutionState:
    db_import_completed: bool = False
    db_import_count: int = 0
    db_import_error: str = ""
    import_cost_before_amounts: dict[str, str] = field(default_factory=dict)
    import_cost_before_details: dict[str, str] = field(default_factory=dict)
    import_cost_rows: list[dict[str, str]] = field(default_factory=list)
    import_cost_total_text: str = ""
    import_cost_summary_text: str = ""
    import_cost_detail_text: str = ""
    import_cost_ready: bool = False
    import_cost_before_loading: bool = False
    import_cost_capture_revision: int = 0
    import_cost_capture_thread: threading.Thread | None = field(default=None, repr=False, compare=False)

    def reset_db_import(self) -> None:
        self.db_import_completed = False
        self.db_import_count = 0
        self.db_import_error = ""

```

```python
    def mark_db_import_completed(self, count: int) -> None:
        self.db_import_completed = True
        self.db_import_count = max(0, int(count))
        self.db_import_error = ""

    def reset_import_cost_tracking(self) -> None:
        self.import_cost_capture_revision += 1
        self.import_cost_before_amounts = {}
        self.import_cost_before_details = {}
        self.import_cost_rows = []
        self.import_cost_total_text = ""
        self.import_cost_summary_text = ""
        self.import_cost_detail_text = ""
        self.import_cost_ready = False
        self.import_cost_before_loading = False
        self.import_cost_capture_thread = None


@dataclass
class ImportWizardSession:
    project_dir: Optional[Path] = None
    repo_path: Optional[Path] = None
    last_export_dir: Optional[Path] = None
    project_name_is_placeholder: bool = False
    dedupe_enabled: bool = True
    dedupe_threshold: float = 0.85
    default_repo_parent_dir_text: str = field(default_factory=lambda: str(default_repo_parent_dir()))
    import_source_dir_text: str = field(default_factory=lambda: str(default_import_source_dir()))
    analysis_enabled: bool = True
    analysis_use_reference_folder: bool = True
    analysis_use_reference_md: bool = False
    analysis_reference_md_path_text: str = ""
    analysis_include_common_mistakes: bool = True
    analysis_provider: str = "deepseek"
    analysis_model_name: str = DEFAULT_ANALYSIS_MODEL_NAME
    export_convertible_multi_mode: str = "keep_combo"
    export_include_answers: bool = DEFAULT_EXPORT_INCLUDE_ANSWERS
    export_include_analysis: bool = DEFAULT_EXPORT_INCLUDE_ANALYSIS
    preferred_textbook_version: str = DEFAULT_PREFERRED_TEXTBOOK_VERSION
    ai_concurrency: int = 3
    question_content_concurrency: int = 3
    analysis_generation_concurrency: int = 3
    source: ImportSourceState = field(default_factory=ImportSourceState)
    refs: ImportQuestionRefState = field(default_factory=ImportQuestionRefState)
    draft: ImportDraftState = field(default_factory=ImportDraftState)
    execution: ImportExecutionState = field(default_factory=ImportExecutionState)

    def apply_question_refs(self, question_refs_by_source: dict[str, list[dict[str, str]]]) -> None:
        self.refs.question_refs_by_source = dict(question_refs_by_source or {})
        self.refs.revision = max(0, int(self.refs.revision)) + 1
        self.draft.clear_questions()
        self.execution.reset_db_import()

    def apply_draft_questions(self, questions: list[Question]) -> None:
        self.draft.replace_questions(questions)
        self.execution.reset_db_import()

    def set_dedupe_hits(self, hits: list[DedupeHit] | None) -> None:
        self.draft.dedupe_hits = hits

    def build_import_session(
        self,
        *,
        source_files: list[Path],
        source_items: list[AiSourceFileItem] | None = None,
        question_refs_by_source: dict[str, list[dict[str, str]]] | None = None,
        question_refs_version: int = 0,
        import_level_path: str = "",
    ) -> ImportWizardSession:
        return build_import_flow_session(
```

```python
            self,
            source_files=source_files,
            source_items=source_items,
            question_refs_by_source=question_refs_by_source,
            question_refs_version=question_refs_version,
            import_level_path=import_level_path,
        )


@dataclass
class WizardState:
    project_dir: Optional[Path] = None
    repo_path: Optional[Path] = None
    last_export_dir: Optional[Path] = None
    start_mode: str = "wizard"
    project_name_is_placeholder: bool = False
    dedupe_enabled: bool = True
    dedupe_threshold: float = 0.85
    default_repo_parent_dir_text: str = field(default_factory=lambda: str(default_repo_parent_dir()))
    import_source_dir_text: str = field(default_factory=lambda: str(default_import_source_dir()))
    analysis_enabled: bool = True
    analysis_use_reference_folder: bool = True
    analysis_use_reference_md: bool = False
    analysis_reference_md_path_text: str = ""
    analysis_include_common_mistakes: bool = True
    analysis_provider: str = "deepseek"
    analysis_model_name: str = DEFAULT_ANALYSIS_MODEL_NAME
    export_convertible_multi_mode: str = "keep_combo"
    export_include_answers: bool = DEFAULT_EXPORT_INCLUDE_ANSWERS
    export_include_analysis: bool = DEFAULT_EXPORT_INCLUDE_ANALYSIS
    preferred_textbook_version: str = DEFAULT_PREFERRED_TEXTBOOK_VERSION
    ai_concurrency: int = 3
    question_content_concurrency: int = 3
    analysis_generation_concurrency: int = 3

    def build_import_session(
        self,
        *,
        source_files: list[Path],
        source_items: list[AiSourceFileItem] | None = None,
        question_refs_by_source: dict[str, list[dict[str, str]]] | None = None,
        question_refs_version: int = 0,
        import_level_path: str = "",
    ) -> ImportWizardSession:
        return build_import_flow_session(
            self,
            source_files=source_files,
            source_items=source_items,
            question_refs_by_source=question_refs_by_source,
            question_refs_version=question_refs_version,
            import_level_path=import_level_path,
        )


def build_import_flow_session(
    base_state: WizardState | ImportWizardSession,
    *,
    source_files: list[Path],
    source_items: list[AiSourceFileItem] | None = None,
    question_refs_by_source: dict[str, list[dict[str, str]]] | None = None,
    question_refs_version: int = 0,
    import_level_path: str = "",
) -> ImportWizardSession:
    cleaned_paths = [Path(path) for path in source_files if str(path).strip()]
    raw_paths = [str(path) for path in cleaned_paths]
    item_map = {
        item.path: item
        for item in (source_items or [])
        if isinstance(item, AiSourceFileItem) and str(item.path or "").strip()
    }
```

```
    default_version = base_state.preferred_textbook_version
    built_items = [
        AiSourceFileItem(
            path=raw_path,
            version=(item_map.get(raw_path).version or default_version) if raw_path in item_map else default_version,
            level_path=(item_map.get(raw_path).level_path or "") if raw_path in item_map else "",
        )
        for raw_path in raw_paths
    ]
    return ImportWizardSession(
        project_dir=base_state.project_dir,
        repo_path=base_state.repo_path,
        last_export_dir=base_state.last_export_dir,
        project_name_is_placeholder=base_state.project_name_is_placeholder,
        dedupe_enabled=base_state.dedupe_enabled,
        dedupe_threshold=base_state.dedupe_threshold,
        default_repo_parent_dir_text=base_state.default_repo_parent_dir_text,
        import_source_dir_text=base_state.import_source_dir_text,
        analysis_enabled=base_state.analysis_enabled,
        analysis_use_reference_folder=base_state.analysis_use_reference_folder,
        analysis_use_reference_md=base_state.analysis_use_reference_md,
        analysis_reference_md_path_text=base_state.analysis_reference_md_path_text,
        analysis_include_common_mistakes=base_state.analysis_include_common_mistakes,
        analysis_provider=base_state.analysis_provider,
        analysis_model_name=base_state.analysis_model_name,
        export_convertible_multi_mode=base_state.export_convertible_multi_mode,
        export_include_answers=base_state.export_include_answers,
        export_include_analysis=base_state.export_include_analysis,
        preferred_textbook_version=base_state.preferred_textbook_version,
        ai_concurrency=base_state.ai_concurrency,
        question_content_concurrency=base_state.question_content_concurrency,
        analysis_generation_concurrency=base_state.analysis_generation_concurrency,
        source=ImportSourceState(
            files=list(cleaned_paths),
            files_text="; ".join(raw_paths),
            file_items=built_items,
            import_level_path=str(import_level_path or "").strip(),
        ),
        refs=ImportQuestionRefState(
            question_refs_by_source=dict(question_refs_by_source or {}),
            revision=max(0, int(question_refs_version)),
        ),
    )

```

sj_generator/domain/__init__.py

022-01

```python
from sj_generator.domain.entities import Question

__all__ = ["Question"]

```

sj_generator/domain/entities/__init__.py

023-01

```python
from sj_generator.domain.entities.question import Question

__all__ = ["Question"]

```

sj_generator/domain/entities/question.py

024-01

```python
from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True)
class Question:
    number: str
    stem: str
    options: str
    answer: str
    analysis: str
    question_type: str = ""
    choice_1: str = ""
    choice_2: str = ""
    choice_3: str = ""
    choice_4: str = ""

```

sj_generator/infrastructure/__init__.py

025-01

```python


```

sj_generator/infrastructure/document/__init__.py

026-01

```python


```

sj_generator/infrastructure/document/source_reader.py

027-01

```python
from __future__ import annotations

from pathlib import Path

from docx import Document


def read_source_text(path: Path) -> str:
    ext = path.suffix.lower()
    if ext == ".docx":
        doc = Document(str(path))
        parts: list[str] = []
        for p in doc.paragraphs:
            t = (p.text or "").strip()
            if t:
                parts.append(t)
        for table in doc.tables:
            for row in table.rows:
                for cell in row.cells:
                    for p in cell.paragraphs:
                        t = (p.text or "").strip()
                        if t:
                            parts.append(t)
        return "\n".join(parts) + ("\n" if parts else "")
    raise ValueError(f"当前仅支持 Word 文档读取：{path.name}")

```

sj_generator/infrastructure/exporting/__init__.py

028-01

```python


```

```python
from __future__ import annotations

import re
from datetime import date

from sj_generator.infrastructure.persistence.excel_repo import try_parse_options_json
from sj_generator.domain.entities import Question


_BR_RE = re.compile(r"<br\s*/?>", flags=re.IGNORECASE)


def export_questions_to_markdown(
    *,
    excel_file_name: str,
    export_date: date,
    questions: list[Question],
    convertible_multi_mode: str = "keep_combo",
    include_answers: bool = True,
    include_analysis: bool = True,
) -> str:
    lines: list[str] = []

    header = (
        f"# 所属部分：{excel_file_name}  "
        f"日期：{export_date.year:04d}年{export_date.month:02d}月{export_date.day:02d}日"
    )
    lines.append(header)
    lines.append("")

    normalized = _normalize_numbers(questions)
    for q in normalized:
        stem_lines = _split_lines(_normalize_breaks(q.stem))
        options_block = _format_options_block(q.options, convertible_multi_mode=convertible_multi_mode)
        if options_block:
            stem_lines = _ensure_choice_blank(stem_lines)

        first = stem_lines[0] if stem_lines else ""
        lines.append(f" {q.number}. {first}".rstrip())
        if len(stem_lines) > 1:
            lines.extend(stem_lines[1:])
        if options_block:
            lines.extend(options_block)

        lines.append("")

    if lines and lines[-1] == "":
        lines.pop()

    if include_answers or include_analysis:
        lines.append("")
        if include_answers and include_analysis:
            lines.append("## 答案与解析")
        elif include_answers:
            lines.append("## 答案")
        else:
            lines.append("## 解析")
        lines.append("")

        for q in normalized:
            row_parts: list[str] = [q.number.strip()]
            if include_answers:
                row_parts.append(q.answer.strip())
            header_text = ". ".join(part for part in row_parts if part).strip()
            if header_text:
                lines.append(f"**{header_text}**")
            if include_analysis:
                analysis_lines = [
                    s for s in _split_lines(_normalize_breaks(q.analysis)) if s.strip()
                ]
```

```python
                lines.extend(analysis_lines)
            lines.append("")

        if lines and lines[-1] == "":
            lines.pop()

    return "\n".join(lines) + "\n"


def _normalize_numbers(questions: list[Question]) -> list[Question]:
    result: list[Question] = []
    used_numbers: set[int] = set()
    for q in questions:
        s = q.number.strip()
        if s.isdigit():
            used_numbers.add(int(s))
    next_no = (max(used_numbers) + 1) if used_numbers else 1
    for q in questions:
        number = q.number.strip()
        if not number.isdigit():
            while next_no in used_numbers:
                next_no += 1
            number = str(next_no)
            used_numbers.add(next_no)
            next_no += 1
        result.append(
            Question(
                number=number,
                stem=q.stem,
                options=q.options,
                answer=q.answer,
                analysis=q.analysis,
            )
        )
    return result


def _format_options_block(options: str, *, convertible_multi_mode: str = "keep_combo") -> list[str]:
    options = _normalize_breaks(options)
    parsed = try_parse_options_json(options)
    if parsed is not None:
        keys = sorted(parsed.keys(), key=_option_key_sort_key)
        return [f"{k}. {parsed[k]}".rstrip() for k in keys]

    options = _force_newline_before_markers(options)
    lines = _split_lines(options)
    lines = [line for line in lines if line.strip()]
    return _normalize_convertible_multi_option_lines(lines, convertible_multi_mode=convertible_multi_mode)


def _force_newline_before_markers(text: str) -> str:
    if not text.strip():
        return ""

    letter = r"(?<!^)(?=(?:[A-Z][\.、])"
    circled = r"(?<!^)(?=(?:[\u2460-\u2473]))"
    out_lines: list[str] = []
    for raw_line in text.split("\n"):
        line = raw_line.rstrip()
        if not line:
            out_lines.append("")
            continue
        line = re.sub(letter, "\n", line)
        if not re.match(r"^\s*[A-Z][\.、. :：]", line):
            line = re.sub(circled, "\n", line)
        out_lines.extend(line.split("\n"))
    return "\n".join(out_lines)


def _normalize_breaks(text: str) -> str:
```

```python
    text = _BR_RE.sub("\n", text)
    return text.replace("\r\n", "\n").replace("\r", "\n")


def _split_lines(text: str) -> list[str]:
    if not text:
        return []
    return [line.rstrip() for line in text.split("\n")]


def _normalize_convertible_multi_option_lines(
    lines: list[str],
    *,
    convertible_multi_mode: str = "keep_combo",
) -> list[str]:
    if not lines:
        return []
    statement_lines = [line for line in lines if _is_circled_option_line(line)]
    combo_lines = sorted(
        [line for line in lines if _is_combo_mapping_line(line)],
        key=_combo_line_sort_key,
    )
    if statement_lines and combo_lines:
        other_lines = [line for line in lines if line not in statement_lines and line not in combo_lines]
        if convertible_multi_mode == "as_multi":
            return statement_lines + other_lines
        combo_line = "  ".join(line.strip() for line in combo_lines if line.strip())
        return statement_lines + other_lines + ([combo_line] if combo_line else [])
    return lines


def _is_circled_option_line(text: str) -> bool:
    return bool(re.match(r"^\s*[\u2460-\u2473][\.、. :：]?\s*", text))


def _is_combo_mapping_line(text: str) -> bool:
    return bool(re.match(r"^\s*[A-D][\.、. :：]\s*[\u2460-\u2473]+", text))


def _combo_line_sort_key(text: str) -> str:
    m = re.match(r"^\s*([A-D])", text)
    return m.group(1) if m else text


def _option_key_sort_key(key: str) -> tuple[int, str]:
    k = key.strip()
    if len(k) >= 1 and "A" <= k[0] <= "Z":
        return (0, k[0])
    return (1, k)


def _ensure_choice_blank(stem_lines: list[str]) -> list[str]:
    if not stem_lines:
        return stem_lines

    idx = None
    for i in range(len(stem_lines) - 1, -1, -1):
        if stem_lines[i].strip():
            idx = i
            break
    if idx is None:
        return stem_lines

    line = stem_lines[idx]
    if re.search(r"( (\s*) |\(\s*\))\s*$", line):
        return stem_lines

    m = re.search(r"([。 ? ! ; ：\.])\s*$", line)
    if m:
        punc = m.group(1)
```

```
        body = line[: m.start(1)].rstrip()
        stem_lines[idx] = f"{body} ( ) {punc}"
    else:
        stem_lines[idx] = f"{line.rstrip()} ( ) "
    return stem_lines

```

```python
from __future__ import annotations

from datetime import date
from pathlib import Path
import re

from sj_generator.infrastructure.exporting.export_md import export_questions_to_markdown
from sj_generator.domain.entities import Question


def _is_pdf_option_line(text: str) -> bool:
    return bool(re.match(r"^\s*(?:[A-D][\.、. :：]|[\u2460-\u2473][\.、. :：]?)\s*", text))


def _prepare_markdown_for_pdf(text: str) -> str:
    # Prevent question numbers like "1. xxx" from being parsed as ordered lists,
    # otherwise QTextDocument may inject extra numbering before option lines.
    escaped = re.sub(r"(?m)^(\s*)(\d+)\.(\s+)", r"\1\2\\.\3", text)
    lines = escaped.splitlines()
    normalized_lines: list[str] = []
    in_answer_section = False
    for line in lines:
        stripped = line.strip()
        if stripped == "## 答案与解析":
            in_answer_section = True
            normalized_lines.append(line)
            continue
        if (not in_answer_section) and _is_pdf_option_line(line):
            normalized_lines.append(f"\u3000{line}")
            continue
        normalized_lines.append(line)
    return "\n".join(f"{line}  " if line.strip() else "" for line in normalized_lines)


def export_questions_to_pdf(
    *,
    excel_file_name: str,
    export_date: date,
    questions: list[Question],
    target_path: Path,
    convertible_multi_mode: str = "keep_combo",
    include_answers: bool = True,
    include_analysis: bool = True,
) -> None:
    from PyQt6.QtCore import QSizeF
    from PyQt6.QtGui import QPageSize, QPdfWriter, QTextDocument

    markdown_text = export_questions_to_markdown(
        excel_file_name=excel_file_name,
        export_date=export_date,
        questions=questions,
        convertible_multi_mode=convertible_multi_mode,
        include_answers=include_answers,
        include_analysis=include_analysis,
    )

    target_path.parent.mkdir(parents=True, exist_ok=True)

    writer = QPdfWriter(str(target_path))
    writer.setPageSize(QPageSize(QPageSize.PageSizeId.A4))
    writer.setResolution(96)

    doc = QTextDocument()
    doc.setDocumentMargin(36)
    if hasattr(doc, "setMarkdown"):
        # QTextDocument treats single newlines in Markdown as soft wraps.
        # Force hard breaks so option lines and analysis lines keep their row layout in PDF.
        doc.setMarkdown(_prepare_markdown_for_pdf(markdown_text))
    else:
        doc.setPlainText(markdown_text)
```

```
    page_size = writer.pageLayout().paintRectPixels(writer.resolution()).size()
    doc.setPageSize(QSizeF(page_size))
    doc.print(writer)

```

sj_generator/infrastructure/llm/__init__.py

031-01

```
1
2
```

sj_generator/infrastructure/llm/balance.py

032-01

```python
from __future__ import annotations

import base64
from dataclasses import dataclass
from decimal import Decimal, InvalidOperation
from datetime import UTC, datetime
import hashlib
import hmac
from urllib.parse import quote
import uuid

import requests

from sj_generator.application.settings import DeepSeekConfig, KimiConfig, QwenConfig


@dataclass(frozen=True)
class ProviderBalanceStatus:
    provider: str
    detail: str


@dataclass(frozen=True)
class ProviderBalanceSnapshot:
    provider: str
    currency: str
    amount: Decimal
    detail: str


def load_provider_balance_statuses(
    *,
    deepseek_cfg: DeepSeekConfig,
    kimi_cfg: KimiConfig,
    qwen_cfg: QwenConfig,
) -> list[ProviderBalanceStatus]:
    return [
        ProviderBalanceStatus("deepseek", _safe_describe(describe_deepseek_balance, deepseek_cfg)),
        ProviderBalanceStatus("kimi", _safe_describe(describe_kimi_balance, kimi_cfg)),
        ProviderBalanceStatus("qwen", _safe_describe(describe_qwen_balance, qwen_cfg)),
    ]


def describe_deepseek_balance(cfg: DeepSeekConfig) -> str:
    if not cfg.is_ready():
        return "未配置"

    url = _build_deepseek_balance_url(cfg.base_url)
    resp = requests.get(
        url,
        headers={"Authorization": f"Bearer {cfg.api_key.strip()}"},
        timeout=cfg.timeout_s,
    )
    resp.raise_for_status()
    data = resp.json()
    return _describe_deepseek_balance_payload(data)


def describe_kimi_balance(cfg: KimiConfig) -> str:
    if not cfg.is_ready():
        return "未配置"

    url = _build_kimi_balance_url(cfg.base_url)
    resp = requests.get(
        url,
        headers={"Authorization": f"Bearer {cfg.api_key.strip()}"},
        timeout=cfg.timeout_s,
    )
    resp.raise_for_status()
    data = resp.json()
```

```python
    return _describe_kimi_balance_payload(data)


def describe_qwen_balance(cfg: QwenConfig) -> str:
    if cfg.has_account_balance_credentials():
        data = _query_aliyun_account_balance(cfg)
        return _describe_aliyun_account_balance_payload(data)
    if cfg.is_ready():
        return "已配置模型 API；未配置阿里云 AccessKey，无法查询账户余额"
    if not cfg.is_ready():
        return "未配置"
    return "未配置"


def query_provider_balance_snapshots(
    *,
    deepseek_cfg: DeepSeekConfig,
    kimi_cfg: KimiConfig,
    qwen_cfg: QwenConfig,
) -> list[ProviderBalanceSnapshot]:
    snapshots: list[ProviderBalanceSnapshot] = []
    snapshot = _safe_snapshot("deepseek", query_deepseek_balance_snapshot, deepseek_cfg)
    if snapshot is not None:
        snapshots.append(snapshot)
    snapshot = _safe_snapshot("kimi", query_kimi_balance_snapshot, kimi_cfg)
    if snapshot is not None:
        snapshots.append(snapshot)
    snapshot = _safe_snapshot("qwen", query_qwen_balance_snapshot, qwen_cfg)
    if snapshot is not None:
        snapshots.append(snapshot)
    return snapshots


def query_deepseek_balance_snapshot(cfg: DeepSeekConfig) -> ProviderBalanceSnapshot | None:
    if not cfg.is_ready():
        return None
    url = _build_deepseek_balance_url(cfg.base_url)
    resp = requests.get(
        url,
        headers={"Authorization": f"Bearer {cfg.api_key.strip()}"},
        timeout=cfg.timeout_s,
    )
    resp.raise_for_status()
    data = resp.json()
    balance_infos = [item for item in (data.get("balance_infos") or []) if isinstance(item, dict)]
    detail = _describe_deepseek_balance_payload(data)
    amount, currency = _sum_balance_infos(balance_infos)
    return ProviderBalanceSnapshot("deepseek", currency, amount, detail)


def query_kimi_balance_snapshot(cfg: KimiConfig) -> ProviderBalanceSnapshot | None:
    if not cfg.is_ready():
        return None
    url = _build_kimi_balance_url(cfg.base_url)
    resp = requests.get(
        url,
        headers={"Authorization": f"Bearer {cfg.api_key.strip()}"},
        timeout=cfg.timeout_s,
    )
    resp.raise_for_status()
    data = resp.json()
    detail = _describe_kimi_balance_payload(data)
    amount, currency = _extract_kimi_balance_amount_and_currency(data)
    return ProviderBalanceSnapshot("kimi", currency, amount, detail)


def query_qwen_balance_snapshot(cfg: QwenConfig) -> ProviderBalanceSnapshot | None:
    if not cfg.has_account_balance_credentials():
        return None
    data = _query_aliyun_account_balance(cfg)
```

```python
    detail = _describe_aliyun_account_balance_payload(data)
    amount, currency = _extract_aliyun_balance_amount_and_currency(data)
    return ProviderBalanceSnapshot("qwen", currency, amount, detail)


def _build_deepseek_balance_url(base_url: str) -> str:
    base = (base_url or "").strip().rstrip("/")
    if base.endswith("/chat/completions"):
        base = base[: -len("/chat/completions")]
    if base.endswith("/v1"):
        base = base[: -len("/v1")]
    return base + "/user/balance"


def _build_kimi_balance_url(base_url: str) -> str:
    base = (base_url or "").strip().rstrip("/")
    if base.endswith("/chat/completions"):
        base = base[: -len("/chat/completions")]
    if not base.endswith("/v1"):
        base += "/v1"
    return base + "/users/me/balance"


def _query_aliyun_account_balance(cfg: QwenConfig) -> dict:
    params = {
        "Action": "QueryAccountBalance",
        "Version": "2017-12-14",
        "Format": "JSON",
        "AccessKeyId": cfg.account_access_key_id.strip(),
        "SignatureMethod": "HMAC-SHA1",
        "Timestamp": datetime.now(UTC).strftime("%Y-%m-%dT%H:%M:%SZ"),
        "SignatureVersion": "1.0",
        "SignatureNonce": uuid.uuid4().hex,
    }
    params["Signature"] = _sign_aliyun_rpc_params(params, cfg.account_access_key_secret.strip())
    resp = requests.get("https://business.aliyuncs.com/", params=params, timeout=cfg.timeout_s)
    resp.raise_for_status()
    data = resp.json()
    if isinstance(data, dict) and data.get("Success") is False:
        raise RuntimeError(data.get("Message") or data.get("Code") or "阿里云账户余额查询失败")
    return data


def _format_deepseek_balance_infos(balance_infos: list[dict]) -> list[str]:
    parts: list[str] = []
    for item in balance_infos:
        currency = str(item.get("currency") or "").strip().upper() or "UNKNOWN"
        total = _format_money(item.get("total_balance"), currency)
        granted = _format_money(item.get("granted_balance"), currency)
        topped_up = _format_money(item.get("topped_up_balance"), currency)
        parts.append(f"{currency} {total}（赠送 {granted}，充值 {topped_up}）")
    return parts


def _describe_deepseek_balance_payload(data: object) -> str:
    payload = data if isinstance(data, dict) else {}
    parts = _format_deepseek_balance_infos(payload.get("balance_infos") or [])
    if not parts:
        detail = "已配置，余额接口已响应，但未返回可展示余额"
    else:
        detail = "已配置，余额 " + "；".join(parts)
    if payload.get("is_available") is False:
        detail += "（当前账户不可用）"
    return detail


def _sum_balance_infos(balance_infos: list[dict]) -> tuple[Decimal, str]:
    total = Decimal("0")
    currency = "CNY"
    for item in balance_infos:
```

```python
        currency = str(item.get("currency") or currency).strip().upper() or currency
        total += _to_decimal(item.get("total_balance"))
    return total, currency


def _format_money(value: object, currency: str) -> str:
    text = str(value or "0").strip().replace(",", "")
    try:
        amount = Decimal(text)
    except (InvalidOperation, ValueError):
        return text or "0"

    prefix = {"CNY": "￥", "USD": "$"}.get(currency, "")
    return f"{prefix}{amount:.2f}" if prefix else f"{amount:.2f}"


def _describe_kimi_balance_payload(data: object) -> str:
    payload = data
    if isinstance(data, dict) and isinstance(data.get("data"), dict):
        payload = data.get("data")

    if not isinstance(payload, dict):
        return "已配置，余额接口已响应，但返回格式无法识别"

    balances = payload.get("balances")
    if isinstance(balances, list):
        parts = _format_deepseek_balance_infos([item for item in balances if isinstance(item, dict)])
        if parts:
            return "已配置，余额 " + "；".join(parts)

    currency = str(payload.get("currency") or "CNY").strip().upper() or "CNY"
    for key in ("available_balance", "balance", "total_balance", "remaining_balance"):
        if key in payload and str(payload.get(key, "")).strip():
            amount = _format_money(payload.get(key), currency)
            return f"已配置，余额 {currency} {amount}"

    return "已配置，余额接口已响应，但未返回可展示余额"


def _extract_kimi_balance_amount_and_currency(data: object) -> tuple[Decimal, str]:
    payload = data
    if isinstance(data, dict) and isinstance(data.get("data"), dict):
        payload = data.get("data")
    if not isinstance(payload, dict):
        return Decimal("0"), "CNY"
    balances = payload.get("balances")
    if isinstance(balances, list):
        parts = [item for item in balances if isinstance(item, dict)]
        if parts:
            return _sum_balance_infos(parts)
    currency = str(payload.get("currency") or "CNY").strip().upper() or "CNY"
    for key in ("available_balance", "balance", "total_balance", "remaining_balance"):
        if key in payload and str(payload.get(key, "")).strip():
            return _to_decimal(payload.get(key)), currency
    return Decimal("0"), currency


def _describe_aliyun_account_balance_payload(data: object) -> str:
    payload = data.get("Data") if isinstance(data, dict) and isinstance(data.get("Data"), dict) else data
    if not isinstance(payload, dict):
        return "已配置，账户余额接口已响应，但返回格式无法识别"

    currency = str(payload.get("Currency") or "CNY").strip().upper() or "CNY"
    available = payload.get("AvailableAmount")
    if available is not None and str(available).strip():
        amount = _format_money(available, currency)
        return f"已配置，阿里云账户余额 {currency} {amount}"
    return "已配置，账户余额接口已响应，但未返回可展示余额"
```

```python
def _extract_aliyun_balance_amount_and_currency(data: object) -> tuple[Decimal, str]:
    payload = data.get("Data") if isinstance(data, dict) and isinstance(data.get("Data"), dict) else data
    if not isinstance(payload, dict):
        return Decimal("0"), "CNY"
    currency = str(payload.get("Currency") or "CNY").strip().upper() or "CNY"
    return _to_decimal(payload.get("AvailableAmount")), currency


def _sign_aliyun_rpc_params(params: dict[str, object], access_key_secret: str) -> str:
    canonicalized = "&".join(
        f"{_percent_encode(key)}={_percent_encode(str(value))}" for key, value in sorted(params.items(), key=lambda item: item[0])
    )
    string_to_sign = "GET&%2F&" + _percent_encode(canonicalized)
    digest = hmac.new((access_key_secret + "&").encode("utf-8"), string_to_sign.encode("utf-8"), hashlib.sha1).digest()
    return base64.b64encode(digest).decode("utf-8")


def _percent_encode(value: str) -> str:
    return quote(value, safe="~")


def _safe_describe(fn, cfg) -> str:
    try:
        return fn(cfg)
    except Exception as e:
        if getattr(cfg, "is_ready", lambda: False)() or getattr(cfg, "has_account_balance_credentials", lambda: False)():
            return f"已配置，余额查询失败：{e}"
        return "未配置"


def _safe_snapshot(provider: str, fn, cfg) -> ProviderBalanceSnapshot | None:
    try:
        return fn(cfg)
    except Exception as e:
        if getattr(cfg, "is_ready", lambda: False)() or getattr(cfg, "has_account_balance_credentials", lambda: False)():
            return ProviderBalanceSnapshot(provider, "CNY", Decimal("0"), f"已配置，余额查询失败：{e}")
        return None


def _to_decimal(value: object) -> Decimal:
    text = str(value or "0").strip().replace(",", "")
    try:
        return Decimal(text)
    except (InvalidOperation, ValueError):
        return Decimal("0")
```

sj_generator/infrastructure/llm/client.py

033-01

```python
from __future__ import annotations

import json
import time
from dataclasses import dataclass
from typing import Any

import requests


@dataclass(frozen=True)
class LlmConfig:
    base_url: str
    api_key: str
    model: str
    timeout_s: float = 60.0
    max_retries: int = 2
    retry_backoff_s: float = 1.0


class LlmClient:
    def __init__(self, config: LlmConfig) -> None:
        self._config = config
        self._session = requests.Session()

    def chat_json(self, *, system: str, user: str) -> Any:
        content = self.chat_text(system=system, user=user)
        try:
            return json.loads(content)
        except Exception:
            extracted = _extract_json(content)
            return json.loads(extracted)

    def chat_text(self, *, system: str, user: str) -> str:
        data = self._post_chat(system=system, user=user)
        return data["choices"][0]["message"]["content"]

    def _post_chat(self, *, system: str, user: str) -> Any:
        url = _build_chat_completions_url(self._config.base_url)
        headers = {"Authorization": f"Bearer {self._config.api_key}"}
        temperature = _pick_temperature(self._config.model)
        payload = {
            "model": self._config.model,
            "messages": [
                {"role": "system", "content": system},
                {"role": "user", "content": user},
            ],
            "temperature": temperature,
            "stream": False,
        }

        last_err: Exception | None = None
        attempts = max(1, int(self._config.max_retries) + 1)
        for i in range(attempts):
            try:
                resp = self._session.post(
                    url,
                    headers=headers,
                    json=payload,
                    timeout=self._config.timeout_s,
                )
                resp.raise_for_status()
                return resp.json()
            except requests.exceptions.Timeout as e:
                last_err = e
            except requests.exceptions.ConnectionError as e:
                last_err = e
            except requests.exceptions.HTTPError as e:
                last_err = e
                status = getattr(e.response, "status_code", None)
```

```python
                body = ""
                try:
                    body = (e.response.text or "").strip() if e.response is not None else ""
                except Exception:
                    body = ""
                if status is not None and status < 500:
                    friendly = _friendly_http_error_message(status=status, body=body)
                    if friendly:
                        raise RuntimeError(friendly) from e
                    if body:
                        raise RuntimeError(f"HTTP {status}：{body}") from e
                    raise

            if i < attempts - 1:
                time.sleep(self._config.retry_backoff_s * (2**i))

        assert last_err is not None
        raise RuntimeError(f"请求失败：{last_err}") from last_err


def _extract_json(text: str) -> str:
    s = text.strip()
    if not s:
        return s

    starts = []
    for i, ch in enumerate(s):
        if ch in "[{":
            starts.append(i)
            break
    if not starts:
        return s

    start = starts[0]
    stack: list[str] = []
    for i in range(start, len(s)):
        ch = s[i]
        if ch in "[{":
            stack.append(ch)
        elif ch == "]":
            if stack and stack[-1] == "[":
                stack.pop()
                if not stack:
                    return s[start : i + 1]
        elif ch == "}":
            if stack and stack[-1] == "{":
                stack.pop()
                if not stack:
                    return s[start : i + 1]
    return s[start:]


def _build_chat_completions_url(base_url: str) -> str:
    base = (base_url or "").strip().rstrip("/")
    if base.endswith("/chat/completions"):
        return base
    if base.endswith("/v1"):
        return base + "/chat/completions"
    return base + "/v1/chat/completions"


def _pick_temperature(model: str) -> float:
    m = (model or "").strip().lower()
    if m.startswith("kimi-k2.6") or m.startswith("kimi-k2.5"):
        return 1.0
    return 0.0


def _friendly_http_error_message(*, status: int, body: str) -> str:
    body_text = (body or "").strip()
```

```python
    body_lower = body_text.lower()
    if status == 402 or "insufficient balance" in body_lower:
        return "接口调用失败：余额不足，请先充值或切换到有余额的模型配置后再重试。"
    if status == 401:
        return "接口调用失败：API Key 无效或已过期，请检查模型配置。"
    if status == 403:
        return "接口调用失败：当前账号无权访问该模型或接口，请检查权限配置。"
    if status == 429:
        return "接口调用失败：请求过于频繁，请稍后重试。"
    return ""

```

sj_generator/infrastructure/llm/explanations.py

034-01

```python
from __future__ import annotations

import re
from dataclasses import dataclass
from pathlib import Path
from typing import Any

from sj_generator.infrastructure.llm.client import LlmClient
from sj_generator.infrastructure.llm.prompt_templates import render_import_prompt
from sj_generator.shared.paths import common_mistakes_md_path


@dataclass(frozen=True)
class ExplanationInputs:
    question_text: str
    answer_text: str
    reference_md_paths: list[Path] | None = None
    include_common_mistakes: bool = True
    root_dir: Path | None = None


@dataclass(frozen=True)
class ExplanationResult:
    answer_text: str
    analysis_text: str


_NONEMPTY_LINE_RE = re.compile(r".*\S.*")


def generate_explanation(client: LlmClient, inp: ExplanationInputs) -> str:
    return generate_explanation_result(client, inp).analysis_text


def generate_explanation_result(client: LlmClient, inp: ExplanationInputs) -> ExplanationResult:
    reference_text = ""
    if inp.reference_md_paths:
        reference_text = _read_reference_md_text(inp.reference_md_paths)

    mistakes_text = ""
    if inp.include_common_mistakes and inp.root_dir is not None:
        md = common_mistakes_md_path(inp.root_dir)
        if md.exists():
            mistakes_text = _read_text_limited(md, max_chars=12000)

    system = render_import_prompt("explanation_system")
    answer_text = (inp.answer_text or "").strip()
    user = _build_user_prompt(
        question_text=inp.question_text,
        answer_text=answer_text,
        reference_md_paths=inp.reference_md_paths,
        reference_text=reference_text,
        mistakes_text=mistakes_text,
    )
    raw = client.chat_text(system=system, user=user)
    parsed_answer, analysis_body = _extract_answer_and_analysis(raw)
    final_answer = answer_text or _normalize_generated_answer_text(parsed_answer)
    final_analysis = postprocess_explanation(analysis_body or raw)
    return ExplanationResult(answer_text=final_answer, analysis_text=final_analysis)


def _build_user_prompt(
    *,
    question_text: str,
    answer_text: str,
    reference_md_paths: list[Path] | None,
    reference_text: str,
    mistakes_text: str,
) -> str:
    reference_block = ""
```

```python
    if reference_text.strip():
        if reference_md_paths:
            names = "、".join([p.name for p in reference_md_paths])
            title = f"参考资料（{len(reference_md_paths)} 份：{names}）："
        else:
            title = "参考资料："
        reference_block = f"\n\n{title}\n{reference_text.strip()}"

    mistakes_block = ""
    if mistakes_text.strip():
        mistakes_block = (
            "\n\n常见错题归因与答题策略参考（md 原文）：\n"
            f"{mistakes_text.strip()}"
        )

    prompt = render_import_prompt(
        "explanation_user",
        question_text=question_text.strip(),
        answer_text=answer_text.strip(),
        reference_block=reference_block,
        mistakes_block=mistakes_block,
    ).strip()
    if answer_text.strip():
        return prompt
    return (
        prompt
        + "\n\n补充要求：当前“答案文本”为空。请你先判断最可能的正确答案，"
        + "并在第一行单独输出“答案：答案标识”；从第二行开始再输出逐项解析。"
    ).strip()


def _read_reference_md_text(paths: list[Path]) -> str:
    max_chars = 100000
    blocks: list[str] = []
    used = 0
    for p in paths:
        try:
            if not p.exists():
                continue
            text = p.read_text(encoding="utf-8", errors="ignore").strip()
        except Exception:
            continue
        if not text:
            continue
        head = f"[文件：{p.name}]"
        block = head + "\n" + text
        if used + len(block) + 2 > max_chars:
            remain = max_chars - used
            if remain <= len(head) + 1:
                break
            block = head + "\n" + block[len(head) + 1 : len(head) + 1 + remain - len(head) - 1].rstrip() + "\n…"
            blocks.append(block)
            break
        blocks.append(block)
        used += len(block) + 2
    return "\n\n".join(blocks).strip()


def postprocess_explanation(text: str) -> str:
    lines = [ln.rstrip() for ln in text.replace("\r\n", "\n").replace("\r", "\n").split("\n")]
    lines = [ln for ln in lines if _NONEMPTY_LINE_RE.fullmatch(ln)]
    cleaned: list[str] = []
    for ln in lines:
        s = ln.lstrip()
        if s.startswith("- "):
            s = s[2:]
        elif s.startswith("-"):
            s = s[1:].lstrip()
        cleaned.append(s)
```

```python
    return "\n".join(cleaned).strip()


def _extract_answer_and_analysis(text: str) -> tuple[str, str]:
    lines = text.replace("\r\n", "\n").replace("\r", "\n").split("\n")
    first_nonempty_index = -1
    for idx, line in enumerate(lines):
        if line.strip():
            first_nonempty_index = idx
            break
    if first_nonempty_index < 0:
        return "", ""
    first_line = lines[first_nonempty_index].strip()
    first_line = re.sub(r"^[\-\*\d\.\)\s]+", "", first_line)
    first_line = first_line.replace("**", "").strip()
    match = re.match(r"^(?:答案|参考答案|正确答案)\s*[：:]?\s*(.+?)\s*$", first_line)
    if not match:
        return "", text
    answer_text = _normalize_generated_answer_text(match.group(1))
    remaining = lines[:first_nonempty_index] + lines[first_nonempty_index + 1 :]
    return answer_text, "\n".join(remaining).strip()


def _as_str(value: object) -> str:
    if value is None:
        return ""
    return str(value).strip()


def _normalize_generated_answer_text(text: str) -> str:
    raw = _as_str(text).upper()
    if not raw:
        return ""
    for old, new in ((", ", ","), ("；", ","), (";", ","), ("、", ","), ("/", ","), (" ", "")):
        raw = raw.replace(old, new)
    if "," in raw:
        raw = "".join(part.strip() for part in raw.split(",") if part.strip())
    return raw

def _read_common_mistakes_md(path: Path) -> list[tuple[str, str]]:
    text = path.read_text(encoding="utf-8", errors="ignore")
    lines = [ln.strip() for ln in text.replace("\r\n", "\n").replace("\r", "\n").split("\n")]
    rows = [ln for ln in lines if ln.startswith("|") and ln.count("|") >= 4]
    out: list[tuple[str, str]] = []
    for ln in rows:
        if ":---" in ln or ":--" in ln:
            continue
        parts = [p.strip() for p in ln.strip("|").split("|")]
        if len(parts) < 2:
            continue
        a_raw = parts[0]
        b_raw = parts[1] if len(parts) > 1 else ""
        a = _strip_md_inline(a_raw)
        b = _strip_md_inline(b_raw)
        a = a.replace("\n", " ").strip()
        b = b.replace("\n", " ").strip()
        if not a:
            continue
        if a in ("错题表现 (常见错误现象)", "错题表现", "表现"):
            continue
        title, detail = _extract_type_and_detail(a)
        if title and detail:
            out.append((title, detail))
        elif title:
            out.append((title, b))
        else:
            out.append((a, b))
    return out
```

```python
def _strip_md_inline(text: str) -> str:
    s = (text or "").strip()
    s = re.sub(r"<br\\s*/?>", "\n", s, flags=re.IGNORECASE)
    s = re.sub(r"\\*\\*(.*?)\\*\\*", r"\\1", s)
    s = re.sub(r"\\*(.*?)\\*", r"\\1", s)
    s = re.sub(r"`([^`]*)`", r"\\1", s)
    return s.strip()


def _extract_type_and_detail(text: str) -> tuple[str, str]:
    s = (text or "").strip()
    if not s:
        return "", ""
    m = re.search(r"(?:^|\\s)([\\u4e00-\\u9fffA-Za-z0-9]+型)(?:\\s|$)", s)
    title = ""
    if m:
        title = m.group(1).strip()
    if "\n" in s:
        first, rest = s.split("\n", 1)
    else:
        first, rest = s, ""
    if not title:
        title = first.strip()
    detail = (rest or "").strip()
    detail = re.sub(r"\\s+", " ", detail)
    if len(detail) > 220:
        detail = detail[:220].rstrip() + "…"
    return title, detail


def _read_text_limited(path: Path, *, max_chars: int) -> str:
    try:
        text = path.read_text(encoding="utf-8", errors="ignore").strip()
    except Exception:
        return ""
    if len(text) <= max_chars:
        return text
    return text[:max_chars].rstrip() + "\n…"
```

```python
from __future__ import annotations

import ast
from concurrent.futures import ThreadPoolExecutor, as_completed
import json
import math
import re
import time
from dataclasses import dataclass
from datetime import date
from pathlib import Path
from typing import Any, Callable, Iterable

from sj_generator.infrastructure.llm.client import LlmClient, LlmConfig
from sj_generator.infrastructure.llm.prompt_templates import get_import_prompt, render_import_prompt
from sj_generator.infrastructure.llm.task_runner import run_tasks_in_parallel
from sj_generator.application.settings import (
    load_deepseek_config,
    load_kimi_config,
    load_project_parse_model_rows,
    load_qwen_config,
)
from sj_generator.domain.entities import Question


@dataclass(frozen=True)
class ImportResult:
    questions: list[Question]
    raw_items: list[dict[str, Any]]


_WS_RE = re.compile(r"\s+")
_CIRCLED_RE = re.compile(r"[\u2460-\u2473]")
_COMBO_RE = re.compile(r"([A-D])\s*[\.. 、 ]\s*([\u2460-\u2473\s]+)")
_LETTER_ONLY_RE = re.compile(r"^[A-D]+$")
_LETTER_MARKER_RE = re.compile(r"(?<!\n)(?=(?:[A-D][\.、. ]))")
_CIRCLED_MARKER_RE = re.compile(r"(?<!\n)(?=(?:[\u2460-\u2473][\.、. ]?))")
_OPTION_KEYS = ("option_1", "option_2", "option_3", "option_4")
_STEM_NUMBER_PATTERNS = [
    re.compile(r"^\s*(\d+)\s*[\.\. 、 \):：]\s*(.+)$", re.S),
    re.compile(r"^\s*[\( （]\s*(\d+)\s*[\)）]\s*(.+)$", re.S),
    re.compile(r"^\s*第\s*(\d+)\s*题\s*[\.\. 、 :：]?\s*(.+)$", re.S),
]


def _provider_label(provider: str) -> str:
    labels = {"deepseek": "DeepSeek", "kimi": "Kimi", "qwen": "千问"}
    return labels.get(str(provider or "").strip().lower(), provider or "")


class _QuestionRefSpecialCaseError(Exception):
    pass


def question_content_model_specs() -> list[dict[str, str]]:
    rows = load_project_parse_model_rows()
    for row in rows:
        if str(row.get("key") or "").strip() != "question_content_parse":
            continue
        models = row.get("models") or []
        specs: list[dict[str, str]] = []
        for index, item in enumerate(models):
            provider = str(item.get("provider") or "").strip().lower()
            model_name = str(item.get("model_name") or "").strip()
            if not provider or not model_name:
                continue
            specs.append(
                {
                    "key": f"model_{index + 1}",
                    "provider": provider,
```

```python
                    "model_name": model_name,
                    "label": f"{_provider_label(provider)}\n{model_name}",
                }
            )
        if specs:
            return specs
        break

    deepseek_model = load_deepseek_config().model.strip() or "deepseek-chat"
    kimi_model = load_kimi_config().model.strip() or "kimi-k2.6"
    qwen_model = load_qwen_config().model.strip() or "qwen-max"
    return [
        {"key": "model_1", "provider": "deepseek", "model_name": deepseek_model, "label": f"DeepSeek\n{deepseek_model}"},
        {"key": "model_2", "provider": "kimi", "model_name": kimi_model, "label": f"Kimi\n{kimi_model}"},
        {"key": "model_3", "provider": "qwen", "model_name": qwen_model, "label": f"千问\n{qwen_model}"},
    ]


def question_content_round_limit() -> int:
    rows = load_project_parse_model_rows()
    for row in rows:
        if str(row.get("key") or "").strip() != "question_content_parse":
            continue
        try:
            value = int(str(row.get("round") or "2").strip())
        except Exception:
            return 2
        return min(8, max(1, value))
    return 2


def question_content_ratio() -> tuple[int, int]:
    rows = load_project_parse_model_rows()
    for row in rows:
        if str(row.get("key") or "").strip() != "question_content_parse":
            continue
        raw = str(row.get("ratio") or "1/4").strip()
        left, sep, right = raw.partition("/")
        if not sep:
            return 1, 1
        try:
            numerator = int(left.strip())
            denominator = int(right.strip())
        except Exception:
            return 1, 1
        return max(1, numerator), max(1, denominator)
    return 1, 1


def question_content_llm_config(provider: str, model_name: str) -> LlmConfig:
    provider_key = str(provider or "").strip().lower()
    if provider_key == "kimi":
        cfg = load_kimi_config()
        return LlmConfig(
            base_url=cfg.base_url.strip(),
            api_key=cfg.api_key.strip(),
            model=model_name.strip(),
            timeout_s=float(cfg.timeout_s),
        )
    if provider_key == "qwen":
        cfg = load_qwen_config()
        return LlmConfig(
            base_url=cfg.base_url.strip(),
            api_key=cfg.api_key.strip(),
            model=model_name.strip(),
            timeout_s=float(cfg.timeout_s),
        )
    cfg = load_deepseek_config()
    return LlmConfig(
```

```python
        base_url=cfg.base_url.strip(),
        api_key=cfg.api_key.strip(),
        model=model_name.strip(),
        timeout_s=float(cfg.timeout_s),
    )


def question_content_provider_ready(provider: str) -> bool:
    provider_key = str(provider or "").strip().lower()
    if provider_key == "kimi":
        return load_kimi_config().is_ready()
    if provider_key == "qwen":
        return load_qwen_config().is_ready()
    return load_deepseek_config().is_ready()


def import_questions_from_sources(
    *,
    client: LlmClient | None = None,
    kimi_client: LlmClient | None = None,
    qwen_client: LlmClient | None = None,
    client_factory: Callable[[], LlmClient] | None = None,
    kimi_client_factory: Callable[[], LlmClient] | None = None,
    qwen_client_factory: Callable[[], LlmClient] | None = None,
    model_specs: list[dict[str, str]] | None = None,
    client_factories: dict[str, Callable[[], LlmClient]] | None = None,
    sources: list[tuple[Path, str]],
    max_chars_per_chunk: int = 6000,
    strategy: str = "per_question",
    max_question_workers: int = 1,
    progress_cb: Callable[[str], None] | None = None,
    question_cb: Callable[[Question], None] | None = None,
    compare_cb: Callable[[dict[str, Any]], None] | None = None,
    question_ref_compare_cb: Callable[[dict[str, Any]], None] | None = None,
    progress_count_cb: Callable[[int, int], None] | None = None,
    stop_cb: Callable[[], bool] | None = None,
    question_refs_by_source: dict[str, list[dict[str, str]]] | None = None,
) -> ImportResult:
    if strategy == "per_question":
        dynamic_specs = [item for item in (model_specs or []) if isinstance(item, dict)]
        dynamic_factories = dict(client_factories or {})
        if not dynamic_specs:
            dynamic_specs = question_content_model_specs()
        if not dynamic_factories:
            legacy_factories: dict[str, Callable[[], LlmClient]] = {}
            if client_factory is not None:
                legacy_factories["model_1"] = client_factory
            if kimi_client_factory is not None:
                legacy_factories["model_2"] = kimi_client_factory
            if qwen_client_factory is not None:
                legacy_factories["model_3"] = qwen_client_factory
            dynamic_factories = legacy_factories
        return _import_questions_per_question(
            client=client,
            kimi_client=kimi_client,
            qwen_client=qwen_client,
            client_factory=client_factory,
            kimi_client_factory=kimi_client_factory,
            qwen_client_factory=qwen_client_factory,
            model_specs=dynamic_specs,
            client_factories=dynamic_factories,
            sources=sources,
            max_chars_per_chunk=max_chars_per_chunk,
            max_question_workers=max_question_workers,
            progress_cb=progress_cb,
            question_cb=question_cb,
            compare_cb=compare_cb,
            question_ref_compare_cb=question_ref_compare_cb,
            progress_count_cb=progress_count_cb,
            stop_cb=stop_cb,
```

```python
            question_refs_by_source=question_refs_by_source,
        )

    items: list[dict[str, Any]] = []
    for path, text in sources:
        text = text.strip()
        if not text:
            continue
        for chunk in _split_text(text, max_chars_per_chunk=max_chars_per_chunk):
            chunk_items = _extract_questions_with_fallback(
                client=client,
                source_name=path.name,
                chunk_text=chunk,
                depth=3,
            )
            items.extend(chunk_items)

    normalized: list[Question] = []
    seen: set[str] = set()
    for obj in items:
        q = _to_question(obj)
        key = _normalize_key(q.stem)
        if key and key in seen:
            continue
        if key:
            seen.add(key)
        normalized.append(q)

    return ImportResult(questions=normalized, raw_items=items)

def _import_questions_per_question(
    *,
    client: LlmClient | None,
    kimi_client: LlmClient | None,
    qwen_client: LlmClient | None,
    client_factory: Callable[[], LlmClient] | None,
    kimi_client_factory: Callable[[], LlmClient] | None,
    qwen_client_factory: Callable[[], LlmClient] | None,
    model_specs: list[dict[str, str]],
    client_factories: dict[str, Callable[[], LlmClient]],
    sources: list[tuple[Path, str]],
    max_chars_per_chunk: int,
    max_question_workers: int,
    progress_cb: Callable[[str], None] | None,
    question_cb: Callable[[Question], None] | None,
    compare_cb: Callable[[dict[str, Any]], None] | None,
    question_ref_compare_cb: Callable[[dict[str, Any]], None] | None,
    progress_count_cb: Callable[[int, int], None] | None,
    stop_cb: Callable[[], bool] | None,
    question_refs_by_source: dict[str, list[dict[str, str]]] | None,
) -> ImportResult:
    items: list[dict[str, Any]] = []
    normalized: list[Question] = []
    seen: set[str] = set()
    active_specs = [item for item in model_specs if isinstance(item, dict)]
    scheduled_question_count = 0
    missing_ref_source_names: list[str] = []
    for path, text in sources:
        if stop_cb and stop_cb():
            break
        text = text.strip()
        if not text:
            continue
        source_key = str(path)
        if question_refs_by_source is not None:
            question_refs = _normalize_question_ref_list(question_refs_by_source.get(source_key, []))
        else:
            if progress_cb:
                progress_cb(f"{path.name}：统计题数…")
            question_refs = _normalize_question_ref_list(
```

```python
                _get_question_number_list_verified(
                    client=client,
                    kimi_client=kimi_client,
                    qwen_client=qwen_client,
                    source_name=path.name,
                    chunk_text=text,
                    depth=3,
                    compare_cb=question_ref_compare_cb,
                    source_path=source_key,
                )
            )
        if not question_refs:
            if question_refs_by_source is not None:
                missing_ref_source_names.append(path.name)
            continue
        total_steps = len(question_refs)
        scheduled_question_count += total_steps
        step = 0
        if progress_count_cb:
            progress_count_cb(0, total_steps)
        worker_count = max(1, int(max_question_workers))
        if worker_count <= 1:
            shared_clients_by_key = _build_question_content_clients_by_key(
                model_specs=active_specs,
                client_factories=client_factories,
                fallback_client=client,
                fallback_kimi_client=kimi_client,
                fallback_qwen_client=qwen_client,
            )
            for i, question_ref in enumerate(question_refs, start=1):
                if stop_cb and stop_cb():
                    break
                question_number = _as_str(question_ref.get("number", ""))
                question_type = _as_str(question_ref.get("question_type", ""))
                obj, err, meta = _process_one_question(
                    client=client,
                    kimi_client=kimi_client,
                    qwen_client=qwen_client,
                    model_specs=active_specs,
                    clients_by_key=shared_clients_by_key,
                    source_name=path.name,
                    chunk_text=text,
                    index=i,
                    total=total_steps,
                    requested_number=question_number,
                    requested_question_type=question_type,
                    progress_cb=progress_cb,
                    compare_cb=compare_cb,
                    stop_cb=stop_cb,
                )
                step += 1
                if progress_count_cb:
                    progress_count_cb(step, total_steps)
                _collect_question_result(
                    index=i,
                    obj=obj,
                    err=err,
                    meta=meta,
                    progress_cb=progress_cb,
                    source_name=path.name,
                    items=items,
                    normalized=normalized,
                    seen=seen,
                    question_cb=question_cb,
                )
            continue

        if not client_factories:
            raise ValueError("题级并发需要提供模型客户端工厂。")

```

```python
        task_items = [
            (
                i,
                _as_str(question_ref.get("number", "")),
                _as_str(question_ref.get("question_type", "")),
                path.name,
                text,
            )
            for i, question_ref in enumerate(question_refs, start=1)
        ]
        results_by_index: dict[int, tuple[dict[str, Any], bool, dict[str, Any]]] = {}

        def on_task_start(_current: int, _total_count: int, _task: tuple[int, str, str, str, str]) -> None:
            return

        def on_task_done(task: tuple[int, str, str, str, str], result: tuple[dict[str, Any], bool, dict[str, Any]]) -> None:
            nonlocal step
            index, _question_number, _question_type, _source_name, _chunk_text = task
            results_by_index[index] = result
            step += 1
            if progress_count_cb:
                progress_count_cb(step, total_steps)

        def on_task_failed(task: tuple[int, str, str, str, str], exc: Exception) -> None:
            nonlocal step
            index, question_number, question_type, _source_name, _chunk_text = task
            results_by_index[index] = ({}, True, {"index": index, "accepted": False, "reason": str(exc)})
            results_by_index[index][2]["requested_number"] = question_number
            results_by_index[index][2]["requested_question_type"] = question_type
            step += 1
            if progress_count_cb:
                progress_count_cb(step, total_steps)

        def run_one(task: tuple[int, str, str, str, str]) -> tuple[dict[str, Any], bool, dict[str, Any]]:
            index, question_number, question_type, source_name, chunk_text = task
            return _process_one_question(
                client=None,
                kimi_client=None,
                qwen_client=None,
                model_specs=active_specs,
                clients_by_key=_build_question_content_clients_by_key(
                    model_specs=active_specs,
                    client_factories=client_factories,
                    fallback_client=client,
                    fallback_kimi_client=kimi_client,
                    fallback_qwen_client=qwen_client,
                ),
                source_name=source_name,
                chunk_text=chunk_text,
                index=index,
                total=total_steps,
                requested_number=question_number,
                requested_question_type=question_type,
                progress_cb=progress_cb,
                compare_cb=compare_cb,
                stop_cb=stop_cb,
            )

        run_tasks_in_parallel(
            tasks=task_items,
            max_workers=min(worker_count, total_steps),
            stop_cb=(stop_cb or (lambda: False)),
            on_task_start=on_task_start,
            on_task_done=on_task_done,
            on_task_failed=on_task_failed,
            run_one=run_one,
        )

        for i in sorted(results_by_index.keys()):
```

```python
            obj, err, meta = results_by_index[i]
            _collect_question_result(
                index=i,
                obj=obj,
                err=err,
                meta=meta,
                progress_cb=progress_cb,
                source_name=path.name,
                items=items,
                normalized=normalized,
                seen=seen,
                question_cb=question_cb,
            )

    if question_refs_by_source is not None and scheduled_question_count <= 0:
        missing_text = "、".join(missing_ref_source_names) if missing_ref_source_names else "当前资料"
        raise RuntimeError(f"未接收到待解析题号：{missing_text}")

    return ImportResult(questions=normalized, raw_items=items)


def _build_question_content_clients_by_key(
    *,
    model_specs: list[dict[str, str]],
    client_factories: dict[str, Callable[[], LlmClient]],
    fallback_client: LlmClient | None,
    fallback_kimi_client: LlmClient | None,
    fallback_qwen_client: LlmClient | None,
) -> dict[str, LlmClient]:
    clients_by_key: dict[str, LlmClient] = {}
    for spec in model_specs:
        model_key = _as_str(spec.get("key", ""))
        if not model_key:
            continue
        factory = client_factories.get(model_key)
        if factory is not None:
            clients_by_key[model_key] = factory()
            continue
        provider = _as_str(spec.get("provider", "")).lower()
        if provider == "kimi" and fallback_kimi_client is not None:
            clients_by_key[model_key] = fallback_kimi_client
        elif provider == "qwen" and fallback_qwen_client is not None:
            clients_by_key[model_key] = fallback_qwen_client
        elif provider == "deepseek" and fallback_client is not None:
            clients_by_key[model_key] = fallback_client
    return clients_by_key


def _process_one_question(
    *,
    client: LlmClient | None,
    kimi_client: LlmClient | None,
    qwen_client: LlmClient | None,
    model_specs: list[dict[str, str]],
    clients_by_key: dict[str, LlmClient],
    source_name: str,
    chunk_text: str,
    index: int,
    total: int,
    requested_number: str,
    requested_question_type: str,
    progress_cb: Callable[[str], None] | None,
    compare_cb: Callable[[dict[str, Any]], None] | None,
    stop_cb: Callable[[], bool] | None,
) -> tuple[dict[str, Any], bool, dict[str, Any]]:
    _ = kimi_client, qwen_client
    round_limit = question_content_round_limit()

    def bump(model_name: str, round_no: int, check_no: int) -> None:
        if progress_cb:
```

```python
            progress_cb(
                f"{source_name}：第 {index}/{total} 题（题号 {requested_number}，第 {round_no}/{round_limit} 轮，{model_name}）…"
            )

    def mark_round(round_no: int, status: str) -> None:
        if not progress_cb:
            return
        if status == "start":
            progress_cb(
                f"{source_name}：第 {index}/{total} 题（题号 {requested_number}，第 {round_no}/{round_limit} 轮，多模型并行请求中）"
            )
        elif status == "consistent":
            progress_cb(
                f"{source_name}：第 {index}/{total} 题（题号 {requested_number}，第 {round_no}/{round_limit} 轮，达到一致阈值）"
            )
        elif status == "inconsistent":
            progress_cb(
                f"{source_name}：第 {index}/{total} 题（题号 {requested_number}，第 {round_no}/{round_limit} 轮，未达到一致阈值）"
            )

    return _get_question_n_verified(
        client=client,
        kimi_client=None,
        qwen_client=None,
        model_specs=model_specs,
        clients_by_key=clients_by_key,
        source_name=source_name,
        chunk_text=chunk_text,
        index=index,
        requested_number=requested_number,
        requested_question_type=requested_question_type,
        depth=2,
        attempt_cb=bump,
        round_cb=mark_round,
        compare_cb=compare_cb,
        stop_cb=stop_cb,
    )


def _collect_question_result(
    *,
    index: int,
    obj: dict[str, Any],
    err: bool,
    meta: dict[str, Any],
    progress_cb: Callable[[str], None] | None,
    source_name: str,
    items: list[dict[str, Any]],
    normalized: list[Question],
    seen: set[str],
    question_cb: Callable[[Question], None] | None,
) -> None:
    if err:
        if str(meta.get("reason") or "").strip().lower() == "stopped":
            return
        if progress_cb:
            progress_cb(f"{source_name}：第 {index} 题解析有误，已跳过。")
        return
    if not obj:
        return
    items.append(obj)
    q = _to_question(obj)
    key = _normalize_key(q.stem)
    if key and key in seen:
        return
    if key:
```

```python
        seen.add(key)
    normalized.append(q)
    if question_cb:
        question_cb(q)


def _question_extract_prompt_rules() -> str:
    return get_import_prompt("question_extract_system")


def _question_number_list_prompt_rules() -> str:
    return get_import_prompt("question_number_list_system")


def _normalize_question_ref_list(data: object) -> list[dict[str, str]]:
    if not isinstance(data, list):
        return []
    out: list[dict[str, str]] = []
    index_by_number: dict[str, int] = {}
    for item in data:
        if isinstance(item, dict):
            number = _as_str(item.get("number", ""))
            question_type = _normalize_question_type_text(_as_str(item.get("question_type", "")))
            duplicate_warning = _normalize_question_ref_duplicate_warning(_as_str(item.get("duplicate_warning", "")))
        else:
            number = _as_str(item)
            question_type = ""
            duplicate_warning = ""
        if not number:
            continue
        if number in index_by_number:
            row = out[index_by_number[number]]
            if question_type and not row.get("question_type"):
                row["question_type"] = question_type
            if duplicate_warning:
                row["duplicate_warning"] = duplicate_warning
            continue
        index_by_number[number] = len(out)
        row = {"number": number}
        if question_type:
            row["question_type"] = question_type
        if duplicate_warning:
            row["duplicate_warning"] = duplicate_warning
        out.append(row)
    return out


def _normalize_question_ref_duplicate_warning(value: str) -> str:
    text = _normalize_text(value)
    if not text:
        return ""
    if "重复" in text or "duplicate" in text.lower():
        return "存在可疑的题号重复"
    return ""


def _question_ref_compare_list(items: object) -> list[dict[str, str]]:
    out: list[dict[str, str]] = []
    for item in _normalize_question_ref_list(items):
        row = {"number": _as_str(item.get("number", ""))}
        question_type = _normalize_question_type_text(_as_str(item.get("question_type", "")))
        if question_type:
            row["question_type"] = question_type
        out.append(row)
    return out


def _normalize_question_type_text(value: str) -> str:
    text = _normalize_text(value)
```

```python
    if "可转多选" in text:
        return "可转多选"
    if "多选" in text:
        return "多选"
    if "单选" in text:
        return "单选"
    return ""


def _get_question_numbers_in_chunk(
    *,
    client: LlmClient,
    source_name: str,
    chunk_text: str,
) -> list[dict[str, str]]:
    sys_prompt = _question_number_list_prompt_rules()
    user_prompt = render_import_prompt(
        "question_number_list_user",
        source_name=source_name,
        chunk_text=chunk_text,
    )
    raw_text = client.chat_text(system=sys_prompt, user=user_prompt)
    data = _parse_question_ref_response_text(raw_text)
    return _normalize_question_ref_list(data)


def _parse_question_ref_response_text(text: object) -> list[dict[str, str]]:
    if isinstance(text, list):
        return _normalize_question_ref_list(text)
    raw = str(text or "").strip()
    if not raw:
        return []
    if raw == "[题号重复]":
        raise _QuestionRefSpecialCaseError("题号重复")
    if raw == "[所给文本无选择题目]":
        raise _QuestionRefSpecialCaseError("所给文本无选择题目")
    if raw.startswith("```"):
        raw = re.sub(r"^```(?:json)?\s*", "", raw)
        raw = re.sub(r"\s*```$", "", raw)
    try:
        data = json.loads(raw)
    except Exception:
        try:
            data = ast.literal_eval(raw)
        except Exception as e:
            raise RuntimeError(f"题号识别结果不是合法 JSON：{raw}") from e
    if not isinstance(data, list):
        raise RuntimeError(f"题号识别结果不是 JSON 数组：{raw}")
    return _normalize_question_ref_list(data)


def _get_question_numbers_with_fallback(
    *,
    client: LlmClient,
    source_name: str,
    chunk_text: str,
    depth: int,
) -> list[dict[str, str]]:
    try:
        return _get_question_numbers_in_chunk(client=client, source_name=source_name, chunk_text=chunk_text)
    except Exception as e:
        msg = str(e).lower()
        if isinstance(e, _QuestionRefSpecialCaseError):
            raise
        if depth <= 0 or len(chunk_text) < 1500:
            raise
        if "timed out" not in msg and "超时" not in msg:
            raise
        out: list[dict[str, str]] = []
        seen: set[str] = set()
```

```python
        for sub in _split_text(chunk_text, max_chars_per_chunk=max(800, len(chunk_text) // 2)):
            for question_ref in _get_question_numbers_with_fallback(
                client=client,
                source_name=source_name,
                chunk_text=sub,
                depth=depth - 1,
            ):
                number = _as_str(question_ref.get("number", ""))
                if number in seen:
                    continue
                seen.add(number)
                out.append(question_ref)
        return out


def _get_question_number_list_verified(
    *,
    client: LlmClient,
    kimi_client: LlmClient | None,
    qwen_client: LlmClient | None,
    source_name: str,
    chunk_text: str,
    depth: int,
    compare_cb: Callable[[dict[str, Any]], None] | None = None,
    source_path: str = "",
) -> list[dict[str, str]]:
    if kimi_client is None or qwen_client is None:
        raise RuntimeError("缺少 Kimi 或千问客户端，无法校验题号列表。")
    payload = _get_question_number_list_compare_payload(
        client=client,
        kimi_client=kimi_client,
        qwen_client=qwen_client,
        source_name=source_name,
        chunk_text=chunk_text,
        depth=depth,
        source_path=source_path,
    )
    if compare_cb is not None:
        compare_cb(payload)
    if bool(payload.get("accepted")):
        return _normalize_question_ref_list(payload.get("final_refs", []))
    markers = payload.get("markers", {})
    if isinstance(markers, dict) and markers:
        marker_text = "；".join(f"{name}=[{value}]" for name, value in markers.items())
        raise RuntimeError(f"{source_name}：题号识别异常。{marker_text}")
    raise RuntimeError(
        f"{source_name}：三模型识别出的选择题题号列表不一致，无法继续导入。"
        f" DeepSeek={payload.get('deepseek', [])}；Kimi={payload.get('kimi', [])}；千问={payload.get('qwen', [])}"
    )


def resolve_question_refs_from_sources(
    *,
    client: LlmClient,
    kimi_client: LlmClient | None,
    qwen_client: LlmClient | None,
    sources: list[tuple[Path, str]],
    progress_cb: Callable[[str], None] | None = None,
    compare_cb: Callable[[dict[str, Any]], None] | None = None,
    progress_count_cb: Callable[[int, int], None] | None = None,
    stop_cb: Callable[[], bool] | None = None,
) -> dict[str, list[dict[str, str]]]:
    refs_by_source: dict[str, list[dict[str, str]]] = {}
    total_sources = len(sources)
    completed = 0
    if progress_count_cb is not None:
        progress_count_cb(0, total_sources)
    for path, text in sources:
        if stop_cb and stop_cb():
```

```python
            break
        normalized_text = text.strip()
        if not normalized_text:
            completed += 1
            if progress_count_cb is not None:
                progress_count_cb(completed, total_sources)
            continue
        if progress_cb:
            progress_cb(f"{path.name}：题号与题型解析中…")
        question_refs = _get_question_number_list_verified(
            client=client,
            kimi_client=kimi_client,
            qwen_client=qwen_client,
            source_name=path.name,
            chunk_text=normalized_text,
            depth=3,
            compare_cb=compare_cb,
            source_path=str(path),
        )
        refs_by_source[str(path)] = _normalize_question_ref_list(question_refs)
        completed += 1
        if progress_count_cb is not None:
            progress_count_cb(completed, total_sources)
    return refs_by_source


def _get_question_number_list_compare_payload(
    *,
    client: LlmClient,
    kimi_client: LlmClient,
    qwen_client: LlmClient,
    source_name: str,
    chunk_text: str,
    depth: int,
    source_path: str = "",
) -> dict[str, Any]:
    results: dict[str, list[dict[str, str]]] = {}
    markers: dict[str, str] = {}
    with ThreadPoolExecutor(max_workers=3) as ex:
        future_map = {
            ex.submit(
                _get_question_numbers_with_fallback,
                client=client,
                source_name=source_name,
                chunk_text=chunk_text,
                depth=depth,
            ): "DeepSeek",
            ex.submit(
                _get_question_numbers_with_fallback,
                client=kimi_client,
                source_name=source_name,
                chunk_text=chunk_text,
                depth=depth,
            ): "Kimi",
            ex.submit(
                _get_question_numbers_with_fallback,
                client=qwen_client,
                source_name=source_name,
                chunk_text=chunk_text,
                depth=depth,
            ): "千问",
        }
        for future in as_completed(future_map):
            model_name = future_map[future]
            try:
                results[model_name] = future.result()
            except _QuestionRefSpecialCaseError as e:
                markers[model_name] = str(e)
                results[model_name] = []
    deepseek_numbers = _normalize_question_ref_list(results.get("DeepSeek", []))
```

```python
    kimi_numbers = _normalize_question_ref_list(results.get("Kimi", []))
    qwen_numbers = _normalize_question_ref_list(results.get("千问", []))
    accepted = (
        not markers
        and
        _question_ref_compare_list(deepseek_numbers)
        == _question_ref_compare_list(kimi_numbers)
        == _question_ref_compare_list(qwen_numbers)
    )
    return {
        "source_name": source_name,
        "source_path": source_path,
        "deepseek": deepseek_numbers,
        "kimi": kimi_numbers,
        "qwen": qwen_numbers,
        "markers": markers,
        "accepted": accepted,
        "final_refs": deepseek_numbers if accepted else [],
    }


def _extract_questions_from_chunk(
    *,
    client: LlmClient,
    source_name: str,
    chunk_text: str,
) -> list[dict[str, Any]]:
    sys_prompt = (
        _question_extract_prompt_rules()
        +
        "输出格式：JSON 数组，每项为对象：\n"
        "{\n"
        '  "question_type": "题型(单选/多选/可转多选；可转多选示例：可转多选)",\n'
        '  "number": "编号(可为空)",\n'
        '  "stem": "题干(普通题示例：我国社会主义民主政治的本质特征是什么？；可转多选题示例：阅读材料，贯彻绿色发展理念需要坚持哪些做法？)",\n'
        '  "option_1": "第1个选项正文；单选/多选通常对应A项，可转多选对应①项；不要包含选项标识",\n'
        '  "option_2": "第2个选项正文；单选/多选通常对应B项，可转多选对应②项；不要包含选项标识",\n'
        '  "option_3": "第3个选项正文；单选/多选通常对应C项，可转多选对应③项；不要包含选项标识",\n'
        '  "option_4": "第4个选项正文；单选/多选通常对应D项，可转多选对应④项；不要包含选项标识",\n'
        '  "choice_1": "可转多选时 A 对应的数字映射，例如 12；非可转多选留空",\n'
        '  "choice_2": "可转多选时 B 对应的数字映射，例如 14；非可转多选留空",\n'
        '  "choice_3": "可转多选时 C 对应的数字映射，例如 23；非可转多选留空",\n'
        '  "choice_4": "可转多选时 D 对应的数字映射，例如 24；非可转多选留空",\n'
        '  "answer": "答案(单选示例：A；多选示例：ACD；可转多选示例：B)"\n'
        "}\n"
        "如果某一题无法稳定识别，请不要输出半截对象，也不要输出无意义字段。\n"
    )

    user_prompt = (
        f"来源文件：{source_name}\n"
        f"导入日期：{date.today().isoformat()}\n"
        "资料文本如下：\n"
        "-----\n"
        f"{chunk_text}\n"
        "-----\n"
        "请输出严格 JSON 数组。"
    )

    data = client.chat_json(system=sys_prompt, user=user_prompt)
    if isinstance(data, list):
        out: list[dict[str, Any]] = []
        for it in data:
            if isinstance(it, dict):
                out.append(_normalize_question_obj_for_view(it))
        return out
    return []

def _extract_questions_with_fallback(
    *,
```

```python
    client: LlmClient,
    source_name: str,
    chunk_text: str,
    depth: int,
) -> list[dict[str, Any]]:
    try:
        return _extract_questions_from_chunk(client=client, source_name=source_name, chunk_text=chunk_text)
    except Exception as e:
        msg = str(e)
        if depth <= 0 or len(chunk_text) < 1500:
            raise
        if "timed out" not in msg.lower() and "超时" not in msg:
            raise

        sub_items: list[dict[str, Any]] = []
        for sub in _split_text(chunk_text, max_chars_per_chunk=max(800, len(chunk_text) // 2)):
            sub_items.extend(
                _extract_questions_with_fallback(
                    client=client,
                    source_name=source_name,
                    chunk_text=sub,
                    depth=depth - 1,
                )
            )
        return sub_items

def _get_question_by_number_in_chunk(
    *,
    client: LlmClient,
    source_name: str,
    chunk_text: str,
    requested_number: str,
    requested_question_type: str,
) -> dict[str, Any]:
    sys_prompt = _question_extract_prompt_rules()
    user_prompt = render_import_prompt(
        "question_extract_user",
        source_name=source_name,
        requested_number=requested_number,
        requested_question_type=requested_question_type,
        chunk_text=chunk_text,
    )
    data = client.chat_json(system=sys_prompt, user=user_prompt)
    return _normalize_question_obj_for_view(data) if isinstance(data, dict) else {}


def _get_question_by_number_with_fallback(
    *,
    client: LlmClient,
    source_name: str,
    chunk_text: str,
    requested_number: str,
    requested_question_type: str,
    depth: int,
) -> dict[str, Any]:
    try:
        return _get_question_by_number_in_chunk(
            client=client,
            source_name=source_name,
            chunk_text=chunk_text,
            requested_number=requested_number,
            requested_question_type=requested_question_type,
        )
    except Exception as e:
        msg = str(e).lower()
        if depth <= 0 or len(chunk_text) < 1500:
            raise
        if "timed out" not in msg and "超时" not in msg:
            raise
        for sub in _split_text(chunk_text, max_chars_per_chunk=max(800, len(chunk_text) // 2)):
```

```python
            obj = _get_question_by_number_with_fallback(
                client=client,
                source_name=source_name,
                chunk_text=sub,
                requested_number=requested_number,
                requested_question_type=requested_question_type,
                depth=depth - 1,
            )
            if obj:
                return obj
        return {}


def _get_question_n_verified(
    *,
    client: LlmClient | None,
    kimi_client: LlmClient | None,
    qwen_client: LlmClient | None,
    model_specs: list[dict[str, str]],
    clients_by_key: dict[str, LlmClient],
    source_name: str,
    chunk_text: str,
    index: int,
    requested_number: str,
    requested_question_type: str,
    depth: int,
    attempt_cb: Callable[[str, int, int], None] | None = None,
    round_cb: Callable[[int, str], None] | None = None,
    compare_cb: Callable[[dict[str, Any]], None] | None = None,
    stop_cb: Callable[[], bool] | None = None,
) -> tuple[dict[str, Any], bool, dict[str, Any]]:
    _ = client, kimi_client, qwen_client
    active_specs = [item for item in model_specs if isinstance(item, dict) and str(item.get("key") or "").strip()]
    if len(active_specs) < 1:
        return {}, True, {"index": index, "accepted": False, "reason": "missing_clients"}

    last_meta: dict[str, Any] = {"index": index, "accepted": False, "reason": "inconsistent"}
    cumulative_valid_objs: list[dict[str, Any]] = []
    round_limit = question_content_round_limit()
    ratio_numerator, ratio_denominator = question_content_ratio()
    valid_denominator = max(1, ratio_denominator)
    active_model_count = len(active_specs)

    def build_meta(
        *,
        round_no: int,
        results_by_model: dict[str, dict[str, Any]],
        costs_sec_by_model: dict[str, float],
        round_matched_count: int,
        round_valid_count: int,
        required_count: int,
        matched_count: int,
        valid_count: int,
        accepted: bool,
        reason: str,
        accepted_obj: dict[str, Any] | None = None,
        partial: bool = False,
        completed_model_count: int = 0,
    ) -> dict[str, Any]:
        payload: dict[str, Any] = {
            "index": index,
            "requested_number": requested_number,
            "requested_question_type": requested_question_type,
            "round": round_no,
            "model_specs": active_specs,
            "results_by_model": {
                model_key: _normalize_question_obj_for_view(obj)
                for model_key, obj in results_by_model.items()
            },
```

```
            "costs_sec_by_model": dict(costs_sec_by_model),
            "round_matched_count": round_matched_count,
            "round_valid_count": round_valid_count,
            "round_limit": round_limit,
            "active_model_count": active_model_count,
            "completed_model_count": completed_model_count,
            "per_round_required": per_round_required,
            "required_count": required_count,
            "matched_count": matched_count,
            "valid_count": valid_count,
            "ratio_numerator": ratio_numerator,
            "ratio_denominator": valid_denominator,
            "accepted": accepted,
            "reason": reason,
            "partial": partial,
        }
        if accepted_obj:
            payload["accepted_obj"] = accepted_obj
        return payload

    for round_no in range(1, round_limit + 1):
        if stop_cb and stop_cb():
            return {}, True, {"index": index, "accepted": False, "reason": "stopped"}
        if round_cb:
            round_cb(round_no, "start")
        per_round_required = _required_consensus_count(
            numerator=ratio_numerator,
            denominator=valid_denominator,
            active_model_count=active_model_count,
        )
        required_count = max(1, per_round_required * round_no)
        results_by_model: dict[str, dict[str, Any]] = {str(spec.get("key") or ""): {} for spec in active_specs}
        costs_sec_by_model: dict[str, float] = {str(spec.get("key") or ""): 0.0 for spec in active_specs}
        with ThreadPoolExecutor(max_workers=max(1, len(active_specs))) as ex:
            starts: dict[str, float] = {str(spec.get("key") or ""): time.perf_counter() for spec in active_specs}
            fut_map = {}
            for check_no, spec in enumerate(active_specs, start=1):
                model_key = str(spec.get("key") or "")
                model_client = clients_by_key.get(model_key)
                if model_client is None:
                    continue
                fut_map[
                    ex.submit(
                        _safe_get_question_by_number_with_fallback,
                        client=model_client,
                        source_name=source_name,
                        chunk_text=chunk_text,
                        requested_number=requested_number,
                        requested_question_type=requested_question_type,
                        depth=depth,
                    )
                ] = (model_key, check_no)
            for fut in as_completed(fut_map):
                model_key, check_no = fut_map[fut]
                spec = next((item for item in active_specs if str(item.get("key") or "") == model_key), None)
                model_name = str(spec.get("label") or model_key) if isinstance(spec, dict) else model_key
                costs_sec_by_model[model_key] = round(time.perf_counter() - starts[model_key], 3)
                if attempt_cb:
                    attempt_cb(model_name, round_no, check_no)
                try:
                    results_by_model[model_key] = fut.result() or {}
                except Exception:
                    results_by_model[model_key] = {}
                if compare_cb is not None:
                    completed_model_count = 0
                    for current_key in costs_sec_by_model:
                        if costs_sec_by_model.get(current_key, 0.0) > 0 or results_by_model.get(current_key):
                            completed_model_count += 1
```

```
                    partial_round_valid_objs = [obj for obj in results_by_model.values() if _is_valid_question_obj(obj)]
                    _, partial_round_matched_count = _pick_consensus_obj(partial_round_valid_objs, 1)
                    partial_cumulative_valid_objs = cumulative_valid_objs + partial_round_valid_objs
                    _, partial_matched_count = _pick_consensus_obj(partial_cumulative_valid_objs, required_count)
                    compare_cb(
                        build_meta(
                            round_no=round_no,
                            results_by_model=results_by_model,
                            costs_sec_by_model=costs_sec_by_model,
                            round_matched_count=partial_round_matched_count,
                            round_valid_count=len(partial_round_valid_objs),
                            required_count=required_count,
                            matched_count=partial_matched_count,
                            valid_count=len(partial_cumulative_valid_objs),
                            accepted=False,
                            reason="processing",
                            partial=True,
                            completed_model_count=completed_model_count,
                        )
                    )

        if not fut_map:
            return {}, True, {"index": index, "accepted": False, "reason": "missing_clients"}

        round_valid_objs = [obj for obj in results_by_model.values() if _is_valid_question_obj(obj)]
        completed_model_count = 0
        for model_key in costs_sec_by_model:
            if costs_sec_by_model.get(model_key, 0.0) > 0 or results_by_model.get(model_key):
                completed_model_count += 1
        _, round_matched_count = _pick_consensus_obj(round_valid_objs, 1)
        cumulative_valid_objs.extend(round_valid_objs)
        accepted_obj, matched_count = _pick_consensus_obj(cumulative_valid_objs, required_count)
        last_meta = build_meta(
            round_no=round_no,
            results_by_model=results_by_model,
            costs_sec_by_model=costs_sec_by_model,
            round_matched_count=round_matched_count,
            round_valid_count=len(round_valid_objs),
            required_count=required_count,
            matched_count=matched_count,
            valid_count=len(cumulative_valid_objs),
            accepted=False,
            reason="inconsistent",
            partial=False,
            completed_model_count=completed_model_count,
        )
        if compare_cb is not None:
            compare_cb(last_meta)
        if accepted_obj:
            if round_cb:
                round_cb(round_no, "consistent")
            accepted_obj = _normalize_question_obj_for_view(accepted_obj)
            last_meta = build_meta(
                round_no=round_no,
                results_by_model=results_by_model,
                costs_sec_by_model=costs_sec_by_model,
                round_matched_count=round_matched_count,
                round_valid_count=len(round_valid_objs),
                required_count=required_count,
                matched_count=matched_count,
                valid_count=len(cumulative_valid_objs),
                accepted=True,
                reason="consistent",
                accepted_obj=accepted_obj,
                partial=False,
                completed_model_count=completed_model_count,
            )
            if compare_cb is not None:
```

```python
                compare_cb(last_meta)
            return accepted_obj, False, last_meta
        if round_cb:
            round_cb(round_no, "inconsistent")
    return {}, True, last_meta


def _required_consensus_count(*, numerator: int, denominator: int, active_model_count: int) -> int:
    valid_numerator = max(1, int(numerator or 1))
    valid_denominator = max(1, int(denominator or 1))
    valid_model_count = max(1, int(active_model_count or 1))
    return max(1, min(valid_model_count, math.ceil(valid_model_count * valid_numerator / valid_denominator)))


def _safe_get_question_by_number_with_fallback(
    *,
    client: LlmClient,
    source_name: str,
    chunk_text: str,
    requested_number: str,
    requested_question_type: str,
    depth: int,
) -> dict[str, Any]:
    max_attempts = 2
    for _attempt in range(max_attempts):
        try:
            obj = _get_question_by_number_with_fallback(
                client=client,
                source_name=source_name,
                chunk_text=chunk_text,
                requested_number=requested_number,
                requested_question_type=requested_question_type,
                depth=depth,
            )
        except Exception:
            obj = {}
        if obj:
            return obj
    return {}


def _to_question(obj: dict[str, Any]) -> Question:
    number = _as_str(obj.get("number", ""))
    stem = _as_str(obj.get("stem", ""))
    number, stem = _split_number_and_stem(number, stem)
    question_type = _normalize_question_type_value(obj)
    option_values = _option_values_from_obj(obj, question_type=question_type)
    options_str = _build_options_string(option_values, question_type=question_type)
    answer = _as_str(obj.get("answer", ""))
    analysis = _original_analysis_text(obj)
    q = Question(
        number=number,
        stem=stem,
        options=options_str,
        answer=answer,
        analysis=analysis,
        question_type=question_type,
        choice_1=_normalize_choice_digits(_as_str(obj.get("choice_1", ""))),
        choice_2=_normalize_choice_digits(_as_str(obj.get("choice_2", ""))),
        choice_3=_normalize_choice_digits(_as_str(obj.get("choice_3", ""))),
        choice_4=_normalize_choice_digits(_as_str(obj.get("choice_4", ""))),
    )
    return _normalize_combination_question(q)


def _normalize_combination_question(q: Question) -> Question:
    combined = "\n".join([q.stem or "", q.options or ""]).strip()
    if not combined and not _question_choice_map(q):
        return q

```

```python
    statements = _extract_circled_statements(combined)
    combos = _question_choice_map(q) or _extract_combo_map(combined)
    is_convertible = q.question_type == "可转多选" or bool(combos)
    if not is_convertible:
        return q

    if len(statements) < 2 and q.options.strip():
        statements = _split_circled_option_lines(q.options)
    if len(statements) < 2:
        return q

    answer = (q.answer or "").strip().replace(" ", "")
    answer = _normalize_convertible_answer(answer, combos)

    new_options = "\n".join(statements).strip()
    new_stem = q.stem
    first = _CIRCLED_RE.search(new_stem or "")
    if first is not None:
        prefix = (new_stem or "")[: first.start()].strip()
        if prefix:
            new_stem = prefix

    return Question(
        number=q.number,
        stem=new_stem,
        options=new_options,
        answer=answer,
        analysis=q.analysis,
        question_type="可转多选",
        choice_1=combos.get("A", ""),
        choice_2=combos.get("B", ""),
        choice_3=combos.get("C", ""),
        choice_4=combos.get("D", ""),
    )


def _extract_combo_map(text: str) -> dict[str, str]:
    out: dict[str, str] = {}
    for m in _COMBO_RE.finditer(text):
        letter = m.group(1)
        digits = _normalize_choice_digits("".join(_CIRCLED_RE.findall(m.group(2))))
        if digits:
            out[letter] = digits
    return out


def _split_circled_option_lines(text: str) -> list[str]:
    return [line.strip() for line in _normalize_text(text).split("\n") if line.strip() and _CIRCLED_RE.match(line.strip())]


def _question_choice_map(q: Question) -> dict[str, str]:
    return {
        letter: value
        for letter, value in (
            ("A", _normalize_choice_digits(q.choice_1)),
            ("B", _normalize_choice_digits(q.choice_2)),
            ("C", _normalize_choice_digits(q.choice_3)),
            ("D", _normalize_choice_digits(q.choice_4)),
        )
        if value
    }


def _normalize_choice_digits(text: str) -> str:
    if not text:
        return ""
    circled_chars = _CIRCLED_RE.findall(text)
    if circled_chars:
        return "".join(str(ord(ch) - 0x245F) for ch in circled_chars)
```

```python
    return "".join(re.findall(r"\d+", text))


def _normalize_convertible_answer(answer: str, combo_map: dict[str, str]) -> str:
    upper = _as_str(answer).replace(" ", "").upper()
    if not upper:
        return ""
    if _LETTER_ONLY_RE.fullmatch(upper):
        return upper
    digits = _normalize_choice_digits(upper)
    if digits:
        mapped = _choice_digits_to_letter(digits, combo_map)
        return mapped or digits
    return upper


def _choice_digits_to_letter(digits: str, combo_map: dict[str, str]) -> str:
    normalized = _normalize_choice_digits(digits)
    if not normalized:
        return ""
    for letter in ("A", "B", "C", "D"):
        if _normalize_choice_digits(combo_map.get(letter, "")) == normalized:
            return letter
    return ""


def _extract_circled_statements(text: str) -> list[str]:
    circled = list(_CIRCLED_RE.finditer(text))
    if not circled:
        return []

    first_combo = _COMBO_RE.search(text)
    combo_start = first_combo.start() if first_combo is not None else len(text)

    out: list[str] = []
    for i, m in enumerate(circled):
        start = m.start()
        if start >= combo_start:
            break
        end = combo_start
        if i + 1 < len(circled):
            end = min(end, circled[i + 1].start())
        seg = text[m.end() : end].strip()
        if seg:
            out.append(f"{m.group(0)}{seg}")
    return out


def _sort_circled(text: str) -> str:
    order = [chr(c) for c in range(0x2460, 0x2474)]
    present = set(_CIRCLED_RE.findall(text))
    return "".join([c for c in order if c in present])


def _split_text(text: str, *, max_chars_per_chunk: int) -> Iterable[str]:
    if len(text) <= max_chars_per_chunk:
        yield text
        return

    lines = text.splitlines()
    buf: list[str] = []
    size = 0
    for line in lines:
        ln = line.rstrip()
        if not ln:
            if buf:
                buf.append("")
                size += 1
            continue
        if size + len(ln) + 1 > max_chars_per_chunk and buf:
```

```python
            yield "\n".join(buf).strip() + "\n"
            buf = []
            size = 0
        buf.append(ln)
        size += len(ln) + 1
    if buf:
        yield "\n".join(buf).strip() + "\n"


def _normalize_key(stem: str) -> str:
    s = stem.strip()
    s = s.replace("\r", "\n")
    s = _WS_RE.sub(" ", s)
    return s.strip().lower()


def _as_str(v: Any) -> str:
    if v is None:
        return ""
    if isinstance(v, str):
        return v.strip()
    return str(v).strip()


def _all_same(objs: list[dict[str, Any]]) -> bool:
    if not objs:
        return False
    fp0 = _fingerprint_question_obj(objs[0])
    return all(_fingerprint_question_obj(o) == fp0 for o in objs[1:])


def _pick_consensus_obj(objs: list[dict[str, Any]], min_count: int) -> tuple[dict[str, Any], int]:
    if min_count <= 0:
        return {}, 0
    counter: dict[str, tuple[int, dict[str, Any]]] = {}
    for obj in objs:
        fp = _fingerprint_question_obj(obj)
        if not fp:
            continue
        if fp not in counter:
            counter[fp] = (1, obj)
            continue
        count, sample = counter[fp]
        counter[fp] = (count + 1, sample)
    best_obj: dict[str, Any] = {}
    best_count = 0
    for count, sample in counter.values():
        if count > best_count:
            best_count = count
            best_obj = sample
    if best_count >= min_count:
        return best_obj, best_count
    return {}, best_count


def _is_valid_question_obj(obj: dict[str, Any]) -> bool:
    if not isinstance(obj, dict) or not obj:
        return False
    stem = _normalize_text(_as_str(obj.get("stem", "")))
    answer = _as_str(obj.get("answer", "")).replace(" ", "").upper()
    question_type = _normalize_question_type_value(obj)
    opt_text = _normalize_text(_canonical_options_text(obj, question_type=question_type))
    has_choice_fields = _has_choice_fields_obj(obj)
    if _has_circled_only_options(opt_text) and _has_letter_only_answer(answer) and not has_choice_fields:
        return False
    if question_type == "可转多选" and not has_choice_fields and not _extract_combo_map("\n".join([stem, opt_text])):
        return False
    return bool(stem and opt_text and question_type)
```

```python

def _has_circled_only_options(options_text: str) -> bool:
    text = _normalize_text(options_text)
    if not text:
        return False
    lines = [ln.strip() for ln in text.split("\n") if ln.strip()]
    if len(lines) < 2:
        return False
    circled_lines = 0
    letter_lines = 0
    for line in lines:
        if line and _CIRCLED_RE.match(line):
            circled_lines += 1
        if re.match(r"^[A-D][\.. 、]", line):
            letter_lines += 1
    return circled_lines >= 2 and letter_lines == 0


def _has_letter_only_answer(answer: str) -> bool:
    return bool(answer) and bool(_LETTER_ONLY_RE.fullmatch(answer))


def _normalize_options_dict(d: dict[str, Any]) -> dict[str, str]:
    out: dict[str, str] = {}
    for k, v in d.items():
        ks = _as_str(k)
        vs = _normalize_text(_as_str(v))
        if ks and vs:
            out[ks] = vs
    return out


def _has_option_fields_obj(obj: dict[str, Any]) -> bool:
    return any(_as_str(obj.get(key, "")) for key in _OPTION_KEYS)


def _option_values_from_obj(obj: dict[str, Any], *, question_type: str) -> list[str]:
    option_values = [_normalize_option_value(_as_str(obj.get(key, ""))) for key in _OPTION_KEYS]
    if any(option_values):
        return option_values
    return _legacy_option_values(obj.get("options", ""), question_type=question_type)


def _legacy_option_values(options: Any, *, question_type: str) -> list[str]:
    if isinstance(options, dict):
        values = [
            _normalize_option_value(value)
            for _key, value in sorted(_normalize_options_dict(options).items(), key=lambda item: item[0])
        ]
        return (values + ["", "", "", ""])[:4]
    text = _normalize_text(_options_to_string(options))
    if not text:
        return ["", "", "", ""]
    normalized = _force_newline_before_option_markers(text)
    lines = [line.strip() for line in normalized.split("\n") if line.strip()]
    values: list[str] = []
    current_parts: list[str] = []
    for line in lines:
        if _starts_with_option_marker(line):
            if current_parts:
                values.append(_normalize_option_value(" ".join(current_parts)))
            current_parts = [_strip_option_marker(line)]
            continue
        if current_parts:
            current_parts.append(line)
        else:
            current_parts = [_strip_option_marker(line)]
    if current_parts:
        values.append(_normalize_option_value(" ".join(current_parts)))
    values = [value for value in values if value]
```

```python
    return (values + ["", "", "", ""])[:4]


def _canonical_options_text(obj: dict[str, Any], *, question_type: str) -> str:
    if _has_option_fields_obj(obj):
        return _build_options_string(_option_values_from_obj(obj, question_type=question_type), question_type=question_type)
    options = obj.get("options", "")
    if isinstance(options, dict):
        return json.dumps(_normalize_options_dict(options), ensure_ascii=False, sort_keys=True, separators=(", ", ":"))
    return _normalize_text(_options_to_string(options))


def _build_options_string(option_values: list[str], *, question_type: str) -> str:
    qtype = question_type if question_type in ("单选", "多选", "可转多选") else "单选"
    markers = ["①", "②", "③", "④"] if qtype == "可转多选" else ["A", "B", "C", "D"]
    lines = [
        f"{marker}. {value}".rstrip()
        for marker, value in zip(markers, option_values)
        if _normalize_option_value(value)
    ]
    return "\n".join(lines).strip()


def _force_newline_before_option_markers(text: str) -> str:
    out = (text or "").replace("\r\n", "\n").replace("\r", "\n")
    out = _LETTER_MARKER_RE.sub("\n", out)
    out = _CIRCLED_MARKER_RE.sub("\n", out)
    return out.strip()


def _starts_with_option_marker(text: str) -> bool:
    s = text.strip()
    return bool(re.match(r"^[A-D][\.\u3001\uFF0E:：]", s) or re.match(r"^[\u2460-\u2473][\.\u3001\uFF0E:：]?", s))


def _strip_option_marker(text: str) -> str:
    s = _normalize_text(text)
    s = re.sub(r"^\s*[A-D][\.\u3001\uFF0E:：]\s*", "", s)
    s = re.sub(r"^\s*[\u2460-\u2473][\.\u3001\uFF0E:：]?\s*", "", s)
    return s.strip()


def _normalize_option_value(text: str) -> str:
    return _strip_option_marker(text)


def _options_to_string(v: Any) -> str:
    if isinstance(v, list):
        lines = [_as_str(it) for it in v]
        return "\n".join([ln for ln in lines if ln]).strip()
    if isinstance(v, tuple):
        lines = [_as_str(it) for it in v]
        return "\n".join([ln for ln in lines if ln]).strip()
    if isinstance(v, str):
        parsed = _parse_options_list_text(v)
        if parsed is not None:
            return _options_to_string(parsed)
    return _as_str(v)


def _parse_options_list_text(text: str) -> list[Any] | None:
    s = _as_str(text)
    if not (s.startswith("[") and s.endswith("]")):
        return None
    try:
        data = json.loads(s)
        if isinstance(data, list):
```

```python
            return data
    except Exception:
        pass
    try:
        data = ast.literal_eval(s)
        if isinstance(data, list):
            return data
    except Exception:
        return None
    return None


def _split_number_and_stem(number: str, stem: str) -> tuple[str, str]:
    num = _as_str(number)
    s = _as_str(stem)
    if not s:
        return num, s
    for pat in _STEM_NUMBER_PATTERNS:
        m = pat.match(s)
        if not m:
            continue
        candidate_num = _as_str(m.group(1))
        candidate_stem = _as_str(m.group(2))
        if not candidate_stem:
            continue
        if not num:
            return candidate_num, candidate_stem
        if num == candidate_num:
            return num, candidate_stem
    return num, s


def _normalize_question_obj_for_view(obj: dict[str, Any]) -> dict[str, Any]:
    if not isinstance(obj, dict) or not obj:
        return {}
    out = dict(obj)
    q = _to_question(out)
    option_values = _option_values_from_obj(out, question_type=q.question_type)
    out["number"] = q.number
    out["stem"] = q.stem
    out["answer"] = q.answer
    out["question_type"] = q.question_type or _normalize_question_type_value(out)
    for idx, value in enumerate(option_values, start=1):
        out[f"option_{idx}"] = value
    out.pop("options", None)
    out["choice_1"] = q.choice_1
    out["choice_2"] = q.choice_2
    out["choice_3"] = q.choice_3
    out["choice_4"] = q.choice_4
    out.pop("original_analysis", None)
    out.pop("analysis", None)
    return out


def _normalize_text(text: str) -> str:
    s = (text or "").replace("\r\n", "\n").replace("\r", "\n")
    lines = [ln.strip() for ln in s.split("\n")]
    s = "\n".join([ln for ln in lines if ln])
    s = _WS_RE.sub(" ", s)
    return s.strip()


def _fingerprint_question_obj(obj: dict[str, Any]) -> str:
    if not isinstance(obj, dict):
        return ""
    question_type = _normalize_question_type_value(obj)
    stem = _normalize_text(_as_str(obj.get("stem", "")))
    answer = _as_str(obj.get("answer", "")).replace(" ", "").upper()
    option_values = _option_values_from_obj(obj, question_type=question_type)
    payload = {
```

```python
        "question_type": question_type,
        "stem": stem,
        "option_1": option_values[0],
        "option_2": option_values[1],
        "option_3": option_values[2],
        "option_4": option_values[3],
        "answer": answer,
        "choice_1": _normalize_choice_digits(_as_str(obj.get("choice_1", ""))),
        "choice_2": _normalize_choice_digits(_as_str(obj.get("choice_2", ""))),
        "choice_3": _normalize_choice_digits(_as_str(obj.get("choice_3", ""))),
        "choice_4": _normalize_choice_digits(_as_str(obj.get("choice_4", ""))),
    }
    return json.dumps(payload, ensure_ascii=False, sort_keys=True, separators=(",", ":"))


def _original_analysis_text(obj: dict[str, Any]) -> str:
    return _as_str(obj.get("original_analysis", "")) or _as_str(obj.get("analysis", ""))


def _normalize_question_type_value(obj: dict[str, Any]) -> str:
    raw = _as_str(obj.get("question_type", ""))
    if raw in ("单选", "多选", "可转多选"):
        return raw
    if _has_choice_fields_obj(obj):
        return "可转多选"
    options_text = _normalize_text(_canonical_options_text(obj, question_type=raw))
    answer = _as_str(obj.get("answer", "")).replace(" ", "").upper()
    if _has_circled_only_options(options_text):
        return "可转多选"
    if _has_multi_answer(answer):
        return "多选"
    return "单选"


def _has_choice_fields_obj(obj: dict[str, Any]) -> bool:
    return any(
        _normalize_choice_digits(_as_str(obj.get(key, "")))
        for key in ("choice_1", "choice_2", "choice_3", "choice_4")
    )


def _has_multi_answer(answer: str) -> bool:
    if not answer:
        return False
    if "," in answer:
        return len([part.strip() for part in answer.split(",") if part.strip()]) > 1
    if _LETTER_ONLY_RE.fullmatch(answer):
        return len(answer) > 1
    circled_chars = _CIRCLED_RE.findall(answer)
    return len(circled_chars) > 1
```

```python
from __future__ import annotations

import json
import os
from dataclasses import dataclass
from pathlib import Path


@dataclass(frozen=True)
class PromptField:
    key: str
    title: str
    description: str
    placeholders: tuple[str, ...] = ()


PROMPT_FIELDS: tuple[PromptField, ...] = (
    PromptField(
        key="question_number_list_system",
        title="题号识别 System Prompt",
        description="三模型识别选择题题号列表时使用的系统提示词。",
    ),
    PromptField(
        key="question_number_list_user",
        title="题号识别 User Prompt",
        description="识别题号列表时发送给模型的用户提示词模板。",
        placeholders=("{{source_name}}", "{{chunk_text}}"),
    ),
    PromptField(
        key="question_extract_system",
        title="逐题提取 System Prompt",
        description="按题号与题型从整篇资料文本中提取目标 JSON 时使用的系统提示词。",
    ),
    PromptField(
        key="question_extract_user",
        title="逐题提取 User Prompt",
        description="按题号与题型提取时发送给模型的用户提示词模板。",
        placeholders=("{{source_name}}", "{{requested_number}}", "{{requested_question_type}}", "{{chunk_text}}"),
    ),
    PromptField(
        key="explanation_system",
        title="解析生成 System Prompt",
        description="补解析阶段生成答案解析时使用的系统提示词。",
    ),
    PromptField(
        key="explanation_user",
        title="解析生成 User Prompt",
        description="补解析阶段发送给模型的用户提示词模板。",
        placeholders=("{{question_text}}", "{{answer_text}}", "{{reference_block}}", "{{mistakes_block}}"),
    ),
    PromptField(
        key="explanation_fill_answer_system",
        title="补答案解析 System Prompt",
        description="缺少答案时，先补答案再生成解析使用的系统提示词。",
    ),
    PromptField(
        key="explanation_fill_answer_user",
        title="补答案解析 User Prompt",
        description="缺少答案时发送给模型的用户提示词模板。",
        placeholders=("{{question_text}}", "{{reference_block}}", "{{mistakes_block}}"),
    ),
)


DEFAULT_IMPORT_PROMPTS: dict[str, str] = {
    "question_number_list_system": (
        "你是一个题库整理助手。你必须仅输出严格 JSON，不要输出任何解释、markdown、前后缀、代码块或额外文本。\n"
        "任务：从提供的资料文本中识别所有选择题的题号列表。\n"
        "题型范围只包括：”单选“、”多选“、”可转多选“。\n"
```

```
        "其中“可转多选”是指：题干或材料先给出 ①②③④ 等若干表述，再给出 A/B/C/D 表示这些表述的不同组合，本质上仍属于单选，但是给出了四种可能的多选，你需要把这种题视为”可转多选“。\n"
        "”可转多选“类型示例：\n"
        "某班同学在学习“中国特色社会主义”时，对以下观点进行讨论，其中正确的观点是（ ）\n"
        "①坚持党的领导是中国特色社会主义最本质的特征；\n"
        "②坚持以人民为中心是中国特色社会主义的根本任务；\n"
        "③坚持依法治国是中国特色社会主义的本质要求；\n"
        "④坚持改革开放是中国特色社会主义的唯一动力。\n"
        "A.①② B.①③ C.②④ D.③④\n"
        "必须输出 JSON 数组，数组每一项都是对象，至少包含 number 和 question_type 两个字段。\n"
        "示例输出：[{\"number\":\"1\",\"question_type\":\"单选\"},{\"number\":\"2\",\"question_type\":\"多选\"},{\"number\":\"5\",\"question_type\":\"可转多选\"}]。\n"
        "保持原始出现顺序，不要重排。\n"
        "如果某题没有明确题号，不要编造。\n"
        "question_type 只能填写“单选”“多选”“可转多选”三者之一；无法确定时不要猜测，跳过该题。\n"
        "如果没有识别到任何带题号的选择题，输出 []。\n"
        "异常情况只返回[错误类型]\n"
        "错误类型包括：题号重复、所给文本无选择题目\n"
    ),
    "question_number_list_user": (
        "来源文件：{{source_name}}\n"
        "资料文本如下：\n"
        "-----\n"
        "{{chunk_text}}\n"
        "-----\n"
        "请输出所有选择题题号列表。"
    ),
    "question_extract_system": (
        "你是一个题库整理助手。你必须仅输出严格 JSON，不要输出任何解释、markdown、前后缀、代码块或额外文本。\n"
        "任务：根据给定的题号和已知选择题类型，从整篇 Word 文本中精确定位并提取目标题目的 JSON 信息。\n"
        "你拿到的是整篇资料文本，而不是只包含目标题目的片段；你必须先在全文中找到与指定题号对应、且与给定题型一致的那一道题，再输出该题的结构化 JSON。\n"
        "如果全文中同一题号出现多个候选内容，优先选择与给定题型一致、题干和选项结构最完整、最像一道独立选择题的那一项。\n"
        "如果找到同题号但题型与给定题型明显不一致，不要强行抽取错误题目；应继续在全文中查找更匹配的候选。\n"
        "如果最终无法在全文中稳定定位到“指定题号 + 指定题型”的目标题，请输出空对象 {}。\n"
        "题目可能是以下三类之一：\n"
        "- 单选：选项标识通常为 A/B/C/D 或 A. / A、 等，答案是单个选项标识。\n"
        "- 多选：答案本身是多个选项标识的组合。\n"
        "- 可转多选：题干或材料中先给出 ①②③④ 等若干表述，再由 A/B/C/D 表示这些表述的不同组合；这类题本质上是组合型选择题。\n"
        "如果原文没有明确答案，answer 允许为空字符串。\n"
        "字段规则必须严格遵守：\n"
        '1. question_type 必须输出且只能是 "单选"、"多选"、"可转多选" 三者之一。\n'
        '2. number 只保留题号本身，例如 "12"；不要保留 "第12题"、"12."、"12、"、括号等；没有明确题号时填空字符串。\n'
        '3. stem 只保留题干正文；不要包含题号、选项、答案、解析、"答案："、"解析：" 等内容。\n'
        '4. 选项必须拆成 option_1、option_2、option_3、option_4 四个字段分别输出；字段值只保留选项正文，不要包含 A/B/C/D/①②③④ 等标识。\n'
        '5. answer 只保留答案标识本身；不要包含 "答案" 二字、冒号、句号、解释文字或空格。\n'
        '6. 普通单选答案统一输出单个标识，如 A / B / C / D。\n'
        '7. 普通多选答案统一输出紧凑组合，不加顿号、逗号、空格或斜杠，例如 ACD、ABD。\n'
        '8. 可转多选题中，若原文先给出 ①②③④ 等表述，再给出 A/B/C/D 代表不同组合，则 option_1 到 option_4 只保留 ①②③④ 对应的四条表述本身，不保留 A/B/C/D 组合项。\n'
        '9. 可转多选题必须额外输出 choice_1、choice_2、choice_3、choice_4 四个字段，分别对应 A、B、C、D 的组合映射；字段值只保留数字序号本身，例如 A 对应 ①②，则 choice_1 输出 "12"。\n'
        '10. 可转多选题的 answer 必须输出正常字母答案 A/B/C/D，不要输出圆圈序号。\n'
        "10.1 结构一致性强约束：如果 question_type 是可转多选，那么必须同时满足“option_1..option_4 为 ①②③④ 表述”“choice_1..choice_4 为数字映射”“answer 为 A/B/C/D”。\n"
        "10.2 如果原文在题干或选项中使用了“ ( ) ”“ ( ) ”“ () ”等括号空位来表示待填答案或占位，返回的 stem、option_1、option_2、option_3、option_4 中必须保留这些原有括号空位，不得省略、删除或改写成其他符号。\n"
        "11. 如果原文出现材料、案例或引导语，且该内容属于该题题干的一部分，应保留在 stem 中。\n"
        "12. 不要凭空编造不存在的题目、选项或答案；无法确定时宁可留空，也不要猜测。\n"
        "13. 如遇图片题、表格题或信息缺失题，只提取文本中能够明确确认的内容。\n"
        '14. 所有 JSON 字段值都不允许包含实际换行、字符串 "\\n"、"<br>"、"<br/>"、"<br />" 等换行符号；如原文是多行内容，必须合并成单行，并用普通空格连接。\n'
        "如果题目出现“组合选项”形式（例如先给出 ①②③④ 四个表述，然后给出 A/B/C/D 代表不同组合，如“ A. ①② B. ①④ … ”），请按以下方式转换：\n"
        "- option_1 到 option_4 只输出 ①②③④ 对应的每条表述（不要包含 A/B/C/D 这些组合项）\n"
        '- question_type 必须输出为 "可转多选"\n'
        '- choice_1 到 choice_4 分别输出 A 到 D 对应的数字映射，例如 A. ①② 则 choice_1 输出 "12"\n'
```

```
        "- answer 必须输出正确选项字母，例如 B\n"
        "这里的任务是：从提供的整篇资料文本中抽取“指定题号 + 指定题型”的那一题。\n"
        "输出格式：JSON 对象：\n"
        "{\n"
        '  "question_type": "题型(单选/多选/可转多选)",\n'
        '  "number": "编号(必须是请求题号本身)",\n'
        '  "stem": "题干正文",\n'
        '  "option_1": "第1个选项正文",\n'
        '  "option_2": "第2个选项正文",\n'
        '  "option_3": "第3个选项正文",\n'
        '  "option_4": "第4个选项正文",\n'
        '  "choice_1": "可转多选时 A 对应数字映射；否则留空",\n'
        '  "choice_2": "可转多选时 B 对应数字映射；否则留空",\n'
        '  "choice_3": "可转多选时 C 对应数字映射；否则留空",\n'
        '  "choice_4": "可转多选时 D 对应数字映射；否则留空",\n'
        '  "answer": "答案标识"\n'
        "}\n"
        "如果无法确定该题，请输出空对象 {}。不要猜测，不要补全文本。"
    ),
    "question_extract_user": (
        "来源文件：{{source_name}}\n"
        "目标题号：{{requested_number}}\n"
        "目标题型：{{requested_question_type}}\n"
        "请根据“目标题号 + 目标题型”，从下面整篇资料文本中精确定位该题，并只输出这一题的 JSON 对象。\n"
        "资料文本如下：\n"
        "-----\n"
        "{{chunk_text}}\n"
        "-----\n"
    ),
    "explanation_system": (
        "你是一个专业的教育助手，擅长讲解选择题。请严格遵循用户的格式要求。"
    ),
    "explanation_user": (
        "请严格按照以下格式输出解析：\n"
        "如果“答案文本”为空，先根据题目、参考资料与常见错题参考判断最可能的正确答案，并在第一行单独输出“答案：答案标识”。\n"
        "如果“答案文本”不为空，直接基于给定答案输出逐项解析，不要重复输出“答案：...”。\n"
        "每行分析一个选项。\n"
        "必须沿用题目中出现的选项标识（例如 A/B/C/D 或 ①/②/③/④），不要擅自改成另一种编号方式。\n"
        "如果是错误选项：必须先指出“具体的错误原因类型”（可以多个，用中文分号“；”分隔），然后再做详细分析。\n"
        "如果是正确选项：必须标记为“正确”，并说明理由。\n"
        "同一选项允许多个错误原因，需要一并列出。\n"
        "\n"
        "示例格式：\n"
        "答案：AC\n"
        "- A：**知识模块归类错误** ...\n"
        "- B：**正确** ...\n"
        "- C：**偷换概念；范围扩大** ...\n"
        "或：\n"
        "答案：B\n"
        "- ①：**知识模块归类错误** ...\n"
        "- ②：**正确** ...\n"
        "- ③：**偷换概念；范围扩大** ...\n"
        "\n"
        "题目文本：\n"
        "{{question_text}}\n"
        "\n"
        "答案文本：\n"
        "{{answer_text}}\n"
        "{{reference_block}}"
        "{{mistakes_block}}"
    ),
    "explanation_fill_answer_system": (
        "你是一个思政选择题解析助手。"
        "当题目原文没有提供答案时，你需要先根据题干、选项和参考资料判断最可能的正确答案，再输出逐项解析。"
        "你必须仅输出严格 JSON，不要输出 markdown、代码块或额外说明。\n"
        "输出格式固定为："
        '{"answer_text":"答案标识","analysis_text":"逐项解析"}' "\n"
        "answer_text 规则：\n"
        "- 单选输出单个字母，如 A。\n"
```

```python
        "- 多选输出紧凑字母组合，如 ACD。\n"
        "- 可转多选输出正常字母答案 A/B/C/D，不要输出圆圈序号。\n"
        "- 不要包含“答案”二字、冒号、句号、空格、逗号、顿号或解释文字。\n"
        "analysis_text 规则：\n"
        "- 每行分析一个选项。\n"
        "- 必须沿用题目中出现的选项标识。\n"
        "- 正确选项标记为“正确”并说明理由；错误选项先写错误原因类型，再分析。\n"
        "- 只输出纯文本解析，不要再包一层 JSON 或 markdown。\n"
    ),
    "explanation_fill_answer_user": (
        "请先判断这道题最可能的正确答案，再输出答案和逐项解析。\n"
        "题目文本：\n"
        "{{question_text}}\n"
        "{{reference_block}}"
        "{{mistakes_block}}"
    ),
}


_CACHE_PATH: Path | None = None
_CACHE_MTIME_NS: int | None = None
_CACHE_PROMPTS: dict[str, str] | None = None


def import_prompt_config_path() -> Path:
    env = os.getenv("SJ_GENERATOR_IMPORT_PROMPTS_PATH", "").strip()
    if env:
        return Path(env)
    app_data = os.getenv("APPDATA", "").strip()
    if app_data:
        return Path(app_data) / "sj_generator" / "import_prompts.json"
    return Path.home() / ".sj_generator" / "import_prompts.json"


def load_import_prompts(*, force_reload: bool = False) -> dict[str, str]:
    global _CACHE_PATH, _CACHE_MTIME_NS, _CACHE_PROMPTS

    path = import_prompt_config_path()
    mtime_ns: int | None = None
    if path.exists():
        try:
            mtime_ns = path.stat().st_mtime_ns
        except OSError:
            mtime_ns = None

    if (
        not force_reload
        and _CACHE_PROMPTS is not None
        and _CACHE_PATH == path
        and _CACHE_MTIME_NS == mtime_ns
    ):
        return dict(_CACHE_PROMPTS)

    raw: dict[str, object] = {}
    if path.exists():
        try:
            loaded = json.loads(path.read_text(encoding="utf-8"))
            if isinstance(loaded, dict):
                raw = loaded
        except Exception:
            raw = {}

    prompts = _normalize_prompt_map(raw)
    _CACHE_PATH = path
    _CACHE_MTIME_NS = mtime_ns
    _CACHE_PROMPTS = dict(prompts)
    return prompts


def save_import_prompts(prompts: dict[str, str]) -> None:
```

```python
    global _CACHE_PATH, _CACHE_MTIME_NS, _CACHE_PROMPTS

    path = import_prompt_config_path()
    path.parent.mkdir(parents=True, exist_ok=True)
    normalized = _normalize_prompt_map(prompts)
    path.write_text(json.dumps(normalized, ensure_ascii=False, indent=2), encoding="utf-8")
    try:
        mtime_ns = path.stat().st_mtime_ns
    except OSError:
        mtime_ns = None
    _CACHE_PATH = path
    _CACHE_MTIME_NS = mtime_ns
    _CACHE_PROMPTS = dict(normalized)


def get_import_prompt(key: str) -> str:
    prompts = load_import_prompts()
    if key not in DEFAULT_IMPORT_PROMPTS:
        raise KeyError(f"未知提示词键：{key}")
    return prompts.get(key, DEFAULT_IMPORT_PROMPTS[key])


def render_import_prompt(key: str, **kwargs: object) -> str:
    text = get_import_prompt(key)
    for name, value in kwargs.items():
        text = text.replace("{{" + str(name) + "}}", str(value))
    return text


def default_import_prompts() -> dict[str, str]:
    return dict(DEFAULT_IMPORT_PROMPTS)


def _normalize_prompt_map(data: dict[str, object]) -> dict[str, str]:
    prompts: dict[str, str] = {}
    for key, default_value in DEFAULT_IMPORT_PROMPTS.items():
        raw = data.get(key, default_value)
        prompts[key] = raw if isinstance(raw, str) else default_value
    return prompts
```

```python
from __future__ import annotations

import json
import time
from concurrent.futures import ThreadPoolExecutor, as_completed
from pathlib import Path
from typing import Callable

from sj_generator.infrastructure.llm.client import LlmClient, LlmConfig
from sj_generator.infrastructure.llm.import_questions import (
    _QuestionRefSpecialCaseError,
    _as_str,
    _get_question_numbers_with_fallback,
    _normalize_question_ref_list,
)
from sj_generator.application.settings import (
    load_deepseek_config,
    load_kimi_config,
    load_project_parse_model_rows,
    load_qwen_config,
    with_capped_timeout,
)

QUESTION_REF_TEST_TIMEOUT_S = 300.0


def question_ref_numbers(items: list[dict[str, str]]) -> list[str]:
    return [_as_str(item.get("number", "")) for item in items if _as_str(item.get("number", ""))]


def question_ref_type_map(items: list[dict[str, str]]) -> dict[str, str]:
    result: dict[str, str] = {}
    for item in _normalize_question_ref_list(items):
        number = _as_str(item.get("number", ""))
        if not number:
            continue
        result[number] = _as_str(item.get("question_type", ""))
    return result


def question_ref_warning_text(items: list[dict[str, str]]) -> str:
    for item in _normalize_question_ref_list(items):
        warning = _as_str(item.get("duplicate_warning", ""))
        if warning == "存在可疑的题号重复":
            return warning
    return ""


def special_marker_text(value: object) -> str:
    return _as_str(value)


def merged_question_ref_numbers(*groups: list[dict[str, str]]) -> list[str]:
    ordered: list[str] = []
    seen: set[str] = set()
    for items in groups:
        for number in question_ref_numbers(items):
            if number in seen:
                continue
            seen.add(number)
            ordered.append(number)
    return ordered


def question_ref_header_labels(model_specs: list[dict[str, str]] | None = None) -> list[str]:
    specs = model_specs or question_ref_model_specs()
    return ["题号", *[str(item.get("label") or "") for item in specs], "最终", "行一致性"]


def row_consistency_text(values: list[str]) -> str:
```

```python
    normalized = [str(value or "").strip() for value in values]
    total = len(normalized)
    if total <= 0:
        return "0/0"
    counts: dict[str, int] = {}
    for value in normalized:
        key = value if value else "__EMPTY__"
        counts[key] = counts.get(key, 0) + 1
    best_count = max(counts.values()) if counts else 0
    return f"{best_count}/{total} {'一致' if best_count == total else '不一致'}"


def question_ref_model_specs() -> list[dict[str, str]]:
    row = _question_number_parse_row()
    models = row.get("models") or []
    if models:
        specs: list[dict[str, str]] = []
        for index, item in enumerate(models):
            provider = str(item.get("provider") or "").strip().lower()
            model_name = str(item.get("model_name") or "").strip()
            if not provider or not model_name:
                continue
            specs.append(
                {
                    "key": f"model_{index + 1}",
                    "provider": provider,
                    "model_name": model_name,
                    "label": f"{_provider_label(provider)}\n{model_name}",
                }
            )
        if specs:
            return specs

    deepseek_model = load_deepseek_config().number_model.strip() or "deepseek-chat"
    kimi_model = load_kimi_config().number_model.strip() or "kimi-k2.6"
    qwen_model = load_qwen_config().number_model.strip() or "qwen-max"
    return [
        {"key": "model_1", "provider": "deepseek", "model_name": deepseek_model, "label": f"DeepSeek\n{deepseek_model}"},
        {"key": "model_2", "provider": "kimi", "model_name": kimi_model, "label": f"Kimi\n{kimi_model}"},
        {"key": "model_3", "provider": "qwen", "model_name": qwen_model, "label": f"千问\n{qwen_model}"},
    ]


def run_question_ref_scan(
    *,
    text: str,
    source_name: str,
    progress_cb: Callable[[dict[str, object]], None] | None = None,
) -> dict[str, object]:
    model_specs = question_ref_model_specs()
    round_limit = _question_ref_round_limit()
    ratio_numerator, ratio_denominator = _question_ref_ratio()
    for spec in model_specs:
        provider = str(spec.get("provider") or "").strip()
        if not _question_ref_provider_ready(provider):
            raise RuntimeError(f"{_provider_label(provider)} 未配置，无法识别题号。")

    clients = {
        str(spec["key"]): LlmClient(
            _question_ref_llm_config(
                str(spec.get("provider") or ""),
                str(spec.get("model_name") or ""),
            )
        )
        for spec in model_specs
    }
    all_results: list[dict[str, object]] = []
    last_round_results: dict[str, list[dict[str, str]]] = {}
    last_round_markers: dict[str, str] = {}
```

```python
    last_round_elapsed_s: dict[str, int] = {}
    accepted = False
    best_count = 0
    executed_rounds = 0
    best_items: list[dict[str, str]] = []

    for round_index in range(1, round_limit + 1):
        round_results: dict[str, list[dict[str, str]]] = {}
        round_markers: dict[str, str] = {}
        round_elapsed_s: dict[str, int] = {}
        round_started_at = time.monotonic()
        if progress_cb is not None:
            progress_cb(
                {
                    "event": "round_started",
                    "round": round_index,
                    "model_specs": model_specs,
                }
            )
        with ThreadPoolExecutor(max_workers=max(1, len(clients))) as executor:
            future_map = {
                executor.submit(
                    _get_question_numbers_with_fallback,
                    client=client,
                    source_name=source_name,
                    chunk_text=text,
                    depth=2,
                ): model_key
                for model_key, client in clients.items()
            }
            for future in as_completed(future_map):
                model_key = future_map[future]
                try:
                    items = _normalize_question_ref_list(future.result())
                except _QuestionRefSpecialCaseError as e:
                    round_markers[model_key] = str(e)
                    items = []
                except Exception as e:
                    round_markers[model_key] = f"识别失败：{e}"
                    items = []
                if progress_cb is not None:
                    elapsed_s = int(max(0.0, time.monotonic() - round_started_at))
                    round_elapsed_s[model_key] = elapsed_s
                    progress_cb(
                        {
                            "event": "model_finished",
                            "round": round_index,
                            "model_key": model_key,
                            "elapsed_s": elapsed_s,
                            "marker": round_markers.get(model_key, ""),
                            "item_count": len(items),
                        }
                    )
                round_results[model_key] = items
                all_results.append(
                    {
                        "round": round_index,
                        "model_key": model_key,
                        "items": items,
                        "fingerprint": special_marker_text(round_markers.get(model_key)) or _question_ref_fingerprint(items),
                    }
                )

        last_round_results = round_results
        last_round_markers = round_markers
        last_round_elapsed_s = round_elapsed_s
        executed_rounds = round_index

        if round_markers:
```

```python
            break

        fingerprint_counts: dict[str, int] = {}
        fingerprint_items: dict[str, list[dict[str, str]]] = {}
        for entry in all_results:
            fingerprint = str(entry.get("fingerprint") or "")
            fingerprint_counts[fingerprint] = fingerprint_counts.get(fingerprint, 0) + 1
            fingerprint_items.setdefault(fingerprint, entry.get("items") or [])

        if fingerprint_counts:
            best_fingerprint = max(
                fingerprint_counts,
                key=lambda value: (fingerprint_counts[value], len(value), value),
            )
            best_count = fingerprint_counts[best_fingerprint]
            best_items = _normalize_question_ref_list(fingerprint_items.get(best_fingerprint, []))

        required_count = ratio_numerator * round_index
        if best_count >= required_count:
            accepted = True
            break

    total_count = len(all_results)
    consistency_text = f"{best_count}/{total_count}" if total_count else "0/0"
    return {
        "model_specs": model_specs,
        "round_limit": round_limit,
        "executed_rounds": executed_rounds,
        "ratio_numerator": ratio_numerator,
        "ratio_denominator": ratio_denominator,
        "best_count": best_count,
        "total_count": total_count,
        "consistency_text": consistency_text,
        "accepted": accepted,
        "final_refs": best_items,
        "results": last_round_results,
        "markers": last_round_markers,
        "elapsed_s_by_model": last_round_elapsed_s,
    }


def resolve_question_refs_with_scan(
    *,
    sources: list[tuple[Path, str]],
    progress_cb: Callable[[str], None] | None = None,
    compare_cb: Callable[[dict[str, object]], None] | None = None,
    scan_progress_cb: Callable[[dict[str, object]], None] | None = None,
    progress_count_cb: Callable[[int, int], None] | None = None,
    stop_cb: Callable[[], bool] | None = None,
) -> dict[str, object]:
    refs_by_source: dict[str, list[dict[str, str]]] = {}
    payloads_by_source: dict[str, dict[str, object]] = {}
    total_sources = len(sources)
    completed = 0
    if progress_count_cb is not None:
        progress_count_cb(0, total_sources)
    for path, text in sources:
        if stop_cb and stop_cb():
            break
        normalized_text = text.strip()
        if not normalized_text:
            completed += 1
            if progress_count_cb is not None:
                progress_count_cb(completed, total_sources)
            continue
        if progress_cb is not None:
            progress_cb(f"{path.name}：题号与题型解析中…")
        if scan_progress_cb is not None:
            scan_progress_cb(
                {
```

```python
                    "event": "source_started",
                    "source_name": path.name,
                    "source_path": str(path),
                }
            )
        payload = run_question_ref_scan(
            text=normalized_text,
            source_name=path.name,
            progress_cb=(
                (lambda event: scan_progress_cb(
                    {
                        **event,
                        "source_name": path.name,
                        "source_path": str(path),
                    }
                ))
                if scan_progress_cb is not None
                else None
            ),
        )
        payload = {
            **payload,
            "source_name": path.name,
            "source_path": str(path),
        }
        payloads_by_source[str(path)] = payload
        if compare_cb is not None:
            compare_cb(payload)
        if bool(payload.get("accepted")):
            refs_by_source[str(path)] = _normalize_question_ref_list(payload.get("final_refs", []))
        completed += 1
        if progress_count_cb is not None:
            progress_count_cb(completed, total_sources)
    total_refs = sum(len(items) for items in refs_by_source.values())
    return {
        "refs_by_source": refs_by_source,
        "payloads_by_source": payloads_by_source,
        "total_refs": total_refs,
    }


def _question_ref_fingerprint(items: list[dict[str, str]]) -> str:
    normalized = [
        {
            key: value
            for key, value in item.items()
            if key in {"number", "question_type"} and str(value or "").strip()
        }
        for item in _normalize_question_ref_list(items)
    ]
    return json.dumps(normalized, ensure_ascii=False, sort_keys=True, separators=(",", ":"))


def _provider_label(provider: str) -> str:
    labels = {"deepseek": "DeepSeek", "kimi": "Kimi", "qwen": "千问"}
    return labels.get(str(provider or "").strip().lower(), provider or "")


def _question_number_parse_row() -> dict:
    rows = load_project_parse_model_rows()
    for row in rows:
        if str(row.get("key") or "").strip() == "question_number_parse":
            return row
    return {"key": "question_number_parse", "round": "1", "ratio": "1/4", "models": []}


def _question_ref_round_limit() -> int:
    row = _question_number_parse_row()
    try:
        value = int(str(row.get("round") or "1").strip())
```

```python
    except Exception:
        return 1
    return min(8, max(1, value))


def _question_ref_ratio() -> tuple[int, int]:
    row = _question_number_parse_row()
    raw = str(row.get("ratio") or "1/4").strip()
    left, sep, right = raw.partition("/")
    if not sep:
        return 1, 1
    try:
        numerator = int(left.strip())
        denominator = int(right.strip())
    except Exception:
        return 1, 1
    return max(1, numerator), max(1, denominator)


def _question_ref_llm_config(provider: str, model_name: str) -> LlmConfig:
    provider_key = str(provider or "").strip().lower()
    if provider_key == "kimi":
        cfg = with_capped_timeout(load_kimi_config(), QUESTION_REF_TEST_TIMEOUT_S)
        return LlmConfig(
            base_url=cfg.base_url.strip(),
            api_key=cfg.api_key.strip(),
            model=model_name.strip(),
            timeout_s=float(cfg.timeout_s),
        )
    if provider_key == "qwen":
        cfg = with_capped_timeout(load_qwen_config(), QUESTION_REF_TEST_TIMEOUT_S)
        return LlmConfig(
            base_url=cfg.base_url.strip(),
            api_key=cfg.api_key.strip(),
            model=model_name.strip(),
            timeout_s=float(cfg.timeout_s),
        )
    cfg = with_capped_timeout(load_deepseek_config(), QUESTION_REF_TEST_TIMEOUT_S)
    return LlmConfig(
        base_url=cfg.base_url.strip(),
        api_key=cfg.api_key.strip(),
        model=model_name.strip(),
        timeout_s=float(cfg.timeout_s),
    )


def _question_ref_provider_ready(provider: str) -> bool:
    provider_key = str(provider or "").strip().lower()
    if provider_key == "kimi":
        return load_kimi_config().is_ready()
    if provider_key == "qwen":
        return load_qwen_config().is_ready()
    return load_deepseek_config().is_ready()
```

sj_generator/infrastructure/llm/task_runner.py

038-01

```python
from __future__ import annotations

from concurrent.futures import FIRST_COMPLETED, ThreadPoolExecutor, as_completed, wait
from typing import Any, Callable


def run_callables_in_parallel_fail_fast(*, callables: list[Callable[[], None]], max_workers: int) -> None:
    if not callables:
        return
    executor = ThreadPoolExecutor(max_workers=max(1, max_workers))
    futures = []
    try:
        futures = [executor.submit(fn) for fn in callables]
        for future in as_completed(futures):
            future.result()
    except Exception:
        for future in futures:
            future.cancel()
        raise
    finally:
        executor.shutdown(wait=False, cancel_futures=True)


def run_tasks_in_parallel(
    *,
    tasks: list[tuple[int, str, str]],
    max_workers: int,
    stop_cb: Callable[[], bool],
    on_task_start: Callable[[int, int, tuple[int, str, str]], None],
    on_task_done: Callable[[tuple[int, str, str], Any], None],
    on_task_failed: Callable[[tuple[int, str, str], Exception], None],
    run_one: Callable[[tuple[int, str, str]], Any],
) -> None:
    total = len(tasks)
    if total <= 0:
        return
    worker_count = max(1, min(int(max_workers), total))
    it = iter(tasks)
    launched = 0
    in_flight: dict[Any, tuple[int, str, str]] = {}

    def submit_next(executor: ThreadPoolExecutor) -> bool:
        nonlocal launched
        if stop_cb():
            return False
        try:
            task = next(it)
        except StopIteration:
            return False
        launched += 1
        on_task_start(launched, total, task)
        future = executor.submit(run_one, task)
        in_flight[future] = task
        return True

    with ThreadPoolExecutor(max_workers=worker_count) as executor:
        for _ in range(worker_count):
            if not submit_next(executor):
                break
        while in_flight:
            done_futures, _ = wait(tuple(in_flight.keys()), return_when=FIRST_COMPLETED)
            for future in done_futures:
                task = in_flight.pop(future)
                try:
                    result = future.result()
                except Exception as e:
                    on_task_failed(task, e)
                else:
                    on_task_done(task, result)
                if not stop_cb():
```

```
                    submit_next(executor)

```

sj_generator/infrastructure/persistence/__init__.py

039-01

```
1
2
```

```python
from __future__ import annotations

import re
import uuid
from dataclasses import dataclass
from datetime import datetime
from pathlib import Path
from typing import Iterable

from sj_generator.infrastructure.persistence.excel_repo import try_parse_options_json
from sj_generator.infrastructure.persistence.sqlite_repo import DbQuestionRecord, append_questions
from sj_generator.domain.entities import Question

LETTER_MARKER_RE = re.compile(r"(?<!\n)(?=(?:[A-D][\.、. ]))")
CIRCLED_MARKER_RE = re.compile(r"(?<!\n)(?=(?:[\u2460-\u2473]))")
LETTER_LINE_RE = re.compile(r"^([A-D])[\.\u3001\uFF0E]\s*(.*)$")
CIRCLED_LINE_RE = re.compile(r"^([\u2460-\u2473])\s*(.*)$")
COMBO_RE = re.compile(r"([A-D])[\.\u3001\uFF0E]\s*([\u2460-\u2473\s]+)")
LETTER_ONLY_RE = re.compile(r"^[A-D]+$")
DIGIT_TOKEN_RE = re.compile(r"\d+")
CIRCLED_CHAR_TO_INDEX = {
    "①": "1",
    "②": "2",
    "③": "3",
    "④": "4",
    "⑤": "5",
    "⑥": "6",
    "⑦": "7",
    "⑧": "8",
    "⑨": "9",
    "⑩": "10",
}


@dataclass(frozen=True)
class ParsedQuestion:
    stem: str
    options: list[str]
    choice_map: dict[str, str]


def import_draft_questions_to_db(
    *,
    db_path: Path,
    questions: list[Question],
    level_path: str,
    source_files: Iterable[Path] | None = None,
    textbook_version: str = "",
) -> int:
    records = draft_questions_to_db_records(
        questions=questions,
        level_path=level_path,
        source_files=source_files,
        textbook_version=textbook_version,
    )
    append_questions(db_path, records)
    return len(records)


def draft_questions_to_db_records(
    *,
    questions: list[Question],
    level_path: str,
    source_files: Iterable[Path] | None = None,
    textbook_version: str = "",
) -> list[DbQuestionRecord]:
    normalized_level_path = (level_path or "").strip()
    if not normalized_level_path:
        raise ValueError("level_path 不能为空。")
    if not questions:
```

```python
        raise ValueError("没有可写入数据库的题目。")

    source = _resolve_source(source_files)
    normalized_textbook_version = (textbook_version or "").strip()
    now_text = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    return [
        _convert_question(
            question,
            source=source,
            textbook_version=normalized_textbook_version,
            level_path=normalized_level_path,
            now_text=now_text,
        )
        for question in questions
    ]


def _resolve_source(source_files: Iterable[Path] | None) -> str:
    if source_files is None:
        return ""
    names: list[str] = []
    seen: set[str] = set()
    for path in source_files:
        name = path.name.strip()
        if not name or name in seen:
            continue
        names.append(name)
        seen.add(name)
    if not names:
        return ""
    if len(names) == 1:
        return names[0]
    return "; ".join(names)


def _convert_question(
    question: Question,
    *,
    source: str,
    textbook_version: str,
    level_path: str,
    now_text: str,
) -> DbQuestionRecord:
    parsed = _parse_question_and_options(question)
    question_type = _detect_question_type(question, parsed)
    choice_map = _resolve_choice_map(question, parsed, question_type)
    answer = _normalize_answer(question.answer, question_type, choice_map)
    option_values = (parsed.options + ["", "", "", ""])[:4]
    return DbQuestionRecord(
        id=str(uuid.uuid4()),
        stem=parsed.stem,
        option_1=option_values[0],
        option_2=option_values[1],
        option_3=option_values[2],
        option_4=option_values[3],
        choice_1=choice_map.get("A", ""),
        choice_2=choice_map.get("B", ""),
        choice_3=choice_map.get("C", ""),
        choice_4=choice_map.get("D", ""),
        answer=answer,
        analysis=(question.analysis or "").strip(),
        question_type=question_type,
        textbook_version=textbook_version,
        source=source,
        level_path=level_path,
        difficulty_score=None,
        knowledge_points="",
        abilities="",
        created_at=now_text,
        updated_at=now_text,
```

```python
    )


def _parse_question_and_options(question: Question) -> ParsedQuestion:
    parsed_json = try_parse_options_json(question.options or "")
    if parsed_json is not None:
        keys = sorted(parsed_json.keys())
        return ParsedQuestion(
            stem=(question.stem or "").strip(),
            options=[(parsed_json.get(key) or "").strip() for key in keys],
            choice_map={},
        )

    combined = _join_stem_and_options(question.stem, question.options)
    choice_map = _extract_choice_map_from_text(combined)
    normalized = _force_newline_before_markers(combined)
    lines = [line.strip() for line in normalized.splitlines() if line.strip()]

    stem_lines: list[str] = []
    options: list[str] = []
    current_option_parts: list[str] | None = None
    for line in lines:
        letter_match = LETTER_LINE_RE.match(line)
        circled_match = CIRCLED_LINE_RE.match(line)
        if letter_match:
            letter = letter_match.group(1).strip()
            choice_value = _extract_choice_value(letter_match.group(2))
            if choice_value:
                if current_option_parts is not None:
                    options.append(" ".join(part for part in current_option_parts if part).strip())
                    current_option_parts = None
                choice_map[letter] = choice_value
                continue
            if current_option_parts is not None:
                options.append(" ".join(part for part in current_option_parts if part).strip())
            current_option_parts = [letter_match.group(2).strip()]
            continue
        if circled_match:
            if current_option_parts is not None:
                options.append(" ".join(part for part in current_option_parts if part).strip())
            current_option_parts = [circled_match.group(2).strip()]
            continue
        if current_option_parts is None:
            stem_lines.append(line)
        else:
            current_option_parts.append(line)

    if current_option_parts is not None:
        options.append(" ".join(part for part in current_option_parts if part).strip())

    if choice_map:
        options = _extract_circled_options(lines)

    if options:
        return ParsedQuestion(
            stem="\n".join(stem_lines).strip(),
            options=options,
            choice_map=choice_map,
        )
    return ParsedQuestion(stem=(combined or "").strip(), options=[], choice_map=choice_map)


def _join_stem_and_options(stem: str, options: str) -> str:
    stem = (stem or "").strip()
    options = (options or "").strip()
    if stem and options:
        return f"{stem}\n{options}"
    return stem or options
```

```python
def _force_newline_before_markers(text: str) -> str:
    if not text.strip():
        return ""
    text = CIRCLED_MARKER_RE.sub("\n", text)
    return LETTER_MARKER_RE.sub("\n", text)


def _detect_question_type(question: Question, parsed: ParsedQuestion) -> str:
    raw = (question.question_type or "").strip()
    if raw in ("单选", "多选", "可转多选"):
        return raw
    if _question_choice_map(question) or parsed.choice_map:
        return "可转多选"
    compact_answer = _compact_answer(question.answer)
    if _is_multi_answer(compact_answer):
        return "多选"
    if _is_convertible_multi(parsed.options):
        return "可转多选"
    return "单选"


def _is_multi_answer(answer: str) -> bool:
    if "," in answer:
        numbers = [part.strip() for part in answer.split(",") if part.strip()]
        return len(numbers) > 1
    if LETTER_ONLY_RE.fullmatch(answer):
        return len(answer) > 1
    circled_digits = [CIRCLED_CHAR_TO_INDEX[ch] for ch in answer if ch in CIRCLED_CHAR_TO_INDEX]
    return len(circled_digits) > 1


def _is_convertible_multi(options: list[str]) -> bool:
    if len(options) < 2:
        return False
    circled_leading_count = 0
    for option in options:
        text = option.strip()
        if text and text[0] in CIRCLED_CHAR_TO_INDEX:
            circled_leading_count += 1
    return circled_leading_count >= 2


def _normalize_answer(answer: str, question_type: str, choice_map: dict[str, str]) -> str:
    compact = _compact_answer(answer)
    if question_type == "可转多选":
        return _normalize_convertible_answer(compact, choice_map)
    if question_type != "多选":
        return compact

    if LETTER_ONLY_RE.fullmatch(compact):
        digits = [str(ord(ch) - ord("A") + 1) for ch in compact]
        return ",".join(digits)

    circled_digits = [CIRCLED_CHAR_TO_INDEX[ch] for ch in compact if ch in CIRCLED_CHAR_TO_INDEX]
    if circled_digits:
        return ",".join(circled_digits)

    digits = DIGIT_TOKEN_RE.findall(compact)
    if digits:
        return ",".join(digits)
    return compact


def _compact_answer(answer: str) -> str:
    return (answer or "").strip().replace(" ", "").replace(", ", ",").replace(" ; ", ",").replace(";", ",")


def _extract_choice_value(text: str) -> str:
    if not text:
        return ""
```

```python
    circled_digits = [CIRCLED_CHAR_TO_INDEX[ch] for ch in text if ch in CIRCLED_CHAR_TO_INDEX]
    if circled_digits:
        return "".join(circled_digits)
    digits = DIGIT_TOKEN_RE.findall(text)
    return "".join(digits)


def _extract_circled_options(lines: list[str]) -> list[str]:
    options: list[str] = []
    current_option_parts: list[str] | None = None
    for line in lines:
        circled_match = CIRCLED_LINE_RE.match(line)
        if circled_match:
            if current_option_parts is not None:
                options.append(" ".join(part for part in current_option_parts if part).strip())
            current_option_parts = [circled_match.group(2).strip()]
            continue
        if LETTER_LINE_RE.match(line):
            if current_option_parts is not None:
                options.append(" ".join(part for part in current_option_parts if part).strip())
                current_option_parts = None
            continue
        if current_option_parts is not None:
            current_option_parts.append(line)
    if current_option_parts is not None:
        options.append(" ".join(part for part in current_option_parts if part).strip())
    return options


def _question_choice_map(question: Question) -> dict[str, str]:
    return {
        letter: value
        for letter, value in (
            ("A", _normalize_choice_value(question.choice_1)),
            ("B", _normalize_choice_value(question.choice_2)),
            ("C", _normalize_choice_value(question.choice_3)),
            ("D", _normalize_choice_value(question.choice_4)),
        )
        if value
    }


def _resolve_choice_map(question: Question, parsed: ParsedQuestion, question_type: str) -> dict[str, str]:
    if question_type != "可转多选":
        return {}
    explicit = _question_choice_map(question)
    return explicit or parsed.choice_map


def _normalize_choice_value(value: str) -> str:
    return "".join(ch for ch in _extract_choice_value(value) if ch.isdigit())


def _extract_choice_map_from_text(text: str) -> dict[str, str]:
    out: dict[str, str] = {}
    for match in COMBO_RE.finditer(text or ""):
        value = _normalize_choice_value(match.group(2))
        if value:
            out[match.group(1)] = value
    return out


def _normalize_convertible_answer(answer: str, choice_map: dict[str, str]) -> str:
    if not answer:
        return ""
    upper = answer.upper()
    if LETTER_ONLY_RE.fullmatch(upper):
        return upper
    circled_digits = "".join(CIRCLED_CHAR_TO_INDEX[ch] for ch in upper if ch in CIRCLED_CHAR_TO_INDEX)
    if circled_digits:
```

```python
        mapped = _choice_digits_to_letter(circled_digits, choice_map)
        return mapped or upper
    digits = "".join(DIGIT_TOKEN_RE.findall(upper))
    if digits:
        mapped = _choice_digits_to_letter(digits, choice_map)
        return mapped or digits
    return upper


def _choice_digits_to_letter(digits: str, choice_map: dict[str, str]) -> str:
    normalized = _normalize_choice_value(digits)
    if not normalized:
        return ""
    for letter in ("A", "B", "C", "D"):
        if _normalize_choice_value(choice_map.get(letter, "")) == normalized:
            return letter
    return ""

```

```python
from __future__ import annotations

import json
from pathlib import Path
from typing import Any

from openpyxl import Workbook, load_workbook

from sj_generator.infrastructure.persistence.sqlite_repo import DbQuestionRecord
from sj_generator.domain.entities import Question


SHEET_NAME = "questions"
HEADERS = ["编号", "题目", "选项", "答案", "解析"]
DB_TABLE_SHEET_NAME = "db_questions"
DB_TABLE_HEADERS = [
    "id",
    "stem",
    "option_1",
    "option_2",
    "option_3",
    "option_4",
    "choice_1",
    "choice_2",
    "choice_3",
    "choice_4",
    "answer",
    "analysis",
    "question_type",
    "textbook_version",
    "source",
    "level_path",
    "difficulty_score",
    "knowledge_points",
    "abilities",
    "created_at",
    "updated_at",
]


def create_empty_repo(path: Path) -> None:
    if path.exists():
        raise FileExistsError(f"文件已存在：{path}")
    path.parent.mkdir(parents=True, exist_ok=True)
    wb = Workbook()
    ws = wb.active
    ws.title = SHEET_NAME
    ws.append(HEADERS)
    wb.save(path)


def load_questions(path: Path) -> list[Question]:
    wb = load_workbook(path)
    ws = wb[SHEET_NAME] if SHEET_NAME in wb.sheetnames else wb.active

    header_row, header_map = _get_header_map(ws)
    _ensure_required_headers(header_map)

    result: list[Question] = []
    for row in ws.iter_rows(min_row=2, values_only=True):
        number = _cell_to_str(row[header_map["编号"]])
        stem = _cell_to_str(row[header_map["题目"]])
        options = _cell_to_str(row[header_map["选项"]])
        answer = _cell_to_str(row[header_map["答案"]])
        analysis = _cell_to_str(row[header_map["解析"]])
        if not any([number, stem, options, answer, analysis]):
            continue
        result.append(
            Question(
                number=number,
```

```python
                stem=stem,
                options=options,
                answer=answer,
                analysis=analysis,
            )
        )
    return result


def append_questions(path: Path, questions: list[Question]) -> None:
    wb = load_workbook(path)
    ws = wb[SHEET_NAME] if SHEET_NAME in wb.sheetnames else wb.active

    header_row, header_map = _get_header_map(ws)
    _ensure_required_headers(header_map)

    for q in questions:
        row: list[Any] = [None] * len(header_row)
        row[header_map["编号"]] = q.number
        row[header_map["题目"]] = q.stem
        row[header_map["选项"]] = q.options
        row[header_map["答案"]] = q.answer
        row[header_map["解析"]] = q.analysis
        ws.append(row)

    wb.save(path)

def save_questions(path: Path, questions: list[Question]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    wb = Workbook()
    ws = wb.active
    ws.title = SHEET_NAME
    ws.append(HEADERS)
    for q in questions:
        ws.append([q.number, q.stem, q.options, q.answer, q.analysis])
    wb.save(path)


def save_db_question_records(path: Path, records: list[DbQuestionRecord]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    wb = Workbook()
    ws = wb.active
    ws.title = DB_TABLE_SHEET_NAME
    ws.append(DB_TABLE_HEADERS)
    for record in records:
        ws.append(
            [
                record.id,
                record.stem,
                record.option_1,
                record.option_2,
                record.option_3,
                record.option_4,
                record.choice_1,
                record.choice_2,
                record.choice_3,
                record.choice_4,
                record.answer,
                record.analysis,
                record.question_type,
                record.textbook_version,
                record.source,
                record.level_path,
                "" if record.difficulty_score is None else record.difficulty_score,
                record.knowledge_points,
                record.abilities,
                record.created_at,
                record.updated_at,
            ]
        )
```

```python
    wb.save(path)


def load_db_question_records(path: Path) -> list[DbQuestionRecord]:
    wb = load_workbook(path)
    ws = wb[DB_TABLE_SHEET_NAME] if DB_TABLE_SHEET_NAME in wb.sheetnames else wb.active
    _, header_map = _get_header_map(ws)
    if "source" not in header_map and "source_filename" in header_map:
        header_map["source"] = header_map["source_filename"]
    _ensure_required_db_table_headers(header_map)

    records: list[DbQuestionRecord] = []
    for row in ws.iter_rows(min_row=2, values_only=True):
        values = {header: _cell_to_str(row[header_map[header]]) for header in DB_TABLE_HEADERS}
        if not any(values.values()):
            continue
        difficulty_raw = values["difficulty_score"]
        difficulty_score: int | None = None
        if difficulty_raw:
            try:
                difficulty_score = int(float(difficulty_raw))
            except Exception as e:
                raise ValueError(f"difficulty_score 无法解析为整数：{difficulty_raw}") from e
        records.append(
            DbQuestionRecord(
                id=values["id"],
                stem=values["stem"],
                option_1=values["option_1"],
                option_2=values["option_2"],
                option_3=values["option_3"],
                option_4=values["option_4"],
                choice_1=values["choice_1"],
                choice_2=values["choice_2"],
                choice_3=values["choice_3"],
                choice_4=values["choice_4"],
                answer=values["answer"],
                analysis=values["analysis"],
                question_type=values["question_type"],
                textbook_version=values["textbook_version"],
                source=values["source"],
                level_path=values["level_path"],
                difficulty_score=difficulty_score,
                knowledge_points=values["knowledge_points"],
                abilities=values["abilities"],
                created_at=values["created_at"],
                updated_at=values["updated_at"],
            )
        )
    return records


def try_parse_options_json(options: str) -> dict[str, str] | None:
    raw = options.strip()
    if not raw.startswith("{") or not raw.endswith("}"):
        return None
    try:
        obj = json.loads(raw)
    except Exception:
        return None
    if not isinstance(obj, dict):
        return None
    normalized: dict[str, str] = {}
    for k, v in obj.items():
        if not isinstance(k, str):
            return None
        if not isinstance(v, str):
            v = str(v)
        normalized[k.strip()] = v
    if not normalized:
        return None
```

```python
    return normalized


def validate_repo(path: Path) -> None:
    wb = load_workbook(path)
    ws = wb[SHEET_NAME] if SHEET_NAME in wb.sheetnames else wb.active
    _, header_map = _get_header_map(ws)
    _ensure_required_headers(header_map)


def _ensure_required_headers(header_map: dict[str, int]) -> None:
    missing = [h for h in HEADERS if h not in header_map]
    if missing:
        raise ValueError(f"题库表头缺少列：{', '.join(missing)}")


def _ensure_required_db_table_headers(header_map: dict[str, int]) -> None:
    missing = [h for h in DB_TABLE_HEADERS if h not in header_map]
    if missing:
        raise ValueError(f"数据库字段表表头缺少列：{', '.join(missing)}")


def _get_header_map(ws: Any) -> tuple[list[Any], dict[str, int]]:
    header_row = [cell.value for cell in next(ws.iter_rows(min_row=1, max_row=1))]
    header_map: dict[str, int] = {}
    for idx, v in enumerate(header_row):
        if isinstance(v, str) and v.strip():
            header_map[v.strip()] = idx
    return header_row, header_map


def _cell_to_str(v: Any) -> str:
    if v is None:
        return ""
    if isinstance(v, str):
        return v.strip()
    return str(v).strip()

```

sj_generator/infrastructure/persistence/sqlite_repo.py

042-01

```python
from __future__ import annotations

import sqlite3
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable

TABLE_NAME = "questions"


@dataclass(frozen=True)
class DbQuestionRecord:
    id: str
    stem: str
    option_1: str
    option_2: str
    option_3: str
    option_4: str
    choice_1: str
    choice_2: str
    choice_3: str
    choice_4: str
    answer: str
    analysis: str
    question_type: str
    textbook_version: str
    source: str
    level_path: str
    difficulty_score: int | None
    knowledge_points: str
    abilities: str
    created_at: str
    updated_at: str


def list_level_paths(path: Path) -> list[str]:
    if not path.exists():
        return []
    with sqlite3.connect(path) as conn:
        rows = conn.execute(
            f"""
            SELECT DISTINCT level_path
            FROM {TABLE_NAME}
            WHERE TRIM(COALESCE(level_path, '')) <> ''
            ORDER BY level_path
            """
        ).fetchall()
    return [str(row[0]).strip() for row in rows if row and str(row[0]).strip()]


def load_questions_by_level_path(path: Path, level_path: str) -> list[DbQuestionRecord]:
    if not path.exists():
        return []
    with sqlite3.connect(path) as conn:
        _create_table(conn)
        rows = conn.execute(
            f"""
            SELECT
                id,
                stem,
                option_1,
                option_2,
                option_3,
                option_4,
                choice_1,
                choice_2,
                choice_3,
                choice_4,
                answer,
                analysis,
```

```
                question_type,
                textbook_version,
                source,
                level_path,
                difficulty_score,
                knowledge_points,
                abilities,
                created_at,
                updated_at
            FROM {TABLE_NAME}
            WHERE level_path = ?
            ORDER BY created_at ASC, id ASC
            """,
            (level_path,),
        ).fetchall()
    return [_row_to_record(row) for row in rows]


def load_all_questions(path: Path) -> list[DbQuestionRecord]:
    if not path.exists():
        return []
    with sqlite3.connect(path) as conn:
        _create_table(conn)
        rows = conn.execute(
            f"""
            SELECT
                id,
                stem,
                option_1,
                option_2,
                option_3,
                option_4,
                choice_1,
                choice_2,
                choice_3,
                choice_4,
                answer,
                analysis,
                question_type,
                textbook_version,
                source,
                level_path,
                difficulty_score,
                knowledge_points,
                abilities,
                created_at,
                updated_at
            FROM {TABLE_NAME}
            ORDER BY created_at ASC, id ASC
            """
        ).fetchall()
    return [_row_to_record(row) for row in rows]


def update_question(path: Path, question: DbQuestionRecord) -> None:
    if not path.exists():
        raise FileNotFoundError(path)
    with sqlite3.connect(path) as conn:
        _create_table(conn)
        conn.execute(
            f"""
            UPDATE {TABLE_NAME}
            SET
                stem = ?,
                option_1 = ?,
                option_2 = ?,
                option_3 = ?,
                option_4 = ?,
                choice_1 = ?,
                choice_2 = ?,
```

```
                choice_3 = ?,
                choice_4 = ?,
                answer = ?,
                analysis = ?,
                question_type = ?,
                textbook_version = ?,
                source = ?,
                level_path = ?,
                difficulty_score = ?,
                knowledge_points = ?,
                abilities = ?,
                created_at = ?,
                updated_at = ?
            WHERE id = ?
            """,
            (
                question.stem,
                question.option_1,
                question.option_2,
                question.option_3,
                question.option_4,
                question.choice_1,
                question.choice_2,
                question.choice_3,
                question.choice_4,
                question.answer,
                question.analysis,
                question.question_type,
                question.textbook_version,
                question.source,
                question.level_path,
                question.difficulty_score,
                question.knowledge_points,
                question.abilities,
                question.created_at,
                question.updated_at,
                question.id,
            ),
        )
        conn.commit()


def delete_question_by_id(path: Path, question_id: str) -> int:
    if not path.exists():
        return 0
    normalized_question_id = str(question_id or "").strip()
    if not normalized_question_id:
        return 0
    with sqlite3.connect(path) as conn:
        _create_table(conn)
        cur = conn.execute(
            f"DELETE FROM {TABLE_NAME} WHERE id = ?",
            (normalized_question_id,),
        )
        conn.commit()
        return int(cur.rowcount or 0)


def delete_questions_by_level_path(path: Path, level_path: str) -> int:
    if not path.exists():
        return 0
    normalized_level_path = str(level_path or "").strip()
    if not normalized_level_path:
        return 0
    with sqlite3.connect(path) as conn:
        _create_table(conn)
        cur = conn.execute(
            f"DELETE FROM {TABLE_NAME} WHERE level_path = ?",
            (normalized_level_path,),
        )
```

```python
        conn.commit()
        return int(cur.rowcount or 0)


def count_questions_by_level_prefix(path: Path, level_prefix: str) -> int:
    if not path.exists():
        return 0
    normalized_level_prefix = str(level_prefix or "").strip()
    if not normalized_level_prefix:
        return 0
    like_pattern = normalized_level_prefix + ".%"
    with sqlite3.connect(path) as conn:
        _create_table(conn)
        row = conn.execute(
            f"""
            SELECT COUNT(*)
            FROM {TABLE_NAME}
            WHERE level_path = ? OR level_path LIKE ?
            """,
            (normalized_level_prefix, like_pattern),
        ).fetchone()
    return int(row[0] or 0) if row else 0


def delete_questions_by_level_prefix(path: Path, level_prefix: str) -> int:
    if not path.exists():
        return 0
    normalized_level_prefix = str(level_prefix or "").strip()
    if not normalized_level_prefix:
        return 0
    like_pattern = normalized_level_prefix + ".%"
    with sqlite3.connect(path) as conn:
        _create_table(conn)
        cur = conn.execute(
            f"""
            DELETE FROM {TABLE_NAME}
            WHERE level_path = ? OR level_path LIKE ?
            """,
            (normalized_level_prefix, like_pattern),
        )
        conn.commit()
        return int(cur.rowcount or 0)


def initialize_db(path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with sqlite3.connect(path) as conn:
        _create_table(conn)
        conn.commit()


def replace_questions(path: Path, questions: Iterable[DbQuestionRecord]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with sqlite3.connect(path) as conn:
        conn.execute(f"DROP TABLE IF EXISTS {TABLE_NAME}")
        _create_table(conn)
        _insert_questions(conn, questions)
        conn.commit()


def append_questions(path: Path, questions: Iterable[DbQuestionRecord]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with sqlite3.connect(path) as conn:
        _create_table(conn)
        _insert_questions(conn, questions)
        conn.commit()


def _create_table(conn: sqlite3.Connection) -> None:
    conn.execute(
```

```
        f"""
        CREATE TABLE IF NOT EXISTS {TABLE_NAME} (
            id TEXT PRIMARY KEY,
            stem TEXT NOT NULL,
            option_1 TEXT NOT NULL DEFAULT '',
            option_2 TEXT NOT NULL DEFAULT '',
            option_3 TEXT NOT NULL DEFAULT '',
            option_4 TEXT NOT NULL DEFAULT '',
            choice_1 TEXT NOT NULL DEFAULT '',
            choice_2 TEXT NOT NULL DEFAULT '',
            choice_3 TEXT NOT NULL DEFAULT '',
            choice_4 TEXT NOT NULL DEFAULT '',
            answer TEXT NOT NULL DEFAULT '',
            analysis TEXT NOT NULL DEFAULT '',
            question_type TEXT NOT NULL DEFAULT '',
            textbook_version TEXT NOT NULL DEFAULT '',
            source TEXT NOT NULL DEFAULT '',
            level_path TEXT NOT NULL DEFAULT '',
            difficulty_score INTEGER,
            knowledge_points TEXT NOT NULL DEFAULT '',
            abilities TEXT NOT NULL DEFAULT '',
            created_at TEXT NOT NULL,
            updated_at TEXT NOT NULL
        )
        """
    )
    existing_columns = {
        str(row[1]).strip()
        for row in conn.execute(f"PRAGMA table_info({TABLE_NAME})").fetchall()
        if len(row) > 1 and str(row[1]).strip()
    }
    for column_name in ("choice_1", "choice_2", "choice_3", "choice_4"):
        if column_name in existing_columns:
            continue
        conn.execute(
            f"ALTER TABLE {TABLE_NAME} ADD COLUMN {column_name} TEXT NOT NULL DEFAULT ''"
        )
    if "source" not in existing_columns:
        conn.execute(
            f"ALTER TABLE {TABLE_NAME} ADD COLUMN source TEXT NOT NULL DEFAULT ''"
        )
        existing_columns.add("source")
    if "source_filename" in existing_columns:
        conn.execute(
            f"""
            UPDATE {TABLE_NAME}
            SET source = source_filename
            WHERE TRIM(COALESCE(source, '')) = '' AND TRIM(COALESCE(source_filename, '')) <> ''
            """
        )


def _insert_questions(conn: sqlite3.Connection, questions: Iterable[DbQuestionRecord]) -> None:
    rows = [
        (
            q.id,
            q.stem,
            q.option_1,
            q.option_2,
            q.option_3,
            q.option_4,
            q.choice_1,
            q.choice_2,
            q.choice_3,
            q.choice_4,
            q.answer,
            q.analysis,
            q.question_type,
            q.textbook_version,
            q.source,
```

```python
            q.level_path,
            q.difficulty_score,
            q.knowledge_points,
            q.abilities,
            q.created_at,
            q.updated_at,
        )
        for q in questions
    ]
    if not rows:
        return
    conn.executemany(
        f"""
        INSERT INTO {TABLE_NAME} (
            id,
            stem,
            option_1,
            option_2,
            option_3,
            option_4,
            choice_1,
            choice_2,
            choice_3,
            choice_4,
            answer,
            analysis,
            question_type,
            textbook_version,
            source,
            level_path,
            difficulty_score,
            knowledge_points,
            abilities,
            created_at,
            updated_at
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """,
        rows,
    )


def _row_to_record(row: tuple[object, ...]) -> DbQuestionRecord:
    return DbQuestionRecord(
        id=str(row[0] or ""),
        stem=str(row[1] or ""),
        option_1=str(row[2] or ""),
        option_2=str(row[3] or ""),
        option_3=str(row[4] or ""),
        option_4=str(row[5] or ""),
        choice_1=str(row[6] or ""),
        choice_2=str(row[7] or ""),
        choice_3=str(row[8] or ""),
        choice_4=str(row[9] or ""),
        answer=str(row[10] or ""),
        analysis=str(row[11] or ""),
        question_type=str(row[12] or ""),
        textbook_version=str(row[13] or ""),
        source=str(row[14] or ""),
        level_path=str(row[15] or ""),
        difficulty_score=int(row[16]) if row[16] is not None else None,
        knowledge_points=str(row[17] or ""),
        abilities=str(row[18] or ""),
        created_at=str(row[19] or ""),
        updated_at=str(row[20] or ""),
    )
```

sj_generator/presentation/__init__.py

043-01

```python


```

sj_generator/presentation/qt/__init__.py

044-01

```python
from __future__ import annotations

from importlib import import_module

_EXPORT_TO_MODULE = {
    "GeneratorWizard": ".main_wizard",
    "main": ".main_wizard",
    "ImportFlowWizard": ".import_flow",
    "configure_import_flow_pages": ".import_flow",
}

__all__ = ["GeneratorWizard", "ImportFlowWizard", "configure_import_flow_pages", "main"]


def __getattr__(name: str):
    module_name = _EXPORT_TO_MODULE.get(name)
    if module_name is None:
        raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
    module = import_module(module_name, __name__)
    value = getattr(module, name)
    globals()[name] = value
    return value


def __dir__() -> list[str]:
    return sorted(set(globals()) | set(__all__))

```

```python
from __future__ import annotations

import json
from decimal import Decimal
from typing import TYPE_CHECKING
from typing import Callable

from PyQt6.QtCore import QObject, QThread, Qt, pyqtSignal
from PyQt6.QtGui import QAction
from sj_generator.infrastructure.llm.balance import (
    describe_deepseek_balance,
    describe_kimi_balance,
    describe_qwen_balance,
    query_deepseek_balance_snapshot,
    query_kimi_balance_snapshot,
    query_qwen_balance_snapshot,
)
from sj_generator.infrastructure.llm.client import LlmConfig
from PyQt6.QtWidgets import (
    QAbstractItemView,
    QComboBox,
    QDialog,
    QDialogButtonBox,
    QFormLayout,
    QHBoxLayout,
    QHeaderView,
    QInputDialog,
    QLabel,
    QLineEdit,
    QMenu,
    QMessageBox,
    QPushButton,
    QTableWidget,
    QTableWidgetItem,
    QTabWidget,
    QToolButton,
    QVBoxLayout,
    QWidget,
)

from sj_generator.infrastructure.llm.client import LlmClient
from sj_generator.application.settings import (
    DeepSeekConfig,
    KimiConfig,
    QwenConfig,
    default_project_parse_model_rows,
    load_available_models,
    load_deepseek_config,
    load_kimi_config,
    load_project_parse_model_rows,
    load_program_settings,
    load_qwen_config,
    save_deepseek_config,
    save_kimi_config,
    save_available_models,
    save_program_analysis_target,
    save_program_settings_merged,
    save_project_parse_model_rows,
    save_qwen_config,
    sync_deepseek_runtime_env,
    sync_kimi_runtime_env,
    sync_qwen_runtime_env,
    to_kimi_question_number_llm_config,
    to_kimi_llm_config,
    to_question_number_llm_config,
    to_llm_config,
    to_qwen_question_number_llm_config,
    to_qwen_llm_config,
    with_capped_timeout,
)
```

```python
from sj_generator.application.settings.import_cost_history import append_balance_history_for_provider_results
from sj_generator.application.state import (
    AI_CONCURRENCY_OPTIONS,
    normalize_ai_concurrency,
    normalize_analysis_model_name,
    normalize_analysis_provider,
)
from sj_generator.infrastructure.llm.task_runner import run_callables_in_parallel_fail_fast
from sj_generator.presentation.qt.import_cost_history_dialog import ImportCostHistoryDialog
from sj_generator.presentation.qt.table_copy import CopyableTableWidget

if TYPE_CHECKING:
    from sj_generator.application.state import WizardState

BUTTON_MIN_WIDTH = 96
BUTTON_MIN_HEIGHT = 36
FAST_TEST_TIMEOUT_S = 12.0
def _style_dialog_button(button: QPushButton | None, text: str | None = None) -> None:
    if button is None:
        return
    if text:
        button.setText(text)
    button.setMinimumSize(BUTTON_MIN_WIDTH, BUTTON_MIN_HEIGHT)


def _style_message_box_buttons(box: QMessageBox) -> None:
    for button_type, text in (
        (QMessageBox.StandardButton.Ok, "确定"),
        (QMessageBox.StandardButton.Cancel, "取消"),
        (QMessageBox.StandardButton.Yes, "是"),
        (QMessageBox.StandardButton.No, "否"),
    ):
        _style_dialog_button(box.button(button_type), text)


def _show_message_box(
    parent,
    *,
    title: str,
    text: str,
    icon: QMessageBox.Icon,
    buttons: QMessageBox.StandardButton = QMessageBox.StandardButton.Ok,
    default_button: QMessageBox.StandardButton = QMessageBox.StandardButton.NoButton,
) -> QMessageBox.StandardButton:
    box = QMessageBox(parent)
    box.setWindowTitle(title)
    box.setText(text)
    box.setIcon(icon)
    box.setStandardButtons(buttons)
    if default_button != QMessageBox.StandardButton.NoButton:
        box.setDefaultButton(default_button)
    _style_message_box_buttons(box)
    return QMessageBox.StandardButton(box.exec())


class ApiConfigDialog(QDialog):
    def __init__(self, parent=None, *, state: WizardState | None = None) -> None:
        super().__init__(parent)
        self.setWindowTitle("API 配置")
        self.resize(560, 320)
        self._state = state

        self._deepseek_tab = _ApiConfigTab(
            title="DeepSeek",
            cfg=load_deepseek_config(),
            save_fn=save_deepseek_config,
            to_llm_fn=to_llm_config,
            cfg_type=DeepSeekConfig,
        )
        self._kimi_tab = _ApiConfigTab(
```

```python
            title="Kimi",
            cfg=load_kimi_config(),
            save_fn=save_kimi_config,
            to_llm_fn=to_kimi_llm_config,
            cfg_type=KimiConfig,
        )
        self._qwen_tab = _ApiConfigTab(
            title="千问",
            cfg=load_qwen_config(),
            save_fn=save_qwen_config,
            to_llm_fn=to_qwen_llm_config,
            cfg_type=QwenConfig,
        )
        self._project_tab = _ProjectConfigTab(state=self._state)
        self._deepseek_tab.modelListChanged.connect(self._project_tab.refresh_model_candidates)
        self._kimi_tab.modelListChanged.connect(self._project_tab.refresh_model_candidates)
        self._qwen_tab.modelListChanged.connect(self._project_tab.refresh_model_candidates)
        self._test_thread: QThread | None = None
        self._test_worker: _AsyncCallWorker | None = None
        self._testing_api_keys: list[str] = []
        self._testing_project = False

        tabs = QTabWidget()
        tabs.addTab(self._project_tab, "项目配置")
        tabs.addTab(self._deepseek_tab, "DeepSeek")
        tabs.addTab(self._kimi_tab, "Kimi")
        tabs.addTab(self._qwen_tab, "千问")

        buttons = QDialogButtonBox(QDialogButtonBox.StandardButton.Ok | QDialogButtonBox.StandardButton.Cancel)
        self._test_all_btn = QPushButton("统一测试")
        self._view_cost_history_btn = QPushButton("查看余额日志")
        buttons.accepted.connect(self._on_accept)
        buttons.rejected.connect(self.reject)
        self._test_all_btn.clicked.connect(self._on_test_all)
        self._view_cost_history_btn.clicked.connect(self._open_import_cost_history_dialog)
        _style_dialog_button(self._test_all_btn, "统一测试")
        _style_dialog_button(self._view_cost_history_btn, "查看余额日志")
        _style_dialog_button(buttons.button(QDialogButtonBox.StandardButton.Ok), "确定")
        _style_dialog_button(buttons.button(QDialogButtonBox.StandardButton.Cancel), "取消")

        layout = QVBoxLayout()
        layout.addWidget(tabs)
        button_row = QHBoxLayout()
        button_row.addWidget(self._view_cost_history_btn)
        button_row.addStretch(1)
        button_row.addWidget(self._test_all_btn)
        button_row.addWidget(buttons)
        layout.addLayout(button_row)
        self.setLayout(layout)

    def _on_accept(self) -> None:
        if self._is_testing():
            _show_message_box(self, title="测试进行中", text="模型测试仍在进行中，请等待完成后再保存。", icon=QMessageBox.Icon.Warning)
            return
        try:
            self._project_tab.save_if_needed()
            self._deepseek_tab.save_if_needed()
            self._kimi_tab.save_if_needed()
            self._qwen_tab.save_if_needed()
        except _ConfigValidationError as e:
            _show_message_box(self, title="配置未完成", text=str(e), icon=QMessageBox.Icon.Warning)
            return
        except Exception as e:
            _show_message_box(self, title="保存失败", text=str(e), icon=QMessageBox.Icon.Critical)
            return
        self.accept()

    def reject(self) -> None:
```

```python
        if self._is_testing():
            _show_message_box(self, title="测试进行中", text="模型测试仍在进行中，请等待完成后再关闭。", icon=QMessageBox.Icon.Warning)
            return
        super().reject()

    def _is_testing(self) -> bool:
        return (
            (self._test_thread is not None and self._test_thread.isRunning())
            or self._project_tab.is_testing()
            or self._deepseek_tab.is_testing()
            or self._kimi_tab.is_testing()
            or self._qwen_tab.is_testing()
        )

    def _on_test_all(self) -> None:
        if self._is_testing():
            _show_message_box(self, title="测试进行中", text="模型测试仍在进行中，请等待完成。", icon=QMessageBox.Icon.Warning)
            return
        try:
            api_targets: list[tuple[_ApiConfigTab, object]] = []
            api_tabs_with_cfgs: list[tuple[_ApiConfigTab, object]] = []
            override_cfgs: dict[str, object] = {}
            for tab in (self._deepseek_tab, self._kimi_tab, self._qwen_tab):
                cfg = tab.collect_cfg_for_test()
                api_tabs_with_cfgs.append((tab, cfg))
                override_cfgs[tab.provider_key()] = cfg
                if tab.needs_test(cfg):
                    api_targets.append((tab, cfg))
            project_payload = self._project_tab.collect_test_payload()
            project_needs_test = self._project_tab.needs_test(project_payload)
        except _ConfigValidationError as e:
            _show_message_box(self, title="参数不合法", text=str(e), icon=QMessageBox.Icon.Warning)
            return

        for tab, _cfg in api_tabs_with_cfgs:
            tab.set_balance_querying()
        for tab, _cfg in api_targets:
            tab.set_status_text(f"正在统一测试 {tab.title_text()} API，请稍候…")
        if project_needs_test:
            self._project_tab.set_status_text("正在统一测试项目配置模型，请稍候…")
        self._testing_api_keys = [tab.provider_key() for tab, _cfg in api_targets]
        self._testing_project = project_needs_test
        self._test_all_btn.setEnabled(False)

        def run_all() -> dict[str, object]:
            api_results: list[tuple[str, dict[str, object]]] = []
            api_test_provider_keys = {tab.provider_key() for tab, _cfg in api_targets}
            for tab, cfg in api_tabs_with_cfgs:
                if tab.provider_key() in api_test_provider_keys:
                    result = _run_api_connectivity_test(tab.title_text(), cfg, tab.to_llm_fn())
                else:
                    result = _run_api_balance_refresh(cfg)
                api_results.append(
                    (
                        tab.provider_key(),
                        result,
                    )
                )
            project_result = None
            if project_needs_test:
                project_result = _run_project_model_test(
                    provider=str(project_payload["provider"]),
                    model_name=str(project_payload["model_name"]),
                    parse_model_rows=list(project_payload["parse_model_rows"]),
                    override_cfgs=override_cfgs,
                )
            return {
                "api_results": api_results,
```

```python
                "project_result": project_result,
                "api_keys": list(self._testing_api_keys),
                "project_tested": self._testing_project,
            }

        thread = QThread(self)
        worker = _AsyncCallWorker(run_all)
        worker.moveToThread(thread)
        thread.started.connect(worker.run)
        worker.finished.connect(self._on_test_all_success)
        worker.failed.connect(self._on_test_all_error)
        worker.finished.connect(thread.quit)
        worker.failed.connect(thread.quit)
        thread.finished.connect(worker.deleteLater)
        thread.finished.connect(thread.deleteLater)
        thread.finished.connect(self._on_test_all_finished)
        self._test_thread = thread
        self._test_worker = worker
        thread.start()

    def _on_test_all_success(self, payload: object) -> None:
        data = payload if isinstance(payload, dict) else {}
        provider_map = {
            "deepseek": self._deepseek_tab,
            "kimi": self._kimi_tab,
            "qwen": self._qwen_tab,
        }
        tested_labels: list[str] = []
        for provider_key, result in data.get("api_results") or []:
            tab = provider_map.get(str(provider_key))
            if tab is None:
                continue
            if bool((result or {}).get("connectivity_tested")):
                tab.apply_test_success(result, show_message=False)
                tested_labels.append(f"{tab.title_text()} API")
            else:
                tab.apply_balance_refresh(result)
        if data.get("project_tested"):
            self._project_tab.apply_test_success(data.get("project_result"), show_message=False)
            tested_labels.append("项目配置模型")
        append_balance_history_for_provider_results(data.get("api_results") or [], source_label="统一测试")
        if tested_labels:
            summary = "、".join(tested_labels)
            message = f"{summary} 测试通过，余额已刷新。"
        else:
            message = "余额已刷新，当前没有需要重测的配置。"
        _show_message_box(self, title="测试通过", text=message, icon=QMessageBox.Icon.Information)

    def _on_test_all_error(self, message: str) -> None:
        provider_map = {
            "deepseek": self._deepseek_tab,
            "kimi": self._kimi_tab,
            "qwen": self._qwen_tab,
        }
        for provider_key in self._testing_api_keys:
            tab = provider_map.get(provider_key)
            if tab is not None:
                tab.reset_runtime_status_after_failure()
        if self._testing_project:
            self._project_tab.reset_runtime_status_after_failure()
        _show_message_box(self, title="测试失败", text=message, icon=QMessageBox.Icon.Critical)

    def _on_test_all_finished(self) -> None:
        self._test_all_btn.setEnabled(True)
        self._test_thread = None
        self._test_worker = None
        self._testing_api_keys = []
        self._testing_project = False

    def _open_import_cost_history_dialog(self) -> None:
```

```python
        ImportCostHistoryDialog(self).exec()


class _ConfigValidationError(Exception):
    pass


class _AsyncCallWorker(QObject):
    finished = pyqtSignal(object)
    failed = pyqtSignal(str)

    def __init__(self, fn: Callable[[], object]) -> None:
        super().__init__()
        self._fn = fn

    def run(self) -> None:
        try:
            self.finished.emit(self._fn())
        except Exception as e:
            self.failed.emit(str(e))


def _analysis_provider_label(provider: str) -> str:
    labels = {"deepseek": "DeepSeek", "kimi": "Kimi", "qwen": "千问"}
    return labels.get(normalize_analysis_provider(provider), "DeepSeek")


def _analysis_target_text(provider: str, model_name: str) -> str:
    return f"{_analysis_provider_label(provider)} / {normalize_analysis_model_name(model_name)}"


def _analysis_target_candidates() -> list[str]:
    candidates: list[str] = []
    for provider_key, provider_label, models in _project_model_menu_groups():
        if not models:
            continue
        for model_name in models:
            candidates.append(f"{provider_label} / {model_name}")
    return candidates


def _provider_key_from_title(title: str) -> str:
    mapping = {
        "DeepSeek": "deepseek",
        "Kimi": "kimi",
        "千问": "qwen",
    }
    return mapping.get(str(title or "").strip(), "deepseek")


def _parse_analysis_target_text(text: str) -> tuple[str, str]:
    raw = (text or "").strip()
    if not raw:
        return "deepseek", normalize_analysis_model_name("")
    left, sep, right = raw.partition("/")
    if not sep:
        left, sep, right = raw.partition("：")
    if not sep:
        left, sep, right = raw.partition(":")
    if not sep:
        return "deepseek", normalize_analysis_model_name(raw)
    provider_text = left.strip().lower()
    aliases = {
        "deepseek": "deepseek",
        "deep seek": "deepseek",
        "kimi": "kimi",
        "moonshot": "kimi",
        "qwen": "qwen",
        "千问": "qwen",
    }
```

```python
    provider = aliases.get(provider_text, normalize_analysis_provider(provider_text))
    model_name = normalize_analysis_model_name(right.strip())
    return provider, model_name


def _to_fast_test_llm_config(cfg: LlmConfig) -> LlmConfig:
    return LlmConfig(
        base_url=cfg.base_url,
        api_key=cfg.api_key,
        model=cfg.model,
        timeout_s=min(float(cfg.timeout_s), FAST_TEST_TIMEOUT_S),
        max_retries=0,
        retry_backoff_s=0.0,
    )


def _query_balance_text_for_cfg(cfg) -> str:
    return _query_balance_result_for_cfg(cfg)["balance_text"]


def _format_balance_amount(currency: str, amount: Decimal) -> str:
    prefix = {"CNY": "￥", "USD": "$"}.get((currency or "").upper(), "")
    if prefix:
        return f"{prefix}{amount:.4f}"
    return f"{(currency or 'UNKNOWN').upper()} {amount:.4f}"


def _query_balance_result_for_cfg(cfg) -> dict[str, str]:
    cfg = with_capped_timeout(cfg, FAST_TEST_TIMEOUT_S)
    try:
        if isinstance(cfg, DeepSeekConfig):
            if not cfg.is_ready():
                return {"balance_text": "未配置", "balance_value": ""}
            snapshot = query_deepseek_balance_snapshot(cfg)
            if snapshot is None:
                return {"balance_text": "未配置", "balance_value": ""}
            return {
                "balance_text": snapshot.detail,
                "balance_value": _format_balance_amount(snapshot.currency, Decimal(str(snapshot.amount))),
            }
        if isinstance(cfg, KimiConfig):
            if not cfg.is_ready():
                return {"balance_text": "未配置", "balance_value": ""}
            snapshot = query_kimi_balance_snapshot(cfg)
            if snapshot is None:
                return {"balance_text": describe_kimi_balance(cfg), "balance_value": ""}
            return {
                "balance_text": snapshot.detail,
                "balance_value": _format_balance_amount(snapshot.currency, Decimal(str(snapshot.amount))),
            }
        if isinstance(cfg, QwenConfig):
            if not cfg.is_ready() and not cfg.has_account_balance_credentials():
                return {"balance_text": "未配置", "balance_value": ""}
            snapshot = query_qwen_balance_snapshot(cfg)
            if snapshot is None:
                return {"balance_text": describe_qwen_balance(cfg), "balance_value": ""}
            return {
                "balance_text": snapshot.detail,
                "balance_value": _format_balance_amount(snapshot.currency, Decimal(str(snapshot.amount))),
            }
    except Exception as e:
        return {"balance_text": f"查询失败：{e}", "balance_value": ""}
    return {"balance_text": "未配置", "balance_value": ""}


def _run_api_connectivity_test(title: str, cfg, to_llm_fn: Callable[[object], object]) -> dict[str, object]:
    number_cfg = _to_fast_test_llm_config(_number_llm_config_for_cfg(cfg))
    unit_cfg = _to_fast_test_llm_config(to_llm_fn(cfg))
    config_map = {
        f"{number_cfg.base_url}|{number_cfg.model}": number_cfg,
```

```python
        f"{unit_cfg.base_url}|{unit_cfg.model}": unit_cfg,
    }

    def check_one(llm_cfg: LlmConfig) -> None:
        text = LlmClient(llm_cfg).chat_text(system="你是连通性测试助手。", user="请只返回 OK")
        if "OK" not in (text or "").upper():
            raise RuntimeError(f"{title} API 测试未通过，返回：{text}")

    run_callables_in_parallel_fail_fast(
        callables=[lambda llm_cfg=llm_cfg: check_one(llm_cfg) for llm_cfg in config_map.values()],
        max_workers=min(2, len(config_map)),
    )
    balance_result = _query_balance_result_for_cfg(cfg)
    return {
        "cfg": cfg,
        "balance_text": balance_result["balance_text"],
        "balance_value": balance_result["balance_value"],
        "connectivity_tested": True,
    }


def _run_api_balance_refresh(cfg) -> dict[str, object]:
    balance_result = _query_balance_result_for_cfg(cfg)
    return {
        "cfg": cfg,
        "balance_text": balance_result["balance_text"],
        "balance_value": balance_result["balance_value"],
        "connectivity_tested": False,
    }




def _build_analysis_llm_config(
    provider: str,
    model_name: str,
    *,
    override_cfgs: dict[str, object] | None = None,
) -> tuple[str, LlmConfig] | tuple[None, None]:
    provider = normalize_analysis_provider(provider)
    provider_label = _analysis_provider_label(provider)
    cfg = (override_cfgs or {}).get(provider)
    if cfg is None:
        if provider == "kimi":
            cfg = load_kimi_config()
        elif provider == "qwen":
            cfg = load_qwen_config()
        else:
            cfg = load_deepseek_config()
    if provider == "kimi":
        if not cfg.is_ready():
            return None, None
        return provider_label, LlmConfig(
            base_url=cfg.base_url.strip(),
            api_key=cfg.api_key.strip(),
            model=model_name,
            timeout_s=float(cfg.timeout_s),
        )
    if provider == "qwen":
        if not cfg.is_ready():
            return None, None
        return provider_label, LlmConfig(
            base_url=cfg.base_url.strip(),
            api_key=cfg.api_key.strip(),
            model=model_name,
            timeout_s=float(cfg.timeout_s),
        )
    if not cfg.is_ready():
        return None, None
    return provider_label, LlmConfig(
```

```python
        base_url=cfg.base_url.strip(),
        api_key=cfg.api_key.strip(),
        model=model_name,
        timeout_s=float(cfg.timeout_s),
    )


def _run_project_model_test(
    *,
    provider: str,
    model_name: str,
    parse_model_rows: list[dict],
    override_cfgs: dict[str, object] | None = None,
) -> dict[str, object]:
    pairs: list[tuple[str, str]] = []
    seen: set[str] = set()

    def add(current_provider: str, current_model_name: str) -> None:
        normalized_provider = normalize_analysis_provider(current_provider)
        normalized_model = str(current_model_name or "").strip()
        if not normalized_provider or not normalized_model:
            return
        key = f"{normalized_provider}|{normalized_model}"
        if key in seen:
            return
        seen.add(key)
        pairs.append((normalized_provider, normalized_model))

    add(provider, model_name)
    for row in parse_model_rows:
        for item in row.get("models") or []:
            add(str(item.get("provider") or ""), str(item.get("model_name") or ""))

    normalized_provider, normalized_model = _parse_analysis_target_text(
        f"{_analysis_provider_label(provider)} / {model_name}"
    )

    def check_one(current_provider: str, current_model_name: str) -> None:
        provider_label, llm_config = _build_analysis_llm_config(
            current_provider,
            current_model_name,
            override_cfgs=override_cfgs,
        )
        if provider_label is None or llm_config is None:
            raise RuntimeError(f"请先完成 {_analysis_provider_label(current_provider)} 的 API 配置。")
        client = LlmClient(_to_fast_test_llm_config(llm_config))
        text = client.chat_text(system="你是连通性测试助手。", user="请只返回 OK")
        if "OK" not in (text or "").upper():
            raise RuntimeError(f"{provider_label} / {current_model_name} 测试未通过，返回：{text}")

    run_callables_in_parallel_fail_fast(
        callables=[
            lambda p=current_provider, m=current_model_name: check_one(p, m)
            for current_provider, current_model_name in pairs
        ],
        max_workers=max(1, len(pairs)),
    )
    return {
        "provider": normalized_provider,
        "model_name": normalized_model,
        "parse_model_rows": parse_model_rows,
    }


def _project_model_menu_groups() -> list[tuple[str, str, list[str]]]:
    return [
        ("deepseek", "DeepSeek", _model_candidates("DeepSeek")),
        ("kimi", "Kimi", _model_candidates("Kimi")),
        ("qwen", "千问", _model_candidates("千问")),
    ]
```

```python


class _GroupedModelSelector(QToolButton):
    valueChanged = pyqtSignal()

    def __init__(self, parent: QWidget | None = None) -> None:
        super().__init__(parent)
        self._provider = ""
        self._model_name = ""
        self.setPopupMode(QToolButton.ToolButtonPopupMode.InstantPopup)
        self.setText("选择模型")
        self.setToolTip("请选择预设模型")
        self._rebuild_menu()

    def current_value(self) -> tuple[str, str]:
        return self._provider, self._model_name

    def set_current_value(self, provider: str, model_name: str) -> None:
        next_provider = normalize_analysis_provider(provider) if str(provider or "").strip() else ""
        next_model_name = str(model_name or "").strip()
        if self._provider == next_provider and self._model_name == next_model_name:
            return
        self._provider = next_provider
        self._model_name = next_model_name
        self._refresh_text()
        self.valueChanged.emit()

    def _rebuild_menu(self) -> None:
        root_menu = QMenu(self)
        clear_action = QAction("清空", self)
        clear_action.triggered.connect(lambda: self.set_current_value("", ""))
        root_menu.addAction(clear_action)
        root_menu.addSeparator()

        for provider_key, provider_label, models in _project_model_menu_groups():
            provider_menu = root_menu.addMenu(provider_label)
            for model_name in models:
                action = QAction(model_name, self)
                action.triggered.connect(
                    lambda _checked=False, p=provider_key, m=model_name: self.set_current_value(p, m)
                )
                provider_menu.addAction(action)

        self.setMenu(root_menu)

    def reload_candidates(self) -> None:
        provider = self._provider
        model_name = self._model_name
        self._rebuild_menu()
        self._provider = provider
        self._model_name = model_name
        self._refresh_text()

    def _refresh_text(self) -> None:
        if self._provider and self._model_name:
            label = _analysis_provider_label(self._provider)
            text = f"{label}\n{self._model_name}"
            tooltip = f"{label} / {self._model_name}"
        else:
            text = "选择模型"
            tooltip = text
        self.setText(text)
        self.setToolTip(tooltip)


class _ProjectConfigTab(QWidget):
    def __init__(self, *, state: WizardState | None) -> None:
        super().__init__()
        self._state = state
        self._parse_model_rows = load_project_parse_model_rows()
```

```python
        self._parse_round_combos: list[QComboBox] = []
        self._parse_ratio_edits: list[QLineEdit] = []
        data = load_program_settings()
        initial_provider = normalize_analysis_provider(
            data.get("analysis_provider", getattr(state, "analysis_provider", "deepseek"))
        )
        initial_model_name = normalize_analysis_model_name(
            data.get("analysis_model_name", getattr(state, "analysis_model_name", ""))
        )
        legacy_ai_concurrency = normalize_ai_concurrency(data.get("ai_concurrency", getattr(state, "ai_concurrency", 3)))
        initial_question_content_concurrency = normalize_ai_concurrency(
            data.get(
                "question_content_concurrency",
                getattr(state, "question_content_concurrency", legacy_ai_concurrency),
            )
        )
        initial_analysis_generation_concurrency = normalize_ai_concurrency(
            data.get(
                "analysis_generation_concurrency",
                getattr(state, "analysis_generation_concurrency", legacy_ai_concurrency),
            )
        )

        self._analysis_target_combo = QComboBox()
        self._analysis_target_combo.setEditable(True)
        self._analysis_target_combo.addItems(_analysis_target_candidates())
        self._analysis_target_combo.setCurrentText(_analysis_target_text(initial_provider, initial_model_name))
        self._question_content_concurrency_combo = QComboBox()
        self._analysis_generation_concurrency_combo = QComboBox()
        self._question_content_concurrency_combo.setMinimumWidth(120)
        self._analysis_generation_concurrency_combo.setMinimumWidth(120)
        for value in AI_CONCURRENCY_OPTIONS:
            self._question_content_concurrency_combo.addItem(str(value), value)
            self._analysis_generation_concurrency_combo.addItem(str(value), value)
        question_content_index = self._question_content_concurrency_combo.findData(initial_question_content_concurrency)
        if question_content_index >= 0:
            self._question_content_concurrency_combo.setCurrentIndex(question_content_index)
        analysis_generation_index = self._analysis_generation_concurrency_combo.findData(
            initial_analysis_generation_concurrency
        )
        if analysis_generation_index >= 0:
            self._analysis_generation_concurrency_combo.setCurrentIndex(analysis_generation_index)
        concurrency_row = QWidget()
        concurrency_layout = QHBoxLayout()
        concurrency_layout.setContentsMargins(0, 0, 0, 0)
        concurrency_layout.addWidget(QLabel("题目内容解析"))
        concurrency_layout.addWidget(self._question_content_concurrency_combo)
        concurrency_layout.addSpacing(20)
        concurrency_layout.addWidget(QLabel("答案解析生成"))
        concurrency_layout.addWidget(self._analysis_generation_concurrency_combo)
        concurrency_layout.addStretch(1)
        concurrency_row.setLayout(concurrency_layout)
        self._status = QLabel("未测试")
        self._status.setWordWrap(True)
        self._tested_key = self._project_config_key(initial_provider, initial_model_name, self._parse_model_rows)
        if self._tested_key:
            self._status.setText("当前已加载解析生成模型配置。")
        self._parse_result_table = self._build_parse_result_table()
        self._refresh_parse_result_model_columns()

        form = QFormLayout()
        form.addRow("解析生成模型：", self._analysis_target_combo)
        form.addRow("并发数：", concurrency_row)

        layout = QVBoxLayout()
        layout.addLayout(form)
```

```python
        layout.addWidget(QLabel("解析结果阈值预留"))
        layout.addWidget(self._parse_result_table)
        layout.addWidget(self._status)
        layout.addStretch(1)
        self.setLayout(layout)

        self._analysis_target_combo.currentTextChanged.connect(self._reset_tested)

    def save_if_needed(self) -> None:
        if self.is_testing():
            raise _ConfigValidationError("项目配置模型测试仍在进行中，请等待完成。")
        provider, model_name = self._current_analysis_target()
        parse_model_rows = self._collect_parse_model_rows()
        if self._project_config_key(provider, model_name, parse_model_rows) != self._tested_key:
            raise _ConfigValidationError("项目配置已修改，请先点击“统一测试”并通过。")
        save_program_analysis_target(
            provider=normalize_analysis_provider(provider),
            model_name=normalize_analysis_model_name(model_name),
        )
        self._parse_model_rows = save_project_parse_model_rows(parse_model_rows)["project_parse_model_rows"]
        question_content_concurrency = normalize_ai_concurrency(
            self._question_content_concurrency_combo.currentData()
        )
        analysis_generation_concurrency = normalize_ai_concurrency(
            self._analysis_generation_concurrency_combo.currentData()
        )
        save_program_settings_merged(
            {
                "question_content_concurrency": question_content_concurrency,
                "analysis_generation_concurrency": analysis_generation_concurrency,
            }
        )
        if self._state is not None:
            self._state.analysis_provider = normalize_analysis_provider(provider)
            self._state.analysis_model_name = normalize_analysis_model_name(model_name)
            self._state.question_content_concurrency = question_content_concurrency
            self._state.analysis_generation_concurrency = analysis_generation_concurrency

    def _current_analysis_target(self) -> tuple[str, str]:
        return _parse_analysis_target_text(self._analysis_target_combo.currentText())

    def collect_test_payload(self) -> dict[str, object]:
        provider, model_name = self._current_analysis_target()
        return {
            "provider": provider,
            "model_name": model_name,
            "parse_model_rows": self._collect_parse_model_rows(),
        }

    def needs_test(self, payload: dict[str, object] | None = None) -> bool:
        data = payload or self.collect_test_payload()
        return self._project_config_key(
            str(data.get("provider") or ""),
            str(data.get("model_name") or ""),
            list(data.get("parse_model_rows") or []),
        ) != self._tested_key

    def set_status_text(self, text: str) -> None:
        self._status.setText(str(text or ""))

    def apply_test_success(self, payload: object, *, show_message: bool = True) -> None:
        data = payload if isinstance(payload, dict) else {}
        provider = str(data.get("provider") or "deepseek")
        normalized_model = str(data.get("model_name") or "").strip()
        parse_model_rows = data.get("parse_model_rows") if isinstance(data.get("parse_model_rows"), list) else []
        self._analysis_target_combo.setCurrentText(_analysis_target_text(provider, normalized_model))
        self._tested_key = self._project_config_key(provider, normalized_model, parse_model_rows)
        self._status.setText("项目配置模型测试通过。")
        if show_message:
```

```python
            _show_message_box(
                self,
                title="测试通过",
                text="当前项目配置中的模型可正常使用。",
                icon=QMessageBox.Icon.Information,
            )

    def reset_runtime_status_after_failure(self) -> None:
        self._status.setText("最近一次测试失败。")

    def refresh_model_candidates(self) -> None:
        current_text = self._analysis_target_combo.currentText()
        self._analysis_target_combo.blockSignals(True)
        self._analysis_target_combo.clear()
        self._analysis_target_combo.addItems(_analysis_target_candidates())
        self._analysis_target_combo.setCurrentText(current_text)
        self._analysis_target_combo.blockSignals(False)
        table = self._parse_result_table
        for row in range(table.rowCount()):
            for col in range(2, table.columnCount()):
                selector = table.cellWidget(row, col)
                if isinstance(selector, _GroupedModelSelector):
                    selector.reload_candidates()

    def _analysis_target_key(self, provider: str, model_name: str) -> str:
        return f"{normalize_analysis_provider(provider)}|{normalize_analysis_model_name(model_name)}"

    def _project_config_key(self, provider: str, model_name: str, parse_model_rows: list[dict]) -> str:
        payload = {
            "analysis_target": self._analysis_target_key(provider, model_name),
            "parse_model_rows": parse_model_rows,
        }
        return json.dumps(payload, ensure_ascii=False, sort_keys=True)

    def _reset_tested(self, *_args) -> None:
        self._tested_key = ""
        self._status.setText("项目配置已变更，需重新测试。")

    def is_testing(self) -> bool:
        return False

    def _build_parse_result_table(self) -> QTableWidget:
        table = CopyableTableWidget(2, 6, self)
        table.setVerticalHeaderLabels(["序号题型解析", "题目内容解析"])
        table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
        table.setSelectionMode(QTableWidget.SelectionMode.NoSelection)
        table.setFocusPolicy(Qt.FocusPolicy.NoFocus)
        table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
        table.verticalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
        table.setMinimumHeight(120)
        self._parse_round_combos = []
        self._parse_ratio_edits = []

        for row in range(table.rowCount()):
            row_cfg = self._parse_model_row(row)
            combo = QComboBox(table)
            combo.addItems(["1", "2", "3", "4", "5", "6", "7", "8"])
            combo.setEditable(True)
            combo.lineEdit().setReadOnly(True)
            combo.lineEdit().setAlignment(Qt.AlignmentFlag.AlignCenter)
            combo.setCurrentText("1" if row == 0 else str(row_cfg.get("round") or (row + 1)))
            if row == 0:
                combo.setEnabled(False)
            combo.currentTextChanged.connect(self._reset_tested)
            table.setCellWidget(row, 0, combo)
            self._parse_round_combos.append(combo)

            ratio_edit = QLineEdit(table)
            ratio_edit.setPlaceholderText("例如 2/4")
            ratio_edit.setAlignment(Qt.AlignmentFlag.AlignCenter)
```

```python
            ratio_edit.setText(str(row_cfg.get("ratio") or "1/4"))
            ratio_edit.textChanged.connect(self._refresh_parse_result_model_columns)
            ratio_edit.textChanged.connect(self._reset_tested)
            table.setCellWidget(row, 1, ratio_edit)
            self._parse_ratio_edits.append(ratio_edit)

        return table

    def _refresh_parse_result_model_columns(self, *_args) -> None:
        table = self._parse_result_table
        model_count = self._parse_result_model_count()
        table.setColumnCount(2 + model_count)
        table.setHorizontalHeaderLabels(
            ["轮次", "通过比例"] + [f"模型{i}" for i in range(1, model_count + 1)]
        )
        for row in range(table.rowCount()):
            for col in range(2, table.columnCount()):
                if not isinstance(table.cellWidget(row, col), _GroupedModelSelector):
                    selector = _GroupedModelSelector(table)
                    provider, model_name = self._parse_model_value(row, col - 2)
                    if provider and model_name:
                        selector.set_current_value(provider, model_name)
                    selector.valueChanged.connect(self._reset_tested)
                    table.setCellWidget(row, col, selector)

    def _parse_result_model_count(self) -> int:
        counts = [self._model_count_from_ratio(edit.text()) for edit in self._parse_ratio_edits]
        return max([1, *counts])

    def _model_count_from_ratio(self, text: str) -> int:
        raw = (text or "").strip()
        if "/" not in raw:
            return 1
        _numerator, _sep, denominator_text = raw.partition("/")
        try:
            denominator = int(denominator_text.strip())
        except Exception:
            return 1
        return min(8, max(1, denominator))

    def _parse_model_row(self, row_index: int) -> dict:
        defaults = default_project_parse_model_rows()
        if 0 <= row_index < len(self._parse_model_rows):
            return self._parse_model_rows[row_index]
        return defaults[row_index]

    def _parse_model_value(self, row_index: int, model_index: int) -> tuple[str, str]:
        row_cfg = self._parse_model_row(row_index)
        models = row_cfg.get("models") or []
        if 0 <= model_index < len(models):
            item = models[model_index]
            return str(item.get("provider") or "").strip(), str(item.get("model_name") or "").strip()
        return "", ""

    def _collect_parse_model_rows(self) -> list[dict]:
        rows: list[dict] = []
        row_keys = [row["key"] for row in default_project_parse_model_rows()]
        table = self._parse_result_table
        for row_index, row_key in enumerate(row_keys):
            combo = self._parse_round_combos[row_index]
            ratio_edit = self._parse_ratio_edits[row_index]
            visible_count = self._model_count_from_ratio(ratio_edit.text())
            models: list[dict] = []
            for model_index in range(visible_count):
                selector = table.cellWidget(row_index, 2 + model_index)
                if not isinstance(selector, _GroupedModelSelector):
                    continue
                provider, model_name = selector.current_value()
                if not provider or not model_name:
                    continue
```

```python
                models.append({"provider": provider, "model_name": model_name})
            rows.append(
                {
                    "key": row_key,
                    "round": "1" if row_index == 0 else (combo.currentText().strip() or "1"),
                    "ratio": ratio_edit.text().strip() or "1/4",
                    "models": models,
                }
            )
        return rows


class _ApiConfigTab(QWidget):
    modelListChanged = pyqtSignal()

    def __init__(
        self,
        *,
        title: str,
        cfg,
        save_fn: Callable[[object], None],
        to_llm_fn: Callable[[object], object],
        cfg_type,
    ) -> None:
        super().__init__()
        self._title = title
        self._cfg = cfg
        self._save_fn = save_fn
        self._to_llm_fn = to_llm_fn
        self._cfg_type = cfg_type
        self._provider_key = _provider_key_from_title(title)
        self._env_api_key = cfg.api_key.strip()
        self._env_account_access_key_id = getattr(cfg, "account_access_key_id", "").strip()
        self._env_account_access_key_secret = getattr(cfg, "account_access_key_secret", "").strip()
        self._available_models = load_available_models(self._provider_key)

        self._base_url_edit = QLineEdit(cfg.base_url)
        self._api_key_edit = QLineEdit(self._env_api_key)
        self._api_key_edit.setEchoMode(QLineEdit.EchoMode.Password)
        self._api_key_edit.setPlaceholderText("请输入 API Key")
        self._account_access_key_id_edit: QLineEdit | None = None
        self._account_access_key_secret_edit: QLineEdit | None = None
        if isinstance(cfg, QwenConfig):
            self._account_access_key_id_edit = QLineEdit(self._env_account_access_key_id)
            self._account_access_key_id_edit.setPlaceholderText("请输入阿里云 AccessKey ID（非必填）")
            self._account_access_key_secret_edit = QLineEdit(self._env_account_access_key_secret)
            self._account_access_key_secret_edit.setEchoMode(QLineEdit.EchoMode.Password)
            self._account_access_key_secret_edit.setPlaceholderText("请输入阿里云 AccessKey Secret（非必填）")
        self._timeout_edit = QLineEdit(str(cfg.timeout_s))
        self._balance_label = QLabel(self._initial_balance_text(cfg))
        self._balance_label.setWordWrap(True)
        self._status = QLabel("未测试")
        self._status.setWordWrap(True)
        self._tested_key = self._cfg_key(cfg) if getattr(cfg, "is_ready", lambda: False)() else ""
        self._model_table = CopyableTableWidget(0, 1, self)
        self._model_table.setHorizontalHeaderLabels(["可用模型"])
        self._model_table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)
        self._model_table.setSelectionMode(QAbstractItemView.SelectionMode.SingleSelection)
        self._model_table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
        self._model_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
        self._model_table.verticalHeader().setSectionResizeMode(QHeaderView.ResizeMode.ResizeToContents)
        self._model_table.itemDoubleClicked.connect(lambda *_args: self._on_edit_model())
        self._add_model_btn = QPushButton("增加模型")
        self._edit_model_btn = QPushButton("修改模型")
        self._add_model_btn.clicked.connect(self._on_add_model)
        self._edit_model_btn.clicked.connect(self._on_edit_model)
        self._refresh_model_table()

        form = QFormLayout()
        form.addRow("Base URL：", self._base_url_edit)
```

```python
        form.addRow("API Key：", self._api_key_edit)
        if self._account_access_key_id_edit is not None and self._account_access_key_secret_edit is not None:
            form.addRow("阿里云 AccessKey ID：", self._account_access_key_id_edit)
            form.addRow("阿里云 AccessKey Secret：", self._account_access_key_secret_edit)
        form.addRow("超时(秒)：", self._timeout_edit)

        layout = QVBoxLayout()
        layout.addLayout(form)
        layout.addWidget(QLabel("可用模型列表"))
        layout.addWidget(self._model_table)
        model_btn_row = QHBoxLayout()
        model_btn_row.addWidget(self._add_model_btn)
        model_btn_row.addWidget(self._edit_model_btn)
        model_btn_row.addStretch(1)
        layout.addLayout(model_btn_row)
        layout.addWidget(self._balance_label)
        layout.addWidget(self._status)
        layout.addStretch(1)
        self.setLayout(layout)

        self._base_url_edit.textChanged.connect(self._reset_tested)
        self._api_key_edit.textChanged.connect(self._reset_tested)
        self._timeout_edit.textChanged.connect(self._reset_tested)
        if self._account_access_key_id_edit is not None:
            self._account_access_key_id_edit.textChanged.connect(self._reset_tested)
        if self._account_access_key_secret_edit is not None:
            self._account_access_key_secret_edit.textChanged.connect(self._reset_tested)

        if self._tested_key:
            self._status.setText("当前已加载可用配置。")

    def provider_key(self) -> str:
        return self._provider_key

    def title_text(self) -> str:
        return self._title

    def to_llm_fn(self) -> Callable[[object], object]:
        return self._to_llm_fn

    def collect_cfg_for_test(self):
        return self._collect_cfg()

    def needs_test(self, cfg=None) -> bool:
        current_cfg = cfg if cfg is not None else self._collect_cfg()
        return bool(current_cfg.is_ready() and self._cfg_key(current_cfg) != self._tested_key)

    def set_status_text(self, text: str) -> None:
        self._status.setText(str(text or ""))

    def set_balance_querying(self) -> None:
        self._balance_label.setText("余额：查询中…")

    def apply_test_success(self, payload: object, *, show_message: bool = True) -> None:
        data = payload if isinstance(payload, dict) else {}
        cfg = data.get("cfg") if data.get("cfg") is not None else self._cfg
        self._tested_key = self._cfg_key(cfg)
        self._status.setText(f"{self._title} 的 API 连通性测试通过。")
        self._balance_label.setText(f"余额：{str(data.get('balance_text') or '未查询')}")
        if show_message:
            _show_message_box(
                self,
                title="测试通过",
                text=f"{self._title} API 可用性测试通过。",
                icon=QMessageBox.Icon.Information,
            )

    def apply_balance_refresh(self, payload: object) -> None:
        data = payload if isinstance(payload, dict) else {}
        self._balance_label.setText(f"余额：{str(data.get('balance_text') or '未查询')}")
```

```python

    def reset_runtime_status_after_failure(self) -> None:
        self._status.setText("最近一次测试失败。")
        try:
            cfg = self._collect_cfg()
            self._balance_label.setText(self._initial_balance_text(cfg))
        except Exception:
            self._balance_label.setText("余额：未查询")

    def save_if_needed(self) -> None:
        if self.is_testing():
            raise _ConfigValidationError(f"{self._title} 测试仍在进行中，请等待完成。")
        cfg = self._collect_cfg()
        cfg_key = self._cfg_key(cfg)
        if cfg.is_ready() and cfg_key != self._tested_key:
            raise _ConfigValidationError(f"{self._title} 配置已修改，请先点击“统一测试”并通过。")
        self._cfg = cfg
        self._save_env_values(cfg)
        saved_models = save_available_models(self._provider_key, self._available_models)
        if saved_models != self._available_models:
            self._available_models = saved_models
            self._refresh_model_table()
        self._save_fn(cfg)
        self.modelListChanged.emit()

    def _collect_cfg(self):
        try:
            timeout_s = float(self._timeout_edit.text().strip() or "0")
        except Exception as e:
            raise _ConfigValidationError(f"{self._title} 的超时(秒)需要是数字。") from e
        if timeout_s <= 0:
            raise _ConfigValidationError(f"{self._title} 的超时(秒)必须大于 0。")
        kwargs = dict(
            base_url=self._base_url_edit.text().strip(),
            api_key=self._api_key_edit.text().strip(),
            number_model=getattr(self._cfg, "number_model", "").strip(),
            model=self._cfg.model.strip(),
            timeout_s=timeout_s,
        )
        if isinstance(self._cfg, QwenConfig):
            kwargs["account_access_key_id"] = (
                self._account_access_key_id_edit.text().strip() if self._account_access_key_id_edit else ""
            )
            kwargs["account_access_key_secret"] = (
                self._account_access_key_secret_edit.text().strip() if self._account_access_key_secret_edit else ""
            )
        if isinstance(self._cfg, DeepSeekConfig):
            kwargs["analysis_model"] = self._cfg.analysis_model.strip()
        cfg = self._cfg_type(**kwargs)
        if not cfg.base_url or not cfg.number_model or not cfg.model:
            raise _ConfigValidationError(f"{self._title} 的 Base URL 不能为空，项目配置中的模型选择也不能为空。")
        return cfg

    def _save_env_values(self, cfg) -> None:
        if isinstance(cfg, DeepSeekConfig):
            sync_deepseek_runtime_env(cfg)
            return
        if isinstance(cfg, KimiConfig):
            sync_kimi_runtime_env(cfg)
            return
        if isinstance(cfg, QwenConfig):
            sync_qwen_runtime_env(cfg)

    def _cfg_key(self, cfg) -> str:
        parts = [cfg.base_url.strip(), cfg.api_key.strip(), getattr(cfg, "number_model", "").strip(), cfg.model.strip()]
        if hasattr(cfg, "analysis_model"):
            parts.append(getattr(cfg, "analysis_model").strip())
        parts.append(str(float(cfg.timeout_s)))
```

```python
        return "|".join(parts)

    def _reset_tested(self, *_args) -> None:
        self._tested_key = ""
        self._status.setText("配置已变更，需重新测试。")
        try:
            cfg = self._collect_cfg()
        except Exception:
            self._balance_label.setText("余额：未查询")
            return
        self._balance_label.setText(self._initial_balance_text(cfg))

    def is_testing(self) -> bool:
        return False

    def _initial_balance_text(self, cfg) -> str:
        if isinstance(cfg, QwenConfig):
            if cfg.has_account_balance_credentials():
                return "余额：待查询"
            if cfg.is_ready():
                return "余额：未配置阿里云 AccessKey，暂无法查询"
            return "余额：未配置"
        if cfg.is_ready():
            return "余额：待查询"
        return "余额：未配置"

    def _refresh_model_table(self) -> None:
        self._model_table.setRowCount(len(self._available_models))
        for row, model_name in enumerate(self._available_models):
            self._model_table.setItem(row, 0, QTableWidgetItem(model_name))

    def _selected_model_row(self) -> int:
        row = self._model_table.currentRow()
        if row >= 0:
            return row
        items = self._model_table.selectedItems()
        if items:
            return items[0].row()
        return -1

    def _prompt_model_name(self, *, title: str, initial_text: str = "") -> str | None:
        text, ok = QInputDialog.getText(self, title, "模型名：", text=initial_text)
        if not ok:
            return None
        model_name = str(text or "").strip()
        if not model_name:
            _show_message_box(self, title="模型名为空", text="模型名不能为空。", icon=QMessageBox.Icon.Warning)
            return None
        return model_name

    def _on_add_model(self) -> None:
        model_name = self._prompt_model_name(title=f"增加 {self._title} 模型")
        if model_name is None:
            return
        existing = {item.lower() for item in self._available_models}
        if model_name.lower() in existing:
            _show_message_box(self, title="模型已存在", text=f"{model_name} 已在可用模型列表中。", icon=QMessageBox.Icon.Warning)
            return
        self._available_models.append(model_name)
        self._refresh_model_table()
        self._model_table.selectRow(len(self._available_models) - 1)
        self.modelListChanged.emit()

    def _on_edit_model(self) -> None:
        row = self._selected_model_row()
        if row < 0 or row >= len(self._available_models):
            _show_message_box(self, title="未选择模型", text="请先在可用模型列表中选择一行。", icon=QMessageBox.Icon.Warning)
            return
```

```python
        original = self._available_models[row]
        model_name = self._prompt_model_name(title=f"修改 {self._title} 模型", initial_text=original)
        if model_name is None:
            return
        existing = {
            item.lower()
            for index, item in enumerate(self._available_models)
            if index != row
        }
        if model_name.lower() in existing:
            _show_message_box(self, title="模型已存在", text=f"{model_name} 已在可用模型列表中。", icon=QMessageBox.Icon.Warning)
            return
        self._available_models[row] = model_name
        self._refresh_model_table()
        self._model_table.selectRow(row)
        self.modelListChanged.emit()


def _model_candidates(title: str) -> list[str]:
    return load_available_models(_provider_key_from_title(title))


def _number_llm_config_for_cfg(cfg):
    if isinstance(cfg, DeepSeekConfig):
        return to_question_number_llm_config(cfg)
    if isinstance(cfg, KimiConfig):
        return to_kimi_question_number_llm_config(cfg)
    return to_qwen_question_number_llm_config(cfg)


def _masked_env_status(exists: bool, env_name: str) -> str:
    if exists:
        return f"已从环境变量 {env_name} 读取"
    return f"未设置环境变量 {env_name}"
```

sj_generator/presentation/qt/compare_highlight.py

046-01

```python
from __future__ import annotations


def compare_highlight_model_styles(
    *,
    model_sigs: dict[str, str],
    round_no: int,
    round_matched_count: int,
) -> dict[str, str]:
    del round_no
    non_empty = {key: sig for key, sig in model_sigs.items() if sig}
    empty_keys = {key for key, sig in model_sigs.items() if not sig}

    if len(non_empty) == len(model_sigs) and len(set(non_empty.values())) == 1:
        return {}

    if round_matched_count >= 2:
        freq: dict[str, int] = {}
        for sig in non_empty.values():
            freq[sig] = freq.get(sig, 0) + 1
        majority_sig = next((sig for sig, count in freq.items() if count >= 2), None)
        if majority_sig is None:
            return {
                key: ("yellow" if key in empty_keys else "red")
                for key in model_sigs.keys()
            }
        styles: dict[str, str] = {key: "yellow" for key in empty_keys}
        for key, sig in non_empty.items():
            if sig != majority_sig:
                styles[key] = "red"
        return styles

    styles = {key: "yellow" for key in empty_keys}
    if len(non_empty) >= 2 and len(set(non_empty.values())) == len(non_empty):
        for key in non_empty.keys():
            styles[key] = "red"
    return styles

```

sj_generator/presentation/qt/constants.py

047-01

```python
PAGE_INTRO = 99
PAGE_WELCOME = 100
PAGE_AI_SELECT = 3
PAGE_AI_IMPORT = 4
PAGE_AI_IMPORT_CONTENT = 5
PAGE_DEDUPE_RESULT = 8
PAGE_AI_ANALYSIS = 10
PAGE_IMPORT_SUCCESS = 11

DEFAULT_WINDOW_WIDTH = 976
DEFAULT_WINDOW_HEIGHT = 575
QT_MAX_WINDOW_SIZE = 16777215

```

```python
from __future__ import annotations

from PyQt6.QtCore import Qt
from PyQt6.QtWidgets import (
    QDialog,
    QDialogButtonBox,
    QHeaderView,
    QHBoxLayout,
    QLabel,
    QMessageBox,
    QPushButton,
    QTableWidgetItem,
    QVBoxLayout,
)

from sj_generator.application.settings.import_cost_history import clear_import_cost_history, load_import_cost_history_rows
from sj_generator.presentation.qt.table_copy import CopyableTableWidget

BUTTON_MIN_WIDTH = 96
BUTTON_MIN_HEIGHT = 36


def _style_dialog_button(button, text: str | None = None) -> None:
    if button is None:
        return
    if text:
        button.setText(text)
    button.setMinimumSize(BUTTON_MIN_WIDTH, BUTTON_MIN_HEIGHT)


def _style_message_box_buttons(box: QMessageBox) -> None:
    for button_type, text in (
        (QMessageBox.StandardButton.Yes, "是"),
        (QMessageBox.StandardButton.No, "否"),
        (QMessageBox.StandardButton.Ok, "确定"),
        (QMessageBox.StandardButton.Cancel, "取消"),
    ):
        _style_dialog_button(box.button(button_type), text)


class ImportCostHistoryDialog(QDialog):
    def __init__(self, parent=None) -> None:
        super().__init__(parent)
        self.setWindowTitle("余额日志")
        self.resize(760, 420)

        self._hint = QLabel("")
        self._hint.setWordWrap(True)

        self._table = CopyableTableWidget()
        self._table.setColumnCount(4)
        self._table.setHorizontalHeaderLabels(["运行日期时间", "DeepSeek", "Kimi", "千问"])
        self._table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
        self._table.verticalHeader().setVisible(False)
        self._table.setEditTriggers(CopyableTableWidget.EditTrigger.NoEditTriggers)
        self._table.setWordWrap(True)

        self._clear_btn = QPushButton("清空日志")
        self._clear_btn.clicked.connect(self._on_clear_logs)
        _style_dialog_button(self._clear_btn, "清空日志")

        buttons = QDialogButtonBox(QDialogButtonBox.StandardButton.Close)
        buttons.rejected.connect(self.reject)
        buttons.accepted.connect(self.accept)
        _style_dialog_button(buttons.button(QDialogButtonBox.StandardButton.Close), "关闭")

        layout = QVBoxLayout()
        layout.addWidget(self._hint)
        layout.addWidget(self._table, 1)
```

```python
        button_row = QHBoxLayout()
        button_row.addWidget(self._clear_btn)
        button_row.addStretch(1)
        button_row.addWidget(buttons)
        layout.addLayout(button_row)
        self.setLayout(layout)

        self._reload_rows()

    def _on_clear_logs(self) -> None:
        box = QMessageBox(self)
        box.setWindowTitle("确认清空")
        box.setText("确定要清空全部余额日志吗？")
        box.setIcon(QMessageBox.Icon.Warning)
        box.setStandardButtons(QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No)
        box.setDefaultButton(QMessageBox.StandardButton.No)
        _style_message_box_buttons(box)
        if box.exec() != QMessageBox.StandardButton.Yes:
            return
        clear_import_cost_history()
        self._reload_rows()

    def _reload_rows(self) -> None:
        rows = load_import_cost_history_rows(limit=200)
        if not rows:
            self._hint.setText("当前还没有余额记录。")
            self._table.setRowCount(0)
            return
        self._hint.setText(f"共 {len(rows)} 条余额记录。")
        self._table.setRowCount(len(rows))
        for row_index, row in enumerate(rows):
            values = [
                str(row.get("run_at") or ""),
                str(row.get("deepseek_balance") or ""),
                str(row.get("kimi_balance") or ""),
                str(row.get("qwen_balance") or ""),
            ]
            for col_index, value in enumerate(values):
                item = self._table.item(row_index, col_index)
                if item is None:
                    item = QTableWidgetItem()
                    self._table.setItem(row_index, col_index, item)
                item.setText(value)
                alignment = Qt.AlignmentFlag.AlignHCenter | Qt.AlignmentFlag.AlignVCenter
                item.setTextAlignment(int(alignment))
        self._table.resizeRowsToContents()

```

```python
from __future__ import annotations

from decimal import Decimal
from datetime import datetime
import threading

from sj_generator.application.settings.import_cost_history import append_import_cost_history_entry, build_total_balance_text
from sj_generator.infrastructure.llm.import_questions import question_content_model_specs
from sj_generator.infrastructure.llm.question_ref_scan import question_ref_model_specs
from sj_generator.infrastructure.llm.balance import query_provider_balance_snapshots
from sj_generator.application.settings import load_deepseek_config, load_kimi_config, load_qwen_config
from sj_generator.application.state import ImportWizardSession

_APP_IMPORT_COST_LOCK = threading.Lock()
_APP_IMPORT_COST_THREAD: threading.Thread | None = None
_APP_IMPORT_COST_SNAPSHOTS: list = []
_APP_IMPORT_COST_LOADING = False
_APP_IMPORT_COST_STARTED = False
_APP_IMPORT_COST_STARTUP_LOGGED = False


def _format_cost_amount_text(currency: str, amount: Decimal) -> str:
    prefix = {"CNY": "￥", "USD": "$"}.get((currency or "").upper(), "")
    if prefix:
        return f"{prefix}{amount:.4f}"
    return f"{(currency or 'UNKNOWN').upper()} {amount:.4f}"


def _query_import_cost_before_snapshots() -> list:
    return query_provider_balance_snapshots(
        deepseek_cfg=load_deepseek_config(),
        kimi_cfg=load_kimi_config(),
        qwen_cfg=load_qwen_config(),
    )


def begin_app_import_cost_capture(*, retry_if_empty: bool = False) -> None:
    global _APP_IMPORT_COST_THREAD, _APP_IMPORT_COST_LOADING, _APP_IMPORT_COST_STARTED, _APP_IMPORT_COST_SNAPSHOTS
    with _APP_IMPORT_COST_LOCK:
        has_snapshots = bool(_APP_IMPORT_COST_SNAPSHOTS)
        if _APP_IMPORT_COST_LOADING:
            return
        if _APP_IMPORT_COST_STARTED and (has_snapshots or not retry_if_empty):
            return
        _APP_IMPORT_COST_LOADING = True
        _APP_IMPORT_COST_STARTED = True
        _APP_IMPORT_COST_SNAPSHOTS = []

        def worker() -> None:
            global _APP_IMPORT_COST_THREAD, _APP_IMPORT_COST_LOADING, _APP_IMPORT_COST_SNAPSHOTS, _APP_IMPORT_COST_STARTUP_LOGGED
            try:
                snapshots = _query_import_cost_before_snapshots()
            except Exception:
                snapshots = []
            should_append_startup_log = False
            with _APP_IMPORT_COST_LOCK:
                _APP_IMPORT_COST_SNAPSHOTS = list(snapshots)
                _APP_IMPORT_COST_LOADING = False
                _APP_IMPORT_COST_THREAD = None
                if snapshots and not _APP_IMPORT_COST_STARTUP_LOGGED:
                    _APP_IMPORT_COST_STARTUP_LOGGED = True
                    should_append_startup_log = True
            if should_append_startup_log:
                _append_startup_balance_history(snapshots)

        thread = threading.Thread(target=worker, name="app-import-cost-before", daemon=True)
        _APP_IMPORT_COST_THREAD = thread
```

```python
    thread.start()


def _set_state_import_cost_before_snapshots(state: ImportWizardSession, snapshots: list) -> None:
    state.execution.import_cost_before_amounts = {
        snapshot.provider: f"{snapshot.currency}|{snapshot.amount}"
        for snapshot in snapshots
    }
    state.execution.import_cost_before_details = {snapshot.provider: snapshot.detail for snapshot in snapshots}


def _apply_app_import_cost_capture_to_state(state: ImportWizardSession) -> None:
    with _APP_IMPORT_COST_LOCK:
        snapshots = list(_APP_IMPORT_COST_SNAPSHOTS)
        loading = bool(_APP_IMPORT_COST_LOADING)
        thread = _APP_IMPORT_COST_THREAD
    if loading:
        state.execution.import_cost_before_loading = True
        state.execution.import_cost_capture_thread = thread
        return
    _set_state_import_cost_before_snapshots(state, snapshots)
    state.execution.import_cost_before_loading = False
    state.execution.import_cost_capture_thread = None


def _wait_app_import_cost_capture() -> None:
    with _APP_IMPORT_COST_LOCK:
        thread = _APP_IMPORT_COST_THREAD
    if thread is None:
        return
    if thread is threading.current_thread():
        return
    thread.join()


def _update_app_import_cost_snapshots(snapshots: list) -> None:
    global _APP_IMPORT_COST_SNAPSHOTS, _APP_IMPORT_COST_LOADING, _APP_IMPORT_COST_THREAD, _APP_IMPORT_COST_STARTED
    with _APP_IMPORT_COST_LOCK:
        _APP_IMPORT_COST_SNAPSHOTS = list(snapshots)
        _APP_IMPORT_COST_LOADING = False
        _APP_IMPORT_COST_THREAD = None
        _APP_IMPORT_COST_STARTED = True


def _append_startup_balance_history(snapshots: list) -> None:
    provider_balances = _build_after_provider_balances(snapshots)
    if not provider_balances:
        return
    append_import_cost_history_entry(
        run_at=datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        source_label="启动",
        provider_balances=provider_balances,
    )


def capture_import_cost_before(state: ImportWizardSession) -> None:
    state.execution.reset_import_cost_tracking()
    begin_app_import_cost_capture()
    _wait_app_import_cost_capture()
    _apply_app_import_cost_capture_to_state(state)


def capture_import_cost_before_async(state: ImportWizardSession) -> None:
    state.execution.reset_import_cost_tracking()
    begin_app_import_cost_capture()
    _apply_app_import_cost_capture_to_state(state)


```

```python
def wait_import_cost_before_capture(state: ImportWizardSession) -> None:
    _wait_app_import_cost_capture()
    _apply_app_import_cost_capture_to_state(state)

def freeze_import_cost_result(state: ImportWizardSession) -> None:
    if state.execution.import_cost_ready:
        return
    state.execution.import_cost_ready = True
    begin_app_import_cost_capture()
    wait_import_cost_before_capture(state)
    state.execution.import_cost_total_text = ""
    state.execution.import_cost_summary_text = ""
    state.execution.import_cost_detail_text = ""
    used_models = _collect_import_used_models()
    before_amounts = dict(getattr(state.execution, "import_cost_before_amounts", {}) or {})
    after_snapshots = query_provider_balance_snapshots(
        deepseek_cfg=load_deepseek_config(),
        kimi_cfg=load_kimi_config(),
        qwen_cfg=load_qwen_config(),
    )
    if not before_amounts:
        state.execution.import_cost_detail_text = "未记录导入前余额，无法计算本次 docx 解析费用。"
        state.execution.import_cost_rows = _build_unavailable_rows(
            used_models,
            before_text="未记录",
            current_text="未查询" if not after_snapshots else "已记录到下方日志",
            cost_text="无法计算",
        )
        _append_import_cost_history(
            before_amounts,
            after_snapshots,
            total_cost="",
            cost_summary="",
            cost_detail=state.execution.import_cost_detail_text,
        )
        if after_snapshots:
            _update_app_import_cost_snapshots(after_snapshots)
        return
    if not after_snapshots:
        state.execution.import_cost_detail_text = "未获取到导入后余额，无法计算本次 docx 解析费用。"
        state.execution.import_cost_rows = _build_unavailable_rows(
            used_models,
            before_text="已记录",
            current_text="未获取",
            cost_text="无法计算",
        )
        _append_import_cost_history(
            before_amounts,
            after_snapshots,
            total_cost="",
            cost_summary="",
            cost_detail=state.execution.import_cost_detail_text,
        )
        return
    after_map = {snapshot.provider: snapshot for snapshot in after_snapshots}
    deltas: list[tuple[str, str, Decimal]] = []
    detail_lines: list[str] = []
    provider_labels = {"deepseek": "DeepSeek", "kimi": "Kimi", "qwen": "千问"}
    rows: list[dict[str, str]] = []
    for provider, models in used_models.items():
        snapshot = after_map.get(provider)
        raw_before = str(before_amounts.get(provider) or "").strip()
        if "|" not in raw_before:
            rows.append(
                {
                    "provider": provider_labels.get(provider, provider),
                    "models": "\n".join(models) if models else "-",
                    "before_balance": "未记录",
                    "current_balance": snapshot.detail if snapshot is not None else "未查询",
                    "cost": "无法计算",
```

```
                }
            )
            continue
        before_currency, _, before_amount_text = raw_before.partition("|")
        if snapshot is None:
            rows.append(
                {
                    "provider": provider_labels.get(provider, provider),
                    "models": "\n".join(models) if models else "-",
                    "before_balance": _format_cost_amount_text(before_currency.strip().upper() or "CNY", Decimal(before_amount_text.strip() or "0")),
                    "current_balance": "未查询",
                    "cost": "无法计算",
                }
            )
            continue
        before_currency = before_currency.strip().upper() or snapshot.currency
        try:
            before_amount = Decimal(before_amount_text.strip())
        except Exception:
            rows.append(
                {
                    "provider": provider_labels.get(provider, provider),
                    "models": "\n".join(models) if models else "-",
                    "before_balance": "格式异常",
                    "current_balance": snapshot.detail,
                    "cost": "无法计算",
                }
            )
            continue
        if before_currency != snapshot.currency:
            rows.append(
                {
                    "provider": provider_labels.get(provider, provider),
                    "models": "\n".join(models) if models else "-",
                    "before_balance": _format_cost_amount_text(before_currency, before_amount),
                    "current_balance": _format_cost_amount_text(snapshot.currency, snapshot.amount),
                    "cost": "币种不一致",
                }
            )
            continue
        delta = before_amount - snapshot.amount
        if delta < Decimal("0"):
            delta = Decimal("0")
        deltas.append((provider, snapshot.currency, delta))
        detail_lines.append(f"{provider_labels.get(provider, provider)}：{_format_cost_amount_text(snapshot.currency, delta)}")
        rows.append(
            {
                "provider": provider_labels.get(provider, provider),
                "models": "\n".join(models) if models else "-",
                "before_balance": _format_cost_amount_text(snapshot.currency, before_amount),
                "current_balance": _format_cost_amount_text(snapshot.currency, snapshot.amount),
                "cost": _format_cost_amount_text(snapshot.currency, delta),
            }
        )
    state.execution.import_cost_rows = rows
    if not deltas:
        state.execution.import_cost_detail_text = "导入前后余额币种不一致或缺少可比较快照，无法计算本次 docx 解析费用。"
        _append_import_cost_history(
            before_amounts,
            after_snapshots,
            total_cost="",
            cost_summary="",
            cost_detail=state.execution.import_cost_detail_text,
        )
        _update_app_import_cost_snapshots(after_snapshots)
        return
    totals_by_currency: dict[str, Decimal] = {}
```

```python
    for _provider, currency, delta in deltas:
        totals_by_currency[currency] = totals_by_currency.get(currency, Decimal("0")) + delta
    non_zero_deltas = [item for item in deltas if item[2] > Decimal("0")]
    summary_parts = [
        f"{provider_labels.get(provider, provider)} {_format_cost_amount_text(currency, delta)}"
        for provider, currency, delta in non_zero_deltas
    ]
    total_parts = [
        _format_cost_amount_text(currency, amount)
        for currency, amount in sorted(totals_by_currency.items(), key=lambda item: item[0])
    ]
    state.execution.import_cost_total_text = "；".join(total_parts) if total_parts else "0"
    state.execution.import_cost_summary_text = "；".join(summary_parts) if summary_parts else "0"
    state.execution.import_cost_detail_text = "；".join(detail_lines)
    _append_import_cost_history(
        before_amounts,
        after_snapshots,
        total_cost=state.execution.import_cost_total_text,
        cost_summary=state.execution.import_cost_summary_text,
        cost_detail=state.execution.import_cost_detail_text,
    )
    _update_app_import_cost_snapshots(after_snapshots)


def _collect_import_used_models() -> dict[str, list[str]]:
    provider_map: dict[str, list[str]] = {}
    seen: dict[str, set[str]] = {}

    def add(provider: str, model_name: str, stage_label: str) -> None:
        provider_key = str(provider or "").strip().lower()
        model_text = str(model_name or "").strip()
        if not provider_key or not model_text:
            return
        if provider_key not in provider_map:
            provider_map[provider_key] = []
            seen[provider_key] = set()
        value = f"{stage_label}：{model_text}"
        if value in seen[provider_key]:
            return
        seen[provider_key].add(value)
        provider_map[provider_key].append(value)

    for spec in question_ref_model_specs():
        add(str(spec.get("provider") or ""), str(spec.get("model_name") or ""), "题号题型")
    for spec in question_content_model_specs():
        add(str(spec.get("provider") or ""), str(spec.get("model_name") or ""), "题目内容")
    return provider_map


def _build_unavailable_rows(
    used_models: dict[str, list[str]],
    *,
    before_text: str,
    current_text: str,
    cost_text: str,
) -> list[dict[str, str]]:
    provider_labels = {"deepseek": "DeepSeek", "kimi": "Kimi", "qwen": "千问"}
    rows: list[dict[str, str]] = []
    for provider, models in used_models.items():
        rows.append(
            {
                "provider": provider_labels.get(provider, provider),
                "models": "\n".join(models) if models else "-",
                "before_balance": before_text,
                "current_balance": current_text,
                "cost": cost_text,
            }
        )
    return rows

```

```python

def _append_import_cost_history(
    before_amounts: dict[str, str],
    after_snapshots: list,
    *,
    total_cost: str,
    cost_summary: str,
    cost_detail: str,
) -> None:
    before_provider_balances = _build_before_provider_balances(before_amounts)
    provider_balances = _build_after_provider_balances(after_snapshots)
    append_import_cost_history_entry(
        run_at=datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        source_label="导入完成",
        before_provider_balances=before_provider_balances,
        provider_balances=provider_balances,
        total_balance=build_total_balance_text(provider_balances),
        total_cost=total_cost,
        cost_summary=cost_summary,
        cost_detail=cost_detail,
    )


def _build_after_provider_balances(after_snapshots: list) -> dict[str, str]:
    values: dict[str, str] = {}
    for snapshot in after_snapshots:
        provider = str(getattr(snapshot, "provider", "") or "").strip().lower()
        if not provider:
            continue
        currency = str(getattr(snapshot, "currency", "") or "").strip().upper() or "CNY"
        amount = getattr(snapshot, "amount", None)
        try:
            values[provider] = _format_cost_amount_text(currency, Decimal(str(amount)))
        except Exception:
            detail = str(getattr(snapshot, "detail", "") or "").strip()
            values[provider] = detail
    return values


def _build_before_provider_balances(before_amounts: dict[str, str]) -> dict[str, str]:
    values: dict[str, str] = {}
    for provider, raw_value in dict(before_amounts or {}).items():
        provider_key = str(provider or "").strip().lower()
        raw_text = str(raw_value or "").strip()
        if not provider_key or "|" not in raw_text:
            continue
        currency, _, amount_text = raw_text.partition("|")
        try:
            values[provider_key] = _format_cost_amount_text(currency.strip().upper() or "CNY", Decimal(amount_text.strip()))
        except Exception:
            continue
    return values


def _reset_app_import_cost_capture_for_tests() -> None:
    global _APP_IMPORT_COST_THREAD, _APP_IMPORT_COST_LOADING, _APP_IMPORT_COST_STARTED, _APP_IMPORT_COST_SNAPSHOTS, _APP_IMPORT_COST_STARTUP_LOGGED
    with _APP_IMPORT_COST_LOCK:
        _APP_IMPORT_COST_THREAD = None
        _APP_IMPORT_COST_LOADING = False
        _APP_IMPORT_COST_STARTED = False
        _APP_IMPORT_COST_SNAPSHOTS = []
        _APP_IMPORT_COST_STARTUP_LOGGED = False

```

sj_generator/presentation/qt/import_flow.py

050-01

```python
from __future__ import annotations

from PyQt6.QtGui import QCloseEvent
from PyQt6.QtWidgets import QWizard

from sj_generator.application.state import ImportWizardSession
from sj_generator.presentation.qt.pages import register_import_flow_pages
from sj_generator.presentation.qt.constants import (
    DEFAULT_WINDOW_HEIGHT,
    DEFAULT_WINDOW_WIDTH,
    PAGE_AI_ANALYSIS,
    PAGE_AI_IMPORT,
    PAGE_AI_IMPORT_CONTENT,
    PAGE_AI_SELECT,
    PAGE_DEDUPE_RESULT,
    PAGE_IMPORT_SUCCESS,
    QT_MAX_WINDOW_SIZE,
)
from sj_generator.presentation.qt.wizard_base import AppWizardBase

BUTTON_MIN_WIDTH = 128
BUTTON_MIN_HEIGHT = 40


def configure_import_flow_pages(wizard: QWizard, state: ImportWizardSession) -> None:
    register_import_flow_pages(wizard, state)


class ImportFlowWizard(AppWizardBase):
    def __init__(self, state: ImportWizardSession, parent=None, *, launcher=None, start_page_id: int = PAGE_AI_SELECT) -> None:
        super().__init__(parent)
        self._state = state
        self._launcher = launcher
        self._start_page_id = start_page_id
        self._deferred_close_requested = False
        self.apply_button_texts(
            {
                QWizard.WizardButton.CustomButton1: "添加文档",
                QWizard.WizardButton.CustomButton2: "打开文档",
                QWizard.WizardButton.CustomButton3: "打开文档",
                QWizard.WizardButton.NextButton: "开始导题",
            }
        )
        self.setOption(QWizard.WizardOption.HaveCustomButton1, True)
        self.setOption(QWizard.WizardOption.HaveCustomButton2, True)
        self.setOption(QWizard.WizardOption.HaveCustomButton3, True)
        self.setButtonLayout(
            [
                QWizard.WizardButton.CustomButton1,
                QWizard.WizardButton.CustomButton2,
                QWizard.WizardButton.CustomButton3,
                QWizard.WizardButton.Stretch,
                QWizard.WizardButton.NextButton,
            ]
        )
        configure_import_flow_pages(self, self._state)
        self.cache_and_hide_page_titles()
        self.setStartId(start_page_id)
        self.currentIdChanged.connect(self.update_window_title)
        self.currentIdChanged.connect(self._sync_window_resizability)
        self.currentIdChanged.connect(self._style_navigation_buttons)
        self.currentIdChanged.connect(self._sync_page_navigation_buttons)
        self.update_window_title(self.startId())
        self._sync_window_resizability(self.startId())
        self._style_navigation_buttons(self.startId())
        self._sync_page_navigation_buttons(self.startId())
        self.resize(DEFAULT_WINDOW_WIDTH, DEFAULT_WINDOW_HEIGHT)

    def _sync_window_resizability(self, _page_id: int) -> None:
```

```python
        self.setMinimumSize(0, 0)
        self.setMaximumSize(QT_MAX_WINDOW_SIZE, QT_MAX_WINDOW_SIZE)

    def _style_navigation_buttons(self, _page_id: int) -> None:
        for which in (
            QWizard.WizardButton.CustomButton1,
            QWizard.WizardButton.CustomButton2,
            QWizard.WizardButton.CustomButton3,
            QWizard.WizardButton.NextButton,
        ):
            button = self.button(which)
            if button is None:
                continue
            button.setMinimumSize(BUTTON_MIN_WIDTH, BUTTON_MIN_HEIGHT)
            button.setStyleSheet("padding: 6px 18px;")

    def _sync_page_navigation_buttons(self, _page_id: int) -> None:
        page = self.currentPage()
        sync = getattr(page, "_sync_wizard_buttons", None)
        if callable(sync):
            sync()
        button = self.button(QWizard.WizardButton.CustomButton1)
        if button is not None:
            button.setVisible(False)
        button2 = self.button(QWizard.WizardButton.CustomButton2)
        if button2 is not None:
            button2.setVisible(False)
        button3 = self.button(QWizard.WizardButton.CustomButton3)
        if button3 is not None:
            button3.setVisible(False)
        back_button = self.button(QWizard.WizardButton.BackButton)
        if back_button is not None:
            back_button.setVisible(False)
        cancel_button = self.button(QWizard.WizardButton.CancelButton)
        if cancel_button is not None:
            cancel_button.setVisible(False)
        finish_button = self.button(QWizard.WizardButton.FinishButton)
        if finish_button is not None:
            finish_button.setVisible(False)
        handler = getattr(page, "_sync_custom_wizard_button", None)
        if callable(handler):
            handler()

    def back(self) -> None:
        # 导入向导进入下一页后不允许回退到上一步。
        return

    def accept(self) -> None:
        super().accept()

    def reject(self) -> None:
        if not self._can_close_current_page():
            self._deferred_close_requested = True
            self.hide()
            return
        super().reject()

    def closeEvent(self, event: QCloseEvent) -> None:
        if not self._can_close_current_page():
            self._deferred_close_requested = True
            self.hide()
            event.ignore()
            return
        self._deferred_close_requested = False
        super().closeEvent(event)

    def _can_close_current_page(self) -> bool:
        page_ids = (
            PAGE_AI_SELECT,
            PAGE_AI_IMPORT,
```

```python
            PAGE_AI_IMPORT_CONTENT,
            PAGE_DEDUPE_RESULT,
            PAGE_AI_ANALYSIS,
            PAGE_IMPORT_SUCCESS,
        )
        for page_id in page_ids:
            page = self.page(page_id)
            guard = getattr(page, "prepare_to_close", None)
            if callable(guard) and not bool(guard()):
                return False
        return True

    def open_additional_documents(self, source_paths, *, message_parent=None) -> bool:
        handler = getattr(self._launcher, "_open_import_flow_windows", None)
        if not callable(handler):
            return False
        return bool(handler(source_paths, message_parent=message_parent))

    def open_split_import_flow_windows(self, states, *, start_page_id: int) -> bool:
        handler = getattr(self._launcher, "_open_split_import_flow_windows", None)
        if not callable(handler):
            return False
        return bool(handler(states, start_page_id=start_page_id))

```

```python
import sys

from PyQt6.QtCore import QTimer, Qt
from PyQt6.QtGui import QIcon
from PyQt6.QtWidgets import QApplication, QWizard

from sj_generator.application.settings import load_program_settings
from sj_generator.application.state import (
    WizardState,
    normalize_ai_concurrency,
    normalize_analysis_model_name,
    normalize_analysis_provider,
    normalize_default_repo_parent_dir_text,
    normalize_export_convertible_multi_mode,
    normalize_export_include_answers,
    normalize_export_include_analysis,
    normalize_import_source_dir_text,
    normalize_preferred_textbook_version,
)
from sj_generator.presentation.qt.import_costs import begin_app_import_cost_capture
from sj_generator.presentation.qt.styles import APP_STYLESHEET
from sj_generator.presentation.qt.constants import (
    DEFAULT_WINDOW_HEIGHT,
    DEFAULT_WINDOW_WIDTH,
    PAGE_INTRO,
    PAGE_WELCOME,
    QT_MAX_WINDOW_SIZE,
)
from sj_generator.presentation.qt.pages.intro_page import IntroPage
from sj_generator.presentation.qt.pages.welcome_page import WelcomePage
from sj_generator.presentation.qt.wizard_base import AppWizardBase
from sj_generator.shared.paths import app_paths


class GeneratorWizard(AppWizardBase):
    def __init__(self) -> None:
        super().__init__()
        self.apply_button_texts(
            {
                QWizard.WizardButton.BackButton: "上一步",
                QWizard.WizardButton.NextButton: "下一步",
                QWizard.WizardButton.CancelButton: "取消",
                QWizard.WizardButton.FinishButton: "完成",
            }
        )

        self._state = WizardState()
        self._welcome_page_loaded = False
        self._apply_saved_program_settings()
        begin_app_import_cost_capture()
        QTimer.singleShot(0, lambda: begin_app_import_cost_capture(retry_if_empty=True))
        self.setPage(PAGE_INTRO, IntroPage())
        self.cache_and_hide_page_titles()
        self.setStartId(PAGE_INTRO)
        self.currentIdChanged.connect(self.update_window_title)
        self.currentIdChanged.connect(self._sync_navigation_buttons)
        self.currentIdChanged.connect(self._sync_window_resizability)
        self.update_window_title(self.startId())
        self._sync_navigation_buttons(self.startId())
        self._sync_window_resizability(self.startId())

    def next(self) -> None:
        if self.currentId() == PAGE_INTRO:
            self._ensure_welcome_page_loaded()
        super().next()

    def _sync_navigation_buttons(self, page_id: int) -> None:
        show_nav = page_id not in (PAGE_INTRO, PAGE_WELCOME)
        self.setButtonLayout([] if not show_nav else self._default_button_layout)
        for which in (
```

```python
            QWizard.WizardButton.BackButton,
            QWizard.WizardButton.NextButton,
            QWizard.WizardButton.CancelButton,
            QWizard.WizardButton.FinishButton,
        ):
            button = self.button(which)
            if button is not None:
                button.setVisible(show_nav)

    def _sync_window_resizability(self, page_id: int) -> None:
        if page_id == PAGE_INTRO:
            self.resize(DEFAULT_WINDOW_WIDTH, DEFAULT_WINDOW_HEIGHT)
            self.setFixedSize(DEFAULT_WINDOW_WIDTH, DEFAULT_WINDOW_HEIGHT)
            return

        self.setMinimumSize(0, 0)
        self.setMaximumSize(QT_MAX_WINDOW_SIZE, QT_MAX_WINDOW_SIZE)

    def _apply_saved_program_settings(self) -> None:
        data = load_program_settings()
        legacy_ai_concurrency = normalize_ai_concurrency(data.get("ai_concurrency"))
        self._state.default_repo_parent_dir_text = normalize_default_repo_parent_dir_text(
            data.get("default_repo_parent_dir_text")
        )
        self._state.import_source_dir_text = normalize_import_source_dir_text(
            data.get("import_source_dir_text")
        )
        self._state.ai_concurrency = legacy_ai_concurrency
        self._state.question_content_concurrency = normalize_ai_concurrency(
            data.get("question_content_concurrency", legacy_ai_concurrency)
        )
        self._state.analysis_generation_concurrency = normalize_ai_concurrency(
            data.get("analysis_generation_concurrency", legacy_ai_concurrency)
        )
        self._state.analysis_enabled = bool(data.get("analysis_enabled", self._state.analysis_enabled))
        self._state.dedupe_enabled = bool(data.get("dedupe_enabled", self._state.dedupe_enabled))
        self._state.analysis_provider = normalize_analysis_provider(data.get("analysis_provider"))
        self._state.analysis_model_name = normalize_analysis_model_name(data.get("analysis_model_name"))
        self._state.export_convertible_multi_mode = normalize_export_convertible_multi_mode(
            data.get("export_convertible_multi_mode")
        )
        legacy_include = data.get("export_include_answers_and_analysis")
        self._state.export_include_answers = normalize_export_include_answers(
            data.get("export_include_answers", legacy_include)
        )
        self._state.export_include_analysis = normalize_export_include_analysis(
            data.get("export_include_analysis", legacy_include)
        )
        self._state.preferred_textbook_version = normalize_preferred_textbook_version(
            data.get("preferred_textbook_version")
        )

    def _ensure_welcome_page_loaded(self) -> None:
        begin_app_import_cost_capture(retry_if_empty=True)
        if self._welcome_page_loaded:
            return
        page = WelcomePage(self._state)
        self.setPage(PAGE_WELCOME, page)
        self.cache_and_hide_page_title(PAGE_WELCOME)
        self._welcome_page_loaded = True


def main() -> None:
    QApplication.setAttribute(Qt.ApplicationAttribute.AA_ShareOpenGLContexts, True)
    app = QApplication(sys.argv)
    app.setStyleSheet(APP_STYLESHEET)
    icon_path = app_paths().logo_path
    icon: QIcon | None = None
    if icon_path.exists():
        loaded_icon = QIcon(str(icon_path))
```

```
        if not loaded_icon.isNull():
            icon = loaded_icon
            app.setWindowIcon(icon)
    wizard = GeneratorWizard()
    if icon is not None:
        wizard.setWindowIcon(icon)
    wizard.resize(DEFAULT_WINDOW_WIDTH, DEFAULT_WINDOW_HEIGHT)
    wizard.show()
    raise SystemExit(app.exec())

```

sj_generator/presentation/qt/message_box.py

052-01

```python
from __future__ import annotations

from PyQt6.QtWidgets import QMessageBox, QWidget

MESSAGE_BOX_BUTTON_MIN_WIDTH = 96
MESSAGE_BOX_BUTTON_MIN_HEIGHT = 36


def style_message_box_buttons(box: QMessageBox) -> None:
    for button_type, text in (
        (QMessageBox.StandardButton.Ok, "确定"),
        (QMessageBox.StandardButton.Cancel, "取消"),
        (QMessageBox.StandardButton.Yes, "是"),
        (QMessageBox.StandardButton.No, "否"),
    ):
        button = box.button(button_type)
        if button is None:
            continue
        button.setText(text)
        button.setMinimumSize(MESSAGE_BOX_BUTTON_MIN_WIDTH, MESSAGE_BOX_BUTTON_MIN_HEIGHT)


def show_message_box(
    parent: QWidget | None,
    *,
    title: str,
    text: str,
    icon: QMessageBox.Icon,
    buttons: QMessageBox.StandardButton = QMessageBox.StandardButton.Ok,
    default_button: QMessageBox.StandardButton = QMessageBox.StandardButton.NoButton,
) -> QMessageBox.StandardButton:
    box = QMessageBox(parent)
    box.setWindowTitle(title)
    box.setText(text)
    box.setIcon(icon)
    box.setStandardButtons(buttons)
    if default_button != QMessageBox.StandardButton.NoButton:
        box.setDefaultButton(default_button)
    style_message_box_buttons(box)
    return QMessageBox.StandardButton(box.exec())

```

sj_generator/presentation/qt/pages/__init__.py

053-01

```python
from .intro_page import IntroPage
from .welcome_page import WelcomePage
from .import_flow_registry import build_import_flow_pages, register_import_flow_pages

__all__ = [
    "IntroPage",
    "WelcomePage",
    "build_import_flow_pages",
    "register_import_flow_pages",
]

```

sj_generator/presentation/qt/pages/import_flow/__init__.py


```python
from .analysis_page import AiAnalysisPage
from .content_page import AiImportContentPage
from .dedupe_page import DedupeResultPage
from .question_ref_page import AiImportPage
from .select_page import AiSelectFilesPage
from .success_page import ImportSuccessPage

__all__ = [
    "AiSelectFilesPage",
    "AiImportPage",
    "AiImportContentPage",
    "DedupeResultPage",
    "AiAnalysisPage",
    "ImportSuccessPage",
]

```

```python
import copy
import time
from pathlib import Path

from PyQt6.QtCore import QObject, QThread, QTimer, pyqtSignal
from PyQt6.QtWidgets import (
    QHeaderView,
    QHBoxLayout,
    QLabel,
    QMessageBox,
    QPushButton,
    QProgressBar,
    QTableWidgetItem,
    QVBoxLayout,
    QWizard,
    QWizardPage,
)

from sj_generator.application.settings import (
    load_deepseek_config,
    load_kimi_config,
    load_qwen_config,
)
from sj_generator.application.state import (
    ImportWizardSession,
    normalize_ai_concurrency,
    normalize_analysis_model_name,
    normalize_analysis_provider,
)
from sj_generator.domain.entities import Question
from sj_generator.infrastructure.llm.client import LlmClient, LlmConfig
from sj_generator.infrastructure.llm.explanations import (
    ExplanationInputs,
    generate_explanation_result,
)
from sj_generator.infrastructure.llm.task_runner import run_tasks_in_parallel
from sj_generator.shared.paths import app_paths
from sj_generator.presentation.qt.constants import PAGE_IMPORT_SUCCESS
from sj_generator.presentation.qt.pages.import_flow.import_db_service import commit_draft_questions_to_db
from sj_generator.presentation.qt.table_copy import CopyableTableWidget

COL_NUMBER = 0
COL_STEM = 1
COL_OPTIONS = 2
COL_ANSWER = 3
COL_CHOICE_SUMMARY = 4
COL_ANALYSIS = 5
COL_ELAPSED = 6
BUTTON_MIN_WIDTH = 96
BUTTON_MIN_HEIGHT = 36


def _style_dialog_button(button: QPushButton | None, text: str | None = None) -> None:
    if button is None:
        return
    if text:
        button.setText(text)
    button.setMinimumSize(BUTTON_MIN_WIDTH, BUTTON_MIN_HEIGHT)


def _style_message_box_buttons(box: QMessageBox) -> None:
    for button_type, text in (
        (QMessageBox.StandardButton.Ok, "确定"),
        (QMessageBox.StandardButton.Cancel, "取消"),
        (QMessageBox.StandardButton.Yes, "是"),
        (QMessageBox.StandardButton.No, "否"),
    ):
        _style_dialog_button(box.button(button_type), text)


```

```python
def _show_message_box(
    parent,
    *,
    title: str,
    text: str,
    icon: QMessageBox.Icon,
    buttons: QMessageBox.StandardButton = QMessageBox.StandardButton.Ok,
    default_button: QMessageBox.StandardButton = QMessageBox.StandardButton.NoButton,
) -> QMessageBox.StandardButton:
    box = QMessageBox(parent)
    box.setWindowTitle(title)
    box.setText(text)
    box.setIcon(icon)
    box.setStandardButtons(buttons)
    if default_button != QMessageBox.StandardButton.NoButton:
        box.setDefaultButton(default_button)
    _style_message_box_buttons(box)
    return QMessageBox.StandardButton(box.exec())


def _analysis_provider_label(provider: str) -> str:
    labels = {"deepseek": "DeepSeek", "kimi": "Kimi", "qwen": "千问"}
    return labels.get(normalize_analysis_provider(provider), "DeepSeek")


def _analysis_target_text(provider: str, model_name: str) -> str:
    return f"{_analysis_provider_label(provider)} / {normalize_analysis_model_name(model_name)}"


def _digits_to_circled(text: str) -> str:
    return "".join(
        {
            "1": "①",
            "2": "②",
            "3": "③",
            "4": "④",
            "5": "⑤",
            "6": "⑥",
            "7": "⑦",
            "8": "⑧",
            "9": "⑨",
        }.get(ch, ch)
        for ch in text
    )


def _format_choice_summary(question: Question) -> str:
    lines = [
        f"{letter}：{_digits_to_circled(value.strip())}"
        for letter, value in (
            ("A", question.choice_1),
            ("B", question.choice_2),
            ("C", question.choice_3),
            ("D", question.choice_4),
        )
        if value.strip()
    ]
    return "\n".join(lines)


class AiAnalysisPage(QWizardPage):
    def __init__(self, state: ImportWizardSession) -> None:
        super().__init__()
        self._state = state
        self.setTitle("AI 生成解析")

        self._status_label = QLabel("")
        self._status_label.setWordWrap(True)
        self._current_label = QLabel("")
        self._current_label.setWordWrap(True)
```

```python

        self._table = CopyableTableWidget()
        self._table.setColumnCount(7)
        self._table.setHorizontalHeaderLabels(["编号", "题目", "选项", "答案", "组合属性", "解析", "解析用时"])
        self._table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
        self._table.setRowCount(0)
        self._table.setColumnHidden(COL_NUMBER, True)

        self._progress = QProgressBar()
        self._progress.setRange(0, 0)
        self._progress.setValue(0)

        self._stop_btn = QPushButton("停止生成")
        self._stop_btn.clicked.connect(self._stop)
        self._stop_btn.setEnabled(False)
        self._stop_btn.hide()

        self._retry_btn = QPushButton("重试")
        self._retry_btn.clicked.connect(self._retry)
        self._retry_btn.setEnabled(False)
        self._retry_btn.hide()

        progress_row = QHBoxLayout()
        progress_row.addWidget(self._progress, 1)

        layout = QVBoxLayout()
        layout.addWidget(self._status_label)
        layout.addWidget(self._current_label)
        layout.addLayout(progress_row)
        layout.addWidget(self._table, 3)
        self.setLayout(layout)
        self._thread: QThread | None = None
        self._worker: _AiAnalysisWorker | None = None
        self._running = False
        self._done = False
        self._discard_run_results = False
        self._close_after_stop_requested = False
        self._tasks: list[tuple[int, str, str]] = []
        self._baseline_questions: list[Question] = []
        self._failed_rows: set[int] = set()
        self._current_task_no = 0
        self._total_task_count = 0
        self._completed_task_count = 0
        self._refresh_status()

    def initializePage(self) -> None:
        self._sync_wizard_buttons()
        self._thread = None
        self._worker = None
        self._running = False
        self._done = False
        self._discard_run_results = False
        self._close_after_stop_requested = False
        self._tasks = []
        self._baseline_questions = copy.deepcopy(list(self._state.draft.questions))
        self._failed_rows = set()
        self._current_task_no = 0
        self._total_task_count = 0
        self._completed_task_count = 0
        self._stop_btn.setEnabled(False)
        self._retry_btn.setEnabled(False)
        self._load_repo()
        self._refresh_status()
        self._prepare_tasks()
        if self._tasks and (not self._done) and (not self._running):
            QTimer.singleShot(0, self._start_generation)

    def nextId(self) -> int:
        return PAGE_IMPORT_SUCCESS

```

```python
    def _sync_wizard_buttons(self) -> None:
        wizard = self.wizard()
        if not isinstance(wizard, QWizard):
            return
        next_text = "开始生成解析"
        if self._running:
            next_text = "生成中…"
        elif self._done and not self._running:
            next_text = "写入题库"
        wizard.setButtonText(QWizard.WizardButton.BackButton, "返回查重")
        wizard.setButtonText(QWizard.WizardButton.NextButton, next_text)
        wizard.setButtonText(QWizard.WizardButton.CancelButton, "返回开始页")
        self._sync_custom_wizard_button()

    def _sync_custom_wizard_button(self) -> None:
        wizard = self.wizard()
        if not isinstance(wizard, QWizard):
            return
        stop_button = wizard.button(QWizard.WizardButton.CustomButton1)
        retry_button = wizard.button(QWizard.WizardButton.CustomButton2)
        if stop_button is not None:
            stop_button.setText("停止生成")
            stop_button.setVisible(True)
            stop_button.setEnabled(self._stop_btn.isEnabled())
            try:
                stop_button.clicked.disconnect()
            except TypeError:
                pass
            stop_button.clicked.connect(self._stop)
        if retry_button is not None:
            retry_button.setText("重试")
            retry_button.setVisible(True)
            retry_button.setEnabled(self._retry_btn.isEnabled())
            try:
                retry_button.clicked.disconnect()
            except TypeError:
                pass
            retry_button.clicked.connect(self._retry)

    def isComplete(self) -> bool:
        return self._done and (not self._running)

    def validatePage(self) -> bool:
        if self._running or (not self._done):
            return False
        return commit_draft_questions_to_db(self, self._state)

    def _refresh_status(self) -> None:
        self._stop_btn.setEnabled(self._running)
        self._retry_btn.setEnabled((not self._running) and bool(self._tasks))

    def _load_repo(self) -> None:
        questions = list(self._state.draft.questions)
        self._table.setRowCount(len(questions))
        for row, question in enumerate(questions):
            self._table.setItem(row, COL_NUMBER, QTableWidgetItem(question.number or ""))
            self._table.setItem(row, COL_STEM, QTableWidgetItem(question.stem or ""))
            self._table.setItem(row, COL_OPTIONS, QTableWidgetItem(question.options or ""))
            self._table.setItem(row, COL_ANSWER, QTableWidgetItem(question.answer or ""))
            self._table.setItem(row, COL_CHOICE_SUMMARY, QTableWidgetItem(_format_choice_summary(question)))
            self._table.setItem(row, COL_ANALYSIS, QTableWidgetItem(question.analysis or ""))
            self._table.setItem(row, COL_ELAPSED, QTableWidgetItem(""))

    def _build_inputs_for_row(self, row: int) -> tuple[str, str]:
        stem = self._table.item(row, COL_STEM).text() if self._table.item(row, COL_STEM) else ""
        options = self._table.item(row, COL_OPTIONS).text() if self._table.item(row, COL_OPTIONS) else ""
        answer = self._table.item(row, COL_ANSWER).text() if self._table.item(row, COL_ANSWER) else ""
        choice_summary = (
            self._table.item(row, COL_CHOICE_SUMMARY).text().strip() if self._table.item(row, COL_CHOICE_SUMMARY) else ""
```

```python
        )
        question_text = stem.strip()
        if options.strip():
            question_text = (question_text + "\n" + options.strip()).strip()
        if choice_summary:
            question_text = (question_text + "\n\n" + choice_summary).strip()
        return question_text, answer.strip()

    def _collect_tasks(self) -> list[tuple[int, str, str]]:
        tasks: list[tuple[int, str, str]] = []
        for row in range(self._table.rowCount()):
            analysis = self._table.item(row, COL_ANALYSIS).text().strip() if self._table.item(row, COL_ANALYSIS) else ""
            if analysis:
                continue
            question_text, answer_text = self._build_inputs_for_row(row)
            if question_text:
                tasks.append((row, question_text, answer_text))
        return tasks

    def _prepare_tasks(self) -> None:
        self._tasks = self._collect_tasks()
        self._total_task_count = len(self._tasks)
        self._current_task_no = 0
        self._completed_task_count = 0
        self._progress.setRange(0, len(self._tasks) if self._tasks else 0)
        self._progress.setValue(0)
        self._sync_wizard_buttons()
        if not self._tasks:
            self._status_label.setText("无需生成解析。")
            self._current_label.setText("")
            self._done = True
            self.completeChanged.emit()
        else:
            self._status_label.setText("进入本页后将自动开始生成解析。")
            self._current_label.setText("")

    def cleanupPage(self) -> None:
        self._sync_wizard_buttons()

    def _start_generation(self) -> None:
        if self._running or self._done:
            return

        provider = normalize_analysis_provider(self._state.analysis_provider)
        provider_label = _analysis_provider_label(provider)
        model_name = normalize_analysis_model_name(self._state.analysis_model_name)
        if provider == "kimi":
            cfg = load_kimi_config()
            llm_config = LlmConfig(
                base_url=cfg.base_url.strip(),
                api_key=cfg.api_key.strip(),
                model=model_name,
                timeout_s=float(cfg.timeout_s),
            )
        elif provider == "qwen":
            cfg = load_qwen_config()
            llm_config = LlmConfig(
                base_url=cfg.base_url.strip(),
                api_key=cfg.api_key.strip(),
                model=model_name,
                timeout_s=float(cfg.timeout_s),
            )
        else:
            cfg = load_deepseek_config()
            llm_config = LlmConfig(
                base_url=cfg.base_url.strip(),
                api_key=cfg.api_key.strip(),
                model=model_name,
                timeout_s=float(cfg.timeout_s),
```

```python
            )

        if not cfg.is_ready():
            _show_message_box(self, title="未配置", text=f"请先完成 {provider_label} 配置。", icon=QMessageBox.Icon.Warning)
            self._status_label.setText("未配置：无法开始生成解析。")
            self._sync_wizard_buttons()
            self.completeChanged.emit()
            return

        resource_paths = app_paths()
        ref_paths: list[Path] = []
        if self._state.analysis_use_reference_folder:
            ref_dir = resource_paths.reference_resource_dir
            if ref_dir.exists():
                ref_paths = sorted([path for path in ref_dir.glob("*.md")], key=lambda path: path.name)
        tasks = self._tasks
        if not tasks:
            self._done = True
            self.completeChanged.emit()
            return

        if self._thread is not None and self._thread.isRunning():
            return

        self._running = True
        self._discard_run_results = False
        self.completeChanged.emit()
        self._stop_btn.setEnabled(True)
        self._retry_btn.setEnabled(False)
        self._progress.setRange(0, len(tasks))
        self._progress.setValue(0)
        self._status_label.setText(f"批量生成：准备开始…（模型：{_analysis_target_text(provider, model_name)}）")
        self._current_label.setText("")
        self._sync_wizard_buttons()

        thread = QThread(self)
        worker = _AiAnalysisWorker(
            llm_config=llm_config,
            provider_label=provider_label,
            tasks=tasks,
            reference_md_paths=ref_paths,
            include_common_mistakes=self._state.analysis_include_common_mistakes,
            root_dir=resource_paths.base_dir,
            max_workers=normalize_ai_concurrency(self._state.analysis_generation_concurrency),
        )
        worker.moveToThread(thread)
        thread.started.connect(worker.run)
        worker.processing.connect(self._on_batch_processing)
        worker.progress.connect(self._on_batch_progress)
        worker.row_done.connect(self._on_batch_row_done)
        worker.row_failed.connect(self._on_batch_row_failed)
        worker.done.connect(self._on_batch_done)
        worker.error.connect(self._on_batch_error)
        worker.done.connect(thread.quit)
        worker.error.connect(thread.quit)
        thread.finished.connect(worker.deleteLater)
        thread.finished.connect(thread.deleteLater)
        self._thread = thread
        self._worker = worker
        thread.start()

    def _stop(self) -> None:
        if self._worker is None:
            return
        self._worker.request_stop()
        self._discard_run_results = True
        self._stop_btn.setEnabled(False)
        self._status_label.setText("正在停止…")
```

```python
        self._current_label.setText("")
        self._restore_baseline_questions()
        self._tasks = self._collect_tasks()
        self._refresh_status()
        self._sync_wizard_buttons()

    def prepare_to_close(self) -> bool:
        thread = self._thread
        if thread is None or (not thread.isRunning()):
            return True
        if self._worker is not None:
            self._worker.request_stop()
        self._discard_run_results = True
        self._close_after_stop_requested = True
        self._stop_btn.setEnabled(False)
        self._retry_btn.setEnabled(False)
        self._status_label.setText("正在停止，收尾完成后将自动退出…")
        self._current_label.setText("")
        self._restore_baseline_questions()
        self._tasks = self._collect_tasks()
        self._refresh_status()
        self._sync_wizard_buttons()
        return False

    def _on_batch_progress(self, current: int, total: int) -> None:
        if self._discard_run_results:
            return
        self._progress.setRange(0, total)
        self._progress.setValue(max(0, min(current, total)))
        self._completed_task_count = current
        self._status_label.setText(self._build_running_status())

    def _on_batch_processing(self, current: int, total: int) -> None:
        if self._discard_run_results:
            return
        self._current_task_no = current
        self._total_task_count = total
        self._status_label.setText(self._build_running_status())
        workers = normalize_ai_concurrency(self._state.analysis_generation_concurrency)
        target_text = _analysis_target_text(self._state.analysis_provider, self._state.analysis_model_name)
        self._current_label.setText(f"选择题 {current}/{total} 题已发起解析（模型：{target_text}，并发 {workers} 路）")

    def _on_batch_row_done(self, row: int, answer_text: str, text: str, elapsed_s: float) -> None:
        if self._discard_run_results:
            return
        existing = self._table.item(row, COL_ANALYSIS).text().strip() if self._table.item(row, COL_ANALYSIS) else ""
        if existing:
            return
        if row in self._failed_rows:
            self._failed_rows.remove(row)
        if answer_text:
            self._table.setItem(row, COL_ANSWER, QTableWidgetItem(answer_text))
        self._table.setItem(row, COL_ANALYSIS, QTableWidgetItem(text))
        self._table.setItem(row, COL_ELAPSED, QTableWidgetItem(f"{elapsed_s:.1f}s"))

    def _on_batch_row_failed(self, row: int, msg: str) -> None:
        if self._discard_run_results:
            return
        self._failed_rows.add(row)
        row_no = row + 1
        self._status_label.setText(f"第 {row_no} 行生成失败：{msg}")
        self._current_label.setText(f"第 {row_no} 行处理失败")

    def _on_batch_done(self, stopped: bool) -> None:
        if self._discard_run_results or stopped:
            self._restore_baseline_questions()
            self._tasks = self._collect_tasks()
            self._current_task_no = 0
```

```python
            self._completed_task_count = 0
            self._total_task_count = len(self._tasks)
            self._progress.setRange(0, len(self._tasks) if self._tasks else 0)
            self._progress.setValue(0)
            self._running = False
            self._done = False
            self._thread = None
            self._worker = None
            self._current_label.setText("")
            self._status_label.setText("已停止生成，本轮结果已丢弃，请点击重试重新生成。")
            self._refresh_status()
            self._sync_wizard_buttons()
            self.completeChanged.emit()
            self._finish_close_after_stop_if_needed()
            return
        self._status_label.setText("批量生成：写回当前草稿…")
        self._current_label.setText("")
        questions: list[Question] = []
        for row in range(self._table.rowCount()):
            original = self._state.draft.questions[row] if row < len(self._state.draft.questions) else None
            number = self._table.item(row, COL_NUMBER).text().strip() if self._table.item(row, COL_NUMBER) else ""
            stem = self._table.item(row, COL_STEM).text().strip() if self._table.item(row, COL_STEM) else ""
            options = self._table.item(row, COL_OPTIONS).text().strip() if self._table.item(row, COL_OPTIONS) else ""
            answer = self._table.item(row, COL_ANSWER).text().strip() if self._table.item(row, COL_ANSWER) else ""
            analysis = self._table.item(row, COL_ANALYSIS).text().strip() if self._table.item(row, COL_ANALYSIS) else ""
            if not any([number, stem, options, answer, analysis]):
                continue
            questions.append(
                Question(
                    number=number,
                    stem=stem,
                    options=options,
                    answer=answer,
                    analysis=analysis,
                    question_type=original.question_type if original is not None else "",
                    choice_1=original.choice_1 if original is not None else "",
                    choice_2=original.choice_2 if original is not None else "",
                    choice_3=original.choice_3 if original is not None else "",
                    choice_4=original.choice_4 if original is not None else "",
                )
            )

        self._state.apply_draft_questions(questions)

        self._progress.setValue(self._progress.maximum())
        self._tasks = self._collect_tasks()
        self._current_task_no = 0
        self._completed_task_count = 0
        self._total_task_count = len(self._tasks)
        pending = len(self._tasks)
        failed = len(self._failed_rows)
        if pending > 0:
            self._status_label.setText(
                f"已停止生成（剩余 {pending} 题可重试，失败 {failed} 题）。"
                if stopped
                else f"批量生成完成（剩余 {pending} 题可重试，失败 {failed} 题）。"
            )
        else:
            self._status_label.setText(
                (f"已停止生成（失败 {failed} 题）。" if stopped else f"批量生成完成（失败 {failed} 题）。")
                if failed > 0
                else ("已停止生成。" if stopped else "批量生成完成。")
            )
        self._running = False
        self._done = True
        self._thread = None
```

```python
        self._worker = None
        self._current_label.setText("")
        self._refresh_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()

    def _on_batch_error(self, msg: str) -> None:
        if self._discard_run_results:
            self._restore_baseline_questions()
            self._tasks = self._collect_tasks()
            self._current_task_no = 0
            self._completed_task_count = 0
            self._total_task_count = len(self._tasks)
            self._running = False
            self._done = False
            self._thread = None
            self._worker = None
            self._status_label.setText("已停止生成，本轮结果已丢弃，请点击重试重新生成。")
            self._current_label.setText("")
            self._refresh_status()
            self._sync_wizard_buttons()
            self.completeChanged.emit()
            self._finish_close_after_stop_if_needed()
            return
        _show_message_box(self, title="生成失败", text=msg, icon=QMessageBox.Icon.Critical)
        self._status_label.setText("生成失败。")
        self._current_label.setText("")
        self._running = False
        self._done = False
        self._thread = None
        self._worker = None
        self._tasks = self._collect_tasks()
        self._current_task_no = 0
        self._completed_task_count = 0
        self._total_task_count = len(self._tasks)
        self._refresh_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()

    def _retry(self) -> None:
        if self._running:
            return
        self._discard_run_results = False
        self._close_after_stop_requested = False
        self._restore_baseline_questions()
        self._tasks = self._collect_tasks()
        if self._tasks:
            self._total_task_count = len(self._tasks)
            self._current_task_no = 0
            self._completed_task_count = 0
            self._progress.setRange(0, len(self._tasks))
            self._progress.setValue(0)
            self._done = False
            self._refresh_status()
            self._status_label.setText("准备重试生成解析…")
            self._current_label.setText("")
            QTimer.singleShot(0, self._start_generation)

    def _restore_baseline_questions(self) -> None:
        questions = copy.deepcopy(self._baseline_questions)
        self._state.apply_draft_questions(questions)
        self._failed_rows = set()
        self._load_repo()

    def _build_running_status(self) -> str:
        total = self._total_task_count
        current = self._current_task_no
        completed = self._completed_task_count
        workers = normalize_ai_concurrency(self._state.analysis_generation_concurrency)
        target_text = _analysis_target_text(self._state.analysis_provider, self._state.analysis_model_name)
```

```python
        if total > 0 and current > 0:
            return f"批量生成：模型 {target_text}，并发 {workers} 路，已发起 {current}/{total}，已完成 {completed}/{total}"
        if total > 0:
            return f"批量生成：模型 {target_text}，并发 {workers} 路，已完成 {completed}/{total}"
        return "批量生成中…"

    def _finish_close_after_stop_if_needed(self) -> None:
        if not self._close_after_stop_requested:
            return
        self._close_after_stop_requested = False
        wizard = self.wizard()
        if isinstance(wizard, QWizard):
            QTimer.singleShot(0, wizard.close)


class _AiAnalysisWorker(QObject):
    processing = pyqtSignal(int, int)
    progress = pyqtSignal(int, int)
    row_done = pyqtSignal(int, str, str, float)
    row_failed = pyqtSignal(int, str)
    done = pyqtSignal(bool)
    error = pyqtSignal(str)

    def __init__(
        self,
        *,
        llm_config,
        provider_label: str,
        tasks: list[tuple[int, str, str]],
        reference_md_paths: list[Path],
        include_common_mistakes: bool,
        root_dir: Path,
        max_workers: int,
    ) -> None:
        super().__init__()
        self._llm_config = llm_config
        self._provider_label = provider_label
        self._tasks = tasks
        self._reference_md_paths = reference_md_paths
        self._include_common_mistakes = include_common_mistakes
        self._root_dir = root_dir
        self._max_workers = normalize_ai_concurrency(max_workers)
        self._stop = False

    def request_stop(self) -> None:
        self._stop = True

    def run(self) -> None:
        try:
            total = len(self._tasks)
            done_count = 0

            def run_one(task: tuple[int, str, str]) -> tuple[str, str, float]:
                _, question_text, answer_text = task
                client = LlmClient(self._llm_config)
                started_at = time.perf_counter()
                inp = ExplanationInputs(
                    question_text=question_text,
                    answer_text=answer_text,
                    reference_md_paths=self._reference_md_paths,
                    include_common_mistakes=self._include_common_mistakes,
                    root_dir=self._root_dir,
                )
                result = generate_explanation_result(client, inp)
                elapsed_s = time.perf_counter() - started_at
                return result.answer_text, result.analysis_text, elapsed_s

            def on_task_start(current: int, total_count: int, task: tuple[int, str, str]) -> None:
                self.processing.emit(current, total_count)
```

```python

            def on_task_done(task: tuple[int, str, str], result: tuple[str, str, float]) -> None:
                nonlocal done_count
                row, _, _ = task
                answer_text, text, elapsed_s = result
                self.row_done.emit(row, answer_text, text, elapsed_s)
                done_count += 1
                self.progress.emit(done_count, total)

            def on_task_failed(task: tuple[int, str, str], exc: Exception) -> None:
                nonlocal done_count
                row, _, _ = task
                self.row_failed.emit(row, str(exc))
                done_count += 1
                self.progress.emit(done_count, total)

            run_tasks_in_parallel(
                tasks=self._tasks,
                max_workers=self._max_workers,
                stop_cb=self._should_stop,
                on_task_start=on_task_start,
                on_task_done=on_task_done,
                on_task_failed=on_task_failed,
                run_one=run_one,
            )
            self.done.emit(self._stop)
        except Exception as exc:
            self.error.emit(str(exc))

    def _should_stop(self) -> bool:
        return self._stop

__all__ = ["AiAnalysisPage"]

```

```python
from PyQt6.QtCore import QThread, QTimer, Qt
from PyQt6.QtWidgets import (
    QHeaderView,
    QLabel,
    QMessageBox,
    QPushButton,
    QProgressBar,
    QTableWidget,
    QVBoxLayout,
    QWizard,
    QWizardPage,
)

from sj_generator.application.state import ImportWizardSession, normalize_ai_concurrency
from sj_generator.domain.entities import Question
from sj_generator.infrastructure.llm.import_questions import (
    _to_question,
    question_content_provider_ready,
    question_content_round_limit,
)
from sj_generator.presentation.qt.constants import (
    PAGE_AI_ANALYSIS,
    PAGE_DEDUPE_RESULT,
    PAGE_IMPORT_SUCCESS,
)
from .import_content_detail import (
    effective_content_question_workers,
    question_content_active_model_specs,
    question_content_detail_headers,
    question_content_model_signature,
    question_ref_total_count,
    set_content_detail_item,
)
from .import_page_common import style_busy_progress
from .import_progress import parse_content_progress_message
from .import_db_service import commit_draft_questions_to_db
from .import_workers import AiImportContentWorker
from sj_generator.presentation.qt.message_box import show_message_box
from .content_support import (
    apply_content_compare_payload,
    apply_content_detail_column_widths_if_needed,
    build_content_status_text,
    create_content_worker_bundle,
    missing_content_model_labels,
)
from sj_generator.presentation.qt.table_copy import CopyableTableWidget


class AiImportContentPage(QWizardPage):
    _DETAIL_TABLE_FONT_POINT_SIZE_MIN = 8
    _DETAIL_TABLE_FONT_POINT_SIZE_MAX = 28

    def __init__(self, state: ImportWizardSession) -> None:
        super().__init__()
        self._state = state
        self.setTitle("题目内容解析")
        self._detail_row_resize_pending = False
        self._detail_table_font_point_size = self._load_detail_table_font_point_size()

        self._progress = QProgressBar()
        self._progress.setRange(0, 0)
        self._progress.setFormat("正在统计题数…")
        style_busy_progress(self._progress)

        self._stop_btn = QPushButton("停止解析")
        self._stop_btn.clicked.connect(self._stop)
        self._stop_btn.setEnabled(False)
        self._stop_btn.hide()

        self._retry_btn = QPushButton("重试")
```

```python
        self._retry_btn.clicked.connect(self._retry)
        self._retry_btn.setEnabled(False)
        self._retry_btn.hide()

        self._status_label = QLabel("")
        self._status_label.setWordWrap(True)

        self._detail_table = CopyableTableWidget(zoom_callback=self._adjust_detail_table_font_size)
        self._detail_table.setColumnCount(len(question_content_detail_headers()))
        self._detail_table.setHorizontalHeaderLabels(question_content_detail_headers())
        self._detail_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Interactive)
        self._detail_table.verticalHeader().setSectionResizeMode(QHeaderView.ResizeMode.ResizeToContents)
        self._detail_table.setWordWrap(True)
        self._detail_table.setTextElideMode(Qt.TextElideMode.ElideNone)
        self._detail_table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
        self._detail_table.setRowCount(0)
        self._detail_table.horizontalHeader().sectionResized.connect(
            lambda *_: self._schedule_detail_row_resize()
        )
        self._detail_row_map: dict[int, int] = {}
        self._detail_width_signature = ""
        self._apply_detail_table_font_size()

        layout = QVBoxLayout()
        layout.addWidget(self._status_label)
        layout.addWidget(self._detail_table)
        self.setLayout(layout)

        self._items: list[Question] = []
        self._last_files_text: str = ""
        self._last_ref_version: int = -1
        self._last_content_model_signature: str = ""
        self._thread: QThread | None = None
        self._worker: AiImportContentWorker | None = None
        self._running = False
        self._stopped = False
        self._discard_run_results = False
        self._stop_requested = False
        self._retry_pending = False
        self._close_after_stop_requested = False
        self._force_finish_requested = False
        self._force_finish_detached = False
        self._force_finish_advance_attempts = 0
        self._finished = False
        self._failed = False
        self._progress_cur = 0
        self._progress_total = 0
        self._accepted_count = 0
        self._skipped_count = 0
        self._phase_text = "准备开始解析…"
        self._detail_text = ""
        self._parallel_text = ""
        self._consistency_text = ""
        self._detail_row_map = {}
        self._compare_secs: dict[int, dict[str, dict[int, int]]] = {}
        self._accepted_items_by_index: dict[int, Question] = {}
        self._content_model_specs = question_content_active_model_specs()
        self._content_model_ready: dict[str, bool] = {}
        self._cur_source_name = "-"
        self._cur_question_no = "-"
        self._cur_round_no = "-"
        self._waiting_detail_timer = QTimer(self)
        self._waiting_detail_timer.setInterval(1000)
        self._waiting_detail_timer.timeout.connect(self._refresh_waiting_placeholder_rows)
        self._waiting_elapsed_s = 0

    def initializePage(self) -> None:
        self._sync_wizard_buttons()
        if self._state.source.files:
            self._cur_source_name = self._state.source.files[0].name
```

```python
        ref_version = int(self._state.refs.revision)
        content_model_specs = question_content_active_model_specs()
        content_model_signature = question_content_model_signature(content_model_specs)
        self._content_model_specs = content_model_specs
        self._detail_table.setColumnCount(len(question_content_detail_headers(self._content_model_specs)))
        self._detail_table.setHorizontalHeaderLabels(question_content_detail_headers(self._content_model_specs))
        self._apply_content_detail_column_widths_if_needed(model_specs=self._content_model_specs)
        should_restart = (
            self._state.source.files_text
            and (
                self._state.source.files_text != self._last_files_text
                or ref_version != self._last_ref_version
                or content_model_signature != self._last_content_model_signature
            )
        )
        should_auto_start = bool(
            self._state.source.files_text
            and question_ref_total_count(self._state.refs.question_refs_by_source) > 0
            and not self._running
            and not self._items
            and (self._thread is None or not self._thread.isRunning())
        )
        if should_restart or should_auto_start:
            self._last_files_text = self._state.source.files_text
            self._last_ref_version = ref_version
            self._last_content_model_signature = content_model_signature
            if should_restart:
                self._detail_table.setRowCount(0)
                self._detail_row_map = {}
                self._compare_secs = {}
                self._items = []
                self._progress.setRange(0, 0)
                self._progress.setValue(0)
            self._reset_progress_meta()
            self._phase_text = "正在解析题目内容，请稍候…"
            self._status_label.setText(self._phase_text)
            self._running = True
            self._stopped = False
            self._finished = False
            self._failed = False
            self._render_status()
            self.completeChanged.emit()
            QTimer.singleShot(30, self._start_import)

    def nextId(self) -> int:
        if self._state.dedupe_enabled:
            return PAGE_DEDUPE_RESULT
        return PAGE_AI_ANALYSIS if self._state.analysis_enabled else PAGE_IMPORT_SUCCESS

    def _sync_wizard_buttons(self) -> None:
        wizard = self.wizard()
        if not isinstance(wizard, QWizard):
            return
        back_button = wizard.button(QWizard.WizardButton.BackButton)
        if back_button is not None:
            back_button.setVisible(False)
        next_text = "开始导题"
        if self._stop_requested or self._stopped or self._failed:
            next_text = "重试后继续"
        elif self._force_finish_requested:
            next_text = "强制完成中…"
        elif self._running:
            next_text = "解析中…"
        elif self._items:
            if self._state.dedupe_enabled:
                next_text = "进入查重"
            elif self._state.analysis_enabled:
                next_text = "进入解析"
            else:
```

```python
                next_text = "写入题库"
        wizard.setButtonText(QWizard.WizardButton.NextButton, next_text)
        self._sync_custom_wizard_button()

    def _sync_custom_wizard_button(self) -> None:
        wizard = self.wizard()
        if not isinstance(wizard, QWizard):
            return
        stop_button = wizard.button(QWizard.WizardButton.CustomButton1)
        retry_button = wizard.button(QWizard.WizardButton.CustomButton2)
        if stop_button is not None:
            stop_button.setText("停止解析")
            stop_button.setVisible(True)
            stop_button.setEnabled(self._stop_btn.isEnabled())
            try:
                stop_button.clicked.disconnect()
            except TypeError:
                pass
            stop_button.clicked.connect(self._stop)
        if retry_button is not None:
            retry_button.setText("重试")
            retry_button.setVisible(True)
            retry_button.setEnabled(self._retry_btn.isEnabled())
            try:
                retry_button.clicked.disconnect()
            except TypeError:
                pass
            retry_button.clicked.connect(self._retry)
        force_finish_button = wizard.button(QWizard.WizardButton.CustomButton3)
        if force_finish_button is not None:
            force_finish_button.setText("强制完成")
            force_finish_button.setVisible(True)
            force_finish_button.setEnabled(self._can_force_finish())
            try:
                force_finish_button.clicked.disconnect()
            except TypeError:
                pass
            force_finish_button.clicked.connect(self._force_finish)

    def isComplete(self) -> bool:
        if self._running or self._failed:
            return False
        if self._stopped or not self._finished:
            return False
        return len(self._items) > 0

    def validatePage(self) -> bool:
        if self._running:
            return False
        self._materialize_items_from_accepted_cache()
        if not self._items:
            show_message_box(self, title="暂无结果", text="当前没有可进入下一步的题目。", icon=QMessageBox.Icon.Warning)
            return False
        self._state.apply_draft_questions(self._items)
        if not self._state.dedupe_enabled and not self._state.analysis_enabled:
            return commit_draft_questions_to_db(self, self._state)
        self.completeChanged.emit()
        return True

    def _can_force_finish(self) -> bool:
        return self._running and (not self._force_finish_requested) and bool(
            self._items or self._accepted_items_by_index
        )

    def _start_import(self) -> None:
        if self._thread is not None and self._thread.isRunning():
            return

        paths = self._state.source.files or []
```

```python
        if not paths:
            show_message_box(self, title="未选择文件", text="请先选择待处理的资料文件。", icon=QMessageBox.Icon.Warning)
            return
        if question_ref_total_count(self._state.refs.question_refs_by_source) <= 0:
            show_message_box(self, title="缺少题号", text="请先完成题号与题型解析。", icon=QMessageBox.Icon.Warning)
            return

        self._content_model_specs = question_content_active_model_specs()
        ready_map = {
            str(spec.get("key") or ""): question_content_provider_ready(str(spec.get("provider") or ""))
            for spec in self._content_model_specs
        }
        self._content_model_ready = ready_map
        self._render_status()
        if not self._content_model_specs or not all(ready_map.values()):
            self._sync_wizard_buttons()
            missing_labels = missing_content_model_labels(self._content_model_specs, ready_map)
            missing_text = "、".join(label for label in missing_labels if label) or "题目内容解析模型"
            show_message_box(
                self,
                title="未配置",
                text=f"请先完成以下题目内容解析模型配置并通过可用性测试：{missing_text}",
                icon=QMessageBox.Icon.Warning,
            )
            return

        self._status_label.setText("正在解析题目内容，请稍候…")
        self._reset_progress_meta()
        self._phase_text = "正在解析题目内容，请稍候…"
        self._show_waiting_placeholder_rows()
        self._render_status()
        self._stop_btn.setEnabled(True)
        self._retry_btn.setEnabled(False)
        self._running = True
        self._stopped = False
        self._discard_run_results = False
        self._stop_requested = False
        self._retry_pending = False
        self._force_finish_requested = False
        self._force_finish_detached = False
        self._force_finish_advance_attempts = 0
        self._finished = False
        self._failed = False
        self._sync_wizard_buttons()
        self.completeChanged.emit()

        bundle = create_content_worker_bundle(
            parent=self,
            model_specs=self._content_model_specs,
            paths=paths,
            question_refs_by_source=dict(self._state.refs.question_refs_by_source),
            max_question_workers=effective_content_question_workers(
                normalize_ai_concurrency(self._state.question_content_concurrency),
                len(self._content_model_specs),
            ),
            on_progress=self._on_progress,
            on_progress_count=self._on_progress_count,
            on_question=self._on_question,
            on_compare=self._on_compare,
            on_done=self._on_done,
            on_error=self._on_error,
        )
        self._thread = bundle.thread
        self._worker = bundle.worker
        self._thread.start()

    def _on_progress(self, msg: str) -> None:
        if self._discard_run_results:
```

```python
            return
        self._phase_text = msg
        self._update_detail_from_progress(msg)
        self._render_status()

    def _on_progress_count(self, current: int, total: int) -> None:
        if self._discard_run_results:
            return
        self._progress_cur = max(0, current)
        self._progress_total = max(0, total)
        if total <= 0:
            self._progress.setRange(0, 0)
            self._progress.setFormat("正在统计题数…")
            self._render_status()
            return
        if self._progress.maximum() != total:
            self._progress.setRange(0, total)
        self._progress.setValue(max(0, min(current, total)))
        self._progress.setFormat("选择题 %v/%m题")
        self._render_status()
        self._sync_wizard_buttons()

    def _on_question(self, question: Question) -> None:
        if self._discard_run_results:
            return
        self._items.append(question)
        self._accepted_count = len(self._items)
        self._render_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()

    def _on_compare(self, payload: object) -> None:
        if self._discard_run_results:
            return
        if not isinstance(payload, dict):
            return
        update = apply_content_compare_payload(
            table=self._detail_table,
            payload=payload,
            fallback_specs=self._content_model_specs,
            row_map=self._detail_row_map,
            compare_secs=self._compare_secs,
        )
        if update is None:
            return
        self._content_model_specs = update.model_specs
        self._detail_width_signature = apply_content_detail_column_widths_if_needed(
            table=self._detail_table,
            model_specs=self._content_model_specs,
            current_signature=self._detail_width_signature,
        )
        self._schedule_detail_row_resize()
        self._cur_question_no = str(update.index)
        self._cur_round_no = update.round_no
        self._cache_accepted_question(payload)
        self._phase_text = f"解析中：已回传第 {update.index} 题第 {update.round_no}/{update.round_limit} 轮比对结果"
        self._render_status()
        self._sync_wizard_buttons()

    def _schedule_detail_row_resize(self) -> None:
        if getattr(self, "_detail_row_resize_pending", False):
            return
        self._detail_row_resize_pending = True
        QTimer.singleShot(0, self._resize_detail_rows_to_contents)

    def _resize_detail_rows_to_contents(self) -> None:
        self._detail_row_resize_pending = False
        self._detail_table.resizeRowsToContents()

```

```python
    def _load_detail_table_font_point_size(self) -> int:
        default_size = self.font().pointSize()
        if default_size <= 0:
            default_size = 10
        return max(
            self._DETAIL_TABLE_FONT_POINT_SIZE_MIN,
            min(self._DETAIL_TABLE_FONT_POINT_SIZE_MAX, default_size),
        )

    def _apply_detail_table_font_size(self) -> None:
        font = self._detail_table.font()
        font.setPointSize(self._detail_table_font_point_size)
        self._detail_table.setFont(font)
        self._detail_table.horizontalHeader().setFont(font)
        self._detail_table.verticalHeader().setFont(font)
        self._detail_table.doItemsLayout()
        self._detail_table.viewport().update()
        self._detail_table.horizontalHeader().viewport().update()
        self._schedule_detail_row_resize()

    def _adjust_detail_table_font_size(self, step: int) -> None:
        new_size = max(
            self._DETAIL_TABLE_FONT_POINT_SIZE_MIN,
            min(self._DETAIL_TABLE_FONT_POINT_SIZE_MAX, self._detail_table_font_point_size + int(step)),
        )
        if new_size == self._detail_table_font_point_size:
            return
        self._detail_table_font_point_size = new_size
        self._apply_detail_table_font_size()

    def _apply_content_detail_column_widths_if_needed(self, *, model_specs: list[dict[str, str]]) -> None:
        self._detail_width_signature = apply_content_detail_column_widths_if_needed(
            table=self._detail_table,
            model_specs=model_specs,
            current_signature=self._detail_width_signature,
        )

    def _apply_content_detail_column_widths(self, model_specs: list[dict[str, str]]) -> None:
        self._detail_width_signature = apply_content_detail_column_widths_if_needed(
            table=self._detail_table,
            model_specs=model_specs,
            current_signature="",
        )

    def _set_content_detail_item(self, row: int, col: int, text: str, *, is_json: bool = False) -> None:
        set_content_detail_item(self._detail_table, row, col, text, is_json=is_json)

    def _on_done(self, total: int) -> None:
        self._waiting_detail_timer.stop()
        if self._discard_run_results:
            if self._force_finish_detached:
                self._thread = None
                self._worker = None
                self._finish_close_after_stop_if_needed()
                return
            if self._force_finish_requested:
                self._finish_force_complete()
                return
            self._clear_partial_results()
            self._phase_text = "已停止解析，本轮结果已丢弃，请点击重试重新解析。"
            self._stop_btn.setEnabled(False)
            self._retry_btn.setEnabled(True)
            self._running = False
            self._stop_requested = False
            self._stopped = True
            self._finished = False
            self._thread = None
            self._worker = None
            self._render_status()
            self._sync_wizard_buttons()
```

```python
            self.completeChanged.emit()
            self._finish_close_after_stop_if_needed()
            if self._retry_pending:
                self._retry_pending = False
                QTimer.singleShot(0, self._retry)
            return
        self._phase_text = f"解析完成：{total} 题（可继续进入下一步）"
        if self._progress.maximum() > 0:
            self._progress.setValue(self._progress.maximum())
            self._progress_cur = self._progress.maximum()
            self._progress_total = self._progress.maximum()
        self._stop_btn.setEnabled(False)
        self._retry_btn.setEnabled(True)
        self._running = False
        self._stop_requested = False
        self._stopped = False
        self._finished = True
        self._thread = None
        self._worker = None
        self._render_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()

    def _on_error(self, msg: str) -> None:
        self._waiting_detail_timer.stop()
        if self._discard_run_results:
            if self._force_finish_detached:
                self._thread = None
                self._worker = None
                self._finish_close_after_stop_if_needed()
                return
            if self._force_finish_requested:
                self._finish_force_complete(error_message=msg)
                return
            self._clear_partial_results()
            self._phase_text = "已停止解析，本轮结果已丢弃，请点击重试重新解析。"
            self._stop_btn.setEnabled(False)
            self._retry_btn.setEnabled(True)
            self._running = False
            self._stop_requested = False
            self._failed = False
            self._thread = None
            self._worker = None
            self._render_status()
            self._sync_wizard_buttons()
            self.completeChanged.emit()
            self._finish_close_after_stop_if_needed()
            if self._retry_pending:
                self._retry_pending = False
                QTimer.singleShot(0, self._retry)
            return
        show_message_box(self, title="解析失败", text=msg, icon=QMessageBox.Icon.Critical)
        self._phase_text = f"解析失败：{msg}"
        self._detail_text = msg
        self._append_failure_row(msg)
        self._stop_btn.setEnabled(False)
        self._retry_btn.setEnabled(True)
        self._running = False
        self._stop_requested = False
        self._failed = True
        self._thread = None
        self._worker = None
        self._render_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()

    def _append_failure_row(self, msg: str) -> None:
        row = self._detail_table.rowCount()
        self._detail_table.setRowCount(row + 1)
        text = f"失败：{msg}"
```

```python
        self._set_content_detail_item(row, 0, "-")
        payload_col_start = 1 + len(self._content_model_specs)
        payload_col_end = payload_col_start + len(self._content_model_specs)
        for col in range(payload_col_start, payload_col_end):
            self._set_content_detail_item(row, col, "", is_json=True)
        self._set_content_detail_item(row, payload_col_end, text)
        self._schedule_detail_row_resize()

    def _stop(self) -> None:
        if self._worker is not None:
            self._worker.request_stop()
            self._discard_run_results = True
            self._stop_requested = True
            self._force_finish_requested = False
            self._stop_btn.setEnabled(False)
            self._retry_btn.setEnabled(True)
            self._running = False
            self._phase_text = "正在停止…可点击重试，收尾完成后将重新解析。"
            self._stopped = True
            self._clear_partial_results()
            self._render_status()
            self._sync_wizard_buttons()
            self.completeChanged.emit()

    def prepare_to_close(self) -> bool:
        thread = self._thread
        if thread is None or (not thread.isRunning()):
            return True
        if self._worker is not None:
            self._worker.request_stop()
        self._discard_run_results = True
        self._stop_requested = True
        self._force_finish_requested = False
        self._close_after_stop_requested = True
        self._stop_btn.setEnabled(False)
        self._retry_btn.setEnabled(False)
        self._running = False
        self._phase_text = "正在停止，收尾完成后将自动退出…"
        self._stopped = True
        self._clear_partial_results()
        self._render_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()
        return False

    def _retry(self) -> None:
        if self._thread is not None and self._thread.isRunning():
            self._retry_pending = True
            self._retry_btn.setEnabled(False)
            self._phase_text = "正在停止当前解析，收尾完成后自动重试…"
            self._render_status()
            self._sync_wizard_buttons()
            return
        if self._running:
            return
        self._discard_run_results = False
        self._stop_requested = False
        self._retry_pending = False
        self._close_after_stop_requested = False
        self._force_finish_requested = False
        self._force_finish_detached = False
        self._force_finish_advance_attempts = 0
        self._clear_partial_results()
        self._phase_text = "准备重试解析…"
        self._stopped = False
        self._finished = False
        self._failed = False
        self._retry_btn.setEnabled(False)
        self._render_status()
        self._sync_wizard_buttons()
```

```python
        self.completeChanged.emit()
        QTimer.singleShot(0, self._start_import)

    def _clear_partial_results(self) -> None:
        self._waiting_detail_timer.stop()
        self._waiting_elapsed_s = 0
        self._detail_table.setRowCount(0)
        self._detail_row_map = {}
        self._compare_secs = {}
        self._accepted_items_by_index = {}
        self._items = []
        self._progress.setRange(0, 0)
        self._progress.setValue(0)
        self._progress.setFormat("正在统计题数…")
        self._reset_progress_meta()

    def _show_waiting_placeholder_rows(self) -> None:
        refs = self._current_source_question_refs()
        if not refs:
            return
        self._waiting_detail_timer.stop()
        self._waiting_elapsed_s = 0
        self._detail_table.setRowCount(len(refs))
        self._detail_row_map = {}
        model_count = len(self._content_model_specs)
        payload_col_offset = 1 + model_count
        verdict_col = payload_col_offset + model_count
        for row_index, item in enumerate(refs):
            display_number = str(item.get("number") or row_index + 1)
            self._detail_row_map[row_index + 1] = row_index
            set_content_detail_item(self._detail_table, row_index, 0, display_number)
            for model_index in range(model_count):
                set_content_detail_item(self._detail_table, row_index, 1 + model_index, "等待 0s")
                set_content_detail_item(self._detail_table, row_index, payload_col_offset + model_index, "", is_json=True)
            set_content_detail_item(self._detail_table, row_index, verdict_col, "等待返回")
        self._schedule_detail_row_resize()
        self._waiting_detail_timer.start()

    def _refresh_waiting_placeholder_rows(self) -> None:
        if not self._running:
            return
        self._waiting_elapsed_s += 1
        model_count = len(self._content_model_specs)
        for row_index in range(self._detail_table.rowCount()):
            for model_index in range(model_count):
                item = self._detail_table.item(row_index, 1 + model_index)
                if item is None:
                    continue
                text = item.text().strip()
                if not text.startswith("等待 "):
                    continue
                item.setText(f"等待 {self._waiting_elapsed_s}s")

    def _current_source_question_refs(self) -> list[dict[str, str]]:
        source_files = list(self._state.source.files or [])
        refs_by_source = dict(self._state.refs.question_refs_by_source or {})
        for path in source_files:
            items = refs_by_source.get(str(path))
            if isinstance(items, list) and items:
                return [item for item in items if isinstance(item, dict)]
        for items in refs_by_source.values():
            if isinstance(items, list) and items:
                return [item for item in items if isinstance(item, dict)]
        return []

    def _reset_progress_meta(self) -> None:
        self._progress_cur = 0
        self._progress_total = 0
        self._accepted_count = 0
```

```python
        self._skipped_count = 0
        self._detail_text = ""
        self._parallel_text = ""
        self._consistency_text = ""
        self._cur_question_no = "-"
        self._cur_round_no = "-"
        self._render_status()

    def _update_detail_from_progress(self, msg: str) -> None:
        parsed = parse_content_progress_message(msg)
        self._skipped_count += int(parsed.get("skipped_delta") or 0)
        source_name = str(parsed.get("source_name") or "").strip()
        if source_name:
            self._cur_source_name = source_name
        question_no = str(parsed.get("question_no") or "").strip()
        if question_no:
            self._cur_question_no = question_no
        round_no = str(parsed.get("round_no") or "").strip()
        if round_no:
            self._cur_round_no = round_no
        if "parallel_text" in parsed:
            self._parallel_text = str(parsed.get("parallel_text") or "")
            return
        if "consistency_text" in parsed:
            self._consistency_text = str(parsed.get("consistency_text") or "")
            if "parallel_text" in parsed:
                self._parallel_text = str(parsed.get("parallel_text") or "")
            return
        detail_text = str(parsed.get("detail_text") or "")
        if detail_text:
            self._detail_text = detail_text
            return

    def _render_status(self) -> None:
        self._status_label.setText(
            build_content_status_text(
                progress_cur=self._progress_cur,
                progress_total=self._progress_total,
                round_limit=question_content_round_limit(),
                concurrency=normalize_ai_concurrency(self._state.question_content_concurrency),
                available_count=self._accepted_count,
                running=self._running,
                failed=self._failed,
                stopped=self._stopped,
                finished=self._finished,
            )
        )

    def _force_finish(self) -> None:
        if not self._running:
            return
        if not (self._items or self._accepted_items_by_index):
            show_message_box(
                self,
                title="暂无可用结果",
                text="至少需要已有 1 题完整通过后，才能强制完成并进入下一步。",
                icon=QMessageBox.Icon.Warning,
            )
            return
        if self._worker is None:
            return
        self._worker.request_stop()
        self._discard_run_results = True
        self._force_finish_requested = False
        self._force_finish_detached = True
        self._stop_requested = False
        self._stop_btn.setEnabled(False)
        self._retry_btn.setEnabled(False)
        current_count = max(len(self._items), len(self._accepted_items_by_index))
        self._materialize_items_from_accepted_cache()
```

```python
        self._running = False
        self._stopped = False
        self._finished = bool(self._items)
        self._failed = False
        self._phase_text = f"正在强制完成…已立即保留当前已处理完成的题目（{current_count} 题），马上进入下一步。"
        self._render_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()
        if self._items:
            QTimer.singleShot(0, self._advance_after_force_finish)

    def _finish_force_complete(self, *, error_message: str = "") -> None:
        self._materialize_items_from_accepted_cache()
        self._stop_btn.setEnabled(False)
        self._retry_btn.setEnabled(True)
        self._running = False
        self._discard_run_results = False
        self._stop_requested = False
        self._stopped = False
        self._finished = bool(self._items)
        self._failed = False
        self._force_finish_requested = False
        self._thread = None
        self._worker = None
        self._force_finish_advance_attempts = 0
        if self._items:
            self._phase_text = f"已强制完成：保留当前已完整返回的 {len(self._items)} 题，正在进入下一步…"
        else:
            self._phase_text = "强制完成失败：当前没有可进入下一步的完整题目。"
            if error_message:
                self._detail_text = error_message
        self._render_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()
        if not self._items:
            show_message_box(
                self,
                title="暂无结果",
                text="当前没有可进入下一步的完整题目，无法强制完成。",
                icon=QMessageBox.Icon.Warning,
            )
            return
        QTimer.singleShot(0, self._advance_after_force_finish)

    def _advance_after_force_finish(self) -> None:
        wizard = self.wizard()
        if not isinstance(wizard, QWizard):
            return
        next_button = wizard.button(QWizard.WizardButton.NextButton)
        if next_button is not None and next_button.isEnabled():
            next_button.click()
            return
        self._force_finish_advance_attempts += 1
        if self._force_finish_advance_attempts <= 5 and self._finished and self._items:
            QTimer.singleShot(50, self._advance_after_force_finish)

    def _cache_accepted_question(self, payload: dict[str, object]) -> None:
        if not bool(payload.get("accepted")):
            return
        accepted_obj = payload.get("accepted_obj")
        if not isinstance(accepted_obj, dict):
            return
        try:
            question = _to_question(accepted_obj)
        except Exception:
            return
        index = int(payload.get("index") or 0)
        if index <= 0:
            return
        self._accepted_items_by_index[index] = question
```

```
        self._accepted_count = max(self._accepted_count, len(self._accepted_items_by_index))

    def _materialize_items_from_accepted_cache(self) -> None:
        if self._items or not self._accepted_items_by_index:
            return
        self._items = [self._accepted_items_by_index[index] for index in sorted(self._accepted_items_by_index.keys())]
        self._accepted_count = len(self._items)

    def _finish_close_after_stop_if_needed(self) -> None:
        if not self._close_after_stop_requested:
            return
        self._close_after_stop_requested = False
        wizard = self.wizard()
        if isinstance(wizard, QWizard):
            QTimer.singleShot(0, wizard.close)

__all__ = ["AiImportContentPage"]
```

```python
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

from PyQt6.QtCore import QThread
from PyQt6.QtWidgets import QTableWidget

from .import_content_detail import (
    apply_compare_row_background,
    apply_content_detail_column_widths,
    apply_partial_pass_highlight,
    build_compare_verdict,
    content_detail_width_signature,
    format_json_cell,
    format_round_secs,
    question_content_detail_headers,
    question_content_payload_model_specs,
    record_round_sec,
    set_content_detail_item,
)
from .import_workers import AiImportContentWorker


def missing_content_model_labels(model_specs: list[dict[str, str]], ready_map: dict[str, bool]) -> list[str]:
    return [
        str(spec.get("label") or spec.get("provider") or spec.get("key") or "").replace("\n", " / ")
        for spec in model_specs
        if not ready_map.get(str(spec.get("key") or ""), False)
    ]


def build_content_status_text(
    *,
    progress_cur: int,
    progress_total: int,
    round_limit: int,
    concurrency: int,
    available_count: int,
    running: bool,
    failed: bool,
    stopped: bool,
    finished: bool,
) -> str:
    if progress_total > 0:
        completed = min(max(int(progress_cur), 0), progress_total)
        progress_text = f"选择题 {completed}/{progress_total}题"
    else:
        progress_text = "选择题 统计中"
    line = f"{progress_text}；最高轮次 {round_limit}轮；并发 {concurrency} 路"
    if not running and not failed and not stopped and finished and progress_total > 0 and progress_cur >= progress_total:
        line = f"{line}；可用题目 {max(0, int(available_count))}题"
    return line


def apply_content_detail_column_widths_if_needed(
    *,
    table: QTableWidget,
    model_specs: list[dict[str, str]],
    current_signature: str,
) -> str:
    signature = content_detail_width_signature(model_specs)
    if signature == current_signature:
        return current_signature
    apply_content_detail_column_widths(table, model_specs)
    return signature


@dataclass
```

```python
class ContentCompareUpdate:
    model_specs: list[dict[str, str]]
    index: int
    round_no: str
    round_limit: int


def apply_content_compare_payload(
    *,
    table: QTableWidget,
    payload: dict[str, object],
    fallback_specs: list[dict[str, str]],
    row_map: dict[int, int],
    compare_secs: dict[int, dict[str, dict[int, int]]],
) -> ContentCompareUpdate | None:
    idx = int(payload.get("index") or 0)
    if idx <= 0:
        return None
    model_specs = question_content_payload_model_specs(payload, fallback_specs)
    table.setColumnCount(len(question_content_detail_headers(model_specs)))
    table.setHorizontalHeaderLabels(question_content_detail_headers(model_specs))
    row = row_map.get(idx)
    if row is None:
        row = table.rowCount()
        table.setRowCount(row + 1)
        row_map[idx] = row
    display_number = str(payload.get("requested_number") or idx)
    set_content_detail_item(table, row, 0, display_number)
    round_no_raw = payload.get("round")
    round_no_int = int(round_no_raw or 0)
    costs_sec_by_model = payload.get("costs_sec_by_model") if isinstance(payload.get("costs_sec_by_model"), dict) else {}
    results_by_model = payload.get("results_by_model") if isinstance(payload.get("results_by_model"), dict) else {}
    for spec in model_specs:
        model_key = str(spec.get("key") or "")
        record_round_sec(
            compare_secs,
            idx=idx,
            round_no=round_no_int,
            model_key=model_key,
            sec_value=costs_sec_by_model.get(model_key),
            ms_value=None,
        )
    for col_index, spec in enumerate(model_specs):
        model_key = str(spec.get("key") or "")
        set_content_detail_item(
            table,
            row,
            1 + col_index,
            format_round_secs(compare_secs, idx=idx, model_key=model_key),
        )
    value_col_offset = 1 + len(model_specs)
    for col_index, spec in enumerate(model_specs):
        model_key = str(spec.get("key") or "")
        set_content_detail_item(
            table,
            row,
            value_col_offset + col_index,
            format_json_cell(results_by_model.get(model_key)),
            is_json=True,
        )
    verdict = build_compare_verdict(payload)
    set_content_detail_item(table, row, value_col_offset + len(model_specs), verdict)
    apply_compare_row_background(table, row=row, payload=payload)
    apply_partial_pass_highlight(table, row=row, payload=payload, model_specs=model_specs)
    return ContentCompareUpdate(
        model_specs=model_specs,
        index=idx,
        round_no=str(round_no_raw or "?"),
```

```python
        round_limit=max(1, int(payload.get("round_limit") or len(model_specs) or 1)),
    )


@dataclass
class ContentWorkerBundle:
    thread: QThread
    worker: AiImportContentWorker


def create_content_worker_bundle(
    *,
    parent,
    model_specs: list[dict[str, str]],
    paths: list[Path],
    question_refs_by_source: dict[str, list[dict[str, str]]],
    max_question_workers: int,
    on_progress,
    on_progress_count,
    on_question,
    on_compare,
    on_done,
    on_error,
) -> ContentWorkerBundle:
    thread = QThread(parent)
    worker = AiImportContentWorker(
        model_specs=model_specs,
        paths=paths,
        question_refs_by_source=dict(question_refs_by_source),
        strategy="per_question",
        max_question_workers=max_question_workers,
    )
    worker.moveToThread(thread)
    thread.started.connect(worker.run)
    worker.progress.connect(on_progress)
    worker.progress_count.connect(on_progress_count)
    worker.question.connect(on_question)
    worker.compare.connect(on_compare)
    worker.done.connect(on_done)
    worker.error.connect(on_error)
    worker.done.connect(thread.quit)
    worker.error.connect(thread.quit)
    thread.finished.connect(worker.deleteLater)
    thread.finished.connect(thread.deleteLater)
    return ContentWorkerBundle(thread=thread, worker=worker)
```

```python
import re
from pathlib import Path

from PyQt6.QtCore import QObject, QThread, pyqtSignal
from PyQt6.QtWidgets import (
    QAbstractItemView,
    QDialog,
    QDialogButtonBox,
    QHeaderView,
    QLabel,
    QMessageBox,
    QSplitter,
    QTableWidgetItem,
    QTextEdit,
    QVBoxLayout,
    QWizard,
    QWizardPage,
)

from sj_generator.application.dedupe.service import (
    DedupeHit,
    dedupe_between_questions_and_db,
)
from sj_generator.application.state import (
    ImportWizardSession,
    library_db_path_from_repo_parent_dir_text,
)
from sj_generator.domain.entities import Question
from sj_generator.infrastructure.persistence.sqlite_repo import (
    DbQuestionRecord,
    load_all_questions,
)
from sj_generator.presentation.qt.constants import PAGE_AI_ANALYSIS, PAGE_IMPORT_SUCCESS
from sj_generator.presentation.qt.pages.import_flow.import_db_service import commit_draft_questions_to_db
from sj_generator.presentation.qt.table_copy import CopyableTableWidget

BUTTON_MIN_WIDTH = 96
BUTTON_MIN_HEIGHT = 36


def _style_dialog_button(button, text: str | None = None) -> None:
    if button is None:
        return
    if text:
        button.setText(text)
    button.setMinimumSize(BUTTON_MIN_WIDTH, BUTTON_MIN_HEIGHT)


def _style_message_box_buttons(box: QMessageBox) -> None:
    for button_type, text in (
        (QMessageBox.StandardButton.Ok, "确定"),
        (QMessageBox.StandardButton.Cancel, "取消"),
        (QMessageBox.StandardButton.Yes, "是"),
        (QMessageBox.StandardButton.No, "否"),
    ):
        _style_dialog_button(box.button(button_type), text)


def _show_message_box(
    parent,
    *,
    title: str,
    text: str,
    icon: QMessageBox.Icon,
    buttons: QMessageBox.StandardButton = QMessageBox.StandardButton.Ok,
    default_button: QMessageBox.StandardButton = QMessageBox.StandardButton.NoButton,
) -> QMessageBox.StandardButton:
    box = QMessageBox(parent)
    box.setWindowTitle(title)
    box.setText(text)
```

```python
    box.setIcon(icon)
    box.setStandardButtons(buttons)
    if default_button != QMessageBox.StandardButton.NoButton:
        box.setDefaultButton(default_button)
    _style_message_box_buttons(box)
    return QMessageBox.StandardButton(box.exec())


def _format_db_question_options(record: DbQuestionRecord) -> str:
    option_texts = [record.option_1, record.option_2, record.option_3, record.option_4]
    if record.question_type == "可转多选":
        lines = [
            f"{marker}. {text.strip()}".rstrip()
            for marker, text in zip(["①", "②", "③", "④"], option_texts)
            if text.strip()
        ]
        for letter, digits in (
            ("A", record.choice_1),
            ("B", record.choice_2),
            ("C", record.choice_3),
            ("D", record.choice_4),
        ):
            digits = (digits or "").strip()
            if not digits:
                continue
            lines.append(f"{letter}. {_digits_to_circled(digits)}")
        return "\n".join(lines)
    if record.question_type == "多选":
        markers = ["①", "②", "③", "④"]
    else:
        markers = ["A", "B", "C", "D"]
    return "\n".join(
        f"{marker}. {text.strip()}".rstrip()
        for marker, text in zip(markers, option_texts)
        if text.strip()
    )


def _format_db_question_answer(record: DbQuestionRecord) -> str:
    answer = (record.answer or "").strip()
    if record.question_type == "可转多选" and any(
        value.strip() for value in (record.choice_1, record.choice_2, record.choice_3, record.choice_4)
    ):
        return answer
    if record.question_type not in {"多选", "可转多选"}:
        return answer
    return _digits_to_circled(answer.replace(",", ""))


def _digits_to_circled(text: str) -> str:
    return "".join(
        {
            "1": "①",
            "2": "②",
            "3": "③",
            "4": "④",
            "5": "⑤",
            "6": "⑥",
            "7": "⑦",
            "8": "⑧",
            "9": "⑨",
        }.get(ch, ch)
        for ch in text
    )


class DedupeResultPage(QWizardPage):
    def __init__(self, state: ImportWizardSession) -> None:
        super().__init__()
        self._state = state
```

```python
        self.setTitle("库内查重（结果）")

        self._status_label = QLabel("")
        self._status_label.setWordWrap(True)

        self._table = CopyableTableWidget()
        self._table.setColumnCount(5)
        self._table.setHorizontalHeaderLabels(["当前题目", "当前序号", "库内来源", "所属层级", "相似度"])
        self._table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
        self._table.setRowCount(0)
        self._table.setEditTriggers(QAbstractItemView.EditTrigger.NoEditTriggers)
        self._table.setSelectionBehavior(QAbstractItemView.SelectionBehavior.SelectRows)
        self._table.setSelectionMode(QAbstractItemView.SelectionMode.SingleSelection)
        self._table.cellDoubleClicked.connect(self._open_detail)

        hint = QLabel("双击行可查看当前题目与库内题目的题干、选项与答案。")
        hint.setWordWrap(True)

        layout = QVBoxLayout()
        layout.addWidget(self._status_label)
        layout.addWidget(self._table, 1)
        layout.addWidget(hint)
        self.setLayout(layout)

        self._thread: QThread | None = None
        self._worker: _DedupeWorker | None = None
        self._running = False
        self._done = False
        self._hits: list[DedupeHit] = []
        self._questions_cache: dict[Path, list[Question]] = {}
        self._current_left_source_path = self._resolve_left_source_path()

    def initializePage(self) -> None:
        self._sync_wizard_buttons()
        self._current_left_source_path = self._resolve_left_source_path()
        if self._state.draft.dedupe_hits is not None:
            self._hits = self._state.draft.dedupe_hits
            self._render_hits(self._hits)
            self._running = False
            self._done = True
            self._sync_wizard_buttons()
            self.completeChanged.emit()
            return

        self._hits = []
        self._table.setRowCount(0)
        self._status_label.setText("正在查重…")
        self._running = True
        self._done = False
        self.completeChanged.emit()

        repo = self._current_left_source_path
        left_questions = list(self._state.draft.questions)
        threshold = self._state.dedupe_threshold
        if not left_questions:
            self._on_error("当前题库草稿为空，无法执行查重。")
            return
        db_path = self._library_db_path()
        if not db_path.exists():
            self._on_error(f"当前总库不存在：{db_path}")
            return

        thread = QThread(self)
        worker = _DedupeWorker(left_repo=repo, left_questions=left_questions, db_path=db_path, threshold=threshold)
        worker.moveToThread(thread)
        thread.started.connect(worker.run)
        worker.done.connect(self._on_done)
        worker.error.connect(self._on_error)
        worker.done.connect(thread.quit)
```

```python
        worker.error.connect(thread.quit)
        thread.finished.connect(worker.deleteLater)
        thread.finished.connect(thread.deleteLater)
        self._thread = thread
        self._worker = worker
        thread.start()

    def isComplete(self) -> bool:
        return self._done and (not self._running)

    def validatePage(self) -> bool:
        if not self._state.analysis_enabled:
            return commit_draft_questions_to_db(self, self._state)
        return True

    def nextId(self) -> int:
        if self._state.analysis_enabled:
            return PAGE_AI_ANALYSIS
        return PAGE_IMPORT_SUCCESS

    def _sync_wizard_buttons(self) -> None:
        wizard = self.wizard()
        if not isinstance(wizard, QWizard):
            return
        next_text = "查重中…"
        if self._done and not self._running:
            next_text = "进入解析" if self._state.analysis_enabled else "写入题库"
        wizard.setButtonText(QWizard.WizardButton.BackButton, "返回解析")
        wizard.setButtonText(QWizard.WizardButton.NextButton, next_text)
        wizard.setButtonText(QWizard.WizardButton.CancelButton, "返回开始页")

    def _render_hits(self, hits: list[DedupeHit]) -> None:
        self._table.setRowCount(len(hits))
        for row, hit in enumerate(hits):
            self._table.setItem(row, 0, QTableWidgetItem(hit.left_stem.strip()[:40]))
            self._table.setItem(row, 1, QTableWidgetItem(hit.left_number or ""))
            self._table.setItem(row, 2, QTableWidgetItem(hit.right_file.name))
            self._table.setItem(row, 3, QTableWidgetItem(hit.right_level_path or ""))
            self._table.setItem(row, 4, QTableWidgetItem(f"{hit.similarity:.3f}"))

    def _on_done(self, hits: list[DedupeHit]) -> None:
        self._running = False
        self._done = True
        self._thread = None
        self._worker = None

        self._hits = hits
        self._state.set_dedupe_hits(hits)
        if not hits:
            self._status_label.setText("未发现达到阈值的重复题。")
            self._table.setRowCount(0)
        else:
            self._render_hits(hits)
            self._status_label.setText(f"库内查重完成：{len(hits)} 条结果（双击查看详情）")
        self._sync_wizard_buttons()
        self.completeChanged.emit()

    def _on_error(self, msg: str) -> None:
        self._running = False
        self._done = False
        self._thread = None
        self._worker = None
        _show_message_box(self, title="查重失败", text=msg, icon=QMessageBox.Icon.Critical)
        self._status_label.setText("查重失败。")
        self._sync_wizard_buttons()
        self.completeChanged.emit()

    def _open_detail(self, row: int, _col: int) -> None:
        if row < 0 or row >= len(self._hits):
            return
```

```python
        hit = self._hits[row]
        dialog = _DedupeDetailDialog(self, hit=hit, loader=self._load_questions_cached)
        dialog.exec()

    def _resolve_left_source_path(self) -> Path:
        if self._state.repo_path is not None:
            return self._state.repo_path
        if self._state.project_dir is not None:
            return self._state.project_dir / "当前草稿.xlsx"
        return self._library_db_path().parent / "当前草稿.xlsx"

    def _load_questions_cached(self, path: Path) -> list[Question]:
        resolved_path = path.resolve()
        if resolved_path in self._questions_cache:
            return self._questions_cache[resolved_path]
        if resolved_path == self._current_left_source_path.resolve():
            questions = list(self._state.draft.questions)
        else:
            if resolved_path == self._library_db_path().resolve():
                records = load_all_questions(resolved_path)
                questions = [
                    Question(
                        number=record.id,
                        stem=record.stem,
                        options=_format_db_question_options(record),
                        answer=_format_db_question_answer(record),
                        analysis=record.analysis,
                        question_type=record.question_type,
                        choice_1=record.choice_1,
                        choice_2=record.choice_2,
                        choice_3=record.choice_3,
                        choice_4=record.choice_4,
                    )
                    for record in records
                ]
            else:
                from sj_generator.infrastructure.persistence.excel_repo import load_questions

                questions = load_questions(resolved_path)
        self._questions_cache[resolved_path] = questions
        return questions

    def _library_db_path(self) -> Path:
        return library_db_path_from_repo_parent_dir_text(self._state.default_repo_parent_dir_text)


class _DedupeWorker(QObject):
    done = pyqtSignal(object)
    error = pyqtSignal(str)

    def __init__(self, *, left_repo: Path, left_questions: list[Question], db_path: Path, threshold: float) -> None:
        super().__init__()
        self._left_repo = left_repo
        self._left_questions = left_questions
        self._db_path = db_path
        self._threshold = threshold

    def run(self) -> None:
        try:
            hits = dedupe_between_questions_and_db(
                left_questions=self._left_questions,
                left_file=self._left_repo,
                db_path=self._db_path,
                threshold=self._threshold,
            )
            self.done.emit(hits)
        except Exception as exc:
            self.error.emit(str(exc))

```

```python

class _DedupeDetailDialog(QDialog):
    def __init__(self, parent, *, hit, loader) -> None:
        super().__init__(parent)
        self.setWindowTitle("查重详情")

        left = QTextEdit()
        left.setReadOnly(True)
        right = QTextEdit()
        right.setReadOnly(True)

        splitter = QSplitter()
        splitter.addWidget(left)
        splitter.addWidget(right)
        splitter.setStretchFactor(0, 1)
        splitter.setStretchFactor(1, 1)

        buttons = QDialogButtonBox(QDialogButtonBox.StandardButton.Ok)
        buttons.accepted.connect(self.accept)
        _style_dialog_button(buttons.button(QDialogButtonBox.StandardButton.Ok), "确定")

        layout = QVBoxLayout()
        layout.addWidget(QLabel(f"相似度：{hit.similarity:.3f}"))
        layout.addWidget(splitter)
        layout.addWidget(buttons)
        self.setLayout(layout)

        left_question = _find_question(loader(hit.left_file), hit.left_number, hit.left_stem)
        right_question = _find_question(loader(hit.right_file), hit.right_number, hit.right_stem)
        left.setPlainText(_format_question_block(hit.left_file, left_question, fallback_stem=hit.left_stem))
        right.setPlainText(_format_question_block(hit.right_file, right_question, fallback_stem=hit.right_stem))


def _format_question_block(path: Path, question: Question | None, *, fallback_stem: str) -> str:
    if question is None:
        return "\n".join(
            [
                f"文件：{path.name}",
                "编号：",
                "",
                "题干：",
                (fallback_stem or "").strip(),
            ]
        ).strip() + "\n"
    return "\n".join(
        [
            f"文件：{path.name}",
            f"编号：{(question.number or '').strip()}",
            "",
            "题干：",
            (question.stem or "").strip(),
            "",
            "选项：",
            (question.options or "").strip(),
            "",
            "答案：",
            (question.answer or "").strip(),
        ]
    ).strip() + "\n"


def _find_question(questions: list[Question], number: str, stem: str) -> Question | None:
    normalized_number = (number or "").strip()
    if normalized_number:
        for question in questions:
            if (question.number or "").strip() == normalized_number:
                return question
    key = _norm_text(stem)
    if not key:
```

```
        return None
    for question in questions:
        if _norm_text(question.stem) == key:
            return question
    return None


def _norm_text(text: str) -> str:
    normalized = (text or "").strip().replace("\r", "\n")
    normalized = re.sub(r"\s+", " ", normalized)
    return normalized.strip()

__all__ = ["DedupeResultPage"]
```

```python
from __future__ import annotations

import json

from PyQt6.QtCore import Qt
from PyQt6.QtGui import QBrush, QColor
from PyQt6.QtWidgets import QHeaderView, QTableWidget, QTableWidgetItem

from sj_generator.infrastructure.llm.import_questions import _fingerprint_question_obj, question_content_model_specs
from sj_generator.presentation.qt.compare_highlight import compare_highlight_model_styles
from sj_generator.application.state import normalize_ai_concurrency


def format_json_cell(value: object) -> str:
    if value is None:
        return ""
    try:
        return json.dumps(value, ensure_ascii=False)
    except Exception:
        return str(value)


def question_ref_total_count(value: dict[str, list[dict[str, str]]] | None) -> int:
    if not isinstance(value, dict):
        return 0
    total = 0
    for items in value.values():
        if isinstance(items, list):
            total += len(items)
    return total


def question_content_active_model_specs() -> list[dict[str, str]]:
    return [item for item in question_content_model_specs() if isinstance(item, dict)]


def effective_content_question_workers(total_budget: int, model_count: int) -> int:
    normalized_budget = normalize_ai_concurrency(total_budget)
    _ = model_count
    return max(1, normalized_budget)


def question_content_payload_model_specs(
    payload: dict[str, object],
    fallback_specs: list[dict[str, str]] | None = None,
) -> list[dict[str, str]]:
    raw = payload.get("model_specs")
    if isinstance(raw, list) and raw:
        return [item for item in raw if isinstance(item, dict)]
    return [item for item in (fallback_specs or question_content_active_model_specs()) if isinstance(item, dict)]


def question_content_model_signature(model_specs: list[dict[str, str]] | None = None) -> str:
    specs = [item for item in (model_specs or question_content_active_model_specs()) if isinstance(item, dict)]
    try:
        return json.dumps(specs, ensure_ascii=False, sort_keys=True)
    except Exception:
        return str(specs)


def question_content_detail_headers(model_specs: list[dict[str, str]] | None = None) -> list[str]:
    specs = [item for item in (model_specs or question_content_active_model_specs()) if isinstance(item, dict)]
    headers: list[str] = ["题号"]
    for spec in specs:
        label = str(spec.get("label") or "模型")
        headers.append(f"{label}\n耗时")
```

```python
    for spec in specs:
        label = str(spec.get("label") or "模型")
        headers.append(f"{label}\nJSON结果")
    headers.append("一致性结论")
    return headers


def has_question_content_payload(value: object) -> bool:
    if value is None:
        return False
    if isinstance(value, dict):
        return any(str(item or "").strip() for item in value.values())
    if isinstance(value, list):
        return len(value) > 0
    return bool(str(value).strip())


def sec_int(sec_value: object, ms_value: object) -> int | None:
    if isinstance(sec_value, (int, float)):
        return int(round(float(sec_value)))
    if isinstance(ms_value, (int, float)):
        return int(round(float(ms_value) / 1000.0))
    return None


def record_round_sec(
    compare_secs: dict[int, dict[str, dict[int, int]]],
    *,
    idx: int,
    round_no: int,
    model_key: str,
    sec_value: object,
    ms_value: object,
) -> None:
    if idx <= 0 or round_no <= 0:
        return
    sec_value_int = sec_int(sec_value, ms_value)
    if sec_value_int is None:
        return
    compare_secs.setdefault(idx, {}).setdefault(model_key, {})[round_no] = sec_value_int


def format_round_secs(compare_secs: dict[int, dict[str, dict[int, int]]], *, idx: int, model_key: str) -> str:
    model_rounds = compare_secs.get(idx, {}).get(model_key, {})
    if not model_rounds:
        return ""
    parts = [str(model_rounds[round_no]) for round_no in sorted(model_rounds.keys())]
    return "+".join(parts)


def content_detail_width_signature(model_specs: list[dict[str, str]]) -> str:
    keys = [str(item.get("key") or "") for item in model_specs if isinstance(item, dict)]
    return "|".join(keys)


def calculate_content_detail_column_widths(*, column_count: int, available_width: int, model_count: int) -> list[int]:
    weights = [1] + ([2] * model_count) + ([8] * model_count) + [2]
    if len(weights) != column_count:
        weights = [1] * column_count
    total_weight = max(1, sum(weights))
    min_widths = [80] + ([120] * model_count) + ([320] * model_count) + [160]
    if len(min_widths) != column_count:
        min_widths = [80] * column_count
    widths: list[int] = []
    for weight, min_width in zip(weights, min_widths):
        target_width = max(min_width, int(round(max(1, available_width) * weight / total_weight)))
        widths.append(target_width)
    return widths

```

```python

def apply_content_detail_column_widths(table: QTableWidget, model_specs: list[dict[str, str]]) -> None:
    if table.columnCount() <= 0:
        return
    header = table.horizontalHeader()
    header.setSectionResizeMode(QHeaderView.ResizeMode.Interactive)
    widths = calculate_content_detail_column_widths(
        column_count=table.columnCount(),
        available_width=table.viewport().width(),
        model_count=len([item for item in model_specs if isinstance(item, dict)]),
    )
    for column, width in enumerate(widths):
        table.setColumnWidth(column, width)


def set_content_detail_item(table: QTableWidget, row: int, col: int, text: str, *, is_json: bool = False) -> None:
    item = QTableWidgetItem(text)
    if is_json:
        item.setTextAlignment(int(Qt.AlignmentFlag.AlignLeft | Qt.AlignmentFlag.AlignVCenter))
    else:
        item.setTextAlignment(int(Qt.AlignmentFlag.AlignHCenter | Qt.AlignmentFlag.AlignVCenter))
    table.setItem(row, col, item)


def compare_row_brush(payload: dict[str, object]) -> QBrush:
    if bool(payload.get("partial")):
        return QBrush()
    if bool(payload.get("accepted")):
        return QBrush(QColor("#e6f4ea"))
    return QBrush(QColor("#fde2e1"))


def apply_compare_row_background(table: QTableWidget, *, row: int, payload: dict[str, object]) -> None:
    brush = compare_row_brush(payload)
    for col in range(table.columnCount()):
        item = table.item(row, col)
        if item is not None:
            item.setBackground(brush)


def clear_compare_payload_background(
    table: QTableWidget,
    *,
    row: int,
    payload: dict[str, object],
    model_specs: list[dict[str, str]],
) -> None:
    payload_col_start = 1 + len(model_specs)
    payload_col_end = payload_col_start + len(model_specs)
    brush = compare_row_brush(payload)
    for col in range(payload_col_start, payload_col_end):
        item = table.item(row, col)
        if item is not None:
            item.setBackground(brush)


def highlight_sig_text(obj: object) -> str:
    if isinstance(obj, dict):
        return _fingerprint_question_obj(obj)
    if obj is None:
        return ""
    if isinstance(obj, str):
        return obj.strip()
    if isinstance(obj, list) and not obj:
        return ""
    return str(obj).strip()


def apply_partial_pass_highlight(
```

```python
    table: QTableWidget,
    *,
    row: int,
    payload: dict[str, object],
    model_specs: list[dict[str, str]],
) -> None:
    clear_compare_payload_background(table, row=row, payload=payload, model_specs=model_specs)
    value_col_offset = 1 + len(model_specs)
    model_cols = [
        (str(spec.get("key") or ""), value_col_offset + index)
        for index, spec in enumerate(model_specs)
    ]
    results_by_model = payload.get("results_by_model") if isinstance(payload.get("results_by_model"), dict) else {}
    highlight_styles = compare_highlight_model_styles(
        model_sigs={key: highlight_sig_text(results_by_model.get(key)) for key, _ in model_cols},
        round_no=int(payload.get("round") or 0),
        round_matched_count=int(payload.get("round_matched_count") or 0),
    )
    if not highlight_styles:
        return
    fail_brush = QBrush(QColor(255, 199, 206))
    empty_brush = QBrush(QColor(255, 235, 156))
    for key, col in model_cols:
        style = highlight_styles.get(key)
        if not style:
            continue
        item = table.item(row, col)
        if item is None:
            continue
        value = results_by_model.get(key)
        if style == "yellow":
            item.setBackground(empty_brush)
        elif style == "red" and not has_question_content_payload(value):
            item.setBackground(fail_brush)


def build_compare_verdict(payload: dict[str, object]) -> str:
    round_no = int(payload.get("round") or 0)
    round_labels = {1: "一轮", 2: "二轮", 3: "三轮"}
    label = round_labels.get(round_no, f"第{round_no}轮")
    round_matched = int(payload.get("round_matched_count") or 0)
    matched = int(payload.get("matched_count") or 0)
    ratio_numerator = max(1, int(payload.get("ratio_numerator") or 1))
    ratio_denominator = max(1, int(payload.get("ratio_denominator") or 1))
    completed = int(payload.get("completed_model_count") or 0)
    total_models = int(payload.get("active_model_count") or 0)
    standard_required = max(1, ratio_numerator * max(1, round_no))
    standard_total = max(standard_required, ratio_denominator * max(1, round_no))
    round_standard_total = max(1, ratio_denominator)
    current_text = f"当前一致：{matched}/{standard_total}"
    round_text = f"本轮通过：{round_matched}/{round_standard_total}"
    if bool(payload.get("partial")):
        status_text = f"{label}进行中（已返回 {completed}/{total_models}）"
        return "\n".join(
            [
                status_text,
                current_text,
                round_text,
                f"通过标准：≥{standard_required}/{standard_total}",
            ]
        )
    accepted = bool(payload.get("accepted"))
    status_text = f"{label}结束（{'通过' if accepted else '未通过'}）"
    return "\n".join(
        [
            status_text,
            current_text,
            round_text,
            f"通过标准：≥{standard_required}/{standard_total}",
```

```
        ]
    )

```

sj_generator/presentation/qt/pages/import_flow/import_db_service.py

060-01

```python
from __future__ import annotations

from PyQt6.QtWidgets import QMessageBox, QWidget

from sj_generator.application.importing import commit_questions_to_db
from sj_generator.application.state import ImportWizardSession, library_db_path_from_repo_parent_dir_text
from sj_generator.presentation.qt.import_costs import freeze_import_cost_result
from sj_generator.presentation.qt.message_box import show_message_box


def commit_draft_questions_to_db(parent: QWidget | None, state: ImportWizardSession) -> bool:
    if state.execution.db_import_completed:
        return True
    freeze_import_cost_result(state)

    questions = list(state.draft.questions)
    if not questions:
        show_message_box(parent, title="无法导入数据库", text="当前草稿为空，无法写入数据库。", icon=QMessageBox.Icon.Warning)
        return False

    level_path = state.source.import_level_path.strip()
    if not level_path:
        show_message_box(parent, title="无法导入数据库", text="未填写层级归属，无法写入数据库。", icon=QMessageBox.Icon.Warning)
        return False

    try:
        count = commit_questions_to_db(
            db_path=library_db_path_from_repo_parent_dir_text(state.default_repo_parent_dir_text),
            questions=questions,
            level_path=level_path,
            source_files=state.source.files or [],
            textbook_version=state.preferred_textbook_version,
        )
    except Exception as e:
        state.execution.db_import_error = str(e)
        show_message_box(parent, title="导入数据库失败", text=str(e), icon=QMessageBox.Icon.Critical)
        return False

    state.execution.mark_db_import_completed(count)
    return True

```

```python
from __future__ import annotations

from pathlib import Path
import re

from PyQt6.QtCore import QRegularExpression
from PyQt6.QtGui import QDropEvent, QRegularExpressionValidator
from PyQt6.QtWidgets import QLineEdit, QProgressBar, QPushButton, QStyle, QStyledItemDelegate, QStyleOptionVieWItem

from sj_generator.application.state import ImportWizardSession

BUTTON_MIN_WIDTH = 96
BUTTON_MIN_HEIGHT = 36

LEVEL_PATH_RE = re.compile(r"^\d+\.\d+\.\d+$")
LEVEL_PATH_EDIT_RE = QRegularExpression(r"\d*(?:\.\d*(?:\.\d*)?)?")


def style_dialog_button(button: QPushButton | None, text: str | None = None) -> None:
    if button is None:
        return
    if text:
        button.setText(text)
    button.setMinimumSize(BUTTON_MIN_WIDTH, BUTTON_MIN_HEIGHT)


def style_busy_progress(progress: QProgressBar) -> None:
    progress.setTextVisible(False)
    progress.setStyleSheet(
        """
        QProgressBar {
            min-height: 14px;
            max-height: 14px;
            border: 1px solid #cfd6e4;
            border-radius: 7px;
            background: #eef2f8;
        }
        QProgressBar::chunk {
            width: 36px;
            margin: 0px;
            border-radius: 7px;
            background-color: #5b8def;
        }
        """
    )


def sanitize_filename(name: str) -> str:
    text = (name or "").strip()
    text = re.sub(r'[<>:"/\\\\|?*]+', "_", text)
    text = text.replace("\n", " ").replace("\r", " ").replace("\t", " ")
    text = text.strip(" .")
    return text


def unique_child_dir(parent: Path, base_name: str) -> Path:
    candidate = parent / base_name
    if not candidate.exists():
        return candidate
    index = 2
    while True:
        numbered = parent / f"{base_name}_{index}"
        if not numbered.exists():
            return numbered
        index += 1


def rename_project(state: ImportWizardSession, *, new_name: str) -> bool:
    project_dir = state.project_dir
```

```python
    repo_path = state.repo_path
    if project_dir is None or repo_path is None:
        return False
    safe = sanitize_filename(new_name)
    if not safe:
        return False

    parent = project_dir.parent
    target_dir = unique_child_dir(parent, safe)
    target_name = target_dir.name
    target_repo = target_dir / f"{target_name}.xlsx"

    state.project_dir = target_dir
    state.repo_path = target_repo
    state.project_name_is_placeholder = False
    return True


def extract_paths_from_drop_event(event: QDropEvent) -> list[Path]:
    md = event.mimeData()
    if not md.hasUrls():
        return []
    out: list[Path] = []
    for url in md.urls():
        if not url.isLocalFile():
            continue
        path = Path(url.toLocalFile())
        if path.exists():
            out.append(path)
    return out


def merge_paths_text(existing_text: str, paths: list[Path]) -> str:
    existing = [p.strip() for p in (existing_text or "").split(";") if p.strip()]
    seen: set[str] = set()
    merged: list[str] = []
    for value in existing:
        if value not in seen:
            merged.append(value)
            seen.add(value)
    for path in paths:
        value = str(path)
        if value not in seen:
            merged.append(value)
            seen.add(value)
    return "; ".join(merged)


def is_valid_level_path(value: str) -> bool:
    return bool(LEVEL_PATH_RE.fullmatch((value or "").strip()))


class LevelPathItemDelegate(QStyledItemDelegate):
    def paint(self, painter, option, index):
        opt = QStyleOptionViewItem(option)
        opt.state &= ~QStyle.StateFlag.State_Selected
        super().paint(painter, opt, index)

    def createEditor(self, parent, option, index):
        editor = QLineEdit(parent)
        editor.setPlaceholderText("例如：3.2.2")
        editor.setValidator(QRegularExpressionValidator(LEVEL_PATH_EDIT_RE, editor))
        return editor


class PreserveCellBackgroundDelegate(QStyledItemDelegate):
    def paint(self, painter, option, index):
        opt = QStyleOptionViewItem(option)
        opt.state &= ~QStyle.StateFlag.State_Selected
        super().paint(painter, opt, index)
```

```
140
```

```python
from __future__ import annotations

import re


def extract_question_ref_source_name(msg: str, current_source_name: str) -> str:
    text = str(msg or "")
    current = str(current_source_name or "").strip() or "-"
    by_colon = re.match(r"^([^：]+)：", text)
    if by_colon:
        return by_colon.group(1)
    by_phase = re.search(r"^(.+?)：题号与题型解析中", text)
    if by_phase:
        return by_phase.group(1)
    return current


def parse_content_progress_message(msg: str) -> dict[str, object]:
    text = str(msg or "")
    parsed: dict[str, object] = {
        "skipped_delta": 1 if "已跳过" in text else 0,
    }

    source_match = re.match(r"^([^：]+)：", text)
    if source_match:
        parsed["source_name"] = source_match.group(1)

    parallel_match = re.search(
        r"第\s*(\d+)\s*/\s*(\d+)\s*题（题号\s*([^，]+)，第\s*(\d+)\s*/\s*(\d+)\s*轮，多模型并行请求中）",
        text,
    )
    if parallel_match:
        question_index, total_count, _number, round_no, total_rounds = parallel_match.groups()
        parsed["question_no"] = question_index
        parsed["round_no"] = round_no
        parsed["parallel_text"] = f"并行状态：第 {round_no}/{total_rounds} 轮多模型并行请求中（题目 {question_index}/{total_count}）"
        return parsed

    accepted_match = re.search(
        r"第\s*(\d+)\s*/\s*(\d+)\s*题（题号\s*([^，]+)，第\s*(\d+)\s*/\s*(\d+)\s*轮，达到一致阈值）",
        text,
    )
    if accepted_match:
        question_index, total_count, _number, round_no, total_rounds = accepted_match.groups()
        parsed["consistency_text"] = f"一致性结论：第 {round_no}/{total_rounds} 轮一致（题目 {question_index}/{total_count}）"
        parsed["parallel_text"] = ""
        return parsed

    rejected_match = re.search(
        r"第\s*(\d+)\s*/\s*(\d+)\s*题（题号\s*([^，]+)，第\s*(\d+)\s*/\s*(\d+)\s*轮，未达到一致阈值）",
        text,
    )
    if rejected_match:
        question_index, total_count, _number, round_no, total_rounds = rejected_match.groups()
        parsed["consistency_text"] = f"一致性结论：第 {round_no}/{total_rounds} 轮不一致（题目 {question_index}/{total_count}）"
        return parsed

    detail_match = re.search(
        r"第\s*(\d+)\s*/\s*(\d+)\s*题（题号\s*([^，]+)，第\s*(\d+)\s*/\s*(\d+)\s*轮，([^）]+)）",
        text,
    )
    if detail_match:
        question_index, total_count, number, round_no, total_rounds, model_name = detail_match.groups()
        parsed["question_no"] = question_index
        parsed["round_no"] = round_no
        parsed["detail_text"] = (
            f"当前题：{question_index}/{total_count}；题号：{number}；"
```

```
            f"轮次：{round_no}/{total_rounds}；模型：{model_name}"
        )
    return parsed

```

sj_generator/presentation/qt/pages/import_flow/import_question_ref_detail.py 063-01

```python
from __future__ import annotations

from pathlib import Path

from PyQt6.QtCore import Qt
from PyQt6.QtGui import QBrush, QColor
from PyQt6.QtWidgets import QComboBox, QTableWidget, QTableWidgetItem

from sj_generator.infrastructure.llm.question_ref_scan import (
    merged_question_ref_numbers,
    question_ref_model_specs,
    question_ref_type_map,
    row_consistency_text,
    special_marker_text,
)


def question_ref_row_background(verdict_text: str, *, is_manually_overridden: bool = False) -> QBrush | None:
    if is_manually_overridden:
        return QBrush(QColor("#fff3cd"))
    text = str(verdict_text or "").strip()
    if not text:
        return None
    if "不一致" in text:
        return QBrush(QColor("#fde2e1"))
    if "一致" in text:
        return QBrush(QColor("#e6f4ea"))
    return None


def question_ref_detail_headers(model_specs: list[dict[str, str]] | None = None) -> list[str]:
    specs = model_specs or question_ref_detail_model_specs()
    headers = ["资料名称"]
    duplicate_counts: dict[str, int] = {}
    for spec in specs:
        label = str(spec.get("label") or "").strip() or "模型"
        duplicate_counts[label] = duplicate_counts.get(label, 0) + 1
    duplicate_indexes: dict[str, int] = {}
    for index, spec in enumerate(specs, start=1):
        label = str(spec.get("label") or "").strip() or "模型"
        duplicate_indexes[label] = duplicate_indexes.get(label, 0) + 1
        display_label = label
        if duplicate_counts.get(label, 0) > 1:
            display_label = f"模型{index}\n{label}"
        headers.extend([f"{display_label}\n题号", f"{display_label}\n类型"])
    headers.append("对比结论")
    headers.append("题型")
    return headers


def question_ref_detail_model_specs(payloads: dict[str, dict[str, object]] | None = None) -> list[dict[str, str]]:
    payload_map = payloads if isinstance(payloads, dict) else {}
    specs: list[dict[str, str]] = []
    seen: set[str] = set()
    for payload in payload_map.values():
        if not isinstance(payload, dict):
            continue
        model_specs = payload.get("model_specs")
        if not isinstance(model_specs, list):
            continue
        for item in model_specs:
            if not isinstance(item, dict):
                continue
            key = str(item.get("key") or "").strip()
            if not key or key in seen:
                continue
            seen.add(key)
            specs.append(
                {
```

```python
                    "key": key,
                    "label": str(item.get("label") or "").strip(),
                }
            )
    return specs or [
        {
            "key": str(item.get("key") or ""),
            "label": str(item.get("label") or "").strip(),
        }
        for item in question_ref_model_specs()
    ]


def question_ref_detail_model_values(
    payload: dict[str, object],
    *,
    model_specs: list[dict[str, str]],
) -> list[list[dict[str, str]]]:
    values: list[list[dict[str, str]]] = []
    results = payload.get("results") if isinstance(payload.get("results"), dict) else {}
    for spec in model_specs:
        key = str(spec.get("key") or "").strip()
        raw = results.get(key, []) if isinstance(results, dict) else []
        if isinstance(raw, list):
            normalized: list[dict[str, str]] = []
            for item in raw:
                if not isinstance(item, dict):
                    continue
                normalized.append(
                    {
                        "number": str(item.get("number") or "").strip(),
                        "question_type": str(item.get("question_type") or "").strip(),
                    }
                )
            values.append(normalized)
        else:
            values.append([])
    return values


def question_ref_number_order(model_refs: list[list[dict[str, str]]]) -> list[str]:
    return merged_question_ref_numbers(*model_refs)


def question_type_options() -> list[str]:
    return ["单选", "多选", "可转多选"]


def question_type_from_candidates(values: list[str]) -> str:
    options = question_type_options()
    normalized = [str(value or "").strip() for value in values if str(value or "").strip() in options]
    if not normalized:
        return options[0]
    counts: dict[str, int] = {}
    for value in normalized:
        counts[value] = counts.get(value, 0) + 1
    best_type = max(
        options,
        key=lambda item: (counts.get(item, 0), -options.index(item)),
    )
    return best_type if counts.get(best_type, 0) > 0 else normalized[0]


def question_type_conflicts(values: list[str]) -> bool:
    options = question_type_options()
    normalized = {str(value or "").strip() for value in values if str(value or "").strip() in options}
    return len(normalized) > 1


def build_question_ref_detail_rows(
```

```python
    *,
    payloads: dict[str, dict[str, object]],
    model_specs: list[dict[str, str]],
    resolve_manual_type,
) -> list[dict[str, object]]:
    rows: list[dict[str, object]] = []
    for source_key, payload in payloads.items():
        source_name = str(payload.get("source_name") or Path(source_key).name or source_key)
        model_refs = question_ref_detail_model_values(payload, model_specs=model_specs)
        number_order = question_ref_number_order(model_refs)
        type_maps = [question_ref_type_map(refs) for refs in model_refs]
        marker_map = payload.get("markers") if isinstance(payload.get("markers"), dict) else {}
        elapsed_map = payload.get("elapsed_s_by_model") if isinstance(payload.get("elapsed_s_by_model"), dict) else {}
        marker_values = [
            special_marker_text(marker_map.get(str(spec.get("key") or "")))
            for spec in model_specs
        ]
        if not number_order:
            marker_summary = "；".join(text for text in marker_values if text)
            if marker_summary:
                rows.append(
                    {
                        "cells": [source_name] + [""] * max(0, len(question_ref_detail_headers(model_specs)) - 3) + [f"异常：{marker_summary}", ""],
                        "source_key": source_key,
                        "number": "",
                        "manual_type": "",
                    }
                )
            else:
                rows.append(
                    {
                        "cells": [source_name] + [""] * max(0, len(question_ref_detail_headers(model_specs)) - 3) + ["未识别到题号", ""],
                        "source_key": source_key,
                        "number": "",
                        "manual_type": "",
                    }
                )
            continue
        for number in number_order:
            row_values = [source_name]
            for model_index, refs in enumerate(model_refs):
                model_key = str(model_specs[model_index].get("key") or "")
                type_map = type_maps[model_index]
                row_values.append(number if number in type_map else "")
                question_type = type_map.get(number, "")
                elapsed_s = str(elapsed_map.get(model_key) or "").strip()
                type_text = question_type
                if elapsed_s:
                    type_text = f"{question_type}\n{elapsed_s}s" if question_type else f"{elapsed_s}s"
                row_values.append(type_text)
            types = [type_map.get(number, "").strip() for type_map in type_maps]
            suggested_type = question_type_from_candidates(types) if types else ""
            manual_type = str(
                resolve_manual_type(
                    source_key=source_key,
                    number=number,
                    payload=payload,
                    model_types=types,
                )
                or ""
            ).strip()
            row_values.append(row_consistency_text(types))
            row_values.append(manual_type)
            rows.append(
                {
                    "cells": row_values,
                    "source_key": source_key,
```

```python
                    "number": number,
                    "manual_type": manual_type,
                    "suggested_type": suggested_type,
                }
            )
    return rows


def build_manual_type_combo(
    *,
    table: QTableWidget,
    current_type: str,
    background: QBrush | None,
    on_changed,
) -> QComboBox:
    combo = QComboBox(table)
    options = [""] + question_type_options()
    combo.addItems(options)
    combo.setEditable(True)
    selected = current_type if current_type in options else ""
    combo.setCurrentText(selected)
    line_edit = combo.lineEdit()
    if line_edit is not None:
        line_edit.setReadOnly(True)
        line_edit.setAlignment(Qt.AlignmentFlag.AlignCenter)
    combo.currentTextChanged.connect(on_changed)
    if background is not None:
        combo.setStyleSheet(f"QComboBox {{ background-color: {background.color().name()}; }}")
    return combo


def populate_question_ref_detail_table(
    *,
    table: QTableWidget,
    rows: list[dict[str, object]],
    build_combo,
    manual_col: int,
    overridden_pairs: set[tuple[str, str]],
) -> None:
    table.setRowCount(len(rows))
    for row_index, row_data in enumerate(rows):
        row_values = list(row_data.get("cells", [])) if isinstance(row_data, dict) else []
        source_key = str(row_data.get("source_key") or "") if isinstance(row_data, dict) else ""
        number = str(row_data.get("number") or "") if isinstance(row_data, dict) else ""
        verdict_text = ""
        if 0 <= (manual_col - 1) < len(row_values):
            verdict_text = str(row_values[manual_col - 1] or "").strip()
        is_manually_overridden = bool(source_key and number and (source_key, number) in overridden_pairs)
        background = question_ref_row_background(
            verdict_text,
            is_manually_overridden=is_manually_overridden,
        )
        for col_index, value in enumerate(row_values):
            if col_index == manual_col:
                continue
            item = QTableWidgetItem(str(value or ""))
            if col_index == 0:
                item.setData(Qt.ItemDataRole.UserRole, source_key)
                item.setData(Qt.ItemDataRole.UserRole + 1, number)
            if background is not None:
                item.setBackground(background)
            table.setItem(row_index, col_index, item)
        manual_type = str(row_data.get("manual_type") or "") if isinstance(row_data, dict) else ""
        suggested_type = str(row_data.get("suggested_type") or "") if isinstance(row_data, dict) else ""
        if source_key and number and ("不一致" in verdict_text):
            table.setCellWidget(
                row_index,
                manual_col,
                build_combo(
                    source_key=source_key,
```

```
                    number=number,
                    current_type=manual_type or suggested_type,
                    background=background,
                ),
            )
        else:
            manual_item = QTableWidgetItem(manual_type)
            manual_item.setTextAlignment(int(Qt.AlignmentFlag.AlignCenter))
            if background is not None:
                manual_item.setBackground(background)
            table.setItem(row_index, manual_col, manual_item)

```

```python
from __future__ import annotations

from dataclasses import dataclass, field
from pathlib import Path

from sj_generator.infrastructure.llm.question_ref_scan import question_ref_model_specs, question_ref_type_map
from .import_progress import extract_question_ref_source_name
from .import_question_ref_detail import (
    question_ref_detail_model_values,
    question_ref_number_order,
    question_type_conflicts,
    question_type_from_candidates,
    question_type_options,
)


@dataclass
class QuestionRefRuntimeState:
    question_refs_by_source: dict[str, list[dict[str, str]]] = field(default_factory=dict)
    question_ref_payloads: dict[str, dict[str, object]] = field(default_factory=dict)
    manual_question_type_overrides: dict[tuple[str, str], str] = field(default_factory=dict)
    progress_cur: int = 0
    progress_total: int = 0
    accepted_count: int = 0
    cur_source_name: str = "-"
    waiting_source_key: str = ""
    waiting_source_name: str = ""
    waiting_model_specs: list[dict[str, str]] = field(default_factory=list)
    waiting_model_state: dict[str, dict[str, object]] = field(default_factory=dict)
    wait_elapsed_s: int = 0
    wait_round: int = 0

    def reset(self, *, source_name: str = "-") -> None:
        self.question_refs_by_source = {}
        self.question_ref_payloads = {}
        self.manual_question_type_overrides = {}
        self.cur_source_name = source_name or "-"
        self.reset_progress()
        self.stop_waiting_feedback()

    def reset_progress(self) -> None:
        self.progress_cur = 0
        self.progress_total = 0
        self.accepted_count = 0

    def update_progress(self, cur: int, total: int) -> None:
        self.progress_cur = max(0, int(cur))
        self.progress_total = max(0, int(total))

    def update_source_from_progress(self, msg: str) -> None:
        self.cur_source_name = extract_question_ref_source_name(msg, self.cur_source_name)

    def status_text(self) -> str:
        return (
            f"资料：{self.cur_source_name}；"
            f"{self.question_progress_text()}；"
            f"已识别题号 {self.accepted_count} 个"
        )

    def question_progress_text(self) -> str:
        total = self.progress_total
        if total > 0:
            cur = min(max(self.progress_cur, 0), total)
            return f"资料 {cur}/{total}个"
        return "资料 统计中"

    def apply_compare_payload(self, payload: dict[str, object]) -> str:
        source_key = str(payload.get("source_path") or payload.get("source_name") or "").strip()
        if not source_key:
            return ""
```

```python
        source_name = str(payload.get("source_name") or Path(source_key).name or source_key)
        self.clear_waiting_source(source_key)
        self.question_ref_payloads[source_key] = dict(payload)
        self.cur_source_name = source_name
        if bool(payload.get("accepted")):
            self.question_refs_by_source[source_key] = self.build_question_refs_for_source(source_key)
            self.apply_manual_type_overrides_for_source(source_key)
            self.refresh_accepted_count()
        return source_name

    def apply_done_payload(self, result: object) -> None:
        payload = result if isinstance(result, dict) else {}
        refs_by_source = payload.get("refs_by_source", {})
        payloads_by_source = payload.get("payloads_by_source", {})
        if isinstance(refs_by_source, dict):
            self.question_refs_by_source = {
                str(key): value for key, value in refs_by_source.items() if isinstance(value, list)
            }
            self.apply_all_manual_type_overrides()
        if isinstance(payloads_by_source, dict):
            self.question_ref_payloads = {
                str(key): value for key, value in payloads_by_source.items() if isinstance(value, dict)
            }
            self.apply_all_manual_type_overrides()
        self.refresh_accepted_count()

    def set_manual_question_type(self, source_key: str, number: str, value: str) -> None:
        normalized = str(value or "").strip()
        if not source_key or not number:
            return
        if not normalized:
            self.manual_question_type_overrides.pop((source_key, number), None)
            self.ensure_source_available_after_manual_override(source_key)
            self.refresh_accepted_count()
            return
        if normalized not in question_type_options():
            return
        self.manual_question_type_overrides[(source_key, number)] = normalized
        updated = self.apply_manual_type_override(source_key, number, normalized)
        if (not updated) and source_key not in self.question_refs_by_source:
            payload = self.question_ref_payloads.get(source_key, {})
            final_refs = payload.get("final_refs")
            if isinstance(final_refs, list):
                cloned = []
                for item in final_refs:
                    if not isinstance(item, dict):
                        continue
                    new_item = dict(item)
                    if str(new_item.get("number") or "").strip() == number:
                        new_item["question_type"] = normalized
                    cloned.append(new_item)
                payload["final_refs"] = cloned
        self.ensure_source_available_after_manual_override(source_key)
        self.refresh_accepted_count()

    def resolve_manual_question_type(
        self,
        *,
        source_key: str,
        number: str,
        payload: dict[str, object],
        model_types: list[str],
    ) -> str:
        override = self.manual_question_type_overrides.get((source_key, number), "").strip()
        if override:
            return override
        if question_type_conflicts(model_types):
            return ""
        question_refs = self.question_refs_by_source.get(source_key, [])
        for item in question_refs:
```

```python
            if not isinstance(item, dict):
                continue
            if str(item.get("number") or "").strip() == number:
                existing = str(item.get("question_type") or "").strip()
                if existing:
                    return existing
        final_refs = payload.get("final_refs")
        if isinstance(final_refs, list):
            for item in final_refs:
                if not isinstance(item, dict):
                    continue
                if str(item.get("number") or "").strip() == number:
                    existing = str(item.get("question_type") or "").strip()
                    if existing:
                        return existing
        return question_type_from_candidates(model_types)

    def build_question_refs_for_source(self, source_key: str) -> list[dict[str, str]]:
        payload = self.question_ref_payloads.get(source_key, {})
        if not isinstance(payload, dict):
            return []
        model_specs = payload.get("model_specs")
        if not isinstance(model_specs, list) or not model_specs:
            model_specs = question_ref_model_specs()
        normalized_specs = [item for item in model_specs if isinstance(item, dict)]
        model_refs = question_ref_detail_model_values(payload, model_specs=normalized_specs)
        number_order = question_ref_number_order(model_refs)
        final_refs = payload.get("final_refs")
        if isinstance(final_refs, list):
            for item in final_refs:
                if not isinstance(item, dict):
                    continue
                number = str(item.get("number") or "").strip()
                if number and number not in number_order:
                    number_order.append(number)
        type_maps = [question_ref_type_map(refs) for refs in model_refs]
        built: list[dict[str, str]] = []
        for number in number_order:
            types = [type_map.get(number, "").strip() for type_map in type_maps]
            has_conflict = question_type_conflicts(types)
            question_type = self.resolve_manual_question_type(
                source_key=source_key,
                number=number,
                payload=payload,
                model_types=types,
            )
            if (not question_type) and (not has_conflict):
                question_type = question_type_from_candidates(types)
            if (not question_type) and (not has_conflict):
                question_type = question_type_options()[0]
            built.append({"number": number, "question_type": question_type})
        return built

    def has_unresolved_manual_types(self) -> bool:
        for source_key, payload in self.question_ref_payloads.items():
            if not isinstance(payload, dict):
                continue
            model_specs = payload.get("model_specs")
            if not isinstance(model_specs, list) or not model_specs:
                model_specs = question_ref_model_specs()
            normalized_specs = [item for item in model_specs if isinstance(item, dict)]
            model_refs = question_ref_detail_model_values(payload, model_specs=normalized_specs)
            number_order = question_ref_number_order(model_refs)
            type_maps = [question_ref_type_map(refs) for refs in model_refs]
            for number in number_order:
                types = [type_map.get(number, "").strip() for type_map in type_maps]
                if not question_type_conflicts(types):
                    continue
                if self.manual_question_type_overrides.get((source_key, number), "").strip():
                    continue
```

```python
                return True
        return False

    def apply_all_manual_type_overrides(self) -> None:
        for (source_key, number), question_type in self.manual_question_type_overrides.items():
            self.apply_manual_type_override(source_key, number, question_type)

    def apply_manual_type_overrides_for_source(self, source_key: str) -> None:
        for (override_source, number), question_type in self.manual_question_type_overrides.items():
            if override_source != source_key:
                continue
            self.apply_manual_type_override(source_key, number, question_type)

    def apply_manual_type_override(self, source_key: str, number: str, question_type: str) -> bool:
        refs = self.question_refs_by_source.get(source_key)
        updated = False
        if isinstance(refs, list):
            for item in refs:
                if not isinstance(item, dict):
                    continue
                if str(item.get("number") or "").strip() == number:
                    item["question_type"] = question_type
                    updated = True
        payload = self.question_ref_payloads.get(source_key)
        if isinstance(payload, dict):
            final_refs = payload.get("final_refs")
            if isinstance(final_refs, list):
                for item in final_refs:
                    if not isinstance(item, dict):
                        continue
                    if str(item.get("number") or "").strip() == number:
                        item["question_type"] = question_type
        return updated

    def ensure_source_available_after_manual_override(self, source_key: str) -> None:
        if not source_key:
            return
        refs = self.build_question_refs_for_source(source_key)
        if not refs:
            return
        self.question_refs_by_source[source_key] = refs
        payload = self.question_ref_payloads.get(source_key)
        if isinstance(payload, dict):
            payload["accepted"] = True

    def refresh_accepted_count(self) -> None:
        self.accepted_count = sum(len(items) for items in self.question_refs_by_source.values())

    def handle_scan_progress(self, payload: object) -> str:
        data = payload if isinstance(payload, dict) else {}
        event = str(data.get("event") or "").strip()
        if event == "source_started":
            self.waiting_source_key = str(data.get("source_path") or data.get("source_name") or "").strip()
            self.waiting_source_name = str(data.get("source_name") or self.waiting_source_key or "").strip()
            return ""
        if event == "round_started":
            source_key = str(data.get("source_path") or data.get("source_name") or "").strip()
            self.waiting_source_key = source_key or self.waiting_source_key
            self.waiting_source_name = str(data.get("source_name") or self.waiting_source_name or self.waiting_source_key).strip()
            model_specs = data.get("model_specs")
            if isinstance(model_specs, list) and model_specs:
                self.waiting_model_specs = [item for item in model_specs if isinstance(item, dict)]
            self.wait_round = max(1, int(data.get("round") or 1))
            self.start_waiting_feedback()
            return "start_waiting"
        if event != "model_finished":
            return ""
        model_key = str(data.get("model_key") or "").strip()
        if not model_key:
```

```python
            return ""
        state = self.waiting_model_state.setdefault(
            model_key,
            {"done": False, "elapsed_s": 0, "marker": "", "item_count": 0},
        )
        state["done"] = True
        state["elapsed_s"] = int(data.get("elapsed_s") or 0)
        state["marker"] = str(data.get("marker") or "")
        state["item_count"] = int(data.get("item_count") or 0)
        return "refresh_waiting"

    def start_waiting_feedback(self) -> None:
        self.wait_elapsed_s = 0
        self.wait_round = max(1, self.wait_round or 1)
        self.waiting_model_state = {
            str(spec.get("key") or ""): {"done": False, "elapsed_s": 0, "marker": "", "item_count": 0}
            for spec in self.waiting_model_specs
        }

    def stop_waiting_feedback(self) -> None:
        self.waiting_source_key = ""
        self.waiting_source_name = ""
        self.waiting_model_specs = []
        self.waiting_model_state = {}
        self.wait_elapsed_s = 0
        self.wait_round = 0

    def clear_waiting_source(self, source_key: str) -> bool:
        if source_key and source_key == self.waiting_source_key:
            self.stop_waiting_feedback()
            return True
        return False

    def build_waiting_detail_rows(self, model_specs: list[dict[str, str]]) -> list[dict[str, object]]:
        if not self.waiting_model_specs:
            return []
        if self.waiting_source_key and self.waiting_source_key in self.question_ref_payloads:
            return []
        display_specs = model_specs or self.waiting_model_specs
        waiting_map = {str(spec.get("key") or ""): spec for spec in self.waiting_model_specs}
        finished_count = 0
        row_values = [self.waiting_source_name or "处理中"]
        for spec in display_specs:
            model_key = str(spec.get("key") or "")
            if model_key in waiting_map:
                state = self.waiting_model_state.get(model_key, {})
                if bool(state.get("done")):
                    finished_count += 1
                    elapsed_s = int(state.get("elapsed_s") or 0)
                    marker = str(state.get("marker") or "").strip()
                    status_text = f"异常 {elapsed_s}s" if marker else f"完成 {elapsed_s}s"
                else:
                    status_text = f"等待 {self.wait_elapsed_s}s"
                row_values.extend([status_text, status_text])
            else:
                row_values.extend(["", ""])
        row_values.append(f"第{max(1, self.wait_round)}轮，已返回 {finished_count}/{len(self.waiting_model_specs)}")
        row_values.append("")
        return [{"cells": row_values, "source_key": "", "number": "", "manual_type": ""}]

    def tick_waiting_feedback(self) -> None:
        self.wait_elapsed_s += 1

```

```python
from __future__ import annotations

from pathlib import Path

from docx import Document
from PyQt6.QtCore import Qt
from PyQt6.QtGui import QBrush, QColor
from PyQt6.QtWidgets import QTableWidget, QTableWidgetItem


def inspect_docx_content(path: Path) -> tuple[bool, bool, str]:
    try:
        doc = Document(str(path))
        has_table = len(doc.tables) > 0
        has_image = len(doc.inline_shapes) > 0
        return has_image, has_table, ""
    except Exception as e:
        return False, False, str(e)


def make_import_check_item(text: str, state: str) -> QTableWidgetItem:
    item = QTableWidgetItem(text)
    item.setTextAlignment(int(Qt.AlignmentFlag.AlignCenter))
    item.setFlags(item.flags() & ~Qt.ItemFlag.ItemIsEditable)
    if state == "found":
        item.setForeground(QBrush(QColor(156, 0, 6)))
        item.setBackground(QBrush(QColor(255, 199, 206)))
    elif state == "clear":
        item.setForeground(QBrush(QColor(0, 97, 0)))
        item.setBackground(QBrush(QColor(198, 239, 206)))
    elif state == "error":
        item.setForeground(QBrush(QColor(156, 0, 6)))
        item.setBackground(QBrush(QColor(255, 235, 156)))
    return item


def populate_import_reminder_columns(
    table: QTableWidget,
    paths: list[Path],
    *,
    image_col: int,
    table_col: int,
) -> None:
    if not paths:
        return

    for row, path in enumerate(paths):
        has_image, has_table, error_text = inspect_docx_content(path)
        if error_text:
            fail_text = f"失败：{error_text}"
            table.setItem(row, image_col, make_import_check_item(fail_text, "error"))
            table.setItem(row, table_col, make_import_check_item(fail_text, "error"))
            continue
        table.setItem(
            row,
            image_col,
            make_import_check_item("发现" if has_image else "未发现", "found" if has_image else "clear"),
        )
        table.setItem(
            row,
            table_col,
            make_import_check_item("发现" if has_table else "未发现", "found" if has_table else "clear"),
        )
    table.resizeRowsToContents()

```

```python
from __future__ import annotations

from pathlib import Path
import time

from PyQt6.QtWidgets import QTableWidget


def safe_mtime_ns(path: Path) -> int:
    try:
        return path.stat().st_mtime_ns
    except Exception:
        return 0


def word_lock_path(path: Path) -> Path:
    return path.with_name(f"~${path.name}")


def build_opened_doc_session(path: Path) -> dict[str, object]:
    return {
        "path": path,
        "initial_mtime_ns": safe_mtime_ns(path),
        "lock_seen": word_lock_path(path).exists(),
        "opened_at": time.monotonic(),
    }


def poll_opened_doc_sessions(opened_doc_sessions: dict[str, dict[str, object]]) -> list[Path]:
    closed_paths: list[Path] = []
    for key, session in list(opened_doc_sessions.items()):
        path = session.get("path")
        if not isinstance(path, Path):
            opened_doc_sessions.pop(key, None)
            continue
        lock_exists = word_lock_path(path).exists()
        lock_seen = bool(session.get("lock_seen"))
        if lock_exists:
            session["lock_seen"] = True
            continue
        initial_mtime_ns = int(session.get("initial_mtime_ns") or 0)
        modified = safe_mtime_ns(path) != initial_mtime_ns
        elapsed = max(0.0, time.monotonic() - float(session.get("opened_at") or 0.0))
        if lock_seen or (modified and elapsed >= 1.0):
            closed_paths.append(path)
            opened_doc_sessions.pop(key, None)
    return closed_paths


def select_first_changed_row(table: QTableWidget, changed_paths: list[Path]) -> None:
    if not changed_paths:
        return
    changed_resolved = {path.resolve() for path in changed_paths if path.exists()}
    for row in range(table.rowCount()):
        item = table.item(row, 0)
        if item is None:
            continue
        raw_path = str(item.data(0x0100) or "").strip()
        if not raw_path:
            continue
        try:
            candidate = Path(raw_path).resolve()
        except Exception:
            continue
        if candidate in changed_resolved:
            table.selectRow(row)
            break

```

```python
from __future__ import annotations

from pathlib import Path

from PyQt6.QtCore import QObject, pyqtSignal

from sj_generator.application.importing import (
    import_questions_for_sources,
    resolve_question_refs_for_sources,
)
from sj_generator.domain.entities import Question
from sj_generator.application.state import normalize_ai_concurrency


class AiQuestionRefWorker(QObject):
    progress = pyqtSignal(str)
    scan_progress = pyqtSignal(object)
    compare = pyqtSignal(object)
    progress_count = pyqtSignal(int, int)
    done = pyqtSignal(object)
    error = pyqtSignal(str)

    def __init__(self, *, cfg, paths: list[Path]) -> None:
        super().__init__()
        self._cfg = cfg
        self._paths = paths
        self._stop = False

    def request_stop(self) -> None:
        self._stop = True

    def run(self) -> None:
        try:
            result = resolve_question_refs_for_sources(
                paths=self._paths,
                progress_cb=self.progress.emit,
                compare_cb=self.compare.emit,
                scan_progress_cb=self.scan_progress.emit,
                progress_count_cb=self.progress_count.emit,
                stop_cb=self._should_stop,
            )
            self.done.emit(result)
        except Exception as e:
            self.error.emit(str(e))

    def _should_stop(self) -> bool:
        return self._stop


class AiImportContentWorker(QObject):
    progress = pyqtSignal(str)
    question = pyqtSignal(object)
    compare = pyqtSignal(object)
    progress_count = pyqtSignal(int, int)
    done = pyqtSignal(int)
    error = pyqtSignal(str)

    def __init__(
        self,
        *,
        model_specs: list[dict[str, str]],
        paths: list[Path],
        question_refs_by_source: dict[str, list[dict[str, str]]],
        strategy: str,
        max_question_workers: int,
    ) -> None:
        super().__init__()
        self._model_specs = [item for item in model_specs if isinstance(item, dict)]
        self._paths = paths
        self._question_refs_by_source = question_refs_by_source
```

```python
        self._strategy = strategy
        self._max_question_workers = normalize_ai_concurrency(max_question_workers)
        self._stop = False

    def request_stop(self) -> None:
        self._stop = True

    def run(self) -> None:
        try:
            result = import_questions_for_sources(
                model_specs=self._model_specs,
                paths=self._paths,
                strategy=self._strategy,
                max_question_workers=self._max_question_workers,
                progress_cb=self.progress.emit,
                question_cb=self._emit_question,
                compare_cb=self._emit_compare,
                progress_count_cb=self.progress_count.emit,
                stop_cb=self._should_stop,
                question_refs_by_source=self._question_refs_by_source,
            )
            self.done.emit(len(result.questions))
        except Exception as e:
            self.error.emit(str(e))

    def _emit_question(self, q: Question) -> None:
        self.question.emit(q)

    def _emit_compare(self, payload: dict) -> None:
        self.compare.emit(payload)

    def _should_stop(self) -> bool:
        return self._stop

```

```python
import os
from pathlib import Path

from PyQt6.QtCore import QThread, QTimer, Qt
from PyQt6.QtGui import QBrush
from PyQt6.QtWidgets import (
    QApplication,
    QComboBox,
    QHeaderView,
    QLabel,
    QMessageBox,
    QPushButton,
    QProgressBar,
    QTableWidget,
    QTableWidgetItem,
    QVBoxLayout,
    QWizard,
    QWizardPage,
)

from sj_generator.application.settings import (
    load_deepseek_config,
    load_kimi_config,
    load_qwen_config,
)
from sj_generator.application.state import ImportWizardSession
from sj_generator.infrastructure.llm.question_ref_scan import question_ref_model_specs
from sj_generator.presentation.qt.constants import PAGE_AI_IMPORT_CONTENT
from .import_content_detail import question_ref_total_count
from .import_page_common import style_busy_progress
from .import_question_ref_detail import (
    build_manual_type_combo,
    question_ref_detail_headers,
)
from .import_ref_session import QuestionRefRuntimeState
from .import_workers import AiQuestionRefWorker
from sj_generator.presentation.qt.message_box import show_message_box
from .question_ref_support import (
    apply_question_ref_detail_column_widths,
    create_question_ref_worker_bundle,
    missing_question_ref_provider_labels,
    render_question_ref_detail_table,
)
from sj_generator.presentation.qt.table_copy import CopyableTableWidget


class AiImportPage(QWizardPage):
    def __init__(self, state: ImportWizardSession) -> None:
        super().__init__()
        self._state = state
        self.setTitle("题号与题型解析")
        self._detail_row_resize_pending = False

        self._progress = QProgressBar()
        self._progress.setRange(0, 0)
        self._progress.setFormat("正在统计题数…")
        style_busy_progress(self._progress)
        self._progress.hide()

        self._stop_btn = QPushButton("停止解析")
        self._stop_btn.clicked.connect(self._stop)
        self._stop_btn.setEnabled(False)
        self._stop_btn.hide()

        self._retry_btn = QPushButton("重试")
        self._retry_btn.clicked.connect(self._retry)
        self._retry_btn.setEnabled(False)
        self._retry_btn.hide()

        self._status_label = QLabel("")
```

```python
        self._status_label.setWordWrap(True)
        self._status_label.hide()

        self._detail_table = CopyableTableWidget()
        self._detail_table.setColumnCount(len(question_ref_detail_headers()))
        self._detail_table.setHorizontalHeaderLabels(question_ref_detail_headers())
        self._detail_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
        self._apply_question_ref_detail_column_widths()
        self._detail_table.verticalHeader().setSectionResizeMode(QHeaderView.ResizeMode.ResizeToContents)
        self._detail_table.setShowGrid(True)
        self._detail_table.setWordWrap(True)
        self._detail_table.setTextElideMode(Qt.TextElideMode.ElideNone)
        self._detail_table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
        self._detail_table.setStyleSheet(
            """
            QTableWidget {
                border: 1px solid black;
                gridline-color: black;
                selection-background-color: rgba(0, 0, 0, 0);
                selection-color: black;
            }
            """
        )
        self._detail_table.setRowCount(0)
        self._detail_table.itemSelectionChanged.connect(self._sync_open_doc_button)
        self._detail_table.itemDoubleClicked.connect(self._on_detail_item_double_clicked)
        self._detail_table.horizontalHeader().sectionResized.connect(
            lambda *_: self._schedule_detail_row_resize()
        )
        self._detail_row_map: dict[int, int] = {}
        layout = QVBoxLayout()
        layout.addWidget(self._detail_table)
        self.setLayout(layout)

        self._ref_state = QuestionRefRuntimeState()
        self._last_files_text: str = ""
        self._thread: QThread | None = None
        self._worker: AiQuestionRefWorker | None = None
        self._running = False
        self._stopped = False
        self._discard_run_results = False
        self._stop_requested = False
        self._retry_pending = False
        self._close_after_stop_requested = False
        self._finished = False
        self._failed = False
        self._phase_text = "准备开始解析…"
        self._detail_row_map = {}
        self._deepseek_ready = False
        self._qwen_ready = False
        self._kimi_ready = False
        self._wait_timer = QTimer(self)
        self._wait_timer.setInterval(1000)
        self._wait_timer.timeout.connect(self._refresh_waiting_detail_rows)

    def initializePage(self) -> None:
        self._sync_wizard_buttons()
        if self._state.source.files:
            self._ref_state.cur_source_name = self._state.source.files[0].name
        if self._state.source.files_text and self._state.source.files_text != self._last_files_text:
            self._last_files_text = self._state.source.files_text
            self._detail_table.setRowCount(0)
            self._detail_row_map = {}
            self._wait_timer.stop()
            self._ref_state.reset(source_name=self._ref_state.cur_source_name)
            self._progress.setRange(0, 0)
            self._progress.setValue(0)
            self._reset_progress_meta()
            self._phase_text = "正在解析题号与题型，请稍候…"
            self._status_label.setText(self._phase_text)
```

```python
            self._state.refs.question_refs_by_source = {}
            self._running = True
            self._stopped = False
            self._finished = False
            self._failed = False
            self._render_status()
            self.completeChanged.emit()
            self.repaint()
            QApplication.processEvents()
            QTimer.singleShot(30, self._start_import)

    def nextId(self) -> int:
        return PAGE_AI_IMPORT_CONTENT

    def _sync_wizard_buttons(self) -> None:
        wizard = self.wizard()
        if not isinstance(wizard, QWizard):
            return
        back_button = wizard.button(QWizard.WizardButton.BackButton)
        if back_button is not None:
            back_button.setVisible(False)
        next_text = "开始导题"
        if self._stop_requested or self._stopped or self._failed:
            next_text = "重试后继续"
        elif self._running:
            next_text = "解析中…"
        elif self._ref_state.question_refs_by_source:
            next_text = "进入题目内容解析"
        wizard.setButtonText(QWizard.WizardButton.NextButton, next_text)
        self._sync_custom_wizard_button()

    def _sync_custom_wizard_button(self) -> None:
        wizard = self.wizard()
        if not isinstance(wizard, QWizard):
            return
        stop_button = wizard.button(QWizard.WizardButton.CustomButton1)
        retry_button = wizard.button(QWizard.WizardButton.CustomButton2)
        if stop_button is not None:
            stop_button.setText("停止解析")
            stop_button.setVisible(True)
            stop_button.setEnabled(self._stop_btn.isEnabled())
            try:
                stop_button.clicked.disconnect()
            except TypeError:
                pass
            stop_button.clicked.connect(self._stop)
        if retry_button is not None:
            retry_button.setText("重试")
            retry_button.setVisible(True)
            retry_button.setEnabled(self._retry_btn.isEnabled())
            try:
                retry_button.clicked.disconnect()
            except TypeError:
                pass
            retry_button.clicked.connect(self._retry)
        open_button = wizard.button(QWizard.WizardButton.CustomButton3)
        if open_button is not None:
            open_button.setText("打开文档")
            open_button.setVisible(True)
            open_button.setEnabled(self._detail_table.currentRow() >= 0)
            try:
                open_button.clicked.disconnect()
            except TypeError:
                pass
            open_button.clicked.connect(self._open_selected_source_doc)

    def _sync_open_doc_button(self) -> None:
        wizard = self.wizard()
        if not isinstance(wizard, QWizard):
            return
```

```python
        button = wizard.button(QWizard.WizardButton.CustomButton3)
        if button is None:
            return
        button.setEnabled(self._detail_table.currentRow() >= 0)

    def isComplete(self) -> bool:
        if self._running or self._failed:
            return False
        if self._stopped or not self._finished:
            return False
        if self._ref_state.has_unresolved_manual_types():
            return False
        return self._ref_state.accepted_count > 0

    def validatePage(self) -> bool:
        if self._running:
            return False
        if question_ref_total_count(self._ref_state.question_refs_by_source) <= 0:
            show_message_box(self, title="暂无结果", text="当前没有可进入下一步的题号。", icon=QMessageBox.Icon.Warning)
            return False
        if self._ref_state.has_unresolved_manual_types():
            show_message_box(
                self,
                title="请选择题型",
                text="当前仍有“对比结论不一致”的题目未选择题型，请先手动选择后再继续。",
                icon=QMessageBox.Icon.Warning,
            )
            return False
        self._state.apply_question_refs(self._ref_state.question_refs_by_source)
        self.completeChanged.emit()
        return True

    def _start_import(self) -> None:
        if self._thread is not None and self._thread.isRunning():
            return

        paths = self._state.source.files or []
        if not paths:
            show_message_box(self, title="未选择文件", text="请先选择待处理的资料文件。", icon=QMessageBox.Icon.Warning)
            return

        cfg = load_deepseek_config()
        kimi_cfg = load_kimi_config()
        qwen_cfg = load_qwen_config()
        self._deepseek_ready = cfg.is_ready()
        self._kimi_ready = kimi_cfg.is_ready()
        self._qwen_ready = qwen_cfg.is_ready()
        self._render_status()
        ready_map = {
            "deepseek": cfg.is_ready(),
            "kimi": kimi_cfg.is_ready(),
            "qwen": qwen_cfg.is_ready(),
        }
        missing_labels = missing_question_ref_provider_labels(ready_map)
        if missing_labels:
            self._sync_wizard_buttons()
            show_message_box(
                self,
                title="未配置",
                text="题号与题型解析所需模型尚未全部可用：" + "、".join(missing_labels),
                icon=QMessageBox.Icon.Warning,
            )
            return

        self._status_label.setText("正在解析题号与题型，请稍候…")
        self._reset_progress_meta()
        self._wait_timer.stop()
        self._ref_state.stop_waiting_feedback()
```

```python
        self._phase_text = "正在解析题号与题型，请稍候…"
        self._show_waiting_placeholder(paths)
        self._render_status()
        self._stop_btn.setEnabled(True)
        self._retry_btn.setEnabled(False)
        self._running = True
        self._stopped = False
        self._discard_run_results = False
        self._stop_requested = False
        self._retry_pending = False
        self._finished = False
        self._failed = False
        self._sync_wizard_buttons()
        self.completeChanged.emit()

        bundle = create_question_ref_worker_bundle(
            parent=self,
            cfg=cfg,
            paths=paths,
            on_progress=self._on_progress,
            on_scan_progress=self._on_scan_progress,
            on_progress_count=self._on_progress_count,
            on_compare=self._on_compare,
            on_done=self._on_done,
            on_error=self._on_error,
        )
        self._thread = bundle.thread
        self._worker = bundle.worker
        self._thread.start()

    def _on_progress(self, msg: str) -> None:
        if self._discard_run_results:
            return
        self._phase_text = msg
        self._update_detail_from_progress(msg)
        self._render_status()

    def _on_progress_count(self, current: int, total: int) -> None:
        if self._discard_run_results:
            return
        self._ref_state.update_progress(current, total)
        if total <= 0:
            self._progress.setRange(0, 0)
            self._progress.setFormat("正在统计资料…")
            self._render_status()
            return
        if self._progress.maximum() != total:
            self._progress.setRange(0, total)
        self._progress.setValue(max(0, min(current, total)))
        self._progress.setFormat("资料 %v/%m个")
        self._render_status()

    def _on_compare(self, payload: object) -> None:
        if self._discard_run_results:
            return
        if not isinstance(payload, dict):
            return
        source_name = self._ref_state.apply_compare_payload(payload)
        if not source_name:
            return
        self._render_question_ref_detail_rows()
        self._phase_text = f"题号与题型解析中：{source_name} 已返回结果"
        self._render_status()

    def _schedule_detail_row_resize(self) -> None:
        if getattr(self, "_detail_row_resize_pending", False):
            return
        self._detail_row_resize_pending = True
        QTimer.singleShot(0, self._resize_detail_rows_to_contents)

```

```python
    def _resize_detail_rows_to_contents(self) -> None:
        self._detail_row_resize_pending = False
        self._detail_table.resizeRowsToContents()

    def _on_done(self, result: object) -> None:
        self._wait_timer.stop()
        self._ref_state.stop_waiting_feedback()
        if self._discard_run_results:
            self._clear_partial_results()
            self._phase_text = "已停止解析，本轮结果已丢弃，请点击重试重新解析。"
            self._stop_btn.setEnabled(False)
            self._retry_btn.setEnabled(True)
            self._running = False
            self._stop_requested = False
            self._stopped = True
            self._finished = False
            self._thread = None
            self._worker = None
            self._render_status()
            self._sync_wizard_buttons()
            self.completeChanged.emit()
            self._finish_close_after_stop_if_needed()
            if self._retry_pending:
                self._retry_pending = False
                QTimer.singleShot(0, self._retry)
            return
        self._ref_state.apply_done_payload(result)
        self._render_question_ref_detail_rows()
        total = self._ref_state.accepted_count
        self._phase_text = f"题号与题型解析完成：共 {total} 个题号（可继续进入下一步）"
        if self._progress.maximum() > 0:
            self._progress.setValue(self._progress.maximum())
            self._ref_state.update_progress(self._progress.maximum(), self._progress.maximum())
        self._stop_btn.setEnabled(False)
        self._retry_btn.setEnabled(True)
        self._running = False
        self._stop_requested = False
        self._stopped = False
        self._finished = True
        self._thread = None
        self._worker = None
        self._render_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()

    def _on_error(self, msg: str) -> None:
        self._wait_timer.stop()
        self._ref_state.stop_waiting_feedback()
        if self._discard_run_results:
            self._clear_partial_results()
            self._phase_text = "已停止解析，本轮结果已丢弃，请点击重试重新解析。"
            self._stop_btn.setEnabled(False)
            self._retry_btn.setEnabled(True)
            self._running = False
            self._stop_requested = False
            self._stopped = True
            self._failed = False
            self._thread = None
            self._worker = None
            self._render_status()
            self._sync_wizard_buttons()
            self.completeChanged.emit()
            self._finish_close_after_stop_if_needed()
            if self._retry_pending:
                self._retry_pending = False
                QTimer.singleShot(0, self._retry)
            return
        show_message_box(self, title="解析失败", text=msg, icon=QMessageBox.Icon.Critical)
        self._phase_text = f"题号与题型解析失败：{msg}"
        self._append_failure_row(msg)
```

```python
        self._stop_btn.setEnabled(False)
        self._retry_btn.setEnabled(True)
        self._running = False
        self._stop_requested = False
        self._failed = True
        self._thread = None
        self._worker = None
        self._render_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()
        self._finish_close_after_stop_if_needed()

    def _append_failure_row(self, msg: str) -> None:
        row = self._detail_table.rowCount()
        self._detail_table.setRowCount(row + 1)
        values = ["失败"] + [""] * (self._detail_table.columnCount() - 3) + [f"失败：{msg}", ""]
        for col, value in enumerate(values):
            self._detail_table.setItem(row, col, QTableWidgetItem(value))
        self._schedule_detail_row_resize()

    def _stop(self) -> None:
        if self._worker is not None:
            self._worker.request_stop()
            self._discard_run_results = True
            self._stop_requested = True
            self._stop_btn.setEnabled(False)
            self._retry_btn.setEnabled(True)
            self._running = False
            self._phase_text = "正在停止…可点击重试，收尾完成后将重新解析。"
            self._stopped = True
            self._clear_partial_results()
            self._render_status()
            self._sync_wizard_buttons()
            self.completeChanged.emit()

    def prepare_to_close(self) -> bool:
        thread = self._thread
        if thread is None or (not thread.isRunning()):
            return True
        if self._worker is not None:
            self._worker.request_stop()
        self._discard_run_results = True
        self._stop_requested = True
        self._close_after_stop_requested = True
        self._stop_btn.setEnabled(False)
        self._retry_btn.setEnabled(False)
        self._running = False
        self._phase_text = "正在停止，收尾完成后将自动退出…"
        self._stopped = True
        self._clear_partial_results()
        self._render_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()
        return False

    def _retry(self) -> None:
        if self._thread is not None and self._thread.isRunning():
            self._retry_pending = True
            self._retry_btn.setEnabled(False)
            self._phase_text = "正在停止当前解析，收尾完成后自动重试…"
            self._render_status()
            self._sync_wizard_buttons()
            return
        if self._running:
            return
        self._discard_run_results = False
        self._stop_requested = False
        self._retry_pending = False
        self._close_after_stop_requested = False
        self._clear_partial_results()
```

```python
        self._phase_text = "准备重试题号与题型解析…"
        self._stopped = False
        self._finished = False
        self._failed = False
        self._retry_btn.setEnabled(False)
        self._render_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()
        QTimer.singleShot(0, self._start_import)

    def _reset_progress_meta(self) -> None:
        self._ref_state.reset_progress()
        self._render_status()

    def _update_detail_from_progress(self, msg: str) -> None:
        self._ref_state.update_source_from_progress(msg)

    def _render_status(self) -> None:
        self._status_label.setText(self._ref_state.status_text())

    def _build_question_progress_text(self) -> str:
        return self._ref_state.question_progress_text()

    def _render_question_ref_detail_rows(self) -> None:
        self._render_question_ref_detail_rows_with_view_restore()

    def _render_question_ref_detail_rows_with_view_restore(self) -> None:
        view_state = self._capture_detail_table_view_state()
        render_question_ref_detail_table(
            table=self._detail_table,
            ref_state=self._ref_state,
            build_combo=self._build_manual_type_combo,
        )
        self._restore_detail_table_view_state(view_state)
        self._schedule_detail_row_resize()

    def _apply_question_ref_detail_column_widths(self) -> None:
        apply_question_ref_detail_column_widths(self._detail_table)

    def _build_manual_type_combo(
        self,
        *,
        source_key: str,
        number: str,
        current_type: str,
        background: QBrush | None,
    ) -> QComboBox:
        return build_manual_type_combo(
            table=self._detail_table,
            current_type=current_type,
            background=background,
            on_changed=lambda text, sk=source_key, num=number: self._on_manual_question_type_changed(sk, num, text),
        )

    def _open_source_doc(self, source_key: str) -> None:
        path = Path(str(source_key or "").strip())
        if not path.exists():
            show_message_box(self, title="文件不存在", text=f"未找到文档：{path}", icon=QMessageBox.Icon.Warning)
            return
        try:
            os.startfile(str(path))
        except Exception as exc:
            show_message_box(self, title="打开失败", text=f"无法打开 Word 文档：{exc}", icon=QMessageBox.Icon.Critical)

    def _open_selected_source_doc(self) -> None:
        self._sync_open_doc_button()
        row = self._detail_table.currentRow()
```

```python
        if row < 0:
            show_message_box(self, title="未选择题目", text="请先在表格中选择一行。", icon=QMessageBox.Icon.Warning)
            return
        item = self._detail_table.item(row, 0)
        source_key = str(item.data(Qt.ItemDataRole.UserRole) or "").strip() if item is not None else ""
        if not source_key:
            show_message_box(self, title="无法打开", text="当前行没有可打开的来源文档。", icon=QMessageBox.Icon.Warning)
            return
        self._open_source_doc(source_key)

    def _on_detail_item_double_clicked(self, item: QTableWidgetItem) -> None:
        if item is None:
            return
        if item.column() != 0:
            return
        source_key = str(item.data(Qt.ItemDataRole.UserRole) or "").strip()
        if not source_key:
            return
        self._open_source_doc(source_key)

    def _on_manual_question_type_changed(self, source_key: str, number: str, value: str) -> None:
        self._ref_state.set_manual_question_type(source_key, number, value)
        self._render_status()
        self._sync_wizard_buttons()
        self.completeChanged.emit()
        self._render_question_ref_detail_rows_with_view_restore()

    def _on_scan_progress(self, payload: object) -> None:
        if self._discard_run_results:
            return
        action = self._ref_state.handle_scan_progress(payload)
        if action == "start_waiting":
            self._render_question_ref_detail_rows()
            self._wait_timer.start()
            return
        if action == "refresh_waiting":
            self._refresh_waiting_detail_rows()

    def _refresh_waiting_detail_rows(self) -> None:
        if self._discard_run_results:
            return
        if not self._ref_state.waiting_model_specs:
            return
        self._render_question_ref_detail_rows()
        self._ref_state.tick_waiting_feedback()

    def _show_waiting_placeholder(self, paths: list[Path]) -> None:
        if not paths:
            return
        current_path = paths[0]
        self._ref_state.waiting_source_key = str(current_path)
        self._ref_state.waiting_source_name = current_path.name
        self._ref_state.waiting_model_specs = [item for item in question_ref_model_specs() if isinstance(item, dict)]
        self._ref_state.wait_round = 1
        self._ref_state.start_waiting_feedback()
        self._render_question_ref_detail_rows()
        self._wait_timer.start()

    def _clear_partial_results(self) -> None:
        self._wait_timer.stop()
        self._ref_state.reset(source_name=self._ref_state.cur_source_name)
        self._detail_table.setRowCount(0)
        self._detail_row_map = {}
        self._progress.setRange(0, 0)
        self._progress.setValue(0)
        self._progress.setFormat("正在统计资料…")
        self._reset_progress_meta()
```

```python

    def _capture_detail_table_view_state(self) -> dict[str, object]:
        current_row = self._detail_table.currentRow()
        anchor_item = self._detail_table.item(current_row, 0) if current_row >= 0 else None
        return {
            "vertical_scroll": self._detail_table.verticalScrollBar().value(),
            "horizontal_scroll": self._detail_table.horizontalScrollBar().value(),
            "source_key": str(anchor_item.data(Qt.ItemDataRole.UserRole) or "").strip() if anchor_item is not None else "",
            "number": str(anchor_item.data(Qt.ItemDataRole.UserRole + 1) or "").strip() if anchor_item is not None else "",
        }

    def _restore_detail_table_view_state(self, view_state: dict[str, object]) -> None:
        source_key = str(view_state.get("source_key") or "").strip()
        number = str(view_state.get("number") or "").strip()
        if source_key:
            for row in range(self._detail_table.rowCount()):
                item = self._detail_table.item(row, 0)
                if item is None:
                    continue
                row_source_key = str(item.data(Qt.ItemDataRole.UserRole) or "").strip()
                row_number = str(item.data(Qt.ItemDataRole.UserRole + 1) or "").strip()
                if row_source_key == source_key and row_number == number:
                    self._detail_table.setCurrentCell(row, 0)
                    break
        self._detail_table.verticalScrollBar().setValue(int(view_state.get("vertical_scroll") or 0))
        self._detail_table.horizontalScrollBar().setValue(int(view_state.get("horizontal_scroll") or 0))

    def _finish_close_after_stop_if_needed(self) -> None:
        if not self._close_after_stop_requested:
            return
        self._close_after_stop_requested = False
        wizard = self.wizard()
        if isinstance(wizard, QWizard):
            QTimer.singleShot(0, wizard.close)

__all__ = ["AiImportPage"]

```

```python
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Callable

from PyQt6.QtCore import QThread
from PyQt6.QtWidgets import QHeaderView, QTableWidget

from sj_generator.infrastructure.llm.question_ref_scan import question_ref_model_specs
from .import_question_ref_detail import (
    build_question_ref_detail_rows,
    populate_question_ref_detail_table,
    question_ref_detail_headers,
    question_ref_detail_model_specs,
)
from .import_ref_session import QuestionRefRuntimeState
from .import_workers import AiQuestionRefWorker

_PROVIDER_LABELS = {
    "deepseek": "DeepSeek",
    "kimi": "Kimi",
    "qwen": "千问",
}


def missing_question_ref_provider_labels(ready_map: dict[str, bool]) -> list[str]:
    required_providers = {
        str(item.get("provider") or "").strip().lower()
        for item in question_ref_model_specs()
        if str(item.get("provider") or "").strip()
    }
    return [
        _PROVIDER_LABELS.get(provider, provider)
        for provider in required_providers
        if not ready_map.get(provider, False)
    ]


def render_question_ref_detail_table(
    *,
    table: QTableWidget,
    ref_state: QuestionRefRuntimeState,
    build_combo: Callable[..., object],
) -> None:
    model_specs = question_ref_detail_model_specs(ref_state.question_ref_payloads)
    header_labels = question_ref_detail_headers(model_specs)
    table.setColumnCount(len(header_labels))
    table.setHorizontalHeaderLabels(header_labels)
    apply_question_ref_detail_column_widths(table)
    table.clearContents()
    rows = build_question_ref_detail_rows(
        payloads=ref_state.question_ref_payloads,
        model_specs=model_specs,
        resolve_manual_type=ref_state.resolve_manual_question_type,
    )
    rows.extend(ref_state.build_waiting_detail_rows(model_specs))
    populate_question_ref_detail_table(
        table=table,
        rows=rows,
        manual_col=table.columnCount() - 1,
        overridden_pairs=set(ref_state.manual_question_type_overrides.keys()),
        build_combo=build_combo,
    )


def apply_question_ref_detail_column_widths(table: QTableWidget) -> None:
    header = table.horizontalHeader()
    column_count = table.columnCount()
    if column_count <= 0:
```

```python
        return
    for col in range(max(0, column_count - 1)):
        header.setSectionResizeMode(col, QHeaderView.ResizeMode.Stretch)
    for col in range(max(0, column_count - 1), column_count):
        header.setSectionResizeMode(col, QHeaderView.ResizeMode.Fixed)
        table.setColumnWidth(col, 120)


@dataclass
class QuestionRefWorkerBundle:
    thread: QThread
    worker: AiQuestionRefWorker


def create_question_ref_worker_bundle(
    *,
    parent,
    cfg,
    paths: list[Path],
    on_progress,
    on_scan_progress,
    on_progress_count,
    on_compare,
    on_done,
    on_error,
) -> QuestionRefWorkerBundle:
    thread = QThread(parent)
    worker = AiQuestionRefWorker(cfg=cfg, paths=paths)
    worker.moveToThread(thread)
    thread.started.connect(worker.run)
    worker.progress.connect(on_progress)
    worker.scan_progress.connect(on_scan_progress)
    worker.progress_count.connect(on_progress_count)
    worker.compare.connect(on_compare)
    worker.done.connect(on_done)
    worker.error.connect(on_error)
    worker.done.connect(thread.quit)
    worker.error.connect(thread.quit)
    thread.finished.connect(worker.deleteLater)
    thread.finished.connect(thread.deleteLater)
    return QuestionRefWorkerBundle(thread=thread, worker=worker)

```

```python
import os
from pathlib import Path

from PyQt6.QtCore import QTimer, Qt
from PyQt6.QtGui import QDragEnterEvent, QDropEvent
from PyQt6.QtWidgets import (
    QFileDialog,
    QHeaderView,
    QMessageBox,
    QTableWidget,
    QTableWidgetItem,
    QVBoxLayout,
    QWizard,
    QWizardPage,
)

from sj_generator.application.state import AiSourceFileItem, ImportWizardSession
from sj_generator.presentation.qt.constants import PAGE_AI_IMPORT
from .import_page_common import (
    LevelPathItemDelegate,
    PreserveCellBackgroundDelegate,
    extract_paths_from_drop_event,
    is_valid_level_path,
    merge_paths_text,
    rename_project,
)
from .import_select_session import (
    build_opened_doc_session,
    poll_opened_doc_sessions,
)
from sj_generator.presentation.qt.message_box import show_message_box
from .select_support import (
    build_split_import_states_for_paths,
    collect_table_items,
    find_invalid_level_paths,
    rebuild_selected_paths_table,
    refresh_after_external_doc_edit,
    reminder_doc_path,
    remove_file_row,
    serialize_selected_paths,
    update_import_reminder,
)
from sj_generator.presentation.qt.table_copy import CopyableTableWidget


class AiSelectFilesPage(QWizardPage):
    def __init__(self, state: ImportWizardSession) -> None:
        super().__init__()
        self._state = state
        self.setTitle("确认导入资料")
        self.setAcceptDrops(True)

        self._files_table = CopyableTableWidget()
        self._files_table.setColumnCount(6)
        self._files_table.setHorizontalHeaderLabels(["名称", "版本", "层级", "图片", "表格", "操作"])
        header = self._files_table.horizontalHeader()
        for column in range(5):
            header.setSectionResizeMode(column, QHeaderView.ResizeMode.Stretch)
        header.setSectionResizeMode(5, QHeaderView.ResizeMode.Fixed)
        self._files_table.setColumnWidth(5, 220)
        self._files_table.verticalHeader().setVisible(False)
        self._files_table.setShowGrid(True)
        self._files_table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)
        self._files_table.setSelectionMode(QTableWidget.SelectionMode.SingleSelection)
        self._files_table.setStyleSheet(
            """
            QTableWidget {
                gridline-color: black;
                selection-background-color: rgba(0, 0, 0, 0);
                selection-color: black;
```

```python
            }
            """
        )
        self._files_table.setEditTriggers(
            QTableWidget.EditTrigger.DoubleClicked | QTableWidget.EditTrigger.EditKeyPressed
        )
        self._files_table.setItemDelegate(PreserveCellBackgroundDelegate(self._files_table))
        self._files_table.setItemDelegateForColumn(2, LevelPathItemDelegate(self._files_table))
        self._files_table.itemChanged.connect(self._handle_file_table_item_changed)
        self._opened_doc_sessions: dict[str, dict[str, object]] = {}
        self._doc_refresh_timer = QTimer(self)
        self._doc_refresh_timer.setInterval(1000)
        self._doc_refresh_timer.timeout.connect(self._poll_opened_docs)

        layout = QVBoxLayout()
        layout.addWidget(self._files_table, 1)
        self.setLayout(layout)

    def initializePage(self) -> None:
        self._sync_wizard_buttons()
        if self._state.source.files_text:
            paths = self._state.source.files or [
                Path(part.strip()) for part in self._state.source.files_text.split(";") if part.strip()
            ]
            self._set_selected_paths(paths)
            self._update_import_reminder(paths)
        else:
            self._set_selected_paths([])
            self._update_import_reminder([])

    def dragEnterEvent(self, event: QDragEnterEvent) -> None:
        if event.mimeData().hasUrls():
            event.acceptProposedAction()
        else:
            event.ignore()

    def dropEvent(self, event: QDropEvent) -> None:
        paths = extract_paths_from_drop_event(event)
        paths = [path for path in paths if path.suffix.lower() == ".docx"]
        if paths:
            merged = merge_paths_text(self._serialize_paths_text(), paths)
            merged_paths = [Path(part.strip()) for part in merged.split(";") if part.strip()]
            self._set_selected_paths(merged_paths, selected_path=paths[0])
            self._update_import_reminder(merged_paths)
            event.acceptProposedAction()
        else:
            event.ignore()

    def validatePage(self) -> bool:
        raw = self._serialize_paths_text()
        if not raw:
            show_message_box(self, title="未选择文件", text="请选择待处理的资料文件。", icon=QMessageBox.Icon.Warning)
            return False
        paths = [Path(part.strip()) for part in raw.split(";") if part.strip()]
        paths = [path for path in paths if path.exists()]
        if not paths:
            show_message_box(self, title="文件不存在", text="请选择存在的资料文件。", icon=QMessageBox.Icon.Warning)
            return False
        invalid_levels = self._find_invalid_level_paths()
        if invalid_levels:
            names = "、".join(invalid_levels)
            show_message_box(
                self,
                title="层级格式无效",
                text=f"以下文件的层级不是三级数字格式：{names}\n请输入如 3.2.2 的形式。",
                icon=QMessageBox.Icon.Warning,
            )
            return False
```

```python
        items = self._collect_table_items()
        level_paths = [item.level_path.strip() for item in items if item.level_path.strip()]
        if not level_paths:
            show_message_box(self, title="未填写层级", text="请在“确认导入资料”页填写层级。", icon=QMessageBox.Icon.Warning)
            return False
        self._state.source.files = list(paths)
        self._state.source.files_text = raw
        self._state.source.file_items = items
        self._state.source.import_level_path = level_paths[0] if len(set(level_paths)) == 1 else ""
        if len(paths) > 1:
            split_states = self._build_split_import_states(paths=paths, items=items)
            wizard = self.wizard()
            open_split = getattr(wizard, "open_split_import_flow_windows", None)
            if not callable(open_split) or (not open_split(split_states, start_page_id=PAGE_AI_IMPORT)):
                show_message_box(
                    self,
                    title="打开窗口失败",
                    text="无法打开拆分后的导入窗口。",
                    icon=QMessageBox.Icon.Warning,
                )
                return False
            if isinstance(wizard, QWizard):
                QTimer.singleShot(0, wizard.close)
            return False
        if self._state.project_name_is_placeholder and self._state.project_dir is not None:
            rename_project(self._state, new_name=paths[0].stem)
        return True

    def nextId(self) -> int:
        return PAGE_AI_IMPORT

    def _sync_wizard_buttons(self) -> None:
        wizard = self.wizard()
        if not isinstance(wizard, QWizard):
            return
        back_button = wizard.button(QWizard.WizardButton.BackButton)
        if back_button is not None:
            back_button.setVisible(False)
        wizard.setButtonText(QWizard.WizardButton.NextButton, "开始导题")

    def _sync_custom_wizard_button(self) -> None:
        wizard = self.wizard()
        if not isinstance(wizard, QWizard):
            return
        add_button = wizard.button(QWizard.WizardButton.CustomButton1)
        if add_button is None:
            return
        add_button.setText("添加文档")
        add_button.setVisible(True)
        try:
            add_button.clicked.disconnect()
        except TypeError:
            pass
        add_button.clicked.connect(self._choose_and_add_docs)

    def _choose_and_add_docs(self) -> None:
        start_dir = (self._state.import_source_dir_text or "").strip()
        if not start_dir:
            start_dir = os.getcwd()
        files, _ = QFileDialog.getOpenFileNames(
            self,
            "选择资料文档",
            start_dir,
            "Word 文档 (*.docx)",
        )
        if not files:
            return
        paths = [Path(value) for value in files if value]
        if not paths:
```

```python
            return
        merged = merge_paths_text(self._serialize_paths_text(), paths)
        merged_paths = [Path(part.strip()) for part in merged.split(";") if part.strip()]
        self._state.import_source_dir_text = str(paths[0].parent)
        self._set_selected_paths(merged_paths, selected_path=paths[0])
        self._update_import_reminder(merged_paths)

    def _serialize_paths_text(self) -> str:
        return serialize_selected_paths(self._files_table)

    def _set_selected_paths(self, paths: list[Path], *, selected_path: Path | None = None) -> None:
        rebuild_selected_paths_table(
            table=self._files_table,
            state=self._state,
            paths=paths,
            selected_path=selected_path,
            on_open_row=self._open_saved_doc_for_row,
            on_remove_row=self._remove_file_row,
        )

    def _handle_file_table_item_changed(self, item: QTableWidgetItem) -> None:
        row = item.row()
        path_item = self._files_table.item(row, 0)
        if path_item is None:
            return
        raw_path = str(path_item.data(Qt.ItemDataRole.UserRole) or "").strip()
        if not raw_path:
            return
        version_item = self._files_table.item(row, 1)
        level_item = self._files_table.item(row, 2)
        version = (version_item.text() if version_item else "").strip() or self._state.preferred_textbook_version
        if version_item is not None and version_item.text().strip() != version:
            self._files_table.blockSignals(True)
            version_item.setText(version)
            self._files_table.blockSignals(False)
        level_path = (level_item.text() if level_item else "").strip()
        if item.column() == 2 and level_path and not self._is_valid_level_path(level_path):
            self._files_table.blockSignals(True)
            item.setText("")
            self._files_table.blockSignals(False)
            show_message_box(
                self,
                title="层级格式无效",
                text="层级只允许输入三级点连接的数字形式，例如 3.2.2。",
                icon=QMessageBox.Icon.Warning,
            )
            level_path = ""
        self._upsert_source_file_item(raw_path, version=version, level_path=level_path)

    def _collect_table_items(self) -> list[AiSourceFileItem]:
        return collect_table_items(self._files_table, self._state.preferred_textbook_version)

    def _upsert_source_file_item(self, raw_path: str, *, version: str, level_path: str) -> None:
        updated = False
        for item in self._state.source.file_items:
            if item.path == raw_path:
                item.version = version
                item.level_path = level_path
                updated = True
                break
        if not updated:
            self._state.source.file_items.append(
                AiSourceFileItem(path=raw_path, version=version, level_path=level_path)
            )

    def _find_invalid_level_paths(self) -> list[str]:
        return find_invalid_level_paths(self._files_table, self._is_valid_level_path)

    def _is_valid_level_path(self, value: str) -> bool:
```

```python
        return is_valid_level_path(value)

    def _build_split_import_states(
        self,
        *,
        paths: list[Path],
        items: list[AiSourceFileItem],
    ) -> list[ImportWizardSession]:
        return build_split_import_states_for_paths(
            base_state=self._state,
            paths=paths,
            items=items,
            rename_project=rename_project,
        )

    def _update_import_reminder(self, paths: list[Path]) -> None:
        update_import_reminder(self._files_table, paths, image_col=3, table_col=4)

    def _open_saved_doc_for_row(self, row: int) -> None:
        path = self._reminder_doc_path(row)
        if path is None:
            return
        if not path.exists():
            show_message_box(self, title="文件不存在", text=f"未找到文档：{path}", icon=QMessageBox.Icon.Warning)
            return
        try:
            os.startfile(str(path))
        except Exception as exc:
            show_message_box(self, title="打开失败", text=f"无法打开 Word 文档：{exc}", icon=QMessageBox.Icon.Critical)
            return
        session = build_opened_doc_session(path)
        self._opened_doc_sessions[str(path.resolve())] = session
        if not self._doc_refresh_timer.isActive():
            self._doc_refresh_timer.start()

    def _remove_file_row(self, row: int) -> None:
        remaining_paths, removed_path = remove_file_row(self._files_table, row)
        if removed_path is None:
            return
        self._opened_doc_sessions.pop(str(removed_path.resolve()), None)
        next_selected = None
        if remaining_paths:
            next_selected = remaining_paths[min(row, len(remaining_paths) - 1)]
        self._set_selected_paths(remaining_paths, selected_path=next_selected)
        self._update_import_reminder(remaining_paths)

    def _reminder_doc_path(self, row: int) -> Path | None:
        return reminder_doc_path(self._files_table, row)

    def _poll_opened_docs(self) -> None:
        if not self._opened_doc_sessions:
            self._doc_refresh_timer.stop()
            return
        closed_paths = poll_opened_doc_sessions(self._opened_doc_sessions)
        if closed_paths:
            self._refresh_after_external_doc_edit(closed_paths)
        if not self._opened_doc_sessions:
            self._doc_refresh_timer.stop()

    def _refresh_after_external_doc_edit(self, changed_paths: list[Path]) -> None:
        refresh_after_external_doc_edit(
            table=self._files_table,
            items=self._collect_table_items(),
            changed_paths=changed_paths,
            image_col=3,
            table_col=4,
        )

```

sj_generator/presentation/qt/pages/import_flow/select_page.py

070-06

```python
__all__ = ["AiSelectFilesPage"]

```

```python
from __future__ import annotations

from pathlib import Path

from PyQt6.QtCore import Qt
from PyQt6.QtWidgets import QHBoxLayout, QPushButton, QTableWidget, QTableWidgetItem, QWidget

from sj_generator.application.state import AiSourceFileItem, ImportWizardSession
from .import_page_common import style_dialog_button
from .import_select_reminder import populate_import_reminder_columns
from .import_select_session import select_first_changed_row


def serialize_selected_paths(table: QTableWidget) -> str:
    parts: list[str] = []
    for row in range(table.rowCount()):
        item = table.item(row, 0)
        raw = item.data(Qt.ItemDataRole.UserRole) if item is not None else None
        if raw:
            parts.append(str(raw))
    return "; ".join(parts)


def rebuild_selected_paths_table(
    *,
    table: QTableWidget,
    state: ImportWizardSession,
    paths: list[Path],
    selected_path: Path | None,
    on_open_row,
    on_remove_row,
) -> None:
    existing_map = {item.path: item for item in state.source.file_items}
    table.blockSignals(True)
    table.setRowCount(0)
    selected_row = 0
    updated_items: list[AiSourceFileItem] = []
    for index, path in enumerate(paths):
        raw_path = str(path)
        existing = existing_map.get(raw_path, AiSourceFileItem(path=raw_path))
        default_version = existing.version or state.preferred_textbook_version
        updated_items.append(
            AiSourceFileItem(path=raw_path, version=default_version, level_path=existing.level_path)
        )
        table.insertRow(index)
        name_item = QTableWidgetItem(path.name)
        name_item.setToolTip(raw_path)
        name_item.setData(Qt.ItemDataRole.UserRole, raw_path)
        name_item.setFlags(name_item.flags() & ~Qt.ItemFlag.ItemIsEditable)
        version_item = QTableWidgetItem(default_version)
        version_item.setTextAlignment(int(Qt.AlignmentFlag.AlignCenter))
        level_item = QTableWidgetItem(existing.level_path)
        level_item.setTextAlignment(int(Qt.AlignmentFlag.AlignCenter))
        image_item = QTableWidgetItem("")
        image_item.setFlags(image_item.flags() & ~Qt.ItemFlag.ItemIsEditable)
        image_item.setTextAlignment(int(Qt.AlignmentFlag.AlignCenter))
        table_item = QTableWidgetItem("")
        table_item.setFlags(table_item.flags() & ~Qt.ItemFlag.ItemIsEditable)
        table_item.setTextAlignment(int(Qt.AlignmentFlag.AlignCenter))
        open_button = QPushButton("打开并编辑")
        style_dialog_button(open_button)
        remove_button = QPushButton("移除")
        style_dialog_button(remove_button)
        open_button.clicked.connect(lambda _checked=False, row_index=index: on_open_row(row_index))
        remove_button.clicked.connect(lambda _checked=False, row_index=index: on_remove_row(row_index))
        action_layout = QHBoxLayout()
        action_layout.setContentsMargins(4, 2, 4, 2)
        action_layout.setSpacing(6)
        action_layout.addWidget(open_button)
        action_layout.addWidget(remove_button)
```

```python
        action_widget = QWidget(table)
        action_widget.setLayout(action_layout)
        table.setItem(index, 0, name_item)
        table.setItem(index, 1, version_item)
        table.setItem(index, 2, level_item)
        table.setItem(index, 3, image_item)
        table.setItem(index, 4, table_item)
        table.setCellWidget(index, 5, action_widget)
        if selected_path is not None and path == selected_path:
            selected_row = index
    state.source.file_items = updated_items
    table.blockSignals(False)
    if table.rowCount() > 0:
        table.selectRow(selected_row)


def collect_table_items(table: QTableWidget, preferred_textbook_version: str) -> list[AiSourceFileItem]:
    items: list[AiSourceFileItem] = []
    for row in range(table.rowCount()):
        name_item = table.item(row, 0)
        if name_item is None:
            continue
        raw_path = str(name_item.data(Qt.ItemDataRole.UserRole) or "").strip()
        if not raw_path:
            continue
        version = (
            (table.item(row, 1).text() if table.item(row, 1) else "").strip() or preferred_textbook_version
        )
        level_path = (table.item(row, 2).text() if table.item(row, 2) else "").strip()
        items.append(AiSourceFileItem(path=raw_path, version=version, level_path=level_path))
    return items


def find_invalid_level_paths(table: QTableWidget, is_valid_level_path) -> list[str]:
    invalid_names: list[str] = []
    for row in range(table.rowCount()):
        name_item = table.item(row, 0)
        level_item = table.item(row, 2)
        if name_item is None or level_item is None:
            continue
        level_path = level_item.text().strip()
        if level_path and not is_valid_level_path(level_path):
            invalid_names.append(name_item.text().strip() or f"第{row + 1}行")
    return invalid_names


def build_split_import_states_for_paths(
    *,
    base_state: ImportWizardSession,
    paths: list[Path],
    items: list[AiSourceFileItem],
    rename_project,
) -> list[ImportWizardSession]:
    item_map = {item.path: item for item in items if str(item.path or "").strip()}
    split_states: list[ImportWizardSession] = []
    for path in paths:
        raw_path = str(path)
        source_item = item_map.get(raw_path, AiSourceFileItem(path=raw_path, version=base_state.preferred_textbook_version))
        child_state = base_state.build_import_session(
            source_files=[path],
            source_items=[source_item],
            import_level_path=source_item.level_path,
        )
        child_state.preferred_textbook_version = source_item.version or base_state.preferred_textbook_version
        if child_state.project_name_is_placeholder and child_state.project_dir is not None:
            rename_project(child_state, new_name=path.stem)
        split_states.append(child_state)
    return split_states

```

```python

def update_import_reminder(table: QTableWidget, paths: list[Path], *, image_col: int, table_col: int) -> None:
    table.blockSignals(True)
    populate_import_reminder_columns(table, paths, image_col=image_col, table_col=table_col)
    table.blockSignals(False)


def reminder_doc_path(table: QTableWidget, row: int) -> Path | None:
    if row < 0 or row >= table.rowCount():
        return None
    item = table.item(row, 0)
    if item is None:
        return None
    raw = item.data(Qt.ItemDataRole.UserRole)
    if not raw:
        return None
    return Path(str(raw))


def remove_file_row(table: QTableWidget, row: int) -> tuple[list[Path], Path | None]:
    if row < 0 or row >= table.rowCount():
        return [], None
    remaining_paths: list[Path] = []
    removed_path: Path | None = None
    for row_index in range(table.rowCount()):
        item = table.item(row_index, 0)
        if item is None:
            continue
        raw = item.data(Qt.ItemDataRole.UserRole)
        if not raw:
            continue
        path = Path(str(raw))
        if row_index == row:
            removed_path = path
            continue
        remaining_paths.append(path)
    return remaining_paths, removed_path


def refresh_after_external_doc_edit(
    *,
    table: QTableWidget,
    items: list[AiSourceFileItem],
    changed_paths: list[Path],
    image_col: int,
    table_col: int,
) -> None:
    paths = [Path(item.path) for item in items]
    update_import_reminder(table, paths, image_col=image_col, table_col=table_col)
    select_first_changed_row(table, changed_paths)

```

sj_generator/presentation/qt/pages/import_flow/success_page.py

072-01

```python
from PyQt6.QtCore import Qt
from PyQt6.QtWidgets import QLabel, QVBoxLayout, QWizard, QWizardPage

from sj_generator.application.state import ImportWizardSession


class ImportSuccessPage(QWizardPage):
    def __init__(self, state: ImportWizardSession) -> None:
        super().__init__()
        self._state = state
        self.setTitle("完成")
        self.setFinalPage(True)

        self._title = QLabel("导入数据库成功")
        self._title.setAlignment(Qt.AlignmentFlag.AlignHCenter | Qt.AlignmentFlag.AlignVCenter)
        self._title.setStyleSheet("font-size: 22px; font-weight: 600;")

        self._hint = QLabel("")
        self._hint.setWordWrap(True)
        self._hint.setAlignment(Qt.AlignmentFlag.AlignHCenter | Qt.AlignmentFlag.AlignVCenter)

        layout = QVBoxLayout()
        layout.addStretch(1)
        layout.addWidget(self._title)
        layout.addWidget(self._hint)
        layout.addStretch(1)
        self.setLayout(layout)

    def initializePage(self) -> None:
        wizard = self.wizard()
        if isinstance(wizard, QWizard):
            wizard.setButtonText(QWizard.WizardButton.FinishButton, "返回开始页")
            wizard.setButtonText(QWizard.WizardButton.CancelButton, "返回开始页")
        count = self._state.execution.db_import_count
        self._title.setText("导入数据库成功")
        lines = [f"本次已写入当前总库 {count} 道题。"]
        lines.append("Markdown 导出请在开始界面的菜单栏中操作。")
        self._hint.setText("\n".join(lines))

    def cleanupPage(self) -> None:
        wizard = self.wizard()
        if isinstance(wizard, QWizard):
            wizard.setButtonText(QWizard.WizardButton.FinishButton, "返回开始页")
            wizard.setButtonText(QWizard.WizardButton.CancelButton, "返回开始页")

    def nextId(self) -> int:
        return -1

__all__ = ["ImportSuccessPage"]
```

sj_generator/presentation/qt/pages/import_flow_registry.py

073-01

```python
from __future__ import annotations

from dataclasses import dataclass
from typing import Callable

from PyQt6.QtWidgets import QWizard, QWizardPage

from sj_generator.application.state import ImportWizardSession
from sj_generator.presentation.qt.pages.import_flow import (
    AiAnalysisPage,
    AiImportContentPage,
    AiImportPage,
    AiSelectFilesPage,
    DedupeResultPage,
    ImportSuccessPage,
)
from sj_generator.presentation.qt.constants import (
    PAGE_AI_ANALYSIS,
    PAGE_AI_IMPORT,
    PAGE_AI_IMPORT_CONTENT,
    PAGE_AI_SELECT,
    PAGE_DEDUPE_RESULT,
    PAGE_IMPORT_SUCCESS,
)


@dataclass(frozen=True)
class WizardPageRegistration:
    page_id: int
    build: Callable[[ImportWizardSession], QWizardPage]


def build_import_flow_pages() -> tuple[WizardPageRegistration, ...]:
    return (
        WizardPageRegistration(PAGE_AI_SELECT, AiSelectFilesPage),
        WizardPageRegistration(PAGE_AI_IMPORT, AiImportPage),
        WizardPageRegistration(PAGE_AI_IMPORT_CONTENT, AiImportContentPage),
        WizardPageRegistration(PAGE_DEDUPE_RESULT, DedupeResultPage),
        WizardPageRegistration(PAGE_AI_ANALYSIS, AiAnalysisPage),
        WizardPageRegistration(PAGE_IMPORT_SUCCESS, ImportSuccessPage),
    )


def register_import_flow_pages(wizard: QWizard, state: ImportWizardSession) -> None:
    for registration in build_import_flow_pages():
        wizard.setPage(registration.page_id, registration.build(state))

```

```python
from __future__ import annotations

from html import escape
from pathlib import Path
import random

from PyQt6.QtCore import QTimer, Qt
from PyQt6.QtGui import QPixmap
from PyQt6.QtWidgets import (
    QApplication,
    QFormLayout,
    QFrame,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QMessageBox,
    QPushButton,
    QSizePolicy,
    QVBoxLayout,
    QWizard,
    QWizardPage,
)

from sj_generator.shared.paths import app_paths
from sj_generator.presentation.qt.constants import PAGE_WELCOME
from sj_generator.presentation.qt.styles import rounded_panel_stylesheet

_IMAGE_SUFFIXES = (".png", ".jpg", ".jpeg", ".webp", ".gif", ".bmp")
BUTTON_MIN_WIDTH = 96
BUTTON_MIN_HEIGHT = 36


def _style_dialog_button(button: QPushButton | None, text: str | None = None) -> None:
    if button is None:
        return
    if text:
        button.setText(text)
    button.setMinimumSize(BUTTON_MIN_WIDTH, BUTTON_MIN_HEIGHT)


def _style_message_box_buttons(box: QMessageBox) -> None:
    for button_type, text in (
        (QMessageBox.StandardButton.Ok, "确定"),
        (QMessageBox.StandardButton.Cancel, "取消"),
        (QMessageBox.StandardButton.Yes, "是"),
        (QMessageBox.StandardButton.No, "否"),
    ):
        _style_dialog_button(box.button(button_type), text)


def _show_message_box(
    parent,
    *,
    title: str,
    text: str,
    icon: QMessageBox.Icon,
    buttons: QMessageBox.StandardButton = QMessageBox.StandardButton.Ok,
    default_button: QMessageBox.StandardButton = QMessageBox.StandardButton.NoButton,
) -> QMessageBox.StandardButton:
    box = QMessageBox(parent)
    box.setWindowTitle(title)
    box.setText(text)
    box.setIcon(icon)
    box.setStandardButtons(buttons)
    if default_button != QMessageBox.StandardButton.NoButton:
        box.setDefaultButton(default_button)
    _style_message_box_buttons(box)
    return QMessageBox.StandardButton(box.exec())


```

```python
def _quotation_dir() -> Path:
    return app_paths().reference_quotation_dir


def _find_quote_file() -> Path | None:
    quote_dir = _quotation_dir()
    if not quote_dir.exists():
        return None
    txt_files = sorted(path for path in quote_dir.iterdir() if path.is_file() and path.suffix.lower() == ".txt")
    return txt_files[0] if txt_files else None


def _find_photo_file(author_name: str) -> Path | None:
    picture_dir = app_paths().reference_picture_dir
    if not picture_dir.exists():
        return None

    normalized_author = author_name.strip()
    if normalized_author:
        for suffix in _IMAGE_SUFFIXES:
            candidate = picture_dir / f"{normalized_author}{suffix}"
            if candidate.exists():
                return candidate

    image_files = sorted(path for path in picture_dir.iterdir() if path.is_file() and path.suffix.lower() in _IMAGE_SUFFIXES)
    return image_files[0] if image_files else None


def _load_quote_content() -> tuple[str, str, Path | None]:
    quote_file = _find_quote_file()
    if quote_file is None:
        return "名言文件缺失。请在 reference/quotation 目录放入 .txt 文件。", "未提供作者", None

    lines = [line.strip() for line in quote_file.read_text(encoding="utf-8").splitlines() if line.strip()]
    quote_text = random.choice(lines) if lines else "名言文件为空。"
    author_name = quote_file.stem.strip() or "未提供作者"
    return quote_text, author_name, _find_photo_file(author_name)


class IntroPage(QWizardPage):
    def __init__(self) -> None:
        super().__init__()
        self.setTitle("登录")
        self._photo_path: Path | None = None
        self._photo_pixmap: QPixmap | None = None

        self._photo_label = QLabel()
        self._photo_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self._photo_label.setScaledContents(False)
        self._photo_label.setMinimumSize(300, 380)
        self._photo_label.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Expanding)
        self._photo_label.setStyleSheet(rounded_panel_stylesheet(background="#f7f7f7"))
        self._photo_label.setText("正在加载图片…")

        self._quote_label = QLabel()
        self._quote_label.setText("正在加载名言…")
        self._quote_label.setTextFormat(Qt.TextFormat.RichText)
        self._quote_label.setWordWrap(True)
        self._quote_label.setAlignment(Qt.AlignmentFlag.AlignVCenter | Qt.AlignmentFlag.AlignLeft)
        self._quote_label.setStyleSheet(
            "background: transparent; border: none; "
            "font-size: 26px; font-weight: 600; color: #c00000; "
            "font-family: 'KaiTi', 'STKaiti', 'SimSun', 'Songti SC';"
        )
        self._quote_label.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Expanding)
        self._quote_label.setContentsMargins(0, 0, 0, 0)

        username_edit = QLineEdit()
```

```python
        username_edit.setPlaceholderText("请输入用户名")
        username_edit.setMinimumHeight(36)
        username_edit.setStyleSheet("border: 1px solid #000000; border-radius: 0px; background: #ffffff;")

        password_edit = QLineEdit()
        password_edit.setPlaceholderText("请输入密码")
        password_edit.setEchoMode(QLineEdit.EchoMode.Password)
        password_edit.setMinimumHeight(36)
        password_edit.setStyleSheet("border: 1px solid #000000; border-radius: 0px; background: #ffffff;")
        password_edit.returnPressed.connect(self._go_next)

        btn_style = (
            "QPushButton {"
            "font-size: 16px; font-weight: 600; padding: 6px 18px; "
            "background: #ffffff; border: 1px solid #000000; border-radius: 0px;"
            "}"
            "QPushButton:hover {"
            "background: #f2f2f2;"
            "}"
            "QPushButton:pressed {"
            "background: #d9d9d9; padding-top: 8px; padding-bottom: 4px;"
            "}"
        )
        btn_height = 44

        user_login_btn = QPushButton("用户登录")
        user_login_btn.setFixedHeight(btn_height)
        user_login_btn.setStyleSheet(btn_style)
        user_login_btn.clicked.connect(self._show_login_error)

        local_login_btn = QPushButton("本地登录")
        local_login_btn.setFixedHeight(btn_height)
        local_login_btn.setStyleSheet(btn_style)
        local_login_btn.clicked.connect(self._go_next)

        exit_btn = QPushButton("退出程序")
        exit_btn.setFixedHeight(btn_height)
        exit_btn.setStyleSheet(btn_style)
        exit_btn.clicked.connect(self._exit_program)

        login_form = QFormLayout()
        login_form.setLabelAlignment(Qt.AlignmentFlag.AlignRight | Qt.AlignmentFlag.AlignVCenter)
        login_form.setFormAlignment(Qt.AlignmentFlag.AlignHCenter | Qt.AlignmentFlag.AlignTop)
        login_form.setHorizontalSpacing(12)
        login_form.setVerticalSpacing(12)
        username_label = QLabel("用户名：")
        username_label.setStyleSheet("border: none; background: transparent;")
        password_label = QLabel("密　码：")
        password_label.setStyleSheet("border: none; background: transparent;")
        login_form.addRow(username_label, username_edit)
        login_form.addRow(password_label, password_edit)

        self._login_panel = QFrame()
        self._login_panel.setStyleSheet(rounded_panel_stylesheet())
        self._login_panel.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Expanding)
        self._login_layout = QVBoxLayout()
        self._login_layout.setContentsMargins(20, 18, 20, 18)
        self._login_layout.setSpacing(12)
        login_title = QLabel("系统登陆")
        login_title.setAlignment(Qt.AlignmentFlag.AlignHCenter | Qt.AlignmentFlag.AlignVCenter)
        login_title.setStyleSheet("font-size: 18px; font-weight: 600;")
        self._login_layout.addWidget(login_title)
        self._login_layout.addLayout(login_form)
        self._btn_row = QHBoxLayout()
        self._btn_row.setSpacing(10)
        self._btn_row.setContentsMargins(0, 0, 0, 0)
        self._btn_row.addWidget(user_login_btn)
        self._btn_row.addWidget(local_login_btn)
        self._btn_row.addWidget(exit_btn)
        self._login_layout.addLayout(self._btn_row)
```

```python
        self._login_panel.setLayout(self._login_layout)
        self._login_buttons = [user_login_btn, local_login_btn, exit_btn]

        photo_layout = QVBoxLayout()
        photo_layout.addWidget(self._photo_label, 1)

        quote_panel = QFrame()
        quote_panel.setStyleSheet(rounded_panel_stylesheet())
        quote_panel.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Expanding)
        quote_layout = QVBoxLayout()
        quote_layout.setContentsMargins(0, 0, 0, 0)
        quote_layout.addWidget(self._quote_label, 1)
        quote_panel.setLayout(quote_layout)

        right_layout = QVBoxLayout()
        right_layout.setSpacing(18)
        right_layout.addWidget(quote_panel, 5)
        right_layout.addWidget(self._login_panel, 5)

        content_layout = QHBoxLayout()
        content_layout.addLayout(photo_layout, 4)
        content_layout.addLayout(right_layout, 6)

        layout = QVBoxLayout()
        layout.addLayout(content_layout, 1)
        self.setLayout(layout)
        QTimer.singleShot(0, self._load_quote_and_photo)
        QTimer.singleShot(0, self._refresh_photo_display)
        QTimer.singleShot(0, self._sync_login_button_widths)

    def initializePage(self) -> None:
        wizard = self.wizard()
        if wizard is not None:
            wizard.setButtonText(QWizard.WizardButton.NextButton, "下一步")
            QTimer.singleShot(0, lambda: self._set_wizard_nav_visible(False))

    def cleanupPage(self) -> None:
        self._set_wizard_nav_visible(True)

    def nextId(self) -> int:
        return PAGE_WELCOME

    def _go_next(self) -> None:
        wizard = self.wizard()
        if wizard is not None:
            wizard.next()

    def _show_login_error(self) -> None:
        _show_message_box(self, title="登录失败", text="账号密码错误", icon=QMessageBox.Icon.Warning)

    def _load_quote_and_photo(self) -> None:
        quote_text, _author_name, photo_path = _load_quote_content()
        self._quote_label.setText(
            '<div style="margin: 0; line-height: 1.2; white-space: pre-wrap;">'
            f"{escape(f'    {quote_text}')}"
            "</div>"
        )
        self._photo_path = photo_path
        self._apply_photo()

    def _exit_program(self) -> None:
        app = QApplication.instance()
        if app is not None:
            app.quit()

    def _set_wizard_nav_visible(self, visible: bool) -> None:
        wizard = self.wizard()
        if wizard is None:
            return
        for which in (
```

```python
            QWizard.WizardButton.BackButton,
            QWizard.WizardButton.NextButton,
            QWizard.WizardButton.CancelButton,
            QWizard.WizardButton.FinishButton,
        ):
            button = wizard.button(which)
            if button is not None:
                button.setVisible(visible)

    def _apply_photo(self) -> None:
        if self._photo_path is None:
            self._photo_pixmap = None
            self._photo_label.clear()
            self._photo_label.setText("暂无照片\n\n请将图片文件放入 reference/picture 目录\n并优先使用与作者同名的文件名。")
            return

        pixmap = QPixmap(str(self._photo_path))
        if pixmap.isNull():
            self._photo_pixmap = None
            self._photo_label.clear()
            self._photo_label.setText("照片加载失败")
            return

        self._photo_pixmap = pixmap
        self._refresh_photo_display()

    def resizeEvent(self, event) -> None:
        super().resizeEvent(event)
        self._refresh_photo_display()
        self._sync_login_button_widths()

    def _refresh_photo_display(self) -> None:
        if self._photo_pixmap is None:
            return

        contents = self._photo_label.contentsRect()
        target_width = max(1, contents.width())
        target_height = max(1, contents.height())
        scaled = self._photo_pixmap.scaled(
            target_width,
            target_height,
            Qt.AspectRatioMode.KeepAspectRatio,
            Qt.TransformationMode.SmoothTransformation,
        )
        self._photo_label.setPixmap(scaled)

    def _sync_login_button_widths(self) -> None:
        panel = getattr(self, "_login_panel", None)
        layout = getattr(self, "_login_layout", None)
        buttons = getattr(self, "_login_buttons", None)
        if panel is None or layout is None or not buttons:
            return
        margins = layout.contentsMargins()
        spacing = self._btn_row.spacing()
        available_width = panel.contentsRect().width() - margins.left() - margins.right() - spacing * (len(buttons) - 1)
        if available_width <= 0:
            return
        button_width = max(80, available_width // len(buttons))
        for button in buttons:
            button.setFixedWidth(button_width)

__all__ = ["IntroPage"]
```

```python
from __future__ import annotations

from datetime import date
from pathlib import Path
import re

from sj_generator.domain.entities import Question
from sj_generator.infrastructure.exporting.export_md import export_questions_to_markdown
from sj_generator.infrastructure.exporting.export_pdf import export_questions_to_pdf
from sj_generator.infrastructure.persistence.excel_repo import save_db_question_records
from sj_generator.infrastructure.persistence.sqlite_repo import DbQuestionRecord


def sanitize_export_name(name: str) -> str:
    cleaned = re.sub(r'[<>:"/\\\\|?*]+', "_", (name or "").strip())
    cleaned = cleaned.strip(" .")
    return cleaned or "导出结果"


def display_export_level_name(level_path: str) -> str:
    normalized_level_path = str(level_path or "").strip()
    if normalized_level_path == "0":
        return "高中阶段"
    return normalized_level_path


def default_export_dir(last_export_dir: Path | None, db_path: Path) -> Path:
    return last_export_dir or db_path.parent


def db_record_to_question(record: DbQuestionRecord) -> Question:
    return Question(
        number=record.id,
        stem=record.stem,
        options=format_db_options(record),
        answer=format_db_answer(record),
        analysis=record.analysis,
        question_type=record.question_type,
        choice_1=record.choice_1,
        choice_2=record.choice_2,
        choice_3=record.choice_3,
        choice_4=record.choice_4,
    )


def format_db_options(record: DbQuestionRecord) -> str:
    options = [record.option_1, record.option_2, record.option_3, record.option_4]
    if record.question_type == "可转多选":
        lines = [
            f"{marker}. {text.strip()}".rstrip()
            for marker, text in zip(["①", "②", "③", "④"], options)
            if text.strip()
        ]
        choice_lines = [
            format_choice_mapping(letter, value)
            for letter, value in (
                ("A", record.choice_1),
                ("B", record.choice_2),
                ("C", record.choice_3),
                ("D", record.choice_4),
            )
            if value.strip()
        ]
        if choice_lines and lines:
            lines.append("")
        lines.extend(choice_lines)
        return "\n".join(lines)
    if record.question_type == "多选":
        markers = ["①", "②", "③", "④"]
    else:
```

```python
        markers = ["A", "B", "C", "D"]
    lines = [
        f"{markers[idx - 1]}. {text.strip()}".rstrip()
        for idx, text in enumerate(options, start=1)
        if text.strip()
    ]
    return "\n".join(lines)


def format_db_answer(record: DbQuestionRecord) -> str:
    answer = (record.answer or "").strip()
    if record.question_type == "可转多选":
        if any(value.strip() for value in (record.choice_1, record.choice_2, record.choice_3, record.choice_4)):
            return answer
    if record.question_type not in ("多选", "可转多选"):
        return answer
    marker_map = {
        "1": "①",
        "2": "②",
        "3": "③",
        "4": "④",
        "5": "⑤",
        "6": "⑥",
        "7": "⑦",
        "8": "⑧",
        "9": "⑨",
        "10": "⑩",
    }
    tokens = [token.strip() for token in answer.replace(", ", ",").split(",") if token.strip()]
    if not tokens:
        return answer
    return "".join(marker_map.get(token, token) for token in tokens)


def format_choice_mapping(letter: str, value: str) -> str:
    circled = "".join(digit_to_circled(ch) for ch in (value or "").strip() if ch.isdigit())
    if circled:
        return f"{letter}. {circled}"
    return f"{letter}. {(value or '').strip()}".rstrip()


def digit_to_circled(digit: str) -> str:
    return {
        "1": "①",
        "2": "②",
        "3": "③",
        "4": "④",
        "5": "⑤",
        "6": "⑥",
        "7": "⑦",
        "8": "⑧",
        "9": "⑨",
        "0": "⑩",
    }.get(digit, digit)


def export_current_level_to_markdown(
    *,
    records: list[DbQuestionRecord],
    level_path: str,
    target_path: Path,
    convertible_multi_mode: str,
    include_answers: bool,
    include_analysis: bool,
) -> None:
    questions = [db_record_to_question(record) for record in records]
    md_text = export_questions_to_markdown(
        excel_file_name=level_path,
        export_date=date.today(),
```

```python
        questions=questions,
        convertible_multi_mode=convertible_multi_mode,
        include_answers=include_answers,
        include_analysis=include_analysis,
    )
    target_path.parent.mkdir(parents=True, exist_ok=True)
    target_path.write_text(md_text, encoding="utf-8")


def export_current_level_to_pdf(
    *,
    records: list[DbQuestionRecord],
    level_path: str,
    target_path: Path,
    convertible_multi_mode: str,
    include_answers: bool,
    include_analysis: bool,
) -> None:
    questions = [db_record_to_question(record) for record in records]
    export_questions_to_pdf(
        excel_file_name=level_path,
        export_date=date.today(),
        questions=questions,
        target_path=target_path,
        convertible_multi_mode=convertible_multi_mode,
        include_answers=include_answers,
        include_analysis=include_analysis,
    )


def export_db_records_to_xlsx(*, records: list[DbQuestionRecord], target_path: Path) -> None:
    save_db_question_records(target_path, records)

```

```python
from __future__ import annotations

from pathlib import Path

from PyQt6.QtWidgets import QFileDialog, QMessageBox, QTreeWidgetItem, QWidget

from sj_generator.application.state import WizardState
from sj_generator.infrastructure.persistence.sqlite_repo import DbQuestionRecord, load_all_questions
from sj_generator.presentation.qt.message_box import show_message_box
from sj_generator.presentation.qt.pages.welcome_export import (
    default_export_dir,
    display_export_level_name,
    export_current_level_to_markdown,
    export_current_level_to_pdf,
    export_db_records_to_xlsx,
    sanitize_export_name,
)


def selected_tree_level_path(current: QTreeWidgetItem | None, role_level_path: int) -> str:
    if current is None:
        return ""
    return str(current.data(0, role_level_path) or "").strip()


def export_current_level_markdown(
    *,
    parent: QWidget,
    current_item: QTreeWidgetItem | None,
    current_records: list[DbQuestionRecord],
    db_path: Path,
    state: WizardState,
    role_level_path: int,
) -> Path | None:
    if not current_records:
        show_message_box(parent, title="无法导出", text="当前页面没有可导出的题目。", icon=QMessageBox.Icon.Warning)
        return None
    level_path = selected_tree_level_path(current_item, role_level_path)
    if not level_path:
        show_message_box(parent, title="无法导出", text="请先选择要导出的层级。", icon=QMessageBox.Icon.Warning)
        return None
    safe_level_name = sanitize_export_name(display_export_level_name(level_path))
    suggested = str(db_path.parent / f"{safe_level_name}.md")
    file_path, _ = QFileDialog.getSaveFileName(parent, "导出 Markdown", suggested, "Markdown (*.md)")
    if not file_path:
        return None
    target_path = Path(file_path)
    try:
        export_current_level_to_markdown(
            records=list(current_records),
            level_path=level_path,
            target_path=target_path,
            convertible_multi_mode=state.export_convertible_multi_mode,
            include_answers=state.export_include_answers,
            include_analysis=state.export_include_analysis,
        )
    except Exception as exc:
        show_message_box(parent, title="导出失败", text=f"写入 Markdown 失败：{exc}", icon=QMessageBox.Icon.Critical)
        return None
    return target_path


def export_current_level_pdf(
    *,
    parent: QWidget,
    current_item: QTreeWidgetItem | None,
    current_records: list[DbQuestionRecord],
    db_path: Path,
```

```python
    state: WizardState,
    role_level_path: int,
) -> Path | None:
    if not current_records:
        show_message_box(parent, title="无法导出", text="当前页面没有可导出的题目。", icon=QMessageBox.Icon.Warning)
        return None
    level_path = selected_tree_level_path(current_item, role_level_path)
    if not level_path:
        show_message_box(parent, title="无法导出", text="请先选择要导出的层级。", icon=QMessageBox.Icon.Warning)
        return None
    safe_level_name = sanitize_export_name(display_export_level_name(level_path))
    suggested = str(default_export_dir(state.last_export_dir, db_path) / f"{safe_level_name}.pdf")
    file_path, _ = QFileDialog.getSaveFileName(parent, "导出 PDF", suggested, "PDF (*.pdf)")
    if not file_path:
        return None
    target_path = Path(file_path)
    try:
        export_current_level_to_pdf(
            records=list(current_records),
            level_path=level_path,
            target_path=target_path,
            convertible_multi_mode=state.export_convertible_multi_mode,
            include_answers=state.export_include_answers,
            include_analysis=state.export_include_analysis,
        )
    except Exception as exc:
        show_message_box(parent, title="导出失败", text=f"写入 PDF 失败：{exc}", icon=QMessageBox.Icon.Critical)
        return None
    return target_path


def export_current_level_xlsx(
    *,
    parent: QWidget,
    current_item: QTreeWidgetItem | None,
    current_records: list[DbQuestionRecord],
    db_path: Path,
    state: WizardState,
    role_level_path: int,
) -> Path | None:
    if not current_records:
        show_message_box(parent, title="无法导出", text="当前页面没有可导出的题目。", icon=QMessageBox.Icon.Warning)
        return None
    level_path = selected_tree_level_path(current_item, role_level_path)
    if not level_path:
        show_message_box(parent, title="无法导出", text="请先选择要导出的层级。", icon=QMessageBox.Icon.Warning)
        return None
    safe_level_name = sanitize_export_name(display_export_level_name(level_path))
    suggested = str(default_export_dir(state.last_export_dir, db_path) / f"{safe_level_name}.xlsx")
    file_path, _ = QFileDialog.getSaveFileName(parent, "导出当前页面 xlsx", suggested, "Excel (*.xlsx)")
    if not file_path:
        return None
    target_path = Path(file_path)
    try:
        export_db_records_to_xlsx(records=list(current_records), target_path=target_path)
    except Exception as exc:
        show_message_box(parent, title="导出失败", text=f"写入 xlsx 失败：{exc}", icon=QMessageBox.Icon.Critical)
        return None
    return target_path


def export_db_table_xlsx(*, parent: QWidget, db_path: Path, last_export_dir: Path | None) -> Path | None:
    if not db_path.exists():
        show_message_box(parent, title="无法导出", text="当前数据库文件不存在。", icon=QMessageBox.Icon.Warning)
        return None
    records = load_all_questions(db_path)
```

```python
    if not records:
        show_message_box(parent, title="无法导出", text="当前数据库表没有可导出的题目。", icon=QMessageBox.Icon.Warning)
        return None
    suggested_name = f"{sanitize_export_name(db_path.stem)}_整体数据库表.xlsx"
    suggested = str(default_export_dir(last_export_dir, db_path) / suggested_name)
    file_path, _ = QFileDialog.getSaveFileName(parent, "导出整体数据库表 xlsx", suggested, "Excel (*.xlsx)")
    if not file_path:
        return None
    target_path = Path(file_path)
    try:
        export_db_records_to_xlsx(records=records, target_path=target_path)
    except Exception as exc:
        show_message_box(parent, title="导出失败", text=f"写入 xlsx 失败：{exc}", icon=QMessageBox.Icon.Critical)
        return None
    return target_path
```

```python
from __future__ import annotations

import hashlib
from pathlib import Path
import shutil

from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import QMessageBox, QWidget, QWizard

from sj_generator.application.state import ImportWizardSession, WizardState
from sj_generator.presentation.qt.message_box import show_message_box


def effective_import_source_dir(configured_dir: str) -> str:
    configured = Path(configured_dir).expanduser()
    if configured.exists() and configured.is_dir():
        return str(configured)
    return str(Path.home() / "Downloads")


def backup_import_source_files(db_path: Path, paths: list[Path]) -> list[Path]:
    if not paths:
        return []
    target_dir = db_path.parent / "doc"
    target_dir.mkdir(parents=True, exist_ok=True)
    known_hash_paths = build_doc_hash_index(target_dir)
    copied_paths: list[Path] = []
    seen_targets: set[Path] = set()
    for src in paths:
        if not src.exists() or not src.is_file():
            raise FileNotFoundError(src)
        file_hash = file_sha256(src)
        target_path = known_hash_paths.get(file_hash)
        if target_path is None:
            target_path = next_doc_backup_path(target_dir, src.name)
            shutil.copy2(src, target_path)
            known_hash_paths[file_hash] = target_path
        resolved_target = target_path.resolve()
        if resolved_target in seen_targets:
            continue
        seen_targets.add(resolved_target)
        copied_paths.append(target_path)
    return copied_paths


def build_doc_hash_index(target_dir: Path) -> dict[str, Path]:
    hash_index: dict[str, Path] = {}
    for path in sorted(target_dir.glob("*.docx"), key=lambda item: item.name.lower()):
        if not path.is_file():
            continue
        try:
            current_hash = file_sha256(path)
        except Exception:
            continue
        hash_index.setdefault(current_hash, path)
    return hash_index


def file_sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        while True:
            chunk = handle.read(1024 * 1024)
            if not chunk:
                break
            digest.update(chunk)
    return digest.hexdigest()


def next_doc_backup_path(target_dir: Path, file_name: str) -> Path:
```

```python
    candidate = target_dir / file_name
    if not candidate.exists():
        return candidate
    stem = candidate.stem
    suffix = candidate.suffix
    index = 2
    while True:
        numbered = target_dir / f"{stem}_{index}{suffix}"
        if not numbered.exists():
            return numbered
        index += 1


def open_import_flow_windows(
    *,
    base_state: WizardState,
    db_path: Path,
    paths: list[Path],
    message_parent: QWidget,
    launcher,
    owner,
    import_flow_windows: list[QWizard],
    start_page_id: int,
    on_state_finished,
    on_window_forget,
) -> bool:
    selected_paths = [Path(path) for path in paths if str(path).strip()]
    if not selected_paths:
        return False
    base_state.import_source_dir_text = str(selected_paths[0].parent)
    try:
        copied_paths = backup_import_source_files(db_path, selected_paths)
    except Exception as exc:
        show_message_box(
            message_parent,
            title="备份失败",
            text=f"复制资料到 doc 目录失败：{exc}",
            icon=QMessageBox.Icon.Critical,
        )
        return False
    if not copied_paths:
        return False
    shared_state = base_state.build_import_session(source_files=copied_paths)
    return open_import_flow_states(
        launcher=launcher,
        owner=owner,
        import_flow_windows=import_flow_windows,
        states=[shared_state],
        start_page_id=start_page_id,
        on_state_finished=on_state_finished,
        on_window_forget=on_window_forget,
    )


def open_import_flow_states(
    *,
    launcher,
    owner,
    import_flow_windows: list[QWizard],
    states: list[ImportWizardSession],
    start_page_id: int,
    on_state_finished,
    on_window_forget,
) -> bool:
    window_states = [state for state in states if isinstance(state, ImportWizardSession)]
    if not window_states:
        return False

    from sj_generator.presentation.qt.import_flow import ImportFlowWizard

```

```python
    opened_windows: list[QWizard] = []
    for window_state in window_states:
        dlg = ImportFlowWizard(window_state, owner, launcher=launcher, start_page_id=start_page_id)
        dlg.setAttribute(Qt.WidgetAttribute.WA_DeleteOnClose, True)
        dlg.finished.connect(lambda _result, state=window_state: on_state_finished(state))
        dlg.destroyed.connect(lambda _obj=None, wizard=dlg: on_window_forget(wizard))
        import_flow_windows.append(dlg)
        opened_windows.append(dlg)
    for index, dlg in enumerate(opened_windows):
        should_activate = index == (len(opened_windows) - 1)
        QTimer.singleShot(
            0,
            lambda wizard=dlg, activate=should_activate: show_import_flow_window(
                import_flow_windows,
                wizard,
                activate=activate,
            ),
        )
    return True


def show_import_flow_window(import_flow_windows: list[QWizard], wizard: QWizard, *, activate: bool) -> None:
    if wizard not in import_flow_windows:
        return
    wizard.show()
    if activate:
        wizard.raise_()
        wizard.activateWindow()

```

```python
from __future__ import annotations

from dataclasses import dataclass

from PyQt6.QtGui import QAction
from PyQt6.QtWidgets import QMenuBar, QWidget


@dataclass
class WelcomeMenuBundle:
    menu_bar: QMenuBar
    column_actions: dict[int, QAction]
    doc_import_action: QAction
    table_import_action: QAction
    export_md_action: QAction
    export_pdf_action: QAction
    export_current_level_xlsx_action: QAction
    export_db_table_xlsx_action: QAction
    add_question_action: QAction
    general_settings_action: QAction
    import_settings_action: QAction
    export_settings_action: QAction
    api_settings_action: QAction


def build_welcome_menu_bar(
    *,
    parent: QWidget,
    column_defs: list[tuple[str, str, bool]],
    column_visibility: dict[str, bool],
    on_doc_import,
    on_table_import,
    on_export_markdown,
    on_export_pdf,
    on_export_current_xlsx,
    on_export_db_xlsx,
    on_add_question,
    on_open_general_settings,
    on_open_import_settings,
    on_open_export_settings,
    on_open_api_config,
    on_toggle_column,
) -> WelcomeMenuBundle:
    menu_bar = QMenuBar(parent)
    file_menu = menu_bar.addMenu("文件")
    import_menu = file_menu.addMenu("导入")
    doc_import_action = import_menu.addAction("从文档文件解析导入")
    doc_import_action.triggered.connect(on_doc_import)
    table_import_action = import_menu.addAction("从表格文件直接导入")
    table_import_action.triggered.connect(on_table_import)

    export_menu = file_menu.addMenu("导出")
    export_md_action = export_menu.addAction("导出当前页面为 Markdown")
    export_md_action.triggered.connect(on_export_markdown)
    export_pdf_action = export_menu.addAction("导出当前页面为 PDF")
    export_pdf_action.triggered.connect(on_export_pdf)
    export_xlsx_menu = export_menu.addMenu("导出为 xlsx")
    export_current_level_xlsx_action = export_xlsx_menu.addAction("当前页面题目")
    export_current_level_xlsx_action.triggered.connect(on_export_current_xlsx)
    export_db_table_xlsx_action = export_xlsx_menu.addAction("所有题目")
    export_db_table_xlsx_action.triggered.connect(on_export_db_xlsx)

    edit_menu = menu_bar.addMenu("编辑")
    add_question_action = edit_menu.addAction("新增题目")
    add_question_action.triggered.connect(on_add_question)

    view_menu = menu_bar.addMenu("视图")
    column_menu = view_menu.addMenu("表格列显示")
    column_actions: dict[int, QAction] = {}
    for idx, (key, title, _visible) in enumerate(column_defs):
```

```python
        visible = column_visibility.get(key, False)
        action = column_menu.addAction(title)
        action.setCheckable(True)
        action.setChecked(visible)
        action.toggled.connect(lambda checked, col=idx: on_toggle_column(col, checked))
        column_actions[idx] = action

    settings_menu = menu_bar.addMenu("设置")
    general_settings_action = settings_menu.addAction("常规设定")
    general_settings_action.triggered.connect(on_open_general_settings)
    import_settings_action = settings_menu.addAction("导入设定")
    import_settings_action.triggered.connect(on_open_import_settings)
    export_settings_action = settings_menu.addAction("导出设定")
    export_settings_action.triggered.connect(on_open_export_settings)
    api_settings_action = settings_menu.addAction("API 配置")
    api_settings_action.triggered.connect(on_open_api_config)

    return WelcomeMenuBundle(
        menu_bar=menu_bar,
        column_actions=column_actions,
        doc_import_action=doc_import_action,
        table_import_action=table_import_action,
        export_md_action=export_md_action,
        export_pdf_action=export_pdf_action,
        export_current_level_xlsx_action=export_current_level_xlsx_action,
        export_db_table_xlsx_action=export_db_table_xlsx_action,
        add_question_action=add_question_action,
        general_settings_action=general_settings_action,
        import_settings_action=import_settings_action,
        export_settings_action=export_settings_action,
        api_settings_action=api_settings_action,
    )

```

```python
from __future__ import annotations

from pathlib import Path

from PyQt6.QtCore import Qt
from PyQt6.QtGui import QWheelEvent
from PyQt6.QtWidgets import (
    QAbstractItemView,
    QFileDialog,
    QHeaderView,
    QLabel,
    QMessageBox,
    QStackedLayout,
    QSplitter,
    QTableWidget,
    QTreeWidget,
    QTreeWidgetItem,
    QVBoxLayout,
    QWizard,
    QWizardPage,
    QWidget,
)

from sj_generator.application.settings import load_welcome_tree_expanded_prefixes
from sj_generator.application.state import (
    ImportWizardSession,
    WizardState,
    library_db_path_from_repo_parent_dir_text,
)
from sj_generator.infrastructure.persistence.sqlite_repo import DbQuestionRecord
from sj_generator.presentation.qt.program_settings_dialog import (
    SECTION_EXPORT,
    SECTION_GENERAL,
    SECTION_IMPORT,
)
from sj_generator.presentation.qt.constants import PAGE_AI_SELECT
from sj_generator.presentation.qt.message_box import show_message_box
from sj_generator.presentation.qt.styles import rounded_panel_stylesheet
from sj_generator.presentation.qt.table_copy import CopyableTableWidget
from sj_generator.presentation.qt.pages.welcome_export_actions import (
    export_current_level_markdown,
    export_current_level_pdf,
    export_current_level_xlsx,
    export_db_table_xlsx,
)
from sj_generator.presentation.qt.pages.welcome_import import (
    effective_import_source_dir,
    open_import_flow_states,
    open_import_flow_windows,
    show_import_flow_window,
)
from sj_generator.presentation.qt.pages.welcome_menu import build_welcome_menu_bar
from sj_generator.presentation.qt.pages.welcome_page_actions import (
    import_from_table_file,
    open_api_config,
    open_program_settings,
)
from sj_generator.presentation.qt.pages.welcome_question_actions import (
    add_question_manually,
    edit_question_record,
    selected_tree_level_path_for_create,
)
from sj_generator.presentation.qt.pages.welcome_table import populate_db_records_table
from sj_generator.presentation.qt.pages.welcome_table_view import (
    adjust_table_font_size,
    apply_table_font_size,
    apply_table_row_resize,
    apply_table_row_resize_followup,
    load_column_visibility,
    load_table_font_point_size,
```

```python
    persist_table_font_point_size,
    rebalance_visible_columns,
    refresh_table_layout_after_column_change,
    schedule_table_row_resize,
    set_column_visible,
)
from sj_generator.presentation.qt.pages.welcome_tree_actions import (
    delete_level_subtree,
    handle_level_selection_changed,
    refresh_level_tree,
    show_folder_tree_context_menu,
    sync_expanded_level_prefixes_from_tree,
)

LAST_COLUMN_MIN_WIDTH = 140
TABLE_FONT_POINT_SIZE_MIN = 8
TABLE_FONT_POINT_SIZE_MAX = 28
TREE_ROLE_LEVEL_PATH = Qt.ItemDataRole.UserRole
TREE_ROLE_LEVEL_PREFIX = int(Qt.ItemDataRole.UserRole) + 1
TREE_ROLE_LEVEL_DEPTH = int(Qt.ItemDataRole.UserRole) + 2
TABLE_COLUMNS = [
    ("stem", "题目", True),
    ("options", "选项", True),
    ("answer", "答案", True),
    ("analysis", "解析", True),
    ("id", "题目id", False),
    ("question_type", "类型", False),
    ("choice_1", "组合A", False),
    ("choice_2", "组合B", False),
    ("choice_3", "组合C", False),
    ("choice_4", "组合D", False),
    ("level_path", "所属层级", False),
    ("textbook_version", "教材版本", False),
    ("source", "来源", False),
    ("difficulty_score", "难度评级", False),
    ("knowledge_points", "考察知识点", False),
    ("abilities", "考察能力", False),
    ("created_at", "入库时间", False),
    ("updated_at", "最后修改时间", False),
]


class _ZoomableTableWidget(CopyableTableWidget):
    def __init__(self, *args, zoom_callback=None, **kwargs) -> None:
        super().__init__(*args, **kwargs)
        self._zoom_callback = zoom_callback

    def wheelEvent(self, event: QWheelEvent) -> None:
        if event.modifiers() & Qt.KeyboardModifier.ControlModifier:
            if self._zoom_callback is not None:
                delta = event.angleDelta().y()
                if delta != 0:
                    self._zoom_callback(1 if delta > 0 else -1)
                    event.accept()
                    return
        super().wheelEvent(event)


class WelcomePage(QWizardPage):
    def __init__(self, state: WizardState) -> None:
        super().__init__()
        self._state = state
        self._db_path = self._current_db_path()
        self._import_flow_windows: list[QWizard] = []
        self._current_db_records: list[DbQuestionRecord] = []
        self._row_resize_pending = False
        self._row_resize_followup_pending = False
        self._column_defs = TABLE_COLUMNS
        self._column_visibility = load_column_visibility(self._column_defs)
        self._table_font_point_size = load_table_font_point_size(
```

```python
            self,
            min_size=TABLE_FONT_POINT_SIZE_MIN,
            max_size=TABLE_FONT_POINT_SIZE_MAX,
        )
        loaded_expanded_prefixes = load_welcome_tree_expanded_prefixes()
        self._expanded_level_prefixes = set(loaded_expanded_prefixes or [])
        self._has_saved_tree_state = loaded_expanded_prefixes is not None
        self._syncing_tree_expand_state = False
        self.setTitle("开始")

        menu_bundle = build_welcome_menu_bar(
            parent=self,
            column_defs=self._column_defs,
            column_visibility=self._column_visibility,
            on_doc_import=self._enter_main_flow,
            on_table_import=self._import_from_table_file,
            on_export_markdown=self._export_current_level_to_markdown,
            on_export_pdf=self._export_current_level_to_pdf,
            on_export_current_xlsx=self._export_current_level_to_xlsx,
            on_export_db_xlsx=self._export_db_table_to_xlsx,
            on_add_question=self._add_question_manually,
            on_open_general_settings=self._open_general_settings,
            on_open_import_settings=self._open_import_settings,
            on_open_export_settings=self._open_export_settings,
            on_open_api_config=self._open_api_cfg,
            on_toggle_column=self._set_column_visible,
        )
        menu_bar = menu_bundle.menu_bar
        self._doc_import_action = menu_bundle.doc_import_action
        self._table_import_action = menu_bundle.table_import_action
        self._export_md_action = menu_bundle.export_md_action
        self._export_pdf_action = menu_bundle.export_pdf_action
        self._export_current_level_xlsx_action = menu_bundle.export_current_level_xlsx_action
        self._export_db_table_xlsx_action = menu_bundle.export_db_table_xlsx_action
        self._add_question_action = menu_bundle.add_question_action
        self._general_settings_action = menu_bundle.general_settings_action
        self._import_settings_action = menu_bundle.import_settings_action
        self._export_settings_action = menu_bundle.export_settings_action
        self._api_settings_action = menu_bundle.api_settings_action

        self._folder_tree = QTreeWidget()
        self._folder_tree.setHeaderHidden(True)
        self._folder_tree.setAlternatingRowColors(True)
        self._folder_tree.setContextMenuPolicy(Qt.ContextMenuPolicy.CustomContextMenu)
        self._folder_tree.currentItemChanged.connect(self._on_level_selection_changed)
        self._folder_tree.customContextMenuRequested.connect(self._show_folder_tree_context_menu)
        self._folder_tree.itemExpanded.connect(self._on_tree_item_expanded)
        self._folder_tree.itemCollapsed.connect(self._on_tree_item_collapsed)

        left_layout = QVBoxLayout()
        left_layout.setContentsMargins(0, 0, 0, 0)
        left_layout.addWidget(self._folder_tree, 1)
        left_panel = QWidget()
        left_panel.setStyleSheet(rounded_panel_stylesheet(background="#ffffff"))
        left_panel.setLayout(left_layout)

        self._table = _ZoomableTableWidget(0, len(self._column_defs), zoom_callback=self._adjust_table_font_size)
        self._table.setHorizontalHeaderLabels([title for _key, title, _visible in self._column_defs])
        self._table.setEditTriggers(QAbstractItemView.EditTrigger.NoEditTriggers)
        self._table.setSelectionMode(QAbstractItemView.SelectionMode.NoSelection)
        self._table.setFocusPolicy(Qt.FocusPolicy.NoFocus)
        self._table.verticalHeader().setVisible(False)
        self._table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Interactive)
        self._table.horizontalHeader().setStretchLastSection(True)
        self._table.setWordWrap(True)
        self._table.setStyleSheet(
            "QTableWidget {"
            "border: 1px solid #000000; border-radius: 0px; background: #ffffff; gridline-color: #000000; outline: none;"
```

```
            "}"
            "QTableWidget > QWidget {"
            "background: #ffffff; border: none;"
            "}"
            "QHeaderView {"
            "background: transparent; border: none;"
            "}"
            "QHeaderView::section {"
            "background: #ffffff; border: none; border-right: 1px solid #000000; border-bottom: 1px solid #000000;"
            "padding: 4px 6px; font-weight: 600;"
            "}"
        )
        self._table.horizontalHeader().setDefaultAlignment(Qt.AlignmentFlag.AlignCenter)
        self._table.horizontalHeader().sectionResized.connect(self._schedule_table_row_resize)
        self._table.cellDoubleClicked.connect(self._on_table_cell_double_clicked)
        self._column_actions = menu_bundle.column_actions
        for idx, (key, _title, _visible) in enumerate(self._column_defs):
            visible = self._column_visibility.get(key, False)
            self._table.setColumnHidden(idx, not visible)
        rebalance_visible_columns(self._table, last_column_min_width=LAST_COLUMN_MIN_WIDTH)

        self._table_placeholder = QLabel("请先从左侧层级中选择，以显示题目信息")
        self._table_placeholder.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self._table_placeholder.setWordWrap(True)
        self._apply_table_font_size()

        right_layout = QVBoxLayout()
        right_layout.setContentsMargins(0, 0, 0, 0)
        self._table_stack = QStackedLayout()
        self._table_stack.addWidget(self._table_placeholder)
        self._table_stack.addWidget(self._table)
        right_layout.addLayout(self._table_stack, 1)
        right_panel = QWidget()
        right_panel.setStyleSheet(rounded_panel_stylesheet(background="#ffffff"))
        right_panel.setLayout(right_layout)

        self._content_splitter = QSplitter(Qt.Orientation.Horizontal)
        self._content_splitter.addWidget(left_panel)
        self._content_splitter.addWidget(right_panel)
        self._content_splitter.setChildrenCollapsible(False)
        self._content_splitter.setHandleWidth(6)
        self._content_splitter.setStyleSheet("QSplitter::handle { background-color: transparent; }")
        self._content_splitter.setStretchFactor(0, 1)
        self._content_splitter.setStretchFactor(1, 4)
        self._content_splitter.setSizes([180, 720])
        self._content_splitter.splitterMoved.connect(self._schedule_table_row_resize)

        layout = QVBoxLayout()
        layout.setMenuBar(menu_bar)
        layout.addWidget(self._content_splitter, 1)
        self.setLayout(layout)
        self._set_table_placeholder_visible(True)

    def initializePage(self) -> None:
        self._db_path = self._current_db_path()
        self._refresh_level_tree()
        if self._folder_tree.currentItem() is None:
            self._current_db_records = []
            self._table.setRowCount(0)
            self._schedule_table_row_resize()
            self._set_table_placeholder_visible(True)

    def _enter_main_flow(self) -> None:
        self._state.start_mode = "wizard"
        files, _ = QFileDialog.getOpenFileNames(
            self,
            "选择资料文件",
            effective_import_source_dir(self._state.import_source_dir_text),
            "Word (*.docx);;All Files (*)",
```

```python
        )
        if not files:
            return
        self._open_import_flow_windows([Path(value) for value in files], message_parent=self)

    def _open_import_flow_windows(self, paths: list[Path], *, message_parent: QWidget | None = None) -> bool:
        return open_import_flow_windows(
            base_state=self._state,
            db_path=self._db_path,
            paths=paths,
            message_parent=message_parent or self,
            launcher=self,
            owner=self.window(),
            import_flow_windows=self._import_flow_windows,
            start_page_id=PAGE_AI_SELECT,
            on_state_finished=self._handle_import_flow_window_finished,
            on_window_forget=self._forget_import_flow_window,
        )

    def _open_import_flow_states(self, states: list[ImportWizardSession], *, start_page_id: int) -> bool:
        return open_import_flow_states(
            launcher=self,
            owner=self.window(),
            import_flow_windows=self._import_flow_windows,
            states=states,
            start_page_id=start_page_id,
            on_state_finished=self._handle_import_flow_window_finished,
            on_window_forget=self._forget_import_flow_window,
        )

    def _handle_import_flow_window_finished(self, state: ImportWizardSession) -> None:
        if not state.execution.db_import_completed:
            return
        self._state.last_export_dir = state.last_export_dir
        self._refresh_level_tree(preferred_level_path=state.source.import_level_path)

    def _open_split_import_flow_windows(self, states: list[ImportWizardSession], *, start_page_id: int) -> bool:
        return self._open_import_flow_states(states, start_page_id=start_page_id)

    def _forget_import_flow_window(self, wizard: QWizard) -> None:
        self._import_flow_windows = [item for item in self._import_flow_windows if item is not wizard]

    def _show_import_flow_window(self, wizard: QWizard, *, activate: bool) -> None:
        show_import_flow_window(self._import_flow_windows, wizard, activate=activate)

    def _open_api_cfg(self) -> None:
        open_api_config(self, self._state)

    def _open_program_settings(self) -> None:
        if open_program_settings(self, self._state, section=SECTION_GENERAL):
            self._db_path = self._current_db_path()
            self._refresh_level_tree()

    def _open_general_settings(self) -> None:
        self._open_program_settings()

    def _open_import_settings(self) -> None:
        open_program_settings(self, self._state, section=SECTION_IMPORT)

    def _open_export_settings(self) -> None:
        open_program_settings(self, self._state, section=SECTION_EXPORT)

    def _import_from_table_file(self) -> None:
        result = import_from_table_file(self, self._db_path)
        if result is None:
            return
        self._refresh_level_tree(preferred_level_path=result.preferred_level_path)

    def _export_current_level_to_markdown(self) -> None:
```

```python
        md_path = export_current_level_markdown(
            parent=self,
            current_item=self._folder_tree.currentItem(),
            current_records=self._current_db_records,
            db_path=self._db_path,
            state=self._state,
            role_level_path=TREE_ROLE_LEVEL_PATH,
        )
        if md_path is None:
            return
        self._state.last_export_dir = md_path.parent
        show_message_box(self, title="导出完成", text=f"已导出 Markdown：\n{md_path}", icon=QMessageBox.Icon.Information)

    def _export_current_level_to_pdf(self) -> None:
        target_path = export_current_level_pdf(
            parent=self,
            current_item=self._folder_tree.currentItem(),
            current_records=self._current_db_records,
            db_path=self._db_path,
            state=self._state,
            role_level_path=TREE_ROLE_LEVEL_PATH,
        )
        if target_path is None:
            return
        self._state.last_export_dir = target_path.parent
        show_message_box(self, title="导出完成", text=f"已导出 PDF：\n{target_path}", icon=QMessageBox.Icon.Information)

    def _export_current_level_to_xlsx(self) -> None:
        target_path = export_current_level_xlsx(
            parent=self,
            current_item=self._folder_tree.currentItem(),
            current_records=self._current_db_records,
            db_path=self._db_path,
            state=self._state,
            role_level_path=TREE_ROLE_LEVEL_PATH,
        )
        if target_path is None:
            return
        self._state.last_export_dir = target_path.parent
        show_message_box(self, title="导出完成", text=f"已导出当前页面 xlsx：\n{target_path}", icon=QMessageBox.Icon.Information)

    def _export_db_table_to_xlsx(self) -> None:
        target_path = export_db_table_xlsx(parent=self, db_path=self._db_path, last_export_dir=self._state.last_export_dir)
        if target_path is None:
            return
        self._state.last_export_dir = target_path.parent
        show_message_box(self, title="导出完成", text=f"已导出整体数据库表 xlsx：\n{target_path}", icon=QMessageBox.Icon.Information)

    def _selected_tree_level_path_for_create(self) -> str:
        return selected_tree_level_path_for_create(
            self._folder_tree.currentItem(),
            role_level_path=TREE_ROLE_LEVEL_PATH,
            role_level_prefix=TREE_ROLE_LEVEL_PREFIX,
        )

    def _add_question_manually(self, *, default_level_path: str | None = None) -> None:
        if default_level_path is None:
            default_level_path = self._selected_tree_level_path_for_create()
        created = add_question_manually(
            parent=self,
            db_path=self._db_path,
            preferred_textbook_version=self._state.preferred_textbook_version,
            default_level_path=str(default_level_path or "").strip(),
        )
        if created is None:
```

```python
            return
        self._refresh_level_tree(preferred_level_path=created.level_path)

    def _refresh_level_tree(self, preferred_level_path: str | None = None) -> None:
        self._syncing_tree_expand_state = True
        self._expanded_level_prefixes, has_tree = refresh_level_tree(
            tree_widget=self._folder_tree,
            db_path=self._db_path,
            expanded_level_prefixes=self._expanded_level_prefixes,
            preferred_level_path=preferred_level_path,
            role_level_prefix=TREE_ROLE_LEVEL_PREFIX,
            role_level_depth=TREE_ROLE_LEVEL_DEPTH,
            role_level_path=TREE_ROLE_LEVEL_PATH,
            on_no_selection=self._clear_current_table_selection,
        )
        self._syncing_tree_expand_state = False
        if not has_tree:
            self._clear_current_table_selection()
        self._sync_expanded_level_prefixes_from_tree()

    def _on_level_selection_changed(self, current: QTreeWidgetItem | None, _previous: QTreeWidgetItem | None) -> None:
        self._current_db_records = handle_level_selection_changed(
            current=current,
            db_path=self._db_path,
            role_level_path=TREE_ROLE_LEVEL_PATH,
            on_empty_selection=self._clear_current_table_selection,
            on_records_loaded=self._populate_db_records_table,
        )

    def _show_folder_tree_context_menu(self, pos) -> None:
        show_folder_tree_context_menu(
            parent=self,
            tree_widget=self._folder_tree,
            pos=pos,
            role_level_prefix=TREE_ROLE_LEVEL_PREFIX,
            role_level_depth=TREE_ROLE_LEVEL_DEPTH,
            role_level_path=TREE_ROLE_LEVEL_PATH,
            on_add_question=lambda default_level_path: self._add_question_manually(default_level_path=default_level_path),
            on_delete_level_subtree=lambda level_prefix, depth, display_label: self._delete_level_subtree(
                level_prefix=level_prefix,
                depth=depth,
                display_label=display_label,
            ),
        )

    def _on_tree_item_expanded(self, _item: QTreeWidgetItem) -> None:
        if self._syncing_tree_expand_state:
            return
        self._sync_expanded_level_prefixes_from_tree()

    def _on_tree_item_collapsed(self, _item: QTreeWidgetItem) -> None:
        if self._syncing_tree_expand_state:
            return
        self._sync_expanded_level_prefixes_from_tree()

    def _delete_level_subtree(self, *, level_prefix: str, depth: int, display_label: str) -> None:
        delete_level_subtree(
            parent=self,
            db_path=self._db_path,
            level_prefix=level_prefix,
            depth=depth,
            display_label=display_label,
            on_deleted=self._clear_table_and_refresh_tree,
            on_no_data=self._refresh_level_tree,
        )

    def _sync_expanded_level_prefixes_from_tree(self) -> None:
        self._expanded_level_prefixes = sync_expanded_level_prefixes_from_tree(
```

```python
            self._folder_tree,
            role_level_prefix=TREE_ROLE_LEVEL_PREFIX,
        )
        self._has_saved_tree_state = True

    def _populate_db_records_table(self, records: list[DbQuestionRecord]) -> None:
        populate_db_records_table(self._table, records, self._column_defs)
        self._schedule_table_row_resize()
        self._set_table_placeholder_visible(False)

    def _set_table_placeholder_visible(self, visible: bool) -> None:
        self._table_stack.setCurrentWidget(self._table_placeholder if visible else self._table)

    def _clear_current_table_selection(self) -> None:
        self._table.setRowCount(0)
        self._current_db_records = []
        self._schedule_table_row_resize()
        self._set_table_placeholder_visible(True)

    def _clear_table_and_refresh_tree(self) -> None:
        self._clear_current_table_selection()
        self._refresh_level_tree()

    def _current_db_path(self) -> Path:
        return library_db_path_from_repo_parent_dir_text(self._state.default_repo_parent_dir_text)

    def _set_column_visible(self, column: int, visible: bool) -> None:
        set_column_visible(
            table=self._table,
            column_actions=self._column_actions,
            column_defs=self._column_defs,
            column=column,
            visible=visible,
            last_column_min_width=LAST_COLUMN_MIN_WIDTH,
            schedule_table_row_resize=self._schedule_table_row_resize,
        )

    def _schedule_table_row_resize(self, *_args) -> None:
        self._row_resize_pending = schedule_table_row_resize(
            row_resize_pending=self._row_resize_pending,
            apply_cb=self._apply_table_row_resize,
        )

    def _apply_table_row_resize(self) -> None:
        self._row_resize_pending = False
        self._row_resize_followup_pending = apply_table_row_resize(
            table=self._table,
            row_resize_followup_pending=self._row_resize_followup_pending,
            followup_cb=self._apply_table_row_resize_followup,
        )

    def _apply_table_row_resize_followup(self) -> None:
        self._row_resize_followup_pending = False
        apply_table_row_resize_followup(self._table)

    def _refresh_table_layout_after_column_change(self) -> None:
        refresh_table_layout_after_column_change(
            table=self._table,
            last_column_min_width=LAST_COLUMN_MIN_WIDTH,
            schedule_table_row_resize=self._schedule_table_row_resize,
        )

    def _apply_table_font_size(self) -> None:
        apply_table_font_size(
            table=self._table,
            table_placeholder=self._table_placeholder,
            font_point_size=self._table_font_point_size,
            schedule_table_row_resize=self._schedule_table_row_resize,
        )

```

```python
    def _adjust_table_font_size(self, step: int) -> None:
        new_size = adjust_table_font_size(
            self._table_font_point_size,
            step,
            min_size=TABLE_FONT_POINT_SIZE_MIN,
            max_size=TABLE_FONT_POINT_SIZE_MAX,
        )
        if new_size == self._table_font_point_size:
            return
        self._table_font_point_size = new_size
        self._apply_table_font_size()
        persist_table_font_point_size(self._table_font_point_size)

    def _on_table_cell_double_clicked(self, row: int, _column: int) -> None:
        if row < 0 or row >= len(self._current_db_records):
            return
        result = edit_question_record(
            parent=self,
            db_path=self._db_path,
            record=self._current_db_records[row],
        )
        if result.action in {"missing", "deleted"}:
            self._clear_current_table_selection()
            self._refresh_level_tree(preferred_level_path=result.preferred_level_path)
            return
        if result.action != "updated" or result.updated_record is None:
            return
        self._current_db_records[row] = result.updated_record
        self._refresh_level_tree(preferred_level_path=result.preferred_level_path)

__all__ = ["WelcomePage"]
```

```python
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

from PyQt6.QtWidgets import QFileDialog, QMessageBox, QWidget

from sj_generator.application.state import WizardState
from sj_generator.infrastructure.persistence.excel_repo import load_db_question_records
from sj_generator.infrastructure.persistence.sqlite_repo import append_questions
from sj_generator.presentation.qt.api_config_dialog import ApiConfigDialog
from sj_generator.presentation.qt.message_box import show_message_box
from sj_generator.presentation.qt.program_settings_dialog import ProgramSettingsDialog


def open_api_config(parent: QWidget, state: WizardState) -> None:
    dlg = ApiConfigDialog(parent, state=state)
    dlg.exec()


def open_program_settings(parent: QWidget, state: WizardState, *, section: str) -> bool:
    dlg = ProgramSettingsDialog(state, parent, section=section)
    return bool(dlg.exec())


@dataclass
class TableImportResult:
    count: int
    preferred_level_path: str


def import_from_table_file(parent: QWidget, db_path: Path) -> TableImportResult | None:
    file_path, _ = QFileDialog.getOpenFileName(parent, "选择表格文件", "", "Excel (*.xlsx);;All Files (*)")
    if not file_path:
        return None
    path = Path(file_path)
    try:
        records = load_db_question_records(path)
    except Exception as exc:
        show_message_box(parent, title="导入失败", text=f"读取表格文件失败：{exc}", icon=QMessageBox.Icon.Critical)
        return None
    if not records:
        show_message_box(parent, title="无法导入", text="当前 xlsx 中没有可写入数据库的记录。", icon=QMessageBox.Icon.Warning)
        return None
    try:
        append_questions(db_path, records)
    except Exception as exc:
        show_message_box(parent, title="导入失败", text=f"写入数据库失败：{exc}", icon=QMessageBox.Icon.Critical)
        return None
    preferred_level_path = next((record.level_path for record in records if record.level_path.strip()), "")
    show_message_box(parent, title="导入完成", text=f"已从数据库字段表 xlsx 导入 {len(records)} 道题。", icon=QMessageBox.Icon.Information)
    return TableImportResult(count=len(records), preferred_level_path=preferred_level_path)

```

```python
from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime
from uuid import uuid4

from PyQt6.QtWidgets import QMessageBox, QTreeWidgetItem, QWidget

from sj_generator.infrastructure.persistence.sqlite_repo import (
    DbQuestionRecord,
    append_questions,
    delete_question_by_id,
    update_question,
)
from sj_generator.presentation.qt.message_box import show_message_box
from sj_generator.presentation.qt.question_edit_dialog import QuestionEditDialog


def selected_tree_level_path_for_create(
    current: QTreeWidgetItem | None,
    *,
    role_level_path: int,
    role_level_prefix: int,
) -> str:
    if current is None:
        return ""
    exact_level_path = str(current.data(0, role_level_path) or "").strip()
    if exact_level_path:
        return exact_level_path
    return str(current.data(0, role_level_prefix) or "").strip()


def build_new_question_record(*, preferred_textbook_version: str, default_level_path: str) -> DbQuestionRecord:
    now_text = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    return DbQuestionRecord(
        id=str(uuid4()),
        stem="",
        option_1="",
        option_2="",
        option_3="",
        option_4="",
        choice_1="",
        choice_2="",
        choice_3="",
        choice_4="",
        answer="",
        analysis="",
        question_type="单选",
        textbook_version=preferred_textbook_version,
        source="录入",
        level_path=default_level_path,
        difficulty_score=None,
        knowledge_points="",
        abilities="",
        created_at=now_text,
        updated_at=now_text,
    )


def add_question_manually(
    *,
    parent: QWidget,
    db_path,
    preferred_textbook_version: str,
    default_level_path: str,
) -> DbQuestionRecord | None:
    new_record = build_new_question_record(
        preferred_textbook_version=preferred_textbook_version,
        default_level_path=str(default_level_path or "").strip(),
```

```python
    )
    dlg = QuestionEditDialog(new_record, parent, create_mode=True)
    if dlg.exec() != QuestionEditDialog.DialogCode.Accepted:
        return None
    created = dlg.updated_record()
    if not created.stem.strip():
        show_message_box(parent, title="无法新增", text="题目内容不能为空。", icon=QMessageBox.Icon.Warning)
        return None
    try:
        append_questions(db_path, [created])
    except Exception as exc:
        show_message_box(parent, title="新增失败", text=f"写入题目失败：{exc}", icon=QMessageBox.Icon.Critical)
        return None
    show_message_box(parent, title="新增完成", text="题目已新增。", icon=QMessageBox.Icon.Information)
    return created


@dataclass
class QuestionEditResult:
    action: str
    preferred_level_path: str = ""
    updated_record: DbQuestionRecord | None = None


def edit_question_record(
    *,
    parent: QWidget,
    db_path,
    record: DbQuestionRecord,
) -> QuestionEditResult:
    dlg = QuestionEditDialog(record, parent)
    result = dlg.exec()
    if result == QuestionEditDialog.DELETE_RESULT:
        deleted_count = delete_question_by_id(db_path, record.id)
        if deleted_count <= 0:
            show_message_box(parent, title="删除失败", text="未找到要删除的题目，可能已被移除。", icon=QMessageBox.Icon.Warning)
            return QuestionEditResult(action="missing")
        show_message_box(parent, title="删除完成", text="题目已删除。", icon=QMessageBox.Icon.Information)
        return QuestionEditResult(action="deleted", preferred_level_path=record.level_path)
    if result != QuestionEditDialog.DialogCode.Accepted:
        return QuestionEditResult(action="cancelled")
    updated = dlg.updated_record()
    try:
        update_question(db_path, updated)
    except Exception as exc:
        show_message_box(parent, title="保存失败", text=f"更新题目失败：{exc}", icon=QMessageBox.Icon.Critical)
        return QuestionEditResult(action="failed")
    return QuestionEditResult(action="updated", preferred_level_path=updated.level_path, updated_record=updated)

```

sj_generator/presentation/qt/pages/welcome_table.py

082-01

```python
from __future__ import annotations

from PyQt6.QtGui import QColor
from PyQt6.QtWidgets import QTableWidget, QTableWidgetItem

from sj_generator.domain.entities import Question
from sj_generator.infrastructure.persistence.sqlite_repo import DbQuestionRecord
from sj_generator.presentation.qt.pages.welcome_export import format_db_answer, format_db_options


def populate_questions_table(
    table: QTableWidget,
    questions: list[Question],
    column_defs: list[tuple[str, str, bool]],
) -> None:
    table.setRowCount(len(questions))
    for row, question in enumerate(questions):
        values = build_question_row_values(row + 1, question)
        populate_row_by_values(table, row, values, column_defs)


def populate_db_records_table(
    table: QTableWidget,
    records: list[DbQuestionRecord],
    column_defs: list[tuple[str, str, bool]],
) -> None:
    table.setRowCount(len(records))
    for row, record in enumerate(records):
        values = build_db_row_values(row + 1, record)
        populate_row_by_values(table, row, values, column_defs)


def build_question_row_values(sequence: int, question: Question) -> dict[str, str]:
    return {
        "stem": format_stem_with_sequence(sequence, question.stem),
        "options": question.options,
        "answer": question.answer,
        "analysis": format_table_analysis(question.analysis),
        "id": "",
        "question_type": question.question_type,
        "choice_1": question.choice_1,
        "choice_2": question.choice_2,
        "choice_3": question.choice_3,
        "choice_4": question.choice_4,
        "textbook_version": "",
        "source": "",
        "level_path": "",
        "difficulty_score": "",
        "knowledge_points": "",
        "abilities": "",
        "created_at": "",
        "updated_at": "",
    }


def build_db_row_values(sequence: int, record: DbQuestionRecord) -> dict[str, str]:
    return {
        "stem": format_stem_with_sequence(sequence, record.stem),
        "options": format_db_options(record),
        "answer": format_db_answer(record),
        "analysis": format_table_analysis(record.analysis),
        "id": record.id,
        "question_type": record.question_type,
        "choice_1": record.choice_1,
        "choice_2": record.choice_2,
        "choice_3": record.choice_3,
        "choice_4": record.choice_4,
        "textbook_version": record.textbook_version,
        "source": record.source,
        "level_path": record.level_path,
```

```python
        "difficulty_score": "" if record.difficulty_score is None else str(record.difficulty_score),
        "knowledge_points": record.knowledge_points,
        "abilities": record.abilities,
        "created_at": record.created_at,
        "updated_at": record.updated_at,
    }


def populate_row_by_values(
    table: QTableWidget,
    row: int,
    values: dict[str, str],
    column_defs: list[tuple[str, str, bool]],
) -> None:
    highlight_missing = any(
        not str(values.get(key, "") or "").strip() for key in ("stem", "options", "answer", "analysis")
    )
    for col, (key, _title, _visible) in enumerate(column_defs):
        alignment = int(QtAlign.center if key == "answer" else QtAlign.left)
        set_table_item(
            table,
            row,
            col,
            values.get(key, ""),
            alignment=alignment,
            background=QColor("#ffd9d9") if highlight_missing else None,
        )


class QtAlign:
    left = 1 | 128
    center = 4 | 128


def format_stem_with_sequence(sequence: int, stem: str) -> str:
    return f"{sequence}. {stem or ''}".strip()


def format_table_analysis(analysis: str) -> str:
    return (analysis or "").replace("**", "")


def set_table_item(
    table: QTableWidget,
    row: int,
    col: int,
    text: str,
    *,
    alignment: int,
    background: QColor | None = None,
) -> None:
    item = QTableWidgetItem(text or "")
    item.setTextAlignment(alignment)
    if background is not None:
        item.setBackground(background)
    table.setItem(row, col, item)
```

sj_generator/presentation/qt/pages/welcome_table_view.py

083-01

```python
from __future__ import annotations

from PyQt6.QtCore import QTimer
from PyQt6.QtGui import QAction
from PyQt6.QtWidgets import QTableWidget, QWidget

from sj_generator.application.settings import (
    load_welcome_table_column_visibility,
    load_welcome_table_font_point_size,
    save_welcome_table_column_visibility,
    save_welcome_table_font_point_size,
)


def load_column_visibility(column_defs: list[tuple[str, str, bool]]) -> dict[str, bool]:
    saved = load_welcome_table_column_visibility()
    defaults = {key: visible for key, _title, visible in column_defs}
    merged = defaults | {key: value for key, value in saved.items() if key in defaults}
    if not any(merged.values()):
        first_key = column_defs[0][0]
        merged[first_key] = True
    return merged


def save_column_visibility(table: QTableWidget, column_defs: list[tuple[str, str, bool]]) -> None:
    visibility = {
        key: (not table.isColumnHidden(idx))
        for idx, (key, _title, _visible) in enumerate(column_defs)
    }
    save_welcome_table_column_visibility(visibility)


def set_column_visible(
    *,
    table: QTableWidget,
    column_actions: dict[int, QAction | object],
    column_defs: list[tuple[str, str, bool]],
    column: int,
    visible: bool,
    last_column_min_width: int,
    schedule_table_row_resize,
) -> None:
    if visible:
        table.setColumnHidden(column, False)
        save_column_visibility(table, column_defs)
        refresh_table_layout_after_column_change(
            table=table,
            last_column_min_width=last_column_min_width,
            schedule_table_row_resize=schedule_table_row_resize,
        )
        return

    visible_columns = [idx for idx in range(table.columnCount()) if not table.isColumnHidden(idx)]
    if len(visible_columns) <= 1:
        action = column_actions.get(column)
        if action is not None:
            action.blockSignals(True)
            action.setChecked(True)
            action.blockSignals(False)
        return

    table.setColumnHidden(column, True)
    save_column_visibility(table, column_defs)
    refresh_table_layout_after_column_change(
        table=table,
        last_column_min_width=last_column_min_width,
        schedule_table_row_resize=schedule_table_row_resize,
    )


```

```python
def schedule_table_row_resize(*, row_resize_pending: bool, apply_cb) -> bool:
    if row_resize_pending:
        return row_resize_pending
    QTimer.singleShot(0, apply_cb)
    return True


def apply_table_row_resize(
    *,
    table: QTableWidget,
    row_resize_followup_pending: bool,
    followup_cb,
) -> bool:
    if table.rowCount() == 0:
        return row_resize_followup_pending
    table.doItemsLayout()
    table.resizeRowsToContents()
    if row_resize_followup_pending:
        return row_resize_followup_pending
    QTimer.singleShot(30, followup_cb)
    return True


def apply_table_row_resize_followup(table: QTableWidget) -> None:
    if table.rowCount() == 0:
        return
    table.doItemsLayout()
    table.viewport().update()
    table.resizeRowsToContents()


def refresh_table_layout_after_column_change(
    *,
    table: QTableWidget,
    last_column_min_width: int,
    schedule_table_row_resize,
) -> None:
    rebalance_visible_columns(table, last_column_min_width=last_column_min_width)
    table.doItemsLayout()
    table.viewport().update()
    schedule_table_row_resize()


def rebalance_visible_columns(table: QTableWidget, *, last_column_min_width: int) -> None:
    visible_columns = [idx for idx in range(table.columnCount()) if not table.isColumnHidden(idx)]
    if not visible_columns:
        return
    available_width = max(1, table.viewport().width())
    if len(visible_columns) == 1:
        table.setColumnWidth(visible_columns[0], available_width)
        return
    fixed_columns = visible_columns[:-1]
    reserved_last_width = min(available_width, last_column_min_width)
    distributable_width = max(len(fixed_columns), available_width - reserved_last_width)
    base_width = max(1, distributable_width // len(fixed_columns))
    consumed_width = 0
    for column in fixed_columns:
        table.setColumnWidth(column, base_width)
        consumed_width += base_width
    table.setColumnWidth(visible_columns[-1], max(1, available_width - consumed_width))


def load_table_font_point_size(widget: QWidget, *, min_size: int, max_size: int) -> int:
    saved = load_welcome_table_font_point_size()
    default_size = widget.font().pointSize()
    if default_size <= 0:
        default_size = 10
    if saved is None:
        return default_size
    return max(min_size, min(max_size, saved))
```

```python


def apply_table_font_size(
    *,
    table: QTableWidget,
    table_placeholder: QWidget,
    font_point_size: int,
    schedule_table_row_resize,
) -> None:
    font = table.font()
    font.setPointSize(font_point_size)
    table.setFont(font)
    table.horizontalHeader().setFont(font)
    table_placeholder.setFont(font)
    table.doItemsLayout()
    table.viewport().update()
    table.horizontalHeader().viewport().update()
    schedule_table_row_resize()


def adjust_table_font_size(current_size: int, step: int, *, min_size: int, max_size: int) -> int:
    return max(min_size, min(max_size, current_size + int(step)))


def persist_table_font_point_size(font_point_size: int) -> None:
    save_welcome_table_font_point_size(font_point_size)

```

```python
from __future__ import annotations

from pathlib import Path

from PyQt6.QtWidgets import QTreeWidget, QTreeWidgetItem

from sj_generator.infrastructure.persistence.sqlite_repo import DbQuestionRecord, load_all_questions


def delete_action_text(depth: int) -> str:
    if depth == 0:
        return "删除整本书"
    if depth == 1:
        return "删除这一课"
    if depth == 2:
        return "删除这一框"
    return "删除当前层级"


def delete_scope_text(depth: int) -> str:
    if depth == 0:
        return "整本书 "
    if depth == 1:
        return "这一课 "
    if depth == 2:
        return "这一框 "
    return "当前层级 "


def parse_level_parts(level_path: str) -> list[str]:
    return [part.strip() for part in str(level_path).split(".") if part.strip()]


def display_level_parts(level_path: str) -> list[str]:
    raw_parts = parse_level_parts(level_path)
    display_parts: list[str] = []
    for idx, part in enumerate(raw_parts):
        if idx == 0 and part == "0":
            return ["0"]
        if idx > 0 and part == "0":
            return display_parts
        display_parts.append(part)
    return display_parts


def tree_level_key_for_path(level_path: str | None) -> str:
    return ".".join(display_level_parts(level_path or ""))


def load_questions_for_tree_level(db_path: Path, level_key: str) -> list[DbQuestionRecord]:
    normalized_level_key = str(level_key or "").strip()
    if not normalized_level_key:
        return []
    return [
        record
        for record in load_all_questions(db_path)
        if tree_level_key_for_path(record.level_path) == normalized_level_key
    ]


def format_level_label(depth: int, part: str) -> str:
    if not part.isdigit():
        return part
    if depth == 0:
        if part == "0":
            return "高中阶段"
        return f"必修{to_chinese_number(int(part))}"
    if depth == 1:
        return f"第{to_chinese_number(int(part))}课"
    if depth == 2:
```

```python
        return f"第{to_chinese_number(int(part))}框"
    return part


def expand_item_ancestors(item: QTreeWidgetItem) -> None:
    current: QTreeWidgetItem | None = item
    while current is not None:
        current.setExpanded(True)
        current = current.parent()


def collect_expanded_level_prefixes(tree_widget: QTreeWidget, role_level_prefix: int) -> set[str]:
    expanded: set[str] = set()

    def visit(item: QTreeWidgetItem) -> None:
        prefix = str(item.data(0, role_level_prefix) or "").strip()
        if prefix and item.isExpanded():
            expanded.add(prefix)
        for index in range(item.childCount()):
            visit(item.child(index))

    for index in range(tree_widget.topLevelItemCount()):
        visit(tree_widget.topLevelItem(index))
    return expanded


def build_level_tree(
    *,
    tree_widget: QTreeWidget,
    level_paths: list[str],
    expanded_prefixes: set[str],
    current_prefix: str,
    preferred_level_key: str,
    role_level_prefix: int,
    role_level_depth: int,
    role_level_path: int,
) -> tuple[QTreeWidgetItem | None, QTreeWidgetItem | None]:
    item_map: dict[tuple[str, ...], QTreeWidgetItem] = {}
    preferred_item: QTreeWidgetItem | None = None
    selected_prefix_item: QTreeWidgetItem | None = None

    for level_path in level_paths:
        parts = display_level_parts(level_path)
        if not parts:
            continue
        parent_item: QTreeWidgetItem | None = None
        path_parts: list[str] = []
        for idx, part in enumerate(parts):
            path_parts.append(part)
            prefix_path = ".".join(path_parts)
            key = tuple(path_parts)
            item = item_map.get(key)
            if item is None:
                item = QTreeWidgetItem([format_level_label(idx, part)])
                item.setToolTip(0, prefix_path)
                item.setData(0, role_level_prefix, prefix_path)
                item.setData(0, role_level_depth, idx)
                item.setData(0, role_level_path, prefix_path)
                if parent_item is None:
                    tree_widget.addTopLevelItem(item)
                else:
                    parent_item.addChild(item)
                item_map[key] = item
            if prefix_path in expanded_prefixes:
                item.setExpanded(True)
            if current_prefix and prefix_path == current_prefix:
                selected_prefix_item = item
            if idx == len(parts) - 1 and preferred_level_key and prefix_path == preferred_level_key:
                preferred_item = item
            parent_item = item
```

```python
    return preferred_item, selected_prefix_item


def to_chinese_number(value: int) -> str:
    digits = {
        0: "零",
        1: "一",
        2: "二",
        3: "三",
        4: "四",
        5: "五",
        6: "六",
        7: "七",
        8: "八",
        9: "九",
        10: "十",
    }
    if value <= 10:
        return digits[value]
    if value < 20:
        return "十" + digits[value - 10]
    tens, ones = divmod(value, 10)
    if ones == 0:
        return digits[tens] + "十"
    return digits[tens] + "十" + digits[ones]

```

```python
from __future__ import annotations

from pathlib import Path
from typing import Callable

from PyQt6.QtWidgets import QMenu, QMessageBox, QTreeWidget, QTreeWidgetItem, QWidget

from sj_generator.application.settings import save_welcome_tree_expanded_prefixes
from sj_generator.infrastructure.persistence.sqlite_repo import (
    DbQuestionRecord,
    count_questions_by_level_prefix,
    delete_questions_by_level_prefix,
    list_level_paths,
)
from sj_generator.presentation.qt.message_box import show_message_box
from sj_generator.presentation.qt.pages.welcome_tree import (
    build_level_tree,
    collect_expanded_level_prefixes,
    delete_action_text,
    delete_scope_text,
    expand_item_ancestors,
    load_questions_for_tree_level,
    tree_level_key_for_path,
)


def refresh_level_tree(
    *,
    tree_widget: QTreeWidget,
    db_path: Path,
    expanded_level_prefixes: set[str],
    preferred_level_path: str | None,
    role_level_prefix: int,
    role_level_depth: int,
    role_level_path: int,
    on_no_selection: Callable[[], None],
) -> tuple[set[str], bool]:
    current_item = tree_widget.currentItem()
    current_prefix = ""
    if current_item is not None:
        current_prefix = str(current_item.data(0, role_level_prefix) or "").strip()
    next_expanded = set(expanded_level_prefixes)
    if tree_widget.topLevelItemCount() > 0:
        next_expanded = collect_expanded_level_prefixes(tree_widget, role_level_prefix)
    tree_widget.clear()
    if not db_path.exists():
        return next_expanded, False

    preferred_item, selected_prefix_item = build_level_tree(
        tree_widget=tree_widget,
        level_paths=list_level_paths(db_path),
        expanded_prefixes=next_expanded,
        current_prefix=current_prefix,
        preferred_level_key=tree_level_key_for_path(preferred_level_path),
        role_level_prefix=role_level_prefix,
        role_level_depth=role_level_depth,
        role_level_path=role_level_path,
    )
    selected_item = preferred_item or selected_prefix_item
    if selected_item is not None:
        tree_widget.setCurrentItem(selected_item)
        if preferred_item is not None:
            expand_item_ancestors(selected_item)
    else:
        tree_widget.setCurrentItem(None)
        tree_widget.clearSelection()
        on_no_selection()
    return next_expanded, True


```

```python
def handle_level_selection_changed(
    *,
    current: QTreeWidgetItem | None,
    db_path: Path,
    role_level_path: int,
    on_empty_selection: Callable[[], None],
    on_records_loaded: Callable[[list[DbQuestionRecord]], None],
) -> list[DbQuestionRecord]:
    if current is None:
        on_empty_selection()
        return []
    level_key = str(current.data(0, role_level_path) or "").strip()
    if not level_key:
        on_empty_selection()
        return []
    records = load_questions_for_tree_level(db_path, level_key)
    on_records_loaded(records)
    return records


def show_folder_tree_context_menu(
    *,
    parent: QWidget,
    tree_widget: QTreeWidget,
    pos,
    role_level_prefix: int,
    role_level_depth: int,
    role_level_path: int,
    on_add_question: Callable[[str], None],
    on_delete_level_subtree: Callable[[str, int, str], None],
) -> None:
    item = tree_widget.itemAt(pos)
    if item is None:
        return
    level_prefix = str(item.data(0, role_level_prefix) or "").strip()
    if not level_prefix:
        return
    depth = int(item.data(0, role_level_depth) or 0)
    tree_widget.setCurrentItem(item)
    menu = QMenu(parent)
    add_action = menu.addAction("新增題目")
    delete_action = menu.addAction(delete_action_text(depth))
    chosen = menu.exec(tree_widget.viewport().mapToGlobal(pos))
    if chosen == add_action:
        on_add_question(str(item.data(0, role_level_path) or item.data(0, role_level_prefix) or "").strip())
        return
    if chosen == delete_action:
        on_delete_level_subtree(level_prefix, depth, item.text(0).strip() or level_prefix)


def sync_expanded_level_prefixes_from_tree(
    tree_widget: QTreeWidget,
    *,
    role_level_prefix: int,
) -> set[str]:
    expanded_level_prefixes = collect_expanded_level_prefixes(tree_widget, role_level_prefix)
    save_welcome_tree_expanded_prefixes(sorted(expanded_level_prefixes))
    return expanded_level_prefixes


def delete_level_subtree(
    *,
    parent: QWidget,
    db_path: Path,
    level_prefix: str,
    depth: int,
    display_label: str,
    on_deleted: Callable[[], None],
    on_no_data: Callable[[], None],
) -> int:
```

```python
    level_prefix = (level_prefix or "").strip()
    if not level_prefix:
        return 0
    total = count_questions_by_level_prefix(db_path, level_prefix)
    if total <= 0:
        show_message_box(parent, title="无需删除", text="当前节点下没有可删除的题目。", icon=QMessageBox.Icon.Information)
        on_no_data()
        return 0
    scope_text = delete_scope_text(depth)
    answer = show_message_box(
        parent,
        title="确认删除",
        text=f"确定删除{scope_text}{display_label}（层级 {level_prefix}）下的 {total} 道题吗？此操作不可撤销。",
        icon=QMessageBox.Icon.Warning,
        buttons=QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
        default_button=QMessageBox.StandardButton.No,
    )
    if answer != QMessageBox.StandardButton.Yes:
        return 0
    deleted_count = delete_questions_by_level_prefix(db_path, level_prefix)
    on_deleted()
    show_message_box(
        parent,
        title="删除完成",
        text=f"已删除{scope_text}{display_label}下的 {deleted_count} 道题。",
        icon=QMessageBox.Icon.Information,
    )
    return deleted_count
```

```python
from __future__ import annotations

from pathlib import Path

from PyQt6.QtWidgets import (
    QCheckBox,
    QComboBox,
    QDialog,
    QDialogButtonBox,
    QFileDialog,
    QFormLayout,
    QHBoxLayout,
    QLabel,
    QMessageBox,
    QPushButton,
    QLineEdit,
    QVBoxLayout,
)

from sj_generator.application.settings import save_program_settings_merged
from sj_generator.application.state import (
    WizardState,
    normalize_export_convertible_multi_mode,
    normalize_export_include_answers,
    normalize_export_include_analysis,
    normalize_default_repo_parent_dir_text,
    normalize_import_source_dir_text,
    normalize_analysis_model_name,
    normalize_analysis_provider,
    normalize_preferred_textbook_version,
    library_db_path_from_repo_parent_dir_text,
    default_import_source_dir,
    desktop_import_source_dir,
)
from sj_generator.infrastructure.persistence.sqlite_repo import load_all_questions
SECTION_GENERAL = "general"
SECTION_IMPORT = "import"
SECTION_EXPORT = "export"
BUTTON_MIN_WIDTH = 96
BUTTON_MIN_HEIGHT = 36


def _style_dialog_button(button: QPushButton | None, text: str | None = None) -> None:
    if button is None:
        return
    if text:
        button.setText(text)
    button.setMinimumSize(BUTTON_MIN_WIDTH, BUTTON_MIN_HEIGHT)


def _style_message_box_buttons(box: QMessageBox) -> None:
    for button_type, text in (
        (QMessageBox.StandardButton.Ok, "确定"),
        (QMessageBox.StandardButton.Cancel, "取消"),
        (QMessageBox.StandardButton.Yes, "是"),
        (QMessageBox.StandardButton.No, "否"),
    ):
        _style_dialog_button(box.button(button_type), text)


def _show_message_box(
    parent,
    *,
    title: str,
    text: str,
    icon: QMessageBox.Icon,
    buttons: QMessageBox.StandardButton = QMessageBox.StandardButton.Ok,
    default_button: QMessageBox.StandardButton = QMessageBox.StandardButton.NoButton,
) -> QMessageBox.StandardButton:
    box = QMessageBox(parent)
```

```python
    box.setWindowTitle(title)
    box.setText(text)
    box.setIcon(icon)
    box.setStandardButtons(buttons)
    if default_button != QMessageBox.StandardButton.NoButton:
        box.setDefaultButton(default_button)
    _style_message_box_buttons(box)
    return QMessageBox.StandardButton(box.exec())


class ProgramSettingsDialog(QDialog):
    def __init__(self, state: WizardState, parent=None, *, section: str = SECTION_GENERAL) -> None:
        super().__init__(parent)
        self._state = state
        self._section = section if section in {SECTION_GENERAL, SECTION_IMPORT, SECTION_EXPORT} else SECTION_GENERAL
        self.setWindowTitle(self._dialog_title())
        self.resize(500, 260)

        self._dedupe_checkbox = QCheckBox("导入流程默认执行库内查重")
        self._dedupe_checkbox.setChecked(bool(self._state.dedupe_enabled))

        self._import_analysis_combo = QComboBox()
        self._import_analysis_combo.addItem("自动生成", True)
        self._import_analysis_combo.addItem("不生成", False)
        analysis_idx = self._import_analysis_combo.findData(bool(self._state.analysis_enabled))
        if analysis_idx >= 0:
            self._import_analysis_combo.setCurrentIndex(analysis_idx)

        self._convertible_multi_export_combo = QComboBox()
        self._convertible_multi_export_combo.addItem("保留组合映射", "keep_combo")
        self._convertible_multi_export_combo.addItem("按新多选输出", "as_multi")
        export_idx = self._convertible_multi_export_combo.findData(
            normalize_export_convertible_multi_mode(self._state.export_convertible_multi_mode)
        )
        if export_idx >= 0:
            self._convertible_multi_export_combo.setCurrentIndex(export_idx)
        self._export_include_answers_checkbox = QCheckBox("包含答案")
        self._export_include_answers_checkbox.setChecked(
            normalize_export_include_answers(self._state.export_include_answers)
        )
        self._export_include_analysis_checkbox = QCheckBox("包含解析")
        self._export_include_analysis_checkbox.setChecked(
            normalize_export_include_analysis(self._state.export_include_analysis)
        )

        self._default_repo_parent_dir_edit = QLineEdit()
        self._default_repo_parent_dir_edit.setText(
            normalize_default_repo_parent_dir_text(self._state.default_repo_parent_dir_text)
        )
        self._default_repo_parent_dir_edit.setPlaceholderText("例如：C:/Users/你的用户名/Desktop/思政题库")
        self._default_repo_parent_dir_browse_btn = QPushButton("选择…")
        self._default_repo_parent_dir_browse_btn.setMinimumSize(BUTTON_MIN_WIDTH, BUTTON_MIN_HEIGHT)
        self._default_repo_parent_dir_browse_btn.clicked.connect(self._browse_default_repo_parent_dir)
        self._preferred_textbook_version_combo = QComboBox()
        self._preferred_textbook_version_combo.setEditable(True)
        self._preferred_textbook_version_combo.setInsertPolicy(QComboBox.InsertPolicy.NoInsert)
        for version in self._collect_textbook_version_options():
            self._preferred_textbook_version_combo.addItem(version)
        self._preferred_textbook_version_combo.setCurrentText(
            normalize_preferred_textbook_version(self._state.preferred_textbook_version)
        )
        self._import_source_dir_preset_combo = QComboBox()
        self._import_source_dir_preset_combo.addItem("下载", "downloads")
        self._import_source_dir_preset_combo.addItem("桌面", "desktop")
        self._import_source_dir_preset_combo.addItem("自定义", "custom")
        self._import_source_dir_edit = QLineEdit()
        self._import_source_dir_edit.setText(
            normalize_import_source_dir_text(self._state.import_source_dir_text)
        )
```

```python
        self._import_source_dir_edit.setPlaceholderText("例如：C:/Users/你的用户名/Downloads")
        self._import_source_dir_browse_btn = QPushButton("选择…")
        self._import_source_dir_browse_btn.setMinimumSize(BUTTON_MIN_WIDTH, BUTTON_MIN_HEIGHT)
        self._import_source_dir_browse_btn.clicked.connect(self._browse_import_source_dir)
        self._apply_import_source_dir_preset_from_text(self._import_source_dir_edit.text())

        default_repo_row = QHBoxLayout()
        default_repo_row.addWidget(self._default_repo_parent_dir_edit, 1)
        default_repo_row.addWidget(self._default_repo_parent_dir_browse_btn)
        import_source_dir_row = QHBoxLayout()
        import_source_dir_row.addWidget(self._import_source_dir_preset_combo)
        import_source_dir_row.addWidget(self._import_source_dir_edit, 1)
        import_source_dir_row.addWidget(self._import_source_dir_browse_btn)

        form = QFormLayout()
        if self._section == SECTION_GENERAL:
            form.addRow("默认题库保存位置：", default_repo_row)
            form.addRow("题目版本首选项：", self._preferred_textbook_version_combo)
        elif self._section == SECTION_IMPORT:
            form.addRow("默认导入目录：", import_source_dir_row)
            form.addRow("导入文档时解析生成：", self._import_analysis_combo)
            form.addRow("", self._dedupe_checkbox)
        else:
            form.addRow("可转多选 Markdown 导出：", self._convertible_multi_export_combo)
            form.addRow("导出 Markdown/PDF：", self._export_include_answers_checkbox)
            form.addRow("", self._export_include_analysis_checkbox)

        buttons = QDialogButtonBox(QDialogButtonBox.StandardButton.Ok | QDialogButtonBox.StandardButton.Cancel)
        buttons.accepted.connect(self._on_accept)
        buttons.rejected.connect(self.reject)
        self._buttons = buttons
        _style_dialog_button(buttons.button(QDialogButtonBox.StandardButton.Ok), "确定")
        _style_dialog_button(buttons.button(QDialogButtonBox.StandardButton.Cancel), "取消")

        layout = QVBoxLayout()
        layout.addLayout(form)
        layout.addWidget(buttons)
        self.setLayout(layout)

        self._import_source_dir_preset_combo.currentIndexChanged.connect(self._on_import_source_dir_preset_changed)
        self._import_source_dir_edit.textChanged.connect(self._on_import_source_dir_text_changed)

    def _on_accept(self) -> None:
        if self._section == SECTION_GENERAL:
            self._state.default_repo_parent_dir_text = normalize_default_repo_parent_dir_text(
                self._default_repo_parent_dir_edit.text()
            )
            self._state.preferred_textbook_version = normalize_preferred_textbook_version(
                self._preferred_textbook_version_combo.currentText()
            )
        elif self._section == SECTION_IMPORT:
            self._state.import_source_dir_text = normalize_import_source_dir_text(self._import_source_dir_edit.text())
            self._state.analysis_enabled = bool(self._import_analysis_combo.currentData())
            self._state.dedupe_enabled = self._dedupe_checkbox.isChecked()
        else:
            self._state.export_convertible_multi_mode = normalize_export_convertible_multi_mode(
                self._convertible_multi_export_combo.currentData()
            )
            self._state.export_include_answers = normalize_export_include_answers(
                self._export_include_answers_checkbox.isChecked()
            )
            self._state.export_include_analysis = normalize_export_include_analysis(
                self._export_include_analysis_checkbox.isChecked()
            )
        self._save_program_settings()
        self.accept()

```

```python
    def _dialog_title(self) -> str:
        if self._section == SECTION_IMPORT:
            return "导入设定"
        if self._section == SECTION_EXPORT:
            return "导出设定"
        return "常规设定"

    def _save_program_settings(self) -> None:
        save_program_settings_merged(
            {
                "default_repo_parent_dir_text": normalize_default_repo_parent_dir_text(
                    self._state.default_repo_parent_dir_text
                ),
                "import_source_dir_text": normalize_import_source_dir_text(
                    self._state.import_source_dir_text
                ),
                "analysis_enabled": bool(self._state.analysis_enabled),
                "dedupe_enabled": bool(self._state.dedupe_enabled),
                "analysis_provider": normalize_analysis_provider(self._state.analysis_provider),
                "analysis_model_name": normalize_analysis_model_name(self._state.analysis_model_name),
                "export_convertible_multi_mode": normalize_export_convertible_multi_mode(
                    self._state.export_convertible_multi_mode
                ),
                "export_include_answers": normalize_export_include_answers(self._state.export_include_answers),
                "export_include_analysis": normalize_export_include_analysis(self._state.export_include_analysis),
                "preferred_textbook_version": normalize_preferred_textbook_version(
                    self._state.preferred_textbook_version
                ),
            }
        )

    def _collect_textbook_version_options(self) -> list[str]:
        versions: list[str] = []
        seen: set[str] = set()

        def add_version(value: str) -> None:
            normalized = str(value or "").strip()
            if not normalized or normalized in seen:
                return
            seen.add(normalized)
            versions.append(normalized)

        add_version(self._state.preferred_textbook_version)
        db_path = library_db_path_from_repo_parent_dir_text(self._state.default_repo_parent_dir_text)
        if db_path.exists():
            for record in load_all_questions(db_path):
                add_version(record.textbook_version)
        return versions

    def _browse_default_repo_parent_dir(self) -> None:
        folder = QFileDialog.getExistingDirectory(
            self,
            "选择默认题库保存位置",
            normalize_default_repo_parent_dir_text(self._default_repo_parent_dir_edit.text()),
        )
        if folder:
            self._default_repo_parent_dir_edit.setText(folder)

    def _import_source_dir_for_preset(self, preset: str) -> str:
        if preset == "desktop":
            return str(desktop_import_source_dir())
        return str(default_import_source_dir())

    def _apply_import_source_dir_preset_from_text(self, text: str) -> None:
        normalized = str(Path(normalize_import_source_dir_text(text))).strip().lower()
        downloads = str(default_import_source_dir()).strip().lower()
        desktop = str(desktop_import_source_dir()).strip().lower()
        if normalized == downloads:
```

```python
            preset = "downloads"
        elif normalized == desktop:
            preset = "desktop"
        else:
            preset = "custom"
        index = self._import_source_dir_preset_combo.findData(preset)
        if index >= 0:
            self._import_source_dir_preset_combo.blockSignals(True)
            self._import_source_dir_preset_combo.setCurrentIndex(index)
            self._import_source_dir_preset_combo.blockSignals(False)

    def _on_import_source_dir_preset_changed(self) -> None:
        preset = str(self._import_source_dir_preset_combo.currentData() or "downloads")
        if preset == "custom":
            return
        self._import_source_dir_edit.setText(self._import_source_dir_for_preset(preset))

    def _on_import_source_dir_text_changed(self, text: str) -> None:
        self._apply_import_source_dir_preset_from_text(text)

    def _browse_import_source_dir(self) -> None:
        folder = QFileDialog.getExistingDirectory(
            self,
            "选择默认导入目录",
            normalize_import_source_dir_text(self._import_source_dir_edit.text()),
        )
        if not folder:
            return
        self._import_source_dir_edit.setText(folder)
        self._apply_import_source_dir_preset_from_text(folder)

```

```python
from __future__ import annotations

from dataclasses import replace
from datetime import datetime

from PyQt6.QtCore import QRegularExpression, Qt
from PyQt6.QtGui import QKeyEvent, QRegularExpressionValidator
from PyQt6.QtWidgets import (
    QComboBox,
    QDialog,
    QDialogButtonBox,
    QGridLayout,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QMessageBox,
    QPushButton,
    QSizePolicy,
    QStackedLayout,
    QTextEdit,
    QVBoxLayout,
    QWidget,
)

from sj_generator.infrastructure.persistence.sqlite_repo import DbQuestionRecord
from sj_generator.presentation.qt.message_box import show_message_box


class LevelPathPartLineEdit(QLineEdit):
    def __init__(self, text: str = "", parent=None) -> None:
        super().__init__(text, parent)
        self._prev_edit: QLineEdit | None = None
        self._next_edit: QLineEdit | None = None

    def set_neighbors(self, prev_edit: QLineEdit | None, next_edit: QLineEdit | None) -> None:
        self._prev_edit = prev_edit
        self._next_edit = next_edit

    def keyPressEvent(self, event: QKeyEvent) -> None:
        key = event.key()
        if key == Qt.Key.Key_Left and self.cursorPosition() == 0 and self._prev_edit is not None:
            self._prev_edit.setFocus()
            self._prev_edit.setCursorPosition(len(self._prev_edit.text()))
            event.accept()
            return
        if key == Qt.Key.Key_Right and self.cursorPosition() == len(self.text()) and self._next_edit is not None:
            self._next_edit.setFocus()
            self._next_edit.setCursorPosition(0)
            event.accept()
            return
        if key in (Qt.Key.Key_Return, Qt.Key.Key_Enter, Qt.Key.Key_Tab):
            if self._next_edit is not None:
                self._next_edit.setFocus()
                self._next_edit.selectAll()
            else:
                self.focusNextPrevChild(True)
            event.accept()
            return
        if key == Qt.Key.Key_Backtab:
            if self._prev_edit is not None:
                self._prev_edit.setFocus()
                self._prev_edit.selectAll()
            else:
                self.focusNextPrevChild(False)
            event.accept()
            return
        super().keyPressEvent(event)


```

```python
class QuestionEditDialog(QDialog):
    DELETE_RESULT = 2

    def __init__(self, record: DbQuestionRecord, parent=None, *, create_mode: bool = False) -> None:
        super().__init__(parent)
        self._record = record
        self._create_mode = create_mode
        self.setWindowTitle("新增题目" if create_mode else "编辑条目")
        self.resize(760, 484)

        self._type_combo = QComboBox()
        self._type_combo.addItems(["单选", "多选", "可转多选"])
        self._type_combo.setCurrentText(record.question_type or "单选")

        self._stem_edit = QTextEdit()
        self._stem_edit.setPlainText(record.stem)
        self._stem_edit.setPlaceholderText("题目内容")
        self._option_1_edit = QLineEdit(record.option_1)
        self._option_2_edit = QLineEdit(record.option_2)
        self._option_3_edit = QLineEdit(record.option_3)
        self._option_4_edit = QLineEdit(record.option_4)
        self._choice_1_edit = QLineEdit(record.choice_1)
        self._choice_2_edit = QLineEdit(record.choice_2)
        self._choice_3_edit = QLineEdit(record.choice_3)
        self._choice_4_edit = QLineEdit(record.choice_4)
        self._answer_line_edit = QLineEdit(record.answer)
        self._answer_line_edit.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)
        self._answer_combo = QComboBox()
        self._answer_combo.addItems(["", "A", "B", "C", "D"])
        self._answer_combo.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)
        combo_answer = (record.answer or "").strip().upper()
        combo_idx = self._answer_combo.findText(combo_answer)
        self._answer_combo.setCurrentIndex(combo_idx if combo_idx >= 0 else 0)
        answer_parts = self._split_multi_answer(record.answer)
        self._answer_part_1_edit = self._create_answer_part_edit(answer_parts[0])
        self._answer_part_2_edit = self._create_answer_part_edit(answer_parts[1])
        self._answer_part_3_edit = self._create_answer_part_edit(answer_parts[2])
        self._answer_part_4_edit = self._create_answer_part_edit(answer_parts[3])
        self._answer_part_1_edit.set_neighbors(None, self._answer_part_2_edit)
        self._answer_part_2_edit.set_neighbors(self._answer_part_1_edit, self._answer_part_3_edit)
        self._answer_part_3_edit.set_neighbors(self._answer_part_2_edit, self._answer_part_4_edit)
        self._answer_part_4_edit.set_neighbors(self._answer_part_3_edit, None)
        self._analysis_edit = QTextEdit()
        self._analysis_edit.setPlainText(record.analysis)
        self._analysis_edit.setPlaceholderText("解析内容")

        self._source_value = QLineEdit(self._format_source_display(record.source))
        self._source_value.setReadOnly(True)
        self._source_value.setPlaceholderText("题目来源：")
        level_parts = self._split_level_path(record.level_path)
        self._level_prefix_label = QLabel("所属层级：")
        self._level_part_1_edit = self._create_level_part_edit(level_parts[0])
        self._level_part_2_edit = self._create_level_part_edit(level_parts[1])
        self._level_part_3_edit = self._create_level_part_edit(level_parts[2])
        self._level_part_1_edit.set_neighbors(None, self._level_part_2_edit)
        self._level_part_2_edit.set_neighbors(self._level_part_1_edit, self._level_part_3_edit)
        self._level_part_3_edit.set_neighbors(self._level_part_2_edit, None)
        self._version_edit = QComboBox()
        self._version_edit.setEditable(True)
        self._version_edit.setInsertPolicy(QComboBox.InsertPolicy.NoInsert)
        for version in self._collect_textbook_version_options(record.textbook_version):
            self._version_edit.addItem(self._format_prefixed_display("教材版本：", version))
        self._version_edit.setCurrentText(self._format_prefixed_display("教材版本：", record.textbook_version))
        if self._version_edit.lineEdit() is not None:
            self._version_edit.lineEdit().setPlaceholderText("教材版本：")
        if self._version_edit.lineEdit() is not None:
            self._bind_locked_prefix(self._version_edit.lineEdit(), "教材版本：")
        self._option_1_label = QLabel("A：")
        self._option_2_label = QLabel("B：")
        self._option_3_label = QLabel("C：")
```

```python
        self._option_4_label = QLabel("D：")
        self._choice_1_label = QLabel("A：")
        self._choice_2_label = QLabel("B：")
        self._choice_3_label = QLabel("C：")
        self._choice_4_label = QLabel("D：")
        self._stem_edit.setMinimumHeight(180)
        self._analysis_edit.setMinimumHeight(140)

        def wrap_panel(title: str, body_layout: QGridLayout | QVBoxLayout | QHBoxLayout) -> QWidget:
            panel = QWidget()
            layout = QVBoxLayout(panel)
            layout.setContentsMargins(2, 2, 2, 2)
            layout.setSpacing(8)
            if title:
                title_label = QLabel(title)
                title_label.setStyleSheet("font-weight: 600;")
                layout.addWidget(title_label)
            layout.addLayout(body_layout)
            return panel

        stem_layout = QVBoxLayout()
        stem_layout.setContentsMargins(0, 0, 0, 0)
        stem_layout.addWidget(self._stem_edit)
        stem_panel = wrap_panel("", stem_layout)

        option_fields_layout = QGridLayout()
        option_fields_layout.setContentsMargins(0, 0, 0, 0)
        option_fields_layout.setHorizontalSpacing(8)
        option_fields_layout.setVerticalSpacing(8)
        option_fields_layout.addWidget(self._option_1_label, 0, 0)
        option_fields_layout.addWidget(self._option_1_edit, 0, 1)
        option_fields_layout.addWidget(self._option_2_label, 1, 0)
        option_fields_layout.addWidget(self._option_2_edit, 1, 1)
        option_fields_layout.addWidget(self._option_3_label, 2, 0)
        option_fields_layout.addWidget(self._option_3_edit, 2, 1)
        option_fields_layout.addWidget(self._option_4_label, 3, 0)
        option_fields_layout.addWidget(self._option_4_edit, 3, 1)
        option_fields_layout.setColumnStretch(1, 1)

        choice_fields_layout = QGridLayout()
        choice_fields_layout.setContentsMargins(0, 0, 0, 0)
        choice_fields_layout.setHorizontalSpacing(8)
        choice_fields_layout.setVerticalSpacing(8)
        choice_fields_layout.addWidget(self._choice_1_label, 0, 0)
        choice_fields_layout.addWidget(self._choice_1_edit, 0, 1)
        choice_fields_layout.addWidget(self._choice_2_label, 1, 0)
        choice_fields_layout.addWidget(self._choice_2_edit, 1, 1)
        choice_fields_layout.addWidget(self._choice_3_label, 2, 0)
        choice_fields_layout.addWidget(self._choice_3_edit, 2, 1)
        choice_fields_layout.addWidget(self._choice_4_label, 3, 0)
        choice_fields_layout.addWidget(self._choice_4_edit, 3, 1)
        choice_fields_layout.setColumnStretch(1, 1)

        option_fields_widget = QWidget()
        option_fields_widget.setLayout(option_fields_layout)
        self._choice_fields_widget = QWidget()
        self._choice_fields_widget.setLayout(choice_fields_layout)

        options_layout = QHBoxLayout()
        options_layout.setContentsMargins(0, 0, 0, 0)
        options_layout.setSpacing(12)
        options_layout.addWidget(option_fields_widget, 3)
        options_layout.addWidget(self._choice_fields_widget, 2)
        options_panel_layout = QVBoxLayout()
        options_panel_layout.setContentsMargins(0, 0, 0, 0)
        options_panel_layout.setSpacing(8)
        options_panel_layout.addLayout(options_layout)
        options_panel = wrap_panel("", options_panel_layout)

        answer_layout = QGridLayout()
```

```python
        answer_layout.setContentsMargins(0, 0, 0, 0)
        answer_layout.setHorizontalSpacing(8)
        answer_layout.setVerticalSpacing(8)
        self._answer_stack = QStackedLayout()
        self._answer_stack.setContentsMargins(0, 0, 0, 0)
        answer_stack_widget = QWidget()
        answer_stack_widget.setLayout(self._answer_stack)
        answer_stack_widget.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)
        self._answer_stack.addWidget(self._answer_line_edit)
        self._answer_stack.addWidget(self._answer_combo)
        answer_multi_row = QHBoxLayout()
        answer_multi_row.setContentsMargins(0, 0, 0, 0)
        answer_multi_row.setSpacing(6)
        answer_multi_row.addWidget(self._answer_part_1_edit)
        answer_multi_row.addWidget(self._answer_part_2_edit)
        answer_multi_row.addWidget(self._answer_part_3_edit)
        answer_multi_row.addWidget(self._answer_part_4_edit)
        answer_multi_row.addStretch(1)
        answer_multi_widget = QWidget()
        answer_multi_widget.setLayout(answer_multi_row)
        answer_multi_widget.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)
        self._answer_stack.addWidget(answer_multi_widget)
        answer_layout.addWidget(QLabel("答案："), 0, 0)
        answer_layout.addWidget(answer_stack_widget, 0, 1)
        answer_layout.addWidget(QLabel("题型："), 1, 0)
        answer_layout.addWidget(self._type_combo, 1, 1)
        answer_layout.setColumnStretch(1, 1)
        answer_panel = wrap_panel("", answer_layout)

        analysis_layout = QVBoxLayout()
        analysis_layout.setContentsMargins(0, 0, 0, 0)
        analysis_layout.addWidget(self._analysis_edit)
        analysis_panel = wrap_panel("", analysis_layout)

        source_content_layout = QVBoxLayout()
        source_content_layout.setContentsMargins(0, 0, 0, 0)
        source_content_layout.addWidget(self._source_value)
        source_content_panel = wrap_panel("", source_content_layout)

        level_content_layout = QVBoxLayout()
        level_content_layout.setContentsMargins(0, 0, 0, 0)
        level_row = QHBoxLayout()
        level_row.setContentsMargins(0, 0, 0, 0)
        level_row.setSpacing(6)
        level_row.addWidget(self._level_prefix_label)
        level_row.addWidget(self._level_part_1_edit)
        level_row.addWidget(QLabel("."))
        level_row.addWidget(self._level_part_2_edit)
        level_row.addWidget(QLabel("."))
        level_row.addWidget(self._level_part_3_edit)
        level_row.addStretch(1)
        level_content_layout.addLayout(level_row)
        level_content_panel = wrap_panel("", level_content_layout)

        version_content_layout = QVBoxLayout()
        version_content_layout.setContentsMargins(0, 0, 0, 0)
        version_content_layout.addWidget(self._version_edit)
        version_content_panel = wrap_panel("", version_content_layout)

        buttons = QDialogButtonBox(QDialogButtonBox.StandardButton.Ok | QDialogButtonBox.StandardButton.Cancel)
        buttons.accepted.connect(self.accept)
        buttons.rejected.connect(self.reject)
        ok_button = buttons.button(QDialogButtonBox.StandardButton.Ok)
        if ok_button is not None:
            ok_button.setText("确定")
            ok_button.setMinimumSize(96, 36)
        cancel_button = buttons.button(QDialogButtonBox.StandardButton.Cancel)
        if cancel_button is not None:
            cancel_button.setText("取消")
```

```python
            cancel_button.setMinimumSize(96, 36)

        button_row = QHBoxLayout()
        self._delete_btn = QPushButton("删除题目")
        self._delete_btn.setMinimumSize(96, 36)
        self._delete_btn.setStyleSheet("border: 1px solid #c00000;")
        self._delete_btn.clicked.connect(self._on_delete_clicked)
        if not self._create_mode:
            button_row.addWidget(self._delete_btn)
        button_row.addStretch(1)
        button_row.addWidget(buttons)

        content_grid = QGridLayout()
        content_grid.setContentsMargins(0, 0, 0, 0)
        content_grid.setHorizontalSpacing(10)
        content_grid.setVerticalSpacing(10)
        content_grid.addWidget(stem_panel, 0, 0, 5, 2)
        content_grid.addWidget(answer_panel, 0, 2, 1, 2)
        content_grid.addWidget(options_panel, 1, 2, 4, 2)
        content_grid.addWidget(analysis_panel, 5, 0, 2, 4)
        content_grid.addWidget(level_content_panel, 7, 0, 1, 4)
        content_grid.addWidget(version_content_panel, 8, 0, 1, 4)
        content_grid.addWidget(source_content_panel, 9, 0, 1, 4)
        for col in range(4):
            content_grid.setColumnStretch(col, 1)
        for row in range(10):
            content_grid.setRowStretch(row, 0)
        content_grid.setRowStretch(5, 1)
        content_grid.setRowStretch(6, 1)

        layout = QVBoxLayout()
        layout.addLayout(content_grid)
        layout.addLayout(button_row)
        self.setLayout(layout)
        self._type_combo.currentTextChanged.connect(self._sync_choice_fields_visibility)
        self._sync_choice_fields_visibility(self._type_combo.currentText())

    def accept(self) -> None:
        level_parts = [
            self._level_part_1_edit.text().strip(),
            self._level_part_2_edit.text().strip(),
            self._level_part_3_edit.text().strip(),
        ]
        if any(level_parts) and not all(level_parts):
            show_message_box(
                self,
                title="所属层级不完整",
                text="所属层级需要按“数字.数字.数字”完整填写。",
                icon=QMessageBox.Icon.Warning,
            )
            return
        super().accept()

    def updated_record(self) -> DbQuestionRecord:
        return replace(
            self._record,
            stem=self._stem_edit.toPlainText().strip(),
            option_1=self._option_1_edit.text().strip(),
            option_2=self._option_2_edit.text().strip(),
            option_3=self._option_3_edit.text().strip(),
            option_4=self._option_4_edit.text().strip(),
            choice_1=self._choice_1_edit.text().strip(),
            choice_2=self._choice_2_edit.text().strip(),
            choice_3=self._choice_3_edit.text().strip(),
            choice_4=self._choice_4_edit.text().strip(),
            answer=self._current_answer_text(),
            analysis=self._analysis_edit.toPlainText().strip(),
            question_type=self._type_combo.currentText().strip(),
            source=self._parse_source_display(self._source_value.text()),
            level_path=self._compose_level_path(),
```

```python
            textbook_version=self._parse_prefixed_display("教材版本：", self._version_edit.currentText()),
            updated_at=datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        )

    def _format_source_display(self, source: str) -> str:
        return self._format_prefixed_display("题目来源：", source)

    def _parse_source_display(self, text: str) -> str:
        return self._parse_prefixed_display("题目来源：", text)

    def _bind_locked_prefix(self, edit: QLineEdit, prefix: str) -> None:
        edit.textEdited.connect(
            lambda _text, target=edit, locked_prefix=prefix: self._ensure_locked_prefix(target, locked_prefix)
        )
        edit.cursorPositionChanged.connect(
            lambda _old, new, target=edit, locked_prefix=prefix: self._clamp_prefix_cursor(
                target, locked_prefix, new
            )
        )
        self._ensure_locked_prefix(edit, prefix)

    def _create_level_part_edit(self, value: str) -> LevelPathPartLineEdit:
        edit = LevelPathPartLineEdit((value or "").strip()[:1])
        edit.setAlignment(Qt.AlignmentFlag.AlignCenter)
        edit.setMaxLength(1)
        edit.setFixedWidth(36)
        edit.setPlaceholderText("0")
        edit.setValidator(QRegularExpressionValidator(QRegularExpression(r"^\d?$"), edit))
        return edit

    def _split_level_path(self, value: str) -> tuple[str, str, str]:
        parts = [part.strip() for part in str(value or "").split(".")]
        return (
            parts[0][:1] if len(parts) > 0 else "",
            parts[1][:1] if len(parts) > 1 else "",
            parts[2][:1] if len(parts) > 2 else "",
        )

    def _compose_level_path(self) -> str:
        parts = [
            self._level_part_1_edit.text().strip(),
            self._level_part_2_edit.text().strip(),
            self._level_part_3_edit.text().strip(),
        ]
        if not any(parts):
            return ""
        if all(parts):
            return ".".join(parts)
        return ""

    def _current_answer_text(self) -> str:
        answer_mode = self._answer_input_mode(self._type_combo.currentText())
        if answer_mode == "combo":
            return self._answer_combo.currentText().strip()
        if answer_mode == "multi_parts":
            digits = [
                self._circled_to_digit(self._answer_part_1_edit.text()),
                self._circled_to_digit(self._answer_part_2_edit.text()),
                self._circled_to_digit(self._answer_part_3_edit.text()),
                self._circled_to_digit(self._answer_part_4_edit.text()),
            ]
            return ",".join(digit for digit in digits if digit)
        return self._answer_line_edit.text().strip()

    def _set_answer_text(self, value: str, *, mode: str) -> None:
        normalized = (value or "").strip()
        if mode == "combo":
            combo_value = normalized.upper()
            combo_idx = self._answer_combo.findText(combo_value)
            self._answer_combo.setCurrentIndex(combo_idx if combo_idx >= 0 else 0)
```

```python
            return
        if mode == "multi_parts":
            parts = self._split_multi_answer(normalized)
            self._answer_part_1_edit.setText(parts[0])
            self._answer_part_2_edit.setText(parts[1])
            self._answer_part_3_edit.setText(parts[2])
            self._answer_part_4_edit.setText(parts[3])
            return
        self._answer_line_edit.setText(normalized)

    def _answer_input_mode(self, question_type: str) -> str:
        normalized_type = (question_type or "").strip()
        if normalized_type in ("单选", "可转多选"):
            return "combo"
        if normalized_type == "多选":
            return "multi_parts"
        return "line"

    def _create_answer_part_edit(self, value: str) -> LevelPathPartLineEdit:
        edit = LevelPathPartLineEdit((value or "").strip()[:1])
        edit.setAlignment(Qt.AlignmentFlag.AlignCenter)
        edit.setMaxLength(1)
        edit.setFixedWidth(36)
        edit.setPlaceholderText("")
        edit.setValidator(QRegularExpressionValidator(QRegularExpression(r"^[①②③④1-4]?$"), edit))
        edit.textEdited.connect(
            lambda _text, target=edit: target.setText(self._normalize_multi_answer_part(target.text()))
        )
        return edit

    def _split_multi_answer(self, value: str) -> tuple[str, str, str, str]:
        chars = [
            self._normalize_multi_answer_part(ch)
            for ch in str(value or "")
            if self._normalize_multi_answer_part(ch)
        ]
        return (
            chars[0] if len(chars) > 0 else "",
            chars[1] if len(chars) > 1 else "",
            chars[2] if len(chars) > 2 else "",
            chars[3] if len(chars) > 3 else "",
        )

    def _normalize_multi_answer_part(self, value: str) -> str:
        text = (value or "").strip()
        if not text:
            return ""
        return {
            "1": "①",
            "2": "②",
            "3": "③",
            "4": "④",
            "①": "①",
            "②": "②",
            "③": "③",
            "④": "④",
        }.get(text[:1], "")

    def _circled_to_digit(self, value: str) -> str:
        text = (value or "").strip()
        return {
            "①": "1",
            "②": "2",
            "③": "3",
            "④": "4",
            "1": "1",
            "2": "2",
            "3": "3",
            "4": "4",
        }.get(text[:1], "")
```

```python

    def _collect_textbook_version_options(self, current_value: str) -> list[str]:
        versions: list[str] = []
        seen: set[str] = set()

        def add_version(value: str) -> None:
            normalized = str(value or "").strip()
            if not normalized or normalized in seen:
                return
            seen.add(normalized)
            versions.append(normalized)

        add_version(current_value)
        parent = self.parent()
        current_records = getattr(parent, "_current_db_records", None)
        if isinstance(current_records, list):
            for record in current_records:
                add_version(getattr(record, "textbook_version", ""))
        return versions

    def _ensure_locked_prefix(self, edit: QLineEdit, prefix: str) -> None:
        min_pos = len(prefix)
        normalized_text = self._format_prefixed_display(prefix, self._parse_prefixed_display(prefix, edit.text())))
        if edit.text() != normalized_text:
            current_pos = max(min_pos, edit.cursorPosition())
            edit.setText(normalized_text)
            edit.setCursorPosition(min(current_pos, len(normalized_text)))
            return
        if edit.cursorPosition() < min_pos:
            edit.setCursorPosition(min_pos)

    def _clamp_prefix_cursor(self, edit: QLineEdit, prefix: str, new_position: int) -> None:
        min_pos = len(prefix)
        if new_position < min_pos:
            edit.setCursorPosition(min_pos)

    def _format_prefixed_display(self, prefix: str, value: str) -> str:
        normalized_prefix = str(prefix or "").strip()
        normalized_value = (value or "").strip()
        if not normalized_prefix:
            return normalized_value
        if not normalized_value:
            return normalized_prefix
        if normalized_value.startswith(normalized_prefix):
            return normalized_value
        return f"{normalized_prefix}{normalized_value}"

    def _parse_prefixed_display(self, prefix: str, text: str) -> str:
        normalized_prefix = str(prefix or "").strip()
        normalized_value = (text or "").strip()
        if normalized_prefix and normalized_value.startswith(normalized_prefix):
            return normalized_value[len(normalized_prefix):].strip()
        return normalized_value

    def _on_delete_clicked(self) -> None:
        answer = show_message_box(
            self,
            title="确认删除",
            text="确定删除这道题目吗？此操作不可撤销。",
            icon=QMessageBox.Icon.Warning,
            buttons=QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
            default_button=QMessageBox.StandardButton.No,
        )
        if answer == QMessageBox.StandardButton.Yes:
            self.done(self.DELETE_RESULT)

    def _sync_choice_fields_visibility(self, question_type: str) -> None:
        normalized_type = (question_type or "").strip()
        current_answer = self._current_answer_text()
```

```python
        answer_mode = self._answer_input_mode(normalized_type)
        self._answer_stack.setCurrentIndex({"line": 0, "combo": 1, "multi_parts": 2}.get(answer_mode, 0))
        self._set_answer_text(current_answer, mode=answer_mode)
        option_labels = (
            ("①：", "②：", "③：", "④：")
            if normalized_type in ("多选", "可转多选")
            else ("A：", "B：", "C：", "D：")
        )
        for widget, text in (
            (self._option_1_label, option_labels[0]),
            (self._option_2_label, option_labels[1]),
            (self._option_3_label, option_labels[2]),
            (self._option_4_label, option_labels[3]),
        ):
            widget.setText(text)
        visible = normalized_type == "可转多选"
        self._choice_fields_widget.setVisible(visible)
        for widget in (
            self._choice_1_label,
            self._choice_1_edit,
            self._choice_2_label,
            self._choice_2_edit,
            self._choice_3_label,
            self._choice_3_edit,
            self._choice_4_label,
            self._choice_4_edit,
        ):
            widget.setVisible(visible)

```

sj_generator/presentation/qt/styles.py

088-01

```python
from __future__ import annotations


APP_STYLESHEET = """
QPushButton {
    border: 1px solid #000000;
    border-radius: 0px;
}

QLineEdit,
QComboBox,
QTextEdit,
QPlainTextEdit,
QAbstractSpinBox {
    border: 1px solid #000000;
    border-radius: 0px;
    background: #ffffff;
}

QToolButton {
    border: 1px solid #000000;
    border-radius: 0px;
}

QTableWidget,
QTableView {
    border: 1px solid #000000;
    border-radius: 0px;
    background: #ffffff;
    gridline-color: #000000;
    outline: none;
}

QTableCornerButton::section {
    background: #ffffff;
    border: 0px;
    border-right: 1px solid #000000;
    border-bottom: 1px solid #000000;
}

QTableWidget QHeaderView::section,
QTableView QHeaderView::section {
    background: #ffffff;
    border: 0px;
    border-right: 1px solid #000000;
    border-bottom: 1px solid #000000;
    padding: 4px 6px;
}

QPushButton:hover,
QToolButton:hover {
    background: #f2f2f2;
}

QPushButton:pressed,
QToolButton:pressed {
    background: #d9d9d9;
}
""".strip()


def rounded_panel_stylesheet(*, background: str = "#fafafa", border_color: str = "#000000", radius: int = 10) -> str:
    return (
        f"border: 1px solid {border_color}; "
        f"border-radius: {int(radius)}px; "
        f"background: {background};"
    )

```

```python
from __future__ import annotations

import re

from PyQt6.QtCore import Qt
from PyQt6.QtGui import QAction, QGuiApplication, QKeySequence, QWheelEvent
from PyQt6.QtWidgets import (
    QApplication,
    QAbstractButton,
    QComboBox,
    QFrame,
    QLabel,
    QLineEdit,
    QPlainTextEdit,
    QTableWidget,
    QTableWidgetSelectionRange,
    QTextEdit,
    QWidget,
)

_INLINE_WHITESPACE_RE = re.compile(r"[\t\r\n]+")


def _normalize_copy_text(value: object) -> str:
    text = str(value or "")
    text = _INLINE_WHITESPACE_RE.sub(" ", text)
    return text.strip()


def _widget_belongs_to_table(widget: QWidget | None, table: QTableWidget) -> bool:
    current = widget
    while current is not None:
        if current is table:
            return True
        current = current.parentWidget()
    return False


def _copy_focused_text_widget(table: QTableWidget) -> bool:
    widget = QApplication.focusWidget()
    if not isinstance(widget, QWidget) or not _widget_belongs_to_table(widget, table):
        return False
    if isinstance(widget, QLineEdit):
        if widget.hasSelectedText():
            widget.copy()
            return True
        return False
    if isinstance(widget, (QTextEdit, QPlainTextEdit)):
        if widget.textCursor().hasSelection():
            widget.copy()
            return True
        return False
    return False


def _table_cell_widget_text(widget: QWidget | None) -> str:
    if widget is None:
        return ""
    if isinstance(widget, QComboBox):
        return widget.currentText()
    if isinstance(widget, (QLabel, QLineEdit)):
        return widget.text()
    if isinstance(widget, (QTextEdit, QPlainTextEdit)):
        return widget.toPlainText()
    if isinstance(widget, QAbstractButton):
        return widget.text()
    text_attr = getattr(widget, "text", None)
    if callable(text_attr):
        try:
            return str(text_attr())
```

```python
        except Exception:
            return ""
    return ""


def table_cell_text(table: QTableWidget, row: int, col: int) -> str:
    item = table.item(row, col)
    if item is not None:
        display_value = item.data(Qt.ItemDataRole.DisplayRole)
        if display_value is not None:
            return _normalize_copy_text(display_value)
        return _normalize_copy_text(item.text())
    return _normalize_copy_text(_table_cell_widget_text(table.cellWidget(row, col)))


def selected_table_as_tsv(table: QTableWidget, *, fallback_to_all: bool = True) -> str:
    ranges = sorted(table.selectedRanges(), key=lambda rng: (rng.topRow(), rng.leftColumn()))
    if not ranges and fallback_to_all and table.rowCount() > 0 and table.columnCount() > 0:
        ranges = [QTableWidgetSelectionRange(0, 0, table.rowCount() - 1, table.columnCount() - 1)]
    if not ranges:
        return ""

    blocks: list[str] = []
    for selected_range in ranges:
        top_row = selected_range.topRow()
        bottom_row = selected_range.bottomRow()
        left_col = selected_range.leftColumn()
        right_col = selected_range.rightColumn()
        rows: list[str] = []
        for row in range(top_row, bottom_row + 1):
            if table.isRowHidden(row):
                continue
            cells = [
                table_cell_text(table, row, col)
                for col in range(left_col, right_col + 1)
                if not table.isColumnHidden(col)
            ]
            if cells:
                rows.append("\t".join(cells))
        if rows:
            blocks.append("\n".join(rows))
    return "\n".join(blocks)


def copy_table_as_tsv(table: QTableWidget, *, fallback_to_all: bool = True) -> bool:
    if _copy_focused_text_widget(table):
        return True
    text = selected_table_as_tsv(table, fallback_to_all=fallback_to_all)
    if not text:
        return False
    QGuiApplication.clipboard().setText(text)
    return True


class CopyableTableWidget(QTableWidget):
    def __init__(self, *args, fallback_copy_all: bool = True, zoom_callback=None, **kwargs) -> None:
        super().__init__(*args, **kwargs)
        self._fallback_copy_all = fallback_copy_all
        self._zoom_callback = zoom_callback
        self.setShowGrid(True)
        self.setFrameShape(QFrame.Shape.Box)
        self.setFrameShadow(QFrame.Shadow.Plain)
        self.setLineWidth(1)
        self.setMidLineWidth(0)
        copy_action = QAction("复制表格文本", self)
        copy_action.setShortcut(QKeySequence.StandardKey.Copy)
        copy_action.setShortcutContext(Qt.ShortcutContext.WidgetWithChildrenShortcut)
        copy_action.triggered.connect(self.copy_selection_as_tsv)
        self.addAction(copy_action)
        self._copy_action = copy_action
```

```python

    def copy_selection_as_tsv(self) -> bool:
        return copy_table_as_tsv(self, fallback_to_all=self._fallback_copy_all)

    def wheelEvent(self, event: QWheelEvent) -> None:
        if event.modifiers() & Qt.KeyboardModifier.ControlModifier:
            if self._zoom_callback is not None:
                delta = event.angleDelta().y()
                if delta != 0:
                    self._zoom_callback(1 if delta > 0 else -1)
                    event.accept()
                    return
        super().wheelEvent(event)

```

sj_generator/presentation/qt/wizard_base.py

090-01

```python
from __future__ import annotations

from collections.abc import Mapping

from PyQt6.QtWidgets import QWizard

PRODUCT_TITLE = "思政智题云枢"

DEFAULT_WIZARD_BUTTON_LAYOUT = [
    QWizard.WizardButton.Stretch,
    QWizard.WizardButton.BackButton,
    QWizard.WizardButton.NextButton,
    QWizard.WizardButton.FinishButton,
    QWizard.WizardButton.CancelButton,
]


class AppWizardBase(QWizard):
    def __init__(self, parent=None, *, window_title: str = PRODUCT_TITLE) -> None:
        super().__init__(parent)
        self._window_title_text = window_title
        self._default_button_layout = list(DEFAULT_WIZARD_BUTTON_LAYOUT)
        self.setWindowTitle(window_title)
        self.setWizardStyle(QWizard.WizardStyle.ModernStyle)
        self.setButtonLayout(self._default_button_layout)

    def apply_button_texts(self, button_texts: Mapping[QWizard.WizardButton, str]) -> None:
        for button, text in button_texts.items():
            self.setButtonText(button, text)

    def cache_and_hide_page_titles(self) -> None:
        for page_id in self.pageIds():
            page = self.page(page_id)
            if page is None:
                continue
            page.setProperty("_window_title_text", page.title())
            page.setTitle("")

    def cache_and_hide_page_title(self, page_id: int) -> None:
        page = self.page(page_id)
        if page is None:
            return
        page.setProperty("_window_title_text", page.title())
        page.setTitle("")

    def update_window_title(self, _page_id: int) -> None:
        self.setWindowTitle(self._window_title_text)

```

sj_generator/shared/__init__.py

091-01

```python
from sj_generator.shared.paths import AppPaths, app_paths, common_mistakes_md_path

__all__ = ["AppPaths", "app_paths", "common_mistakes_md_path"]

```

sj_generator/shared/paths.py

092-01

```python
from __future__ import annotations

import sys
from dataclasses import dataclass
from pathlib import Path


@dataclass(frozen=True)
class AppPaths:
    base_dir: Path

    @property
    def logo_path(self) -> Path:
        return self.base_dir / "logo.png"

    @property
    def doc_dir(self) -> Path:
        return self.base_dir / "doc"

    @property
    def docs_dir(self) -> Path:
        return self.base_dir / "docs"

    @property
    def reference_dir(self) -> Path:
        return self.base_dir / "reference"

    @property
    def reference_resource_dir(self) -> Path:
        return self.reference_dir / "resource"

    @property
    def reference_mistakes_dir(self) -> Path:
        return self.reference_dir / "mistakes"

    @property
    def reference_quotation_dir(self) -> Path:
        return self.reference_dir / "quotation"

    @property
    def reference_picture_dir(self) -> Path:
        return self.reference_dir / "picture"


def app_base_dir() -> Path:
    if getattr(sys, "frozen", False):
        runtime_dir = Path(sys.executable).resolve().parent
        if (runtime_dir / "reference").exists() or (runtime_dir / "logo.png").exists():
            return runtime_dir
        meipass = getattr(sys, "_MEIPASS", "")
        if meipass:
            return Path(meipass)
        return runtime_dir
    return Path(__file__).resolve().parents[2]


def app_paths(base_dir: Path | None = None) -> AppPaths:
    return AppPaths(base_dir=base_dir or app_base_dir())


def common_mistakes_md_path(base_dir: Path | None = None) -> Path:
    return app_paths(base_dir).reference_mistakes_dir / "选择题常见错题归因.md"
```

```python
from __future__ import annotations

from importlib import import_module

# Compatibility entrypoint for the legacy ui package.
# Keep concrete forwarding logic in each sibling module; this package
# only centralizes stable exports and lazy-loads them on demand.
_MODULE_EXPORTS = {
    'constants': ['PAGE_INTRO', 'PAGE_WELCOME', 'PAGE_AI_SELECT', 'PAGE_AI_IMPORT', 'PAGE_AI_IMPORT_CONTENT', 'PAGE_DEDUPE_RESULT', 'PAGE_AI_ANALYSIS', 'PAGE_IMPORT_SUCCESS', 'DEFAULT_WINDOW_WIDTH', 'DEFAULT_WINDOW_HEIGHT', 'QT_MAX_WINDOW_SIZE'],
    'styles': ['APP_STYLESHEET', 'rounded_panel_stylesheet'],
    'wizard_base': ['PRODUCT_TITLE', 'DEFAULT_WIZARD_BUTTON_LAYOUT', 'AppWizardBase'],
    'message_box': ['MESSAGE_BOX_BUTTON_MIN_WIDTH', 'MESSAGE_BOX_BUTTON_MIN_HEIGHT', 'style_message_box_buttons', 'show_message_box'],
    'table_copy': ['table_cell_text', 'selected_table_as_tsv', 'copy_table_as_tsv', 'CopyableTableWidget'],
    'api_config_dialog': ['ApiConfigDialog'],
    'program_settings_dialog': ['SECTION_GENERAL', 'SECTION_IMPORT', 'SECTION_EXPORT', 'ProgramSettingsDialog'],
    'question_edit_dialog': ['LevelPathPartLineEdit', 'QuestionEditDialog'],
    'import_cost_history_dialog': ['ImportCostHistoryDialog'],
    'compare_highlight': ['compare_highlight_model_styles'],
    'import_page_common': ['LEVEL_PATH_RE', 'LEVEL_PATH_EDIT_RE', 'style_dialog_button', 'style_busy_progress', 'sanitize_filename', 'unique_child_dir', 'rename_project', 'extract_paths_from_drop_event', 'merge_paths_text', 'is_valid_level_path', 'LevelPathItemDelegate', 'PreserveCellBackgroundDelegate'],
    'import_select_reminder': ['inspect_docx_content', 'make_import_check_item', 'populate_import_reminder_columns'],
    'import_select_session': ['safe_mtime_ns', 'word_lock_path', 'build_opened_doc_session', 'poll_opened_doc_sessions', 'select_first_changed_row'],
    'import_question_ref_detail': ['question_ref_row_background', 'question_ref_detail_headers', 'question_ref_detail_model_specs', 'question_ref_detail_model_values', 'question_ref_number_order', 'question_type_options', 'question_type_from_candidates', 'question_type_conflicts', 'build_question_ref_detail_rows', 'build_manual_type_combo', 'populate_question_ref_detail_table'],
    'import_ref_session': ['QuestionRefRuntimeState'],
    'import_content_detail': ['format_json_cell', 'question_ref_total_count', 'question_content_active_model_specs', 'effective_content_question_workers', 'question_content_payload_model_specs', 'question_content_model_signature', 'question_content_detail_headers', 'has_question_content_payload', 'sec_int', 'record_round_sec', 'format_round_secs', 'content_detail_width_signature', 'calculate_content_detail_column_widths', 'apply_content_detail_column_widths', 'set_content_detail_item', 'compare_row_brush', 'apply_compare_row_background', 'clear_compare_payload_background', 'highlight_sig_text', 'apply_partial_pass_highlight', 'build_compare_verdict'],
    'import_progress': ['extract_question_ref_source_name', 'parse_content_progress_message'],
    'import_workers': ['AiQuestionRefWorker', 'AiImportContentWorker'],
    'import_db_service': ['commit_draft_questions_to_db'],
    'import_costs': ['begin_app_import_cost_capture', 'capture_import_cost_before', 'capture_import_cost_before_async', 'wait_import_cost_before_capture', 'freeze_import_cost_result'],
    'welcome_menu': ['WelcomeMenuBundle', 'build_welcome_menu_bar'],
    'welcome_page_actions': ['TableImportResult', 'open_api_config', 'open_program_settings', 'import_from_table_file'],
    'welcome_import': ['effective_import_source_dir', 'backup_import_source_files', 'build_doc_hash_index', 'file_sha256', 'next_doc_backup_path', 'open_import_flow_windows', 'open_import_flow_states', 'show_import_flow_window'],
    'welcome_export': ['sanitize_export_name', 'display_export_level_name', 'default_export_dir', 'db_record_to_question', 'format_db_options', 'format_db_answer', 'format_choice_mapping', 'digit_to_circled', 'export_current_level_to_markdown', 'export_current_level_to_pdf', 'export_db_records_to_xlsx'],
    'welcome_export_actions': ['selected_tree_level_path', 'export_current_level_markdown', 'export_current_level_pdf', 'export_current_level_xlsx', 'export_db_table_xlsx'],
    'welcome_question_actions': ['selected_tree_level_path_for_create', 'build_new_question_record', 'add_question_manually', 'QuestionEditResult', 'edit_question_record'],
    'welcome_table': ['populate_questions_table', 'populate_db_records_table', 'build_question_row_values', 'build_db_row_values', 'populate_row_by_values', 'QtAlign', 'format_stem_with_sequence', 'format_table_analysis', 'set_table_item'],
    'welcome_table_view': ['load_column_visibility', 'save_column_visibility', 'set_column_visible', 'schedule_table_row_resize', 'apply_table_row_resize', 'apply_table_row_resize_followup', 'refresh_table_layout_after_column_change', 'rebalance_visible_columns', 'load_table_font_point_size', 'apply_table_font_size', 'adjust_table_font_size', 'persist_table_font_point_size'],
    'welcome_tree': ['delete_action_text', 'delete_scope_text', 'parse_level_parts', 'display_level_parts', 'tree_level_key_for_path', 'load_questions_for_tree_level', 'format_level_label', 'expand_item_ancestors', 'collect_expanded_level_prefixes', 'build_level_tree', 'to_chinese_number'],
    'welcome_tree_actions': ['refresh_level_tree', 'handle_level_selection_changed', 'show_folder_tree_context_menu', 'sync_expanded_level_prefixes_from_tree', 'delete_level_subtree'],
```

```
    "pages": ['IntroPage', 'WelcomePage', 'AiSelectFilesPage', 'AiImportPage', 'AiImportContentPage', 'DedupeResultPage', 'AiAnalysisPage', 'ImportSuccessPage'],
}

_OMITTED_EXPORTS = ('BUTTON_MIN_HEIGHT', 'BUTTON_MIN_WIDTH')

_EXPORT_TO_MODULE = {
    name: module_name
    for module_name, names in _MODULE_EXPORTS.items()
    for name in names
}

__all__ = [
    'PAGE_INTRO',
    'PAGE_WELCOME',
    'PAGE_AI_SELECT',
    'PAGE_AI_IMPORT',
    'PAGE_AI_IMPORT_CONTENT',
    'PAGE_DEDUPE_RESULT',
    'PAGE_AI_ANALYSIS',
    'PAGE_IMPORT_SUCCESS',
    'DEFAULT_WINDOW_WIDTH',
    'DEFAULT_WINDOW_HEIGHT',
    'QT_MAX_WINDOW_SIZE',
    'APP_STYLESHEET',
    'rounded_panel_stylesheet',
    'PRODUCT_TITLE',
    'DEFAULT_WIZARD_BUTTON_LAYOUT',
    'AppWizardBase',
    'MESSAGE_BOX_BUTTON_MIN_WIDTH',
    'MESSAGE_BOX_BUTTON_MIN_HEIGHT',
    'style_message_box_buttons',
    'show_message_box',
    'table_cell_text',
    'selected_table_as_tsv',
    'copy_table_as_tsv',
    'CopyableTableWidget',
    'ApiConfigDialog',
    'SECTION_GENERAL',
    'SECTION_IMPORT',
    'SECTION_EXPORT',
    'ProgramSettingsDialog',
    'LevelPathPartLineEdit',
    'QuestionEditDialog',
    'ImportCostHistoryDialog',
    'compare_highlight_model_styles',
    'LEVEL_PATH_RE',
    'LEVEL_PATH_EDIT_RE',
    'style_dialog_button',
    'style_busy_progress',
    'sanitize_filename',
    'unique_child_dir',
    'rename_project',
    'extract_paths_from_drop_event',
    'merge_paths_text',
    'is_valid_level_path',
    'LevelPathItemDelegate',
    'PreserveCellBackgroundDelegate',
    'inspect_docx_content',
    'make_import_check_item',
    'populate_import_reminder_columns',
    'safe_mtime_ns',
    'word_lock_path',
    'build_opened_doc_session',
    'poll_opened_doc_sessions',
    'select_first_changed_row',
    'question_ref_row_background',
    'question_ref_detail_headers',
    'question_ref_detail_model_specs',
    'question_ref_detail_model_values',
```

```
    'question_ref_number_order',
    'question_type_options',
    'question_type_from_candidates',
    'question_type_conflicts',
    'build_question_ref_detail_rows',
    'build_manual_type_combo',
    'populate_question_ref_detail_table',
    'QuestionRefRuntimeState',
    'format_json_cell',
    'question_ref_total_count',
    'question_content_active_model_specs',
    'effective_content_question_workers',
    'question_content_payload_model_specs',
    'question_content_model_signature',
    'question_content_detail_headers',
    'has_question_content_payload',
    'sec_int',
    'record_round_sec',
    'format_round_secs',
    'content_detail_width_signature',
    'calculate_content_detail_column_widths',
    'apply_content_detail_column_widths',
    'set_content_detail_item',
    'compare_row_brush',
    'apply_compare_row_background',
    'clear_compare_payload_background',
    'highlight_sig_text',
    'apply_partial_pass_highlight',
    'build_compare_verdict',
    'extract_question_ref_source_name',
    'parse_content_progress_message',
    'AiQuestionRefWorker',
    'AiImportContentWorker',
    'commit_draft_questions_to_db',
    'begin_app_import_cost_capture',
    'capture_import_cost_before',
    'capture_import_cost_before_async',
    'wait_import_cost_before_capture',
    'freeze_import_cost_result',
    'WelcomeMenuBundle',
    'build_welcome_menu_bar',
    'TableImportResult',
    'open_api_config',
    'open_program_settings',
    'import_from_table_file',
    'effective_import_source_dir',
    'backup_import_source_files',
    'build_doc_hash_index',
    'file_sha256',
    'next_doc_backup_path',
    'open_import_flow_windows',
    'open_import_flow_states',
    'show_import_flow_window',
    'sanitize_export_name',
    'display_export_level_name',
    'default_export_dir',
    'db_record_to_question',
    'format_db_options',
    'format_db_answer',
    'format_choice_mapping',
    'digit_to_circled',
    'export_current_level_to_markdown',
    'export_current_level_to_pdf',
    'export_db_records_to_xlsx',
    'selected_tree_level_path',
    'export_current_level_markdown',
    'export_current_level_pdf',
    'export_current_level_xlsx',
    'export_db_table_xlsx',
    'selected_tree_level_path_for_create',
```

```python
    'build_new_question_record',
    'add_question_manually',
    'QuestionEditResult',
    'edit_question_record',
    'populate_questions_table',
    'populate_db_records_table',
    'build_question_row_values',
    'build_db_row_values',
    'populate_row_by_values',
    'QtAlign',
    'format_stem_with_sequence',
    'format_table_analysis',
    'set_table_item',
    'load_column_visibility',
    'save_column_visibility',
    'set_column_visible',
    'schedule_table_row_resize',
    'apply_table_row_resize',
    'apply_table_row_resize_followup',
    'refresh_table_layout_after_column_change',
    'rebalance_visible_columns',
    'load_table_font_point_size',
    'apply_table_font_size',
    'adjust_table_font_size',
    'persist_table_font_point_size',
    'delete_action_text',
    'delete_scope_text',
    'parse_level_parts',
    'display_level_parts',
    'tree_level_key_for_path',
    'load_questions_for_tree_level',
    'format_level_label',
    'expand_item_ancestors',
    'collect_expanded_level_prefixes',
    'build_level_tree',
    'to_chinese_number',
    'refresh_level_tree',
    'handle_level_selection_changed',
    'show_folder_tree_context_menu',
    'sync_expanded_level_prefixes_from_tree',
    'delete_level_subtree',
    'IntroPage',
    'WelcomePage',
    'AiSelectFilesPage',
    'AiImportPage',
    'AiImportContentPage',
    'DedupeResultPage',
    'AiAnalysisPage',
    'ImportSuccessPage',
]


def __getattr__(name: str):
    module_name = _EXPORT_TO_MODULE.get(name)
    if module_name is None:
        raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
    module = import_module(f".{module_name}", __name__)
    value = getattr(module, name)
    globals()[name] = value
    return value


def __dir__() -> list[str]:
    return sorted(set(globals()) | set(__all__))
```

sj_generator/ui/api_config_dialog.py

094-01

```python
from sj_generator.presentation.qt.api_config_dialog import ApiConfigDialog

__all__ = ["ApiConfigDialog"]

```

sj_generator/ui/compare_highlight.py

095-01

```python
from sj_generator.presentation.qt.compare_highlight import compare_highlight_model_styles

__all__ = ["compare_highlight_model_styles"]

```

sj_generator/ui/constants.py

096-01

```python
from sj_generator.presentation.qt.constants import (
    DEFAULT_WINDOW_HEIGHT,
    DEFAULT_WINDOW_WIDTH,
    PAGE_AI_ANALYSIS,
    PAGE_AI_IMPORT,
    PAGE_AI_IMPORT_CONTENT,
    PAGE_AI_SELECT,
    PAGE_DEDUPE_RESULT,
    PAGE_IMPORT_SUCCESS,
    PAGE_INTRO,
    PAGE_WELCOME,
    QT_MAX_WINDOW_SIZE,
)

__all__ = [
    "PAGE_INTRO",
    "PAGE_WELCOME",
    "PAGE_AI_SELECT",
    "PAGE_AI_IMPORT",
    "PAGE_AI_IMPORT_CONTENT",
    "PAGE_DEDUPE_RESULT",
    "PAGE_AI_ANALYSIS",
    "PAGE_IMPORT_SUCCESS",
    "DEFAULT_WINDOW_WIDTH",
    "DEFAULT_WINDOW_HEIGHT",
    "QT_MAX_WINDOW_SIZE",
]
```

sj_generator/ui/import_content_detail.py

097-01

```python
from sj_generator.presentation.qt.pages.import_flow.import_content_detail import (
    apply_compare_row_background,
    apply_content_detail_column_widths,
    apply_partial_pass_highlight,
    build_compare_verdict,
    calculate_content_detail_column_widths,
    clear_compare_payload_background,
    compare_row_brush,
    content_detail_width_signature,
    effective_content_question_workers,
    format_json_cell,
    format_round_secs,
    has_question_content_payload,
    highlight_sig_text,
    question_content_active_model_specs,
    question_content_detail_headers,
    question_content_model_signature,
    question_content_payload_model_specs,
    question_ref_total_count,
    record_round_sec,
    sec_int,
    set_content_detail_item,
)

__all__ = [
    "format_json_cell",
    "question_ref_total_count",
    "question_content_active_model_specs",
    "effective_content_question_workers",
    "question_content_payload_model_specs",
    "question_content_model_signature",
    "question_content_detail_headers",
    "has_question_content_payload",
    "sec_int",
    "record_round_sec",
    "format_round_secs",
    "content_detail_width_signature",
    "calculate_content_detail_column_widths",
    "apply_content_detail_column_widths",
    "set_content_detail_item",
    "compare_row_brush",
    "apply_compare_row_background",
    "clear_compare_payload_background",
    "highlight_sig_text",
    "apply_partial_pass_highlight",
    "build_compare_verdict",
]
```

sj_generator/ui/import_cost_history_dialog.py

098-01

```python
from sj_generator.presentation.qt.import_cost_history_dialog import ImportCostHistoryDialog

__all__ = ["ImportCostHistoryDialog"]

```

sj_generator/ui/import_costs.py

099-01

```python
from sj_generator.presentation.qt.import_costs import (
    begin_app_import_cost_capture,
    capture_import_cost_before,
    capture_import_cost_before_async,
    freeze_import_cost_result,
    wait_import_cost_before_capture,
)

__all__ = [
    "begin_app_import_cost_capture",
    "capture_import_cost_before",
    "capture_import_cost_before_async",
    "wait_import_cost_before_capture",
    "freeze_import_cost_result",
]

```

sj_generator/ui/import_db_service.py

100-01

```python
from sj_generator.presentation.qt.pages.import_flow.import_db_service import commit_draft_questions_to_db

__all__ = ["commit_draft_questions_to_db"]

```

sj_generator/ui/import_page_common.py

101-01

```python
from sj_generator.presentation.qt.pages.import_flow.import_page_common import (
    BUTTON_MIN_HEIGHT,
    BUTTON_MIN_WIDTH,
    LEVEL_PATH_EDIT_RE,
    LEVEL_PATH_RE,
    LevelPathItemDelegate,
    PreserveCellBackgroundDelegate,
    extract_paths_from_drop_event,
    is_valid_level_path,
    merge_paths_text,
    rename_project,
    sanitize_filename,
    style_busy_progress,
    style_dialog_button,
    unique_child_dir,
)

__all__ = [
    "BUTTON_MIN_WIDTH",
    "BUTTON_MIN_HEIGHT",
    "LEVEL_PATH_RE",
    "LEVEL_PATH_EDIT_RE",
    "style_dialog_button",
    "style_busy_progress",
    "sanitize_filename",
    "unique_child_dir",
    "rename_project",
    "extract_paths_from_drop_event",
    "merge_paths_text",
    "is_valid_level_path",
    "LevelPathItemDelegate",
    "PreserveCellBackgroundDelegate",
]
```

sj_generator/ui/import_progress.py

102-01

```python
from sj_generator.presentation.qt.pages.import_flow.import_progress import (
    extract_question_ref_source_name,
    parse_content_progress_message,
)

__all__ = [
    "extract_question_ref_source_name",
    "parse_content_progress_message",
]

```

sj_generator/ui/import_question_ref_detail.py

103-01

```python
from sj_generator.presentation.qt.pages.import_flow.import_question_ref_detail import (
    build_manual_type_combo,
    build_question_ref_detail_rows,
    populate_question_ref_detail_table,
    question_ref_detail_headers,
    question_ref_detail_model_specs,
    question_ref_detail_model_values,
    question_ref_number_order,
    question_ref_row_background,
    question_type_conflicts,
    question_type_from_candidates,
    question_type_options,
)

__all__ = [
    "question_ref_row_background",
    "question_ref_detail_headers",
    "question_ref_detail_model_specs",
    "question_ref_detail_model_values",
    "question_ref_number_order",
    "question_type_options",
    "question_type_from_candidates",
    "question_type_conflicts",
    "build_question_ref_detail_rows",
    "build_manual_type_combo",
    "populate_question_ref_detail_table",
]
```

sj_generator/ui/import_ref_session.py

104-01

```python
from sj_generator.presentation.qt.pages.import_flow.import_ref_session import QuestionRefRuntimeState

__all__ = ["QuestionRefRuntimeState"]

```

sj_generator/ui/import_select_reminder.py

105-01

```python
from sj_generator.presentation.qt.pages.import_flow.import_select_reminder import (
    inspect_docx_content,
    make_import_check_item,
    populate_import_reminder_columns,
)

__all__ = [
    "inspect_docx_content",
    "make_import_check_item",
    "populate_import_reminder_columns",
]

```

sj_generator/ui/import_select_session.py

106-01

```python
from sj_generator.presentation.qt.pages.import_flow.import_select_session import (
    build_opened_doc_session,
    poll_opened_doc_sessions,
    safe_mtime_ns,
    select_first_changed_row,
    word_lock_path,
)

__all__ = [
    "safe_mtime_ns",
    "word_lock_path",
    "build_opened_doc_session",
    "poll_opened_doc_sessions",
    "select_first_changed_row",
]

```

sj_generator/ui/import_workers.py

107-01

```python
from sj_generator.presentation.qt.pages.import_flow.import_workers import AiImportContentWorker, AiQuestionRefWorker

__all__ = ["AiQuestionRefWorker", "AiImportContentWorker"]

```

sj_generator/ui/message_box.py

108-01

```python
from sj_generator.presentation.qt.message_box import (
    MESSAGE_BOX_BUTTON_MIN_HEIGHT,
    MESSAGE_BOX_BUTTON_MIN_WIDTH,
    show_message_box,
    style_message_box_buttons,
)

__all__ = [
    "MESSAGE_BOX_BUTTON_MIN_WIDTH",
    "MESSAGE_BOX_BUTTON_MIN_HEIGHT",
    "style_message_box_buttons",
    "show_message_box",
]

```

sj_generator/ui/pages/__init__.py

109-01

```python
from __future__ import annotations

from importlib import import_module

_MODULE_EXPORTS = {
    "intro_pages": ["IntroPage"],
    "welcome_pages": ["WelcomePage"],
    "import_flow": [
        "AiSelectFilesPage",
        "AiImportPage",
        "AiImportContentPage",
        "DedupeResultPage",
        "AiAnalysisPage",
        "ImportSuccessPage",
    ],
}

_EXPORT_TO_MODULE = {
    name: module_name
    for module_name, names in _MODULE_EXPORTS.items()
    for name in names
}

__all__ = [
    "IntroPage",
    "WelcomePage",
    "AiSelectFilesPage",
    "AiImportPage",
    "AiImportContentPage",
    "DedupeResultPage",
    "AiAnalysisPage",
    "ImportSuccessPage",
]


def __getattr__(name: str):
    module_name = _EXPORT_TO_MODULE.get(name)
    if module_name is None:
        raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
    module = import_module(f".{module_name}", __name__)
    value = getattr(module, name)
    globals()[name] = value
    return value


def __dir__() -> list[str]:
    return sorted(set(globals()) | set(__all__))

```

sj_generator/ui/pages/analysis_pages.py

110-01

```python
from sj_generator.presentation.qt.pages.import_flow.analysis_page import (
    AiAnalysisPage,
)

__all__ = ["AiAnalysisPage"]

```

sj_generator/ui/pages/dedupe_pages.py

111-01

```python
from sj_generator.presentation.qt.pages.import_flow.dedupe_page import (
    DedupeResultPage,
)

__all__ = ["DedupeResultPage"]

```

sj_generator/ui/pages/export_pages.py

112-01

```python
from sj_generator.presentation.qt.pages.import_flow.success_page import (
    ImportSuccessPage,
)

__all__ = ["ImportSuccessPage"]

```

sj_generator/ui/pages/import_flow/__init__.py

113-01

```python
from __future__ import annotations

from importlib import import_module

_EXPORT_TO_MODULE = {
    "AiSelectFilesPage": ".select_page",
    "AiImportPage": ".question_ref_page",
    "AiImportContentPage": ".content_page",
    "DedupeResultPage": "..dedupe_pages",
    "AiAnalysisPage": "..analysis_pages",
    "ImportSuccessPage": "..export_pages",
}

__all__ = [
    "AiSelectFilesPage",
    "AiImportPage",
    "AiImportContentPage",
    "DedupeResultPage",
    "AiAnalysisPage",
    "ImportSuccessPage",
]


def __getattr__(name: str):
    module_name = _EXPORT_TO_MODULE.get(name)
    if module_name is None:
        raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
    module = import_module(module_name, __name__)
    value = getattr(module, name)
    globals()[name] = value
    return value


def __dir__() -> list[str]:
    return sorted(set(globals()) | set(__all__))
```

sj_generator/ui/pages/import_flow/content_page.py

114-01

```python
from sj_generator.presentation.qt.pages.import_flow.content_page import (
    AiImportContentPage,
)

__all__ = ["AiImportContentPage"]

```

sj_generator/ui/pages/import_flow/content_support.py

115-01

```python
from sj_generator.presentation.qt.pages.import_flow.content_support import (
    ContentCompareUpdate,
    ContentWorkerBundle,
    apply_content_compare_payload,
    apply_content_detail_column_widths_if_needed,
    build_content_status_text,
    create_content_worker_bundle,
    missing_content_model_labels,
)

__all__ = [
    "missing_content_model_labels",
    "build_content_status_text",
    "apply_content_detail_column_widths_if_needed",
    "ContentCompareUpdate",
    "apply_content_compare_payload",
    "ContentWorkerBundle",
    "create_content_worker_bundle",
]

```

sj_generator/ui/pages/import_flow/question_ref_page.py

116-01

```python
from sj_generator.presentation.qt.pages.import_flow.question_ref_page import (
    AiImportPage,
)

__all__ = ["AiImportPage"]

```

sj_generator/ui/pages/import_flow/question_ref_support.py

117-01

```python
from sj_generator.presentation.qt.pages.import_flow.question_ref_support import (
    QuestionRefWorkerBundle,
    apply_question_ref_detail_column_widths,
    create_question_ref_worker_bundle,
    missing_question_ref_provider_labels,
    render_question_ref_detail_table,
)

__all__ = [
    "QuestionRefWorkerBundle",
    "missing_question_ref_provider_labels",
    "render_question_ref_detail_table",
    "apply_question_ref_detail_column_widths",
    "create_question_ref_worker_bundle",
]

```

sj_generator/ui/pages/import_flow/select_page.py

118-01

```python
from sj_generator.presentation.qt.pages.import_flow.select_page import (
    AiSelectFilesPage,
)

__all__ = ["AiSelectFilesPage"]

```

sj_generator/ui/pages/import_flow/select_support.py

119-01

```python
from sj_generator.presentation.qt.pages.import_flow.select_support import (
    build_split_import_states_for_paths,
    collect_table_items,
    find_invalid_level_paths,
    rebuild_selected_paths_table,
    refresh_after_external_doc_edit,
    reminder_doc_path,
    remove_file_row,
    serialize_selected_paths,
    update_import_reminder,
)

__all__ = [
    "serialize_selected_paths",
    "rebuild_selected_paths_table",
    "collect_table_items",
    "find_invalid_level_paths",
    "build_split_import_states_for_paths",
    "update_import_reminder",
    "reminder_doc_path",
    "remove_file_row",
    "refresh_after_external_doc_edit",
]

```

sj_generator/ui/pages/import_pages.py

120-01

```python
from .import_flow import (
    AiAnalysisPage,
    AiImportContentPage,
    AiImportPage,
    AiSelectFilesPage,
    DedupeResultPage,
    ImportSuccessPage,
)

__all__ = [
    "AiSelectFilesPage",
    "AiImportPage",
    "AiImportContentPage",
    "DedupeResultPage",
    "AiAnalysisPage",
    "ImportSuccessPage",
]

```

sj_generator/ui/pages/intro_pages.py

121-01

```python
from sj_generator.presentation.qt.pages.intro_page import IntroPage

__all__ = ["IntroPage"]

```

sj_generator/ui/pages/welcome_pages.py

122-01

```python
from sj_generator.presentation.qt.pages.welcome_page import WelcomePage

__all__ = ["WelcomePage"]

```

sj_generator/ui/program_settings_dialog.py

123-01

```python
from sj_generator.presentation.qt.program_settings_dialog import (
    BUTTON_MIN_HEIGHT,
    BUTTON_MIN_WIDTH,
    SECTION_EXPORT,
    SECTION_GENERAL,
    SECTION_IMPORT,
    ProgramSettingsDialog,
)

__all__ = [
    "SECTION_GENERAL",
    "SECTION_IMPORT",
    "SECTION_EXPORT",
    "BUTTON_MIN_WIDTH",
    "BUTTON_MIN_HEIGHT",
    "ProgramSettingsDialog",
]

```

sj_generator/ui/question_edit_dialog.py

124-01

```python
from sj_generator.presentation.qt.question_edit_dialog import (
    LevelPathPartLineEdit,
    QuestionEditDialog,
)

__all__ = ["LevelPathPartLineEdit", "QuestionEditDialog"]

```

sj_generator/ui/styles.py

125-01

```python
from sj_generator.presentation.qt.styles import APP_STYLESHEET, rounded_panel_stylesheet

__all__ = ["APP_STYLESHEET", "rounded_panel_stylesheet"]

```

sj_generator/ui/table_copy.py

126-01

```python
from sj_generator.presentation.qt.table_copy import (
    CopyableTableWidget,
    copy_table_as_tsv,
    selected_table_as_tsv,
    table_cell_text,
)

__all__ = [
    "table_cell_text",
    "selected_table_as_tsv",
    "copy_table_as_tsv",
    "CopyableTableWidget",
]

```

sj_generator/ui/welcome_export.py

127-01

```python
from sj_generator.presentation.qt.pages.welcome_export import (
    db_record_to_question,
    default_export_dir,
    digit_to_circled,
    display_export_level_name,
    export_current_level_to_markdown,
    export_current_level_to_pdf,
    export_db_records_to_xlsx,
    format_choice_mapping,
    format_db_answer,
    format_db_options,
    sanitize_export_name,
)

__all__ = [
    "sanitize_export_name",
    "display_export_level_name",
    "default_export_dir",
    "db_record_to_question",
    "format_db_options",
    "format_db_answer",
    "format_choice_mapping",
    "digit_to_circled",
    "export_current_level_to_markdown",
    "export_current_level_to_pdf",
    "export_db_records_to_xlsx",
]

```

sj_generator/ui/welcome_export_actions.py

128-01

```python
from sj_generator.presentation.qt.pages.welcome_export_actions import (
    export_current_level_markdown,
    export_current_level_pdf,
    export_current_level_xlsx,
    export_db_table_xlsx,
    selected_tree_level_path,
)

__all__ = [
    "selected_tree_level_path",
    "export_current_level_markdown",
    "export_current_level_pdf",
    "export_current_level_xlsx",
    "export_db_table_xlsx",
]

```

sj_generator/ui/welcome_import.py

129-01

```python
from sj_generator.presentation.qt.pages.welcome_import import (
    backup_import_source_files,
    build_doc_hash_index,
    effective_import_source_dir,
    file_sha256,
    next_doc_backup_path,
    open_import_flow_states,
    open_import_flow_windows,
    show_import_flow_window,
)

__all__ = [
    "effective_import_source_dir",
    "backup_import_source_files",
    "build_doc_hash_index",
    "file_sha256",
    "next_doc_backup_path",
    "open_import_flow_windows",
    "open_import_flow_states",
    "show_import_flow_window",
]

```

sj_generator/ui/welcome_menu.py

130-01

```python
from sj_generator.presentation.qt.pages.welcome_menu import (
    WelcomeMenuBundle,
    build_welcome_menu_bar,
)

__all__ = ["WelcomeMenuBundle", "build_welcome_menu_bar"]

```

sj_generator/ui/welcome_page_actions.py

131-01

```python
from sj_generator.presentation.qt.pages.welcome_page_actions import (
    TableImportResult,
    import_from_table_file,
    open_api_config,
    open_program_settings,
)

__all__ = [
    "TableImportResult",
    "open_api_config",
    "open_program_settings",
    "import_from_table_file",
]

```

sj_generator/ui/welcome_question_actions.py

132-01

```python
from sj_generator.presentation.qt.pages.welcome_question_actions import (
    QuestionEditResult,
    add_question_manually,
    build_new_question_record,
    edit_question_record,
    selected_tree_level_path_for_create,
)

__all__ = [
    "selected_tree_level_path_for_create",
    "build_new_question_record",
    "add_question_manually",
    "QuestionEditResult",
    "edit_question_record",
]

```

sj_generator/ui/welcome_table.py

133-01

```python
from sj_generator.presentation.qt.pages.welcome_table import (
    QtAlign,
    build_db_row_values,
    build_question_row_values,
    format_stem_with_sequence,
    format_table_analysis,
    populate_db_records_table,
    populate_questions_table,
    populate_row_by_values,
    set_table_item,
)

__all__ = [
    "populate_questions_table",
    "populate_db_records_table",
    "build_question_row_values",
    "build_db_row_values",
    "populate_row_by_values",
    "QtAlign",
    "format_stem_with_sequence",
    "format_table_analysis",
    "set_table_item",
]

```

sj_generator/ui/welcome_table_view.py

134-01

```python
from sj_generator.presentation.qt.pages.welcome_table_view import (
    adjust_table_font_size,
    apply_table_font_size,
    apply_table_row_resize,
    apply_table_row_resize_followup,
    load_column_visibility,
    load_table_font_point_size,
    persist_table_font_point_size,
    rebalance_visible_columns,
    refresh_table_layout_after_column_change,
    save_column_visibility,
    schedule_table_row_resize,
    set_column_visible,
)

__all__ = [
    "load_column_visibility",
    "save_column_visibility",
    "set_column_visible",
    "schedule_table_row_resize",
    "apply_table_row_resize",
    "apply_table_row_resize_followup",
    "refresh_table_layout_after_column_change",
    "rebalance_visible_columns",
    "load_table_font_point_size",
    "apply_table_font_size",
    "adjust_table_font_size",
    "persist_table_font_point_size",
]
```

sj_generator/ui/welcome_tree.py

135-01

```python
from sj_generator.presentation.qt.pages.welcome_tree import (
    build_level_tree,
    collect_expanded_level_prefixes,
    delete_action_text,
    delete_scope_text,
    display_level_parts,
    expand_item_ancestors,
    format_level_label,
    load_questions_for_tree_level,
    parse_level_parts,
    to_chinese_number,
    tree_level_key_for_path,
)

__all__ = [
    "delete_action_text",
    "delete_scope_text",
    "parse_level_parts",
    "display_level_parts",
    "tree_level_key_for_path",
    "load_questions_for_tree_level",
    "format_level_label",
    "expand_item_ancestors",
    "collect_expanded_level_prefixes",
    "build_level_tree",
    "to_chinese_number",
]
```

sj_generator/ui/welcome_tree_actions.py

136-01

```python
from sj_generator.presentation.qt.pages.welcome_tree_actions import (
    delete_level_subtree,
    handle_level_selection_changed,
    refresh_level_tree,
    show_folder_tree_context_menu,
    sync_expanded_level_prefixes_from_tree,
)

__all__ = [
    "refresh_level_tree",
    "handle_level_selection_changed",
    "show_folder_tree_context_menu",
    "sync_expanded_level_prefixes_from_tree",
    "delete_level_subtree",
]

```

sj_generator/ui/wizard_base.py

137-01

```python
from sj_generator.presentation.qt.wizard_base import (
    AppWizardBase,
    DEFAULT_WIZARD_BUTTON_LAYOUT,
    PRODUCT_TITLE,
)

__all__ = ["PRODUCT_TITLE", "DEFAULT_WIZARD_BUTTON_LAYOUT", "AppWizardBase"]

```